

SIMPY: **SIMULAÇÃO EM** **PYTHON**

um guia prático



AFONSO C. MEDINA

ISBN: 978-65-01-19235-2



9 786501 192352

SimPy: Simulação em Python

Um guia prático

Afonso C. Medina

Este livro está disponível em <https://leanpub.com/simpy>

Esta versão foi publicada em 2025-07-18 ISBN 978-65-01-19235-2



Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Medina, Afonso C.

SimPy [livro eletrônico] : simulação em Python :
um guia prático / Afonso C. Medina. – São Paulo :
Ed. do Autor, 2024.

PDF

ISBN 978-65-01-19235-2

1. Python (Linguagem de programação para computadores)
2. Simulação computacional I. Título.

24-233532 CDD-005.133

Índices para catálogo sistemático:

1. Python : Desenvolvimento de aplicações Web :
Programação : Processamento de dados 005.133

Eliete Marques da Silva - Bibliotecária - CRB-8/9380

© 2025 Afonso C. Medina

Também De Afonso C. Medina

Modelagem e Simulação de Eventos Discretos 5ª ed.

Conteúdo

1 See me, Fell me, README	2
2 Apresentação	4
2.1 Por que utilizar o SimPy?	4
2.2 Prós e contras	5
2.3 Um breve histórico do SimPy	5
2.4 Onde procurar ajuda sobre o SimPy	6
2.5 Como utilizar este livro	6
3 Instalando o SimPy	8
3.1 Passo 1: Anaconda, the easy way	8
3.2 Passo 2: Instalando o Pip (para quem não instalou o Anaconda)	9
3.3 Passo 3: Instalando o SimPy	10
3.4 Passo 4: Instalando algum Ambiente Integrado de Desenvolvimento (IDE)	10
4 Tudo depende do Python	13
4.1 Teste seus conhecimentos em Python: o problema da ruína do apostador	13
4.2 Desafios	13
4.3 Solução do desafio 1	15
4.4 Teste seus conhecimentos	16
5 Primeiros passos em SimPy: criando entidades	17
5.1 Chamada das bibliotecas random e simpy	17
5.2 Criando um environment de simulação	17
5.3 Criando um gerador de chegadas dentro do environment	18
5.4 Criando intervalos de tempo de espera com env.timeout(tempo_de_espera)	19
5.5 Executando o modelo por um tempo determinado com env.run(until=tempo_de_simulacao)	20
5.6 Conceitos desta seção	23
5.7 Desafios	23
5.8 Solução dos desafios 2 e 3	24
5.9 Teste seus conhecimentos	26
6 Criando, ocupando e desocupando recursos	28
6.1 Criando	28
6.2 Ocupando	28
6.3 Desocupando	30
6.4 Status do recurso	32

6.5 Conceitos desta seção	35
7 Juntando tudo em um exemplo: a fila M/M/1	36
7.1 Geração de chegadas de entidades	36
7.2 Realizando o atendimento no servidor	37
7.3 Uma representação alternativa para a ocupação e desocupação de recursos	40
7.4 Conceitos desta seção	41
7.5 Desafios	41
7.6 Solução dos desafios 4, 5 e 6	43
7.7 Teste seus conhecimentos	50
8 Atributos e variáveis: diferenças em SimPy	51
8.1 Atributos em modelos orientados ao objeto	53
8.2 Conceitos desta seção	54
8.3 Desafios	55
8.4 Solução dos desafios 7 e 8	56
8.5 Teste seus conhecimentos	58
9 Environments: controlando a simulação	60
9.1 Controle de execução com env.run(until=fim_da_simulação)	60
9.2 Parada por execução de todos os processos programados	62
9.3 Parada por fim de execução de processo específico por env.run(until=processo)	64
9.4 Simulação passo a passo: peek & step	65
9.5 Conceitos desta seção	66
9.6 Desafios	66
9.7 Solução dos desafios 9 e 10	68
10 Outros tipos de recursos: com prioridade e preemptivos	72
10.1 Recursos com prioridade: PriorityResource	72
10.2 Recursos que podem ser interrompidos: PreemptiveResource	74
10.3 Conceitos desta seção	78
10.4 Desafios	78
10.5 Solução dos desafios 11 e 12	80
11 Interrupções de processos: simpy.Interrupt	83
11.1 Criando quebras de equipamento	83
11.2 Interrompendo um processo sem captura por try...except	86
11.3 Conceitos desta seção	88
11.4 Desafios	88
11.5 Solução dos desafios 13 e 14	90
11.6 Teste seus conhecimentos	94
12 Armazenagem e seleção de objetos específicos com Store, FilterStore e PriorityStore	96
12.1 Construindo um conjunto de objetos com Store	96
12.2 Selecionando um objeto específico com FilterStore()	99
12.3 Criando um Store com prioridade: PriorityStore	101
12.4 Conceitos desta seção	104
12.5 Desafios	105

12.6 Solução dos desafios 15 e 16	106
12.7 Teste seus conhecimentos	114
13 Enchendo ou esvaziando caixas, tanques, estoques ou objetos com Container() . .	115
13.1 Enchendo o meu container yield meuContainer.put(quantidade)	115
13.2 Esvaziando o meu container: yield meuContainer.get(quantidade)	116
13.3 Criando um sensor para o nível atual do container	119
13.4 Conceitos desta seção	121
13.5 Desafios	122
13.6 Solução dos desafios 17 e 18	123
13.7 Teste seus conhecimentos	128
14 Criando lotes (ou agrupando) entidades durante a simulação	129
14.1 Uma tática para agrupamento de lotes utilizando o Container	129
14.2 Agrupando lotes por atributo da entidade utilizando o FilterStore	132
14.3 Desafios	135
14.4 Solução dos desafios 19 e 20	137
14.5 Teste seus conhecimentos	142
15 Criando, manipulando e disparando eventos com event()	143
15.1 Criando um evento isolado com event()	143
15.2 Conceitos desta seção	147
15.3 Desafios	148
15.4 Solução dos desafios 21 e 22	149
15.5 Teste seus conhecimentos	153
16 Aguardando múltiplos eventos ao mesmo tempo com AnyOf e Allof	155
16.1 Aguardando até que, ao menos, um evento termine com AnyOf	156
16.2 Aguardando todos os eventos com Allof	157
16.3 Compreendendo melhor as saídas dos comandos Allof e AnyOf	158
16.4 Conceitos desta seção	159
16.5 Desafios	159
16.6 Solução dos desafios 23 e 24	161
16.7 Teste seus conhecimentos	165
17 Propriedades úteis dos eventos	166
17.1 Conceitos desta seção	170
17.2 Desafios	171
17.3 Solução do desafio 25	172
17.4 Teste seus conhecimentos	172
18 Adicionando callbacks aos eventos	174
18.1 Todo processo é um evento	176
18.2 Conceitos desta seção	177
18.3 Desafios	177
18.4 Solução do desafio 26	179
18.5 Teste seus conhecimentos	181

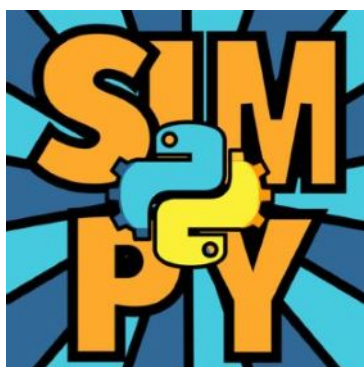
19 Interrupções de eventos	182
19.1 Interrompendo um evento com o método interrupt	182
19.2 Interrompendo um evento com o método fail	186
20 O que são funções geradoras? (ou como funciona o SimPy) - Parte I	187
20.1 Iterador	187
20.2 Funções geradoras	187
21 O que são funções geradoras? (ou como funciona o SimPy?) - Parte II	190
21.1 SimPy vs. funções geradoras	190
22 Simulação Baseada em Agentes usando o SimPy	192
22.1 Como construir um modelo de simulação de agentes: passos básicos	193
22.2 Modelo epidêmico baseado em agentes: o modelo SIR	194
22.3 Modelagem do problema no SimPy	197
22.4 Melhorando o desempenho do código de simulação	211
22.5 Como seguir a partir daqui	216
22.6 Conceitos desta seção	218
22.7 Desafios	218
22.8 Solução dos Desafios 27 e 28	219
22.9 Teste seus conhecimentos	226

1 See me, Fell me, README¹

Este livro foi concebido como uma introdução à biblioteca **SimPy** (*Simulation in Python*) para construção de modelos de simulação de eventos discretos em Python. Ele é produto do trabalho de alguém que pesquisa, ensina e utiliza profissionalmente ferramentas de simulação no seu dia-a-dia.

Trata-se de obra *em desenvolvimento* no ambiente Leanpub. Isto significa que o material é atualizada periodicamente, seja para correções, melhorias ou adições de novos conteúdos.

Propositadamente, o livro é dividido em seções curtas, enfatizando um conceito por vez, facilitando o processo de aprendizado e a busca rápida por material de referência.



Antes de prosseguir, um pequeno aviso:

A obra pressupõe que o leitor possua conhecimentos de Modelagem e Simulação de Eventos Discretos, esteja fazendo algum curso sobre o assunto ou mesmo lendo o prestigioso livro [Modelagem e Simulação de Eventos Discretos \(Chwif & Medina\)](#)².

Portanto, o livro não se pretende como fonte de consulta sobre metodologias de modelagem de sistemas, mas sim, um livro sobre uma das linguagens mais fascinantes disponíveis para se construir modelos de simulação.

Ao caro leitor que inicia aqui sua navegação, uma sugestão: mentalize e reflita sobre um projeto de simulação que gostaria de desenvolver em SimPy e, a cada seção vencida do livro, tente desenvolver em paralelo o seu próprio projeto em harmonia com o conteúdo aprendido.

Un bon voyage!

Afonso C. Medina

¹https://www.youtube.com/watch?v=RC_MS-tG5vw

²<https://leanpub.com/msed>

ATUALIZAÇÃO

Versão atual: 02/2025

Novidades:

- Correções de texto
- Figuras máquinas e lavanderia

Em desenvolvimento:

- Otimização
- Estatísticas de saída

A fazer:

- Aplicações de IA
- Comunicação com planilhas eletrônicas/txt
- Revisão Final de Código
- Revisão Final de Texto

2 Apresentação

Este livro foi elaborado a partir de seções compactadas em textos curtos, para que o leitor, a cada seção, tenha uma visão clara do conteúdo apresentado, construindo o conhecimento de maneira sólida e respeitando a sua curva de aprendizado pessoal.

SimPy (*Simulation in Python*) é um *framework*¹² para a construção de modelos de simulação de eventos discretos em Python e distribuído segundo a *licença MIT*³. Ele se diferencia dos pacotes comerciais usualmente utilizados na simulação de eventos discretos, pois não se trata de uma aplicação com objetos prontos, facilmente conectáveis entre si por simples cliques do mouse. Com o **SimPy**, cabe ao usuário construir um programa de computador em Python que represente seu modelo. Essencialmente, **SimPy** é uma biblioteca de comandos que conferem ao Python o poder de construir modelos a eventos discretos.

Com o **SimPy** pode-se construir, além de modelos de simulação discreta, modelos de simulação em “Real Time”, modelos de agentes e até mesmo modelos de simulação contínua. De fato, e como o leitor notará ao longo deste texto, essas possibilidades estão mais associadas ao Python do que propriamente aos recursos fornecidos pelo **SimPy**.

2.1 Por que utilizar o SimPy?

Talvez a pergunta correta seja: “por que utilizar o Python?”

Python é hoje, talvez, a linguagem mais utilizada no meio científico e uma breve pesquisa pela Internet vai sugerir artigos, *posts* e intermináveis discussões sobre os porquês desse sucesso todo. Eu resumiria o sucesso do Python em 3 grandes razões:

- **Facilidade de codificação.** Engenheiros, matemáticos e pesquisadores em geral querem pensar no problema, nem tanto na linguagem e Python cumpre o que promete quando se fala em facilidade. Se ela é fácil de codificar, mais fácil ainda é ler e interpretar um código feito em Python;
- **Bibliotecas! Bibliotecas!** Um número inacreditável de bibliotecas (particularmente para a área científica) está disponível para o programador (e pesquisador).
- **Scripts.** A funcionalidade de trabalhar com *scripts*⁴⁵ ou pequenos trechos de código interpretado (basicamente, Python é uma “linguagem script”) diminui drasticamente o tempo de desenvolvimento e aprendizado da linguagem.

¹<https://pt.wikipedia.org/wiki/Framework>

²Em português, ainda é comum o estrangeirismo “framework” entre profissionais da Ciência da Computação.

³https://pt.wikipedia.org/wiki/Licen%C3%A7a_MIT

⁴https://pt.wikipedia.org/wiki/Linguagem_de_script

⁵*Script* é uma sequência de comandos executados no interior de algum arquivo por meio de um interpretador. É chamado de *script*, pois eles são lidos e interpretados pelo Python, linha a linha.

Além disso, o SimPy, quando comparado com pacotes comerciais, é gratuito - o que por si só é uma grande vantagem num mercado em que os softwares são precificados a partir de milhares de dólares—e bastante flexível, no sentido de que não é engessado apenas pelos módulos existentes.

Sob o aspecto funcional, SimPy se apresenta como uma *biblioteca* em Python e isso significa que um modelo de simulação desenvolvido com ele, terá à disposição tudo que existe de bom para quem programa em Python: o código fica fácil de ler (e desenvolver), o modelo pode ser distribuído como um pacote (sem a necessidade do usuário final instalar o Python para executá-lo), além das diversas bibliotecas de estatística e otimização disponíveis em Python, que ampliam, em muito, o horizonte de aplicação dos modelos.

Esta disponibilidade de bibliotecas, bem como ser um software livre, torna o SimPy particularmente interessante para quem está desenvolvendo sua pesquisa acadêmica na área de simulação. O seu modelo provavelmente ficará melhor documentado e portanto mais fácil de ser compreendido, potencializando a divulgação dos resultados de sua pesquisa em dissertações, congressos e artigos científicos.

2.2 Prós e contras

Prós:

- Código aberto e livre ([licença MIT](https://pt.wikipedia.org/wiki/Licença_MIT))⁶;
- Diversas funções de bibliotecas de otimização, matemática e estatística podem ser incorporadas ao modelo;
- Permite a programação de lógicas sofisticadas, apoiando-se no Python (e suas bibliotecas);
- Comunidade ativa de desenvolvedores e usuários que mantém a biblioteca atualizada.

Contras:

- Desempenho: embora seja notável a melhora de desempenho do Python a cada nova versão, SimPy é uma biblioteca de simulação, isto é: ela pertence a uma classe de aplicações bastante exigente em termos computacionais. Assim, não conte com tempos de processamento comparáveis àqueles de pacotes comerciais, tais como o AnyLogic ou o Simul8, pois não será;
- Ausência de ferramentas para animação;
- Necessidade de se programar cada processo do modelo;
- Exige conhecimento prévio em Python;
- Não inclui um ambiente visual de desenvolvimento.

⁶https://pt.wikipedia.org/wiki/Licença_MIT

2.3 Um breve histórico do SimPy

O SimPy surgiu em 2002 baseado na combinação das ideias do Simscript e da família de linguagens Simula. Contextualizando, ele combina o SiPy, pacote desenvolvido por Klaus Muller, baseado no Simula, com o SimPy, desenvolvido por Tony Vignaux e Chang Chui e baseado em outra linguagem, o Simscript.

Talvez o que melhor simbolize a gênese do SimPy seja a decisão do Prof. Klaus Muller em utilizar *geradores* ou *funções geradoras* na construção do pacote SiPy. Quando os dois projetos originais foram unidos no SimPy como conhecemos hoje, o trio decidiu por seguir pelo caminho dos *geradores*, até então uma grata novidade no Python 2.2 ([estamos ainda em 2002](#)⁷, pessoal!).

Na sua infância, o SimPy foi utilizado em cursos de introdução à simulação e, aos poucos, foi despertando o interesse de pesquisadores e profissionais pelo mundo. A partir da versão 3.0, o SimPy, já contando com um número maior de desenvolvedores, foi totalmente reescrito e sua importância no mundo da simulação pode ser medida pela infinidade de trabalhos publicados em conferências e revistas especializadas, bem como na sua reimplementação em outras linguagem, tais como: C# ([Sim#](#))⁸, Julia ([ConcurrentSim](#))⁹ e R ([simmer](#))¹⁰.

2.4 Onde procurar ajuda sobre o SimPy

Existe uma variedade de vídeos e tutoriais disponíveis na Internet, mas três são as fontes de consulta que mais utilizo:

- O próprio site do projeto <http://simpy.readthedocs.io>, com exemplos e uma detalhada descrição da Interface de Programação de Aplicações (*Application Programming Interface-API*);
- A [lista de discussão](#)¹¹ de usuários é bastante ativa, com respostas bem elaboradas;
- O [Stack Overflow](#)¹² tem um número razoável de questões e exemplos, mas cuidado, pois boa parte do material ainda se refere à antiga versão 2 do SimPy, bastante diferente da versão 3, base desta obra.

2.5 Como utilizar este livro

Este texto foi planejado em formato de seções compactas, de modo que sejam curtas e didáticas — tendo por meta que cada seção não ultrapasse muito além de 500 palavras. A cada capítulo é apresentado um novo tópico por meio de um ou mais exemplos práticos. A seguir, são propostos desafios complementares para que o leitor avance seu conhecimento praticando no próprio exemplo proposto.

⁷https://gnosis.cx/publish/programming/charming_python_b1.txt

⁸<https://github.com/heal-research/SimSharp>

⁹<https://github.com/JuliaDynamics/ConcurrentSim.jl>

¹⁰<https://github.com/r-simmer/simmer>

¹¹<https://groups.google.com/forum/#!forum/python-simpy>

¹²<http://stackoverflow.com/questions/tagged/simpy>

Como todo livro que apresenta uma linguagem nova de programação, o ideal é que o leitor ou a leitora *pratique os exemplos reescrevendo o código*. Copiar e colar códigos já prontos no texto vai torná-lo um mestre Jedi no uso de teclado e mouse para a cópia e colagem de textos, mas não vai ensiná-lo a programar em Python ou SimPy. Assim, minha sugestão é que o leitor ou a leitora leia cada exemplo apresentado e reescreva-o novamente. Surgirão erros de digitação, *bugs* inesperados e dúvidas que se superadas na revisão do código, tornarão o leitor ou a leitora, de fato, um mestre Jedi da simulação em Python.

Esta é a primeira edição de um livro sobre uma linguagem que vem apresentando interesse crescente nos últimos anos. Naturalmente, o aumento de usuários—e espera-se que este livro contribua para isso—provocará também o surgimento de mais conhecimento, mais soluções criativas e que deverão ser incorporadas neste livro em futuras revisões. De fato, este texto já é a evolução do material que disponibilizei por alguns anos como tutorial no gitBook.

Encontrou algum erro ou mesmo deseja enviar alguma sugestão de melhoria? Por favor, encaminhe um e-mail para: livrosimulacao@gmail.com.

O plano proposto por este texto é caminhar pela seguinte sequência (**em permanente revisão**):

- Introdução ao SimPy;
- Instalação do pacote;
- Conceitos básicos: entidades, recursos, filas etc.;
- Conceitos avançados: prioridade de recursos, compartilhamento de recursos, controle de filas, *Store* de recursos etc.;
- Experimentação (replicações, tempo de *warm-up*, intervalos de confiança etc.);
- Simulação de agentes e otimização;
- Aplicações de IA.

De tempos em tempos, se não chover e não fizer muito Sol, o autor se compromete a atualizar para novas versões do SimPy e do Python.

3 Instalando o SimPy

Nossa jornada começa por um breve tutorial de instalação de alguns programas e bibliotecas úteis para o SimPy. Seleccionamos, para começar o tutorial, os seguintes pacotes:

1. Python 3.4
2. Pip
3. SimPy 3.0.10
4. NumPy

Um breve preâmbulo das nossas escolhas: no momento da elaboração deste tutorial, o **Python** estava na versão 3.5.0, mas o **Anaconda** (explicado adiante) ainda fornecia a versão 3.4. Se você já possui uma instalação com o Python 3.5 ou *adelante*, pode instalar o SimPy sem problemas, pois ele é compatível com as versões mais recentes do Python.

Pip é um instalador de bibliotecas e facilita muito a vida do programador.

O **SimPy** 3.0.10 é a versão mais atual no momento em que este tutorial é escrito e traz grandes modificações em relação à versão 2.0.



Existe vasto material disponível na Internet para o SimPy. Contudo, um cuidado especial deve ser tomado: grande parte deste material refere-se a versão 2, que possui diferenças críticas em relação à versão mais atual. Este texto é relativo à versão 3 em diante.

Quanto ao **NumPy**, vamos aproveitar o embalo para instalá-lo, pois será muito útil nos nossos modelos de simulação. Basicamente, NumPy acrescenta um tipo de dados (*n-dimensional array*) que facilita a codificação de modelos de simulação, particularmente na análise de dados de saída do modelo.

3.1 Passo 1: Anaconda, the easy way



- Se você já tem o Python e o Pip instalados em sua máquina, pule diretamente para o Passo 3: “Instalando o SimPy”;
- Se você já tem o Python instalado, mas não o Pip (quem tem o Python +3.4, já tem o Pip instalado), pule para o Passo 2: “Instalando o Pip”

Se esta é a sua primeira vez com Python+SimPy, sugestão: não perca tempo e instale a distribuição gratuita do [Anaconda](http://continuum.io/downloads)¹.

¹<http://continuum.io/downloads>



Por meio do Anaconda, tudo é mais fácil, limpo e o processo já instala mais de 200 pacotes verificados por toda sorte de compatibilidade, para que você não tenha trabalho algum. (Entre os pacotes instalados está o [NumPy](#)² que, como explicado, será muito útil no desenvolvimento dos seus modelos).

No momento de elaboração deste livro, eles disponibilizavam as versões 2.7, 3.4 e 3.5 do Python (em 32 e 64 bit) na [página de downloads](#)³.

Baixe o arquivo com a versão desejada (mais uma vez: SimPy roda nas duas versões) e siga as instruções do instalador.

3.2 Passo 2: Instalando o Pip (para quem não instalou o Anaconda)



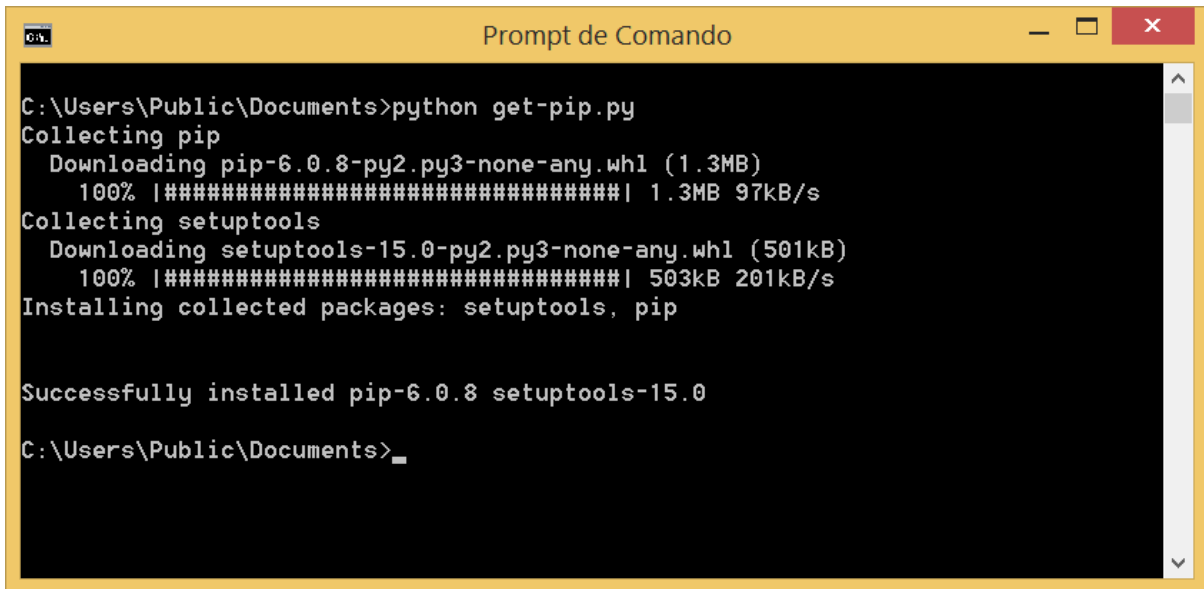
Se a versão instalada do Python for +3.4 ou você fez o passo anterior, pode pular este passo, pois o pip já foi instalado no seu computador.

1. Baixe o pacote `get-pip.py` [por meio deste link](#)⁴, salvando-o em uma pasta de trabalho conveniente.
2. Execute `python get-pip.py` na pasta de trabalho escolhida (note a mensagem ao final, confirmando que o pip foi instalado com sucesso).

²<http://www.numpy.org>

³<http://continuum.io/downloads>

⁴<https://bootstrap.pypa.io/get-pip.py>



```
C:\Users\Public\Documents>python get-pip.py
Collecting pip
  Downloading pip-6.0.8-py2.py3-none-any.whl (1.3MB)
    100% |#####| 1.3MB 97kB/s
Collecting setuptools
  Downloading setuptools-15.0-py2.py3-none-any.whl (501kB)
    100% |#####| 503kB 201kB/s
Installing collected packages: setuptools, pip

Successfully installed pip-6.0.8 setuptools-15.0

C:\Users\Public\Documents>_
```

3.3 Passo 3: Instalando o SimPy

Instalar o SimPy é fácil!

Digite em uma janela cmd:

```
1 pip install -U simpy
```



```
C:\Users\Public>pip install -U simpy
Collecting simpy
  Downloading simpy-3.0.7-py2.py3-none-any.whl (47kB)
    100% |#####| 49kB 121kB/s
Installing collected packages: simpy

Successfully installed simpy-3.0.7

C:\Users\Public>
```

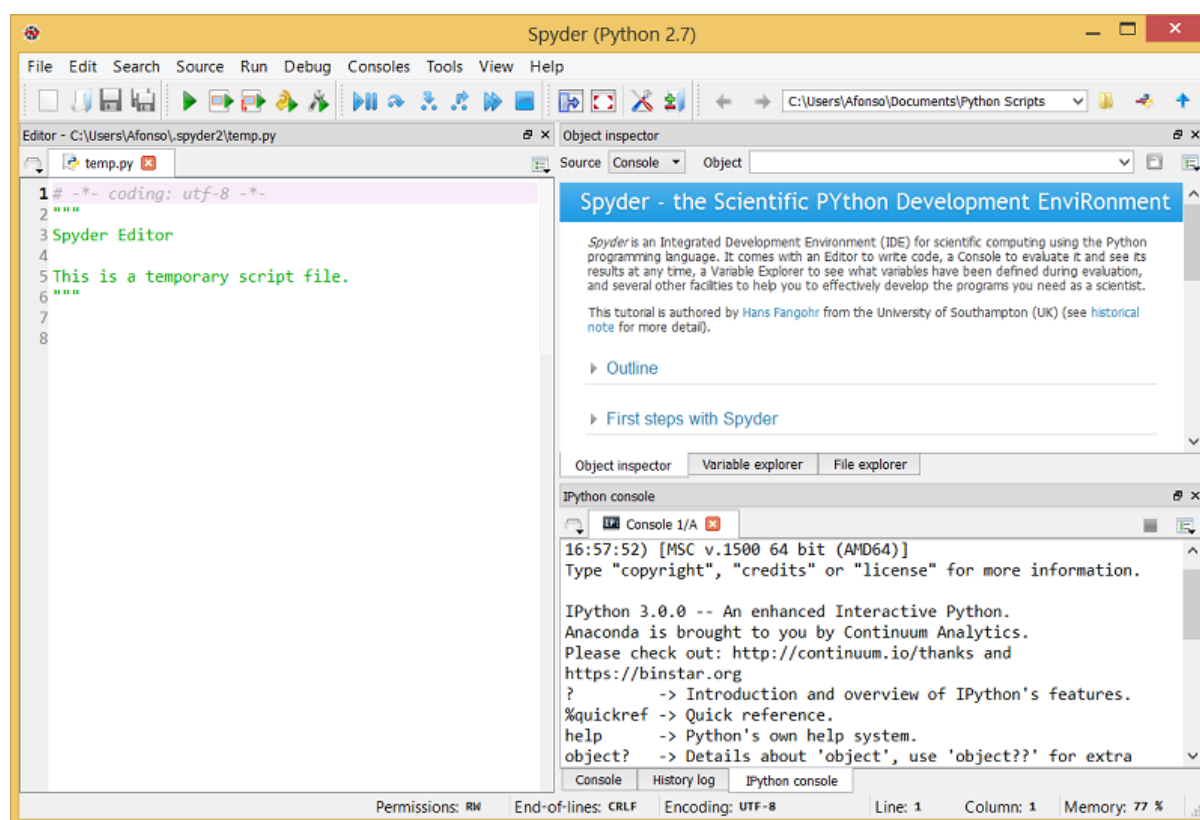
A mensagem “Sucessfully installed simpy-3.0.10” indica que você já está pronto para o SimPy. Mas, antes disso, tenho uma sugestão para você:

3.4 Passo 4: Instalando algum Ambiente Integrado de Desenvolvimento (IDE)

Os IDEs⁵, para quem não conhece, são verdadeiras interfaces de edição de código que facilitam a vida do programador. Geralmente possuem um editor de textos avançado, recursos de verificação de erros, monitores para os estados das variáveis do código, comandos de processamento passo-a-passo etc.

Se você instalou o Anaconda, então já ganhou um dos bons: o Spyder⁶, que já está configurado e pronto para o uso. Geralmente (a depender da sua versão do Sistema Operacional) ele aparece como um ícone na área de trabalho. Se não localizar o ícone, procure por Spyder no seu computador (repare na pegadinha do “y”) ou digite em uma janela de cmd o comando `spyder`.

Aberto, o Spyder fica como a figura a seguir:

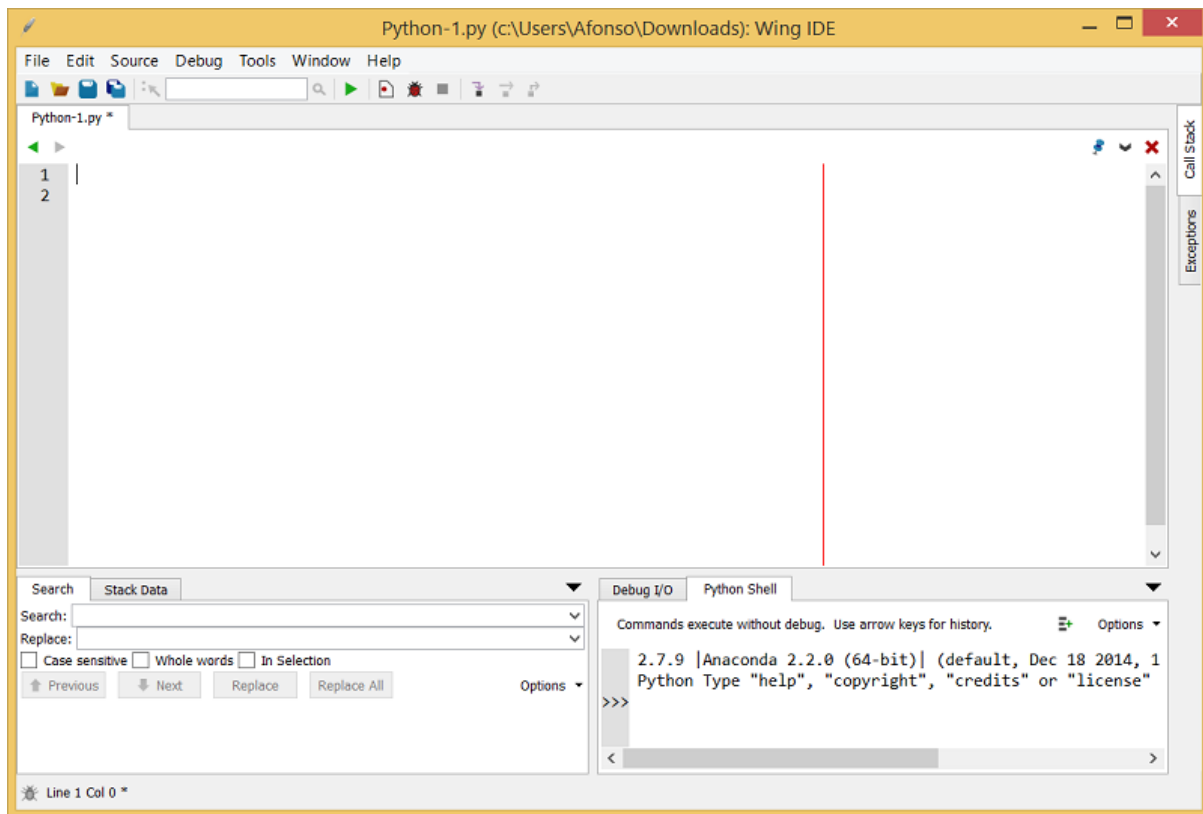


Outro IDE muito bom é o Wing IDE 101⁷ que é gratuito (e também possui uma versão profissional paga):

⁵https://pt.wikipedia.org/wiki/Ambiente_de_desenvolvimento_integrado

⁶[https://en.wikipedia.org/wiki/Spyder_\(software\)](https://en.wikipedia.org/wiki/Spyder_(software))

⁷<http://wingware.com/downloads/wingide-101>



Se você instalou o Anaconda e pretende utilizar algum IDE que não o Spyder, siga as instruções de configurações específicas deste link: [Using IDEs](https://docs.anaconda.com/working-with-conda/ide-tutorials/)⁸.

Se você chegou até aqui e tem tudo instalado, o próximo passo é começar para valer com o SimPy! Na próxima seção, é claro. Precisamos de uma pausa para um chazinho depois de tantos bytes instalados.



⁸<https://docs.anaconda.com/working-with-conda/ide-tutorials/>

4 Tudo depende do Python

Antes de começarmos com o SimPy, precisamos garantir que você tenha algum conhecimento mínimo de Python. Se você julga que seus conhecimentos na linguagem são razoáveis, a recomendação é que você pule para a seção seguinte, “Teste seus conhecimentos em Python”.

Se você nunca teve contato com a linguagem, aviso que não pretendo construir uma “introdução” ou “tutorial” para o Python, simplesmente porque isso é o que mais existe à disposição na internet.

Procure um tutorial rápido (existem tutoriais de até 10 minutos!) e mãos à obra! Nada mais fácil do que aprender o básico de Python.

Sugestões:

1. [Sololearn](https://www.sololearn.com)¹: um dos modos mais rápidos e bacanas de se aprender Python (e outras linguagens). São diversos cursos, divididos por níveis e que podem ser seguidos até pelo celular (sim, aprendi Java viajando pela linha azul do metrô de SP).
2. [Introdução à Programação Interativa em Python \(Parte 1\)](https://pt.coursera.org/course/interactivepython1)²: o Coursera tem um curso que vai ensiná-lo a criar programas que interagem com o usuário, isto é: joguinhos 😊. Eu fiz, é bom!
3. *Summerfield, Mak. Programming in Python 3: a complete introduction. Addison-Wesley Professional. 2012*: eu tenho sempre na minha mesa este ótimo livro de Python. Ele já foi traduzido para o Português e pode ser encontrado na [Amazon](https://www.amazon.com.br/Programa%C3%A7%C3%A3o-Em-Python-Mark-Summerfield/dp/8576083841/ref=sr_1_7?s=books&ie=UTF8&qid=1448738880&sr=1-7&keywords=python)³.

Feito o tutorial, curso ou aprendido mesmo tudo sozinho, teste seus conhecimentos para verificar se você sabe o *básico necessário* de Python para começar com o SimPy.

4.1 Teste seus conhecimentos em Python: o problema da ruína do apostador

O [problema da ruína do apostador](#)⁴ é um problema clássico proposto por Pascal em uma carta para Fermat, em 1656. A versão aqui apresentada é uma simplificação, visando avaliar seus conhecimentos em Python.

4.2 Desafios



Desafio 1: dois apostadores iniciam um jogo de cara ou coroa em que cada um deles aposta \$1 sempre em um mesmo lado da moeda. O vencedor leva a aposta total (\$2). Cada jogador tem inicialmente \$5 disponíveis para apostar. O jogo termina quando um dos jogadores atinge a ruína e não tem mais dinheiro para apostar.

¹<https://www.sololearn.com>

²<https://pt.coursera.org/course/interactivepython1>

³http://www.amazon.com.br/Programa%C3%A7%C3%A3o-Em-Python-Mark-Summerfield/dp/8576083841/ref=sr_1_7?s=books&ie=UTF8&qid=1448738880&sr=1-7&keywords=python

⁴http://en.wikipedia.org/wiki/Gambler%27s_ruin

Construa três funções:

1. `transfer(winner, loser, bankroll, tossCount)`: transfere o valor do jogador perdedor para o vencedor e imprime na tela o nome do vencedor;
2. `coinToss(bankroll, tossCount)`: sorteia o vencedor do cara ou coroa;
3. `run2Ruin(bankroll)`: mantém um laço permanente até que um dos jogadores atinja a ruína

Teste o programa com os parâmetros sugeridos (você pode utilizar o código a seguir como uma máscara para o seu programa):

```
1  import random                # gerador de números aleatórios
2
3  names = ['Chewbacca', 'R2D2'] # jogadores
4
5  def transfer(winner, loser, bankroll, tossCount):
6      # função que transfere o dinheiro do winner para o loser
7      # imprime o vencedor do lançamento e o bankroll de cada jogador
8      pass
9
10 def coinToss(bankroll, tossCount):
11     # função que sorteia a moeda e chama a transfer
12     pass
13
14 def run2Ruin(bankroll):
15     # função que executa o jogo até a ruína de um dos jogadores
16     pass
17
18 bankroll = [5, 5]             # dinheiro disponível para cada jogador
19 run2Ruin(bankroll)            # inicia o jogo
```

Agora é com você: complete o código anterior e descubra se você está pronto para iniciar com o SimPy!

(A próxima seção apresenta uma possível resposta para o desafio e, na sequência, tudo enfim, começa.)

4.3 Solução do desafio 1

Desafio 1: dois apostadores iniciam um jogo de cara ou coroa em que cada um deles aposta \$1 sempre em um mesmo lado da moeda. O vencedor leva a aposta total (\$2). Cada jogador tem inicialmente \$5 disponíveis para apostar. O jogo termina quando um dos jogadores atinge a ruína e não tem mais dinheiro para apostar.

O código a seguir é uma possível solução para o desafio 1 da seção anterior. Naturalmente é possível deixá-lo mais claro, eficiente, obscuro, maligno, elegante, rápido ou lento, como todo código de programação. O importante é que se você fez alguma coisa que funcione, acredito que é o suficiente para começar com o SimPy.

```

1  import random                                # gerador de números aleatórios
2
3  names = ['Chewbacca', 'R2D2']               # jogadores
4
5  def transfer(winner, loser, bankroll, tossCount):
6      # função que transfere o dinheiro do winner para o loser
7      # imprime o vencedor do lançamento e o bankroll de cada jogador
8      bankroll[winner] += 1
9      bankroll[loser] -= 1
10     print("\nLançamento: %d\tVencedor: %s" % (tossCount, names[winner]))
11     print("%s possui: %d e %s possui: %d"
12           % (names[0], bankroll[0], names[1], bankroll[1]))
13
14 def coinToss(bankroll, tossCount):
15     # função que sorteia a moeda e chama a transfer
16     if random.uniform(0, 1) < 0.5:
17         transfer(1, 0, bankroll, tossCount)
18     else:
19         transfer(0, 1, bankroll, tossCount)
20
21 def run2Ruin(bankroll):
22     # função que executa o jogo até a ruína de um dos jogadores
23     tossCount = 0                               # contador de lançamentos
24     while bankroll[0] > 0 and bankroll[1] > 0:
25         tossCount += 1
26         coinToss(bankroll, tossCount)
27         winner = bankroll[1] > bankroll[0]
28         print("\n%s venceu depois de %d lançamentos, fim de jogo!"
29               % (names[winner], tossCount))
30
31 bankroll = [5, 5]                               # dinheiro disponível para cada jogador
32 run2Ruin(bankroll)                             # inicia o jogo

```

No meu computador, o problema anterior fornece o seguinte resultado:

```
1 Lançamento: 1    Vencedor: Chewbacca
2 Chewbacca possui: $6 e R2D2 possui: $4
3
4 Lançamento: 2    Vencedor: Chewbacca
5 Chewbacca possui: $7 e R2D2 possui: $3
6
7 Lançamento: 3    Vencedor: Chewbacca
8 Chewbacca possui: $8 e R2D2 possui: $2
9
10 Lançamento: 4    Vencedor: Chewbacca
11 Chewbacca possui: $9 e R2D2 possui: $1
12
13 Lançamento: 5    Vencedor: Chewbacca
14 Chewbacca possui: $10 e R2D2 possui: $0
15
16 Chewbacca venceu depois de 5 lançamentos, fim de jogo!
```

Note que o resultado fornecido deve ser diferente em seu computador, assim como ele se modifica a cada nova rodada no programa. Para que os resultados entre o meu computador e o seu sejam semelhantes, precisamos garantir que a sequência de números aleatórios nos dois computadores sejam as mesmas. Isso é possível com o comando `random.seed(semente)`, que estabelece um valor inicial fixo para a semente da sequência de números aleatórios gerada (veja o item 1 na seção “Teste seus conhecimentos”, na seção a seguir).

4.4 Teste seus conhecimentos

1. Cada vez que você executa o programa, a função `random.uniform(0,1)` sorteia um novo número aleatório entre 0 e 1, tornando imprevisível o resultado do programa. Utilize a função `random.seed(semente)` para fazer com que a sequência gerada de números aleatórios seja sempre a mesma.
2. Acrescente um laço no programa principal de modo que o jogo possa ser repetido até um número pré definido de vezes. Simule 100 partidas e verifique em quantas partidas cada um dos jogadores venceu.
3. O Problema da Ruína do Apostador é comumente aplicado em situações de apostadores jogando contra Cassinos. Nesta situação, o apostador possui uma quantidade finita de dinheiro e joga contra outro apostador, o Cassino, com quantidade muito grande ou infinita. Considere que R2D2 é um Cassino e tem, portanto, um *bankroll* infinito, enquanto que o do Chewbacca é limitado a \$5. Como você utilizaria o simulador construído para demonstrar que a probabilidade de ruína do apostador Chewbacca é igual a 1? (Note que a afirmação é verdadeira mesmo considerando que o jogo é honesto e não há cobrança de nenhuma taxa por parte do Cassino).

5 Primeiros passos em SimPy: criando entidades

Algo elementar em qualquer pacote de simulação é uma função para criar entidades dentro do modelo. É o “*Alô mundo!*”¹ dos pacotes de simulação. Sua primeira missão, caso decida aceitá-la, será construir uma função que gere entidades com intervalos entre chegadas sucessivas exponencialmente distribuídos, com média de 2 min. Simule o sistema por 10 minutos apenas.

5.1 Chamada das bibliotecas random e simpy

Inicialmente, serão necessárias duas bibliotecas do Python: a *random* – biblioteca de geração de números aleatórios – e a *simpy*, que é o próprio SimPy.

Nosso primeiro modelo de simulação em SimPy começa com as chamadas das respectivas bibliotecas de interesse:

```
1 import random          # gerador de números aleatórios
2 import simpy           # biblioteca de simulação
3
4 random.seed(1000)      # semente do gerador de números aleatórios
```

Note a linha final `random.seed(1000)`. Ela garante que a geração de números aleatórios sempre começará pela mesma semente, de modo que a sequência de números aleatórios gerados a cada execução programa será sempre a mesma, facilitando o processo de verificação do programa.

5.2 Criando um environment de simulação

Tudo no SimPy gira em torno de **eventos** gerados por funções e todos os eventos devem ocorrer em um **environment**, ou um “ambiente” de simulação criado a partir da função `simpy.Environment()`.

Assim, nosso programa deve conter ao menos uma chamada à função `simpy.Environment()`, criando um *environment* “env”:

¹http://pt.wikipedia.org/wiki/Programa_Olá_Mundo


```
1 import random          # gerador de números aleatórios
2 import simpy            # biblioteca de simulação
3
4 random.seed(1000)       # semente do gerador de números aleatórios
5 env = simpy.Environment() # cria o environment do modelo na variável env
```

Se você executar o programa anterior, nada acontece. No momento, você apenas criou um *environment*, mas não criou nenhum processo, portanto, não existe ainda nenhum evento a ser simulado pelo SimPy.

5.3 Criando um gerador de chegadas dentro do environment

Vamos escrever uma função `geraChegadas()` que cria entidades no sistema enquanto durar a simulação. Nosso primeiro gerador de entidades terá três parâmetros de entrada: o *environment*, um atributo que representará o nome da entidade e a taxa desejada de chegadas de entidades por unidade de tempo. Para o SimPy, equivale dizer que você vai construir uma *função geradora de eventos* dentro do *environment* criado. No caso, os eventos gerados serão as chegadas de entidades no sistema.

Assim, nosso código começa a ganhar corpo:

```
1 import random          # gerador de números aleatórios
2 import simpy            # biblioteca de simulação
3
4 def geraChegadas(env, nome, taxa):
5     # função que cria chegadas de entidades no sistema
6     pass
7
8 random.seed(1000)       # semente do gerador de números aleatórios
9 env = simpy.Environment() # cria o environment do modelo
```

Precisamos informar ao SimPy que a função `geraChegadas()` é, de fato, um processo que deve ser executado ao longo de toda a simulação. Um processo é criado dentro do *environment*, pelo comando:

```
1 env.process(função_que_gera_o_processo)
```

A chamada ao processo é sempre feita após a criação do *env*, então basta acrescentar uma nova linha ao nosso código:

```

1  import random                # gerador de números aleatórios
2  import simpy                 # biblioteca de simulação
3
4  def geraChegadas(env, nome, taxa):
5  # função que cria chegadas de entidades no sistema
6      pass
7
8  random.seed(1000)           # semente do gerador de números aleatórios
9  env = simpy.Environment()   # cria o environment do modelo
10 # cria o processo de chegadas
11 env.process(geraChegadas(env, "Cliente", 2))

```

5.4 Criando intervalos de tempo de espera com `env.timeout(tempo_de_espera)`

Inicialmente, precisamos gerar intervalos de tempos aleatórios, exponencialmente distribuídos, para representar os tempos entre chegadas sucessivas das entidades. Para gerar chegadas com intervalos exponenciais, utilizaremos a biblioteca `random`, bem detalhada na sua [documentação](#)², e que possui a função:

```
1 random.expovariate(lambd)
```

Onde `lambd` é a taxa de ocorrência dos eventos ou, matematicamente, o inverso do tempo médio entre eventos sucessivos. No caso, se queremos que as chegadas ocorram entre intervalos médios de 2 min, a função ficaria:

```
1 random.expovariate(lambd=1.0/2.0)
```

A linha anterior é basicamente nosso gerador de números aleatórios exponencialmente distribuídos. O passo seguinte será informar ao SimPy que queremos nossas entidades surgindo no sistema segundo a distribuição definida. Isso é feito pela chamada da palavra reservada `yield` com a função do SimPy `env.timeout(intervalo)`, que nada mais é do que uma função que causa um atraso de tempo, um *delay* no tempo dentro do *environment* `env` criado:

```
1 yield env.timeout(random.expovariate(1.0/2.0))
```

Na linha de código anterior estamos executando `yield env.timeout(0.5)` para que o modelo retarde o processo num tempo aleatório gerado pela função `random.expovariate(0.5)`.

Oportunamente, discutiremos mais a fundo qual o papel do palavra `yield` (*spoiler*: ela não é do SimPy, mas originalmente do próprio Python). Por hora, considere que ela é apenas uma maneira de **criar eventos** dentro do `env` e que, caso uma função represente um processo, obrigatoriamente

²<https://docs.python.org/2/library/random.html>

ela precisará conter o comando `yield` *alguma coisa*, bem como o respectivo `environment` do processo.



Uma função criada no Python (com o comando `def`) só é tratada como um **processo** ou **gerador de eventos** para o SimPy, caso ela contenha ao menos uma linha de código com o comando `yield`. Mais adiante, a seção “O que são funções geradoras”, explica em mais detalhe o funcionamento do `yield`.

Colocando tudo junto na função `geraChegadas()`, temos:

```

1  import random                # gerador de números aleatórios
2  import simpy                 # biblioteca de simulação
3
4  def geraChegadas(env, nome, taxa):
5      # função que cria chegadas de entidades no sistema
6      contaChegada = 0
7      while True:
8          yield env.timeout(random.expovariate(1.0/taxa))
9          contaChegada += 1
10         print("%s %i chega em: %.1f " % (nome, contaChegada, env.now))
11
12 random.seed(1000)            # semente do gerador de números aleatórios
13 env = simpy.Environment()    # cria o environment do modelo
14
15 # cria o processo de chegadas
16 env.process(geraChegadas(env, "Cliente", 2))

```

O código deve ser autoexplicativo: o laço `while` é **infinito** enquanto dure a simulação; um contador, `contaChegada`, armazena o total de entidades geradas e a função `print`, imprime na tela o instante de chegada de cada cliente. Note que, dentro do `print`, existe uma chamada para a **hora atual de simulação** `env.now`.

Por fim, uma chamada a função `random.seed()` garante que os números aleatórios a cada execução do programa serão os mesmos.

5.5 Executando o modelo por um tempo determinado com `env.run(until=tempo_de_simulacao)`

Se você executar o código anterior, nada acontece novamente, pois ainda falta informarmos ao SimPy qual o tempo de duração da simulação. Isto é feito pelo comando:

```

1  env.run(until=tempo_de_simulacao)

```

No exemplo proposto, o tempo de simulação deve ser de 10 min, como representado na linha 15 do código a seguir:

```
1 import random          # gerador de números aleatórios
2 import simpy            # biblioteca de simulação
3
4 def geraChegadas(env, nome, taxa):
5     # função que cria chegadas de entidades no sistema
6     contaChegada = 0
7     while True:
8         yield env.timeout(random.expovariate(1/taxa))
9         contaChegada += 1
10        print("%s %i chega em: %.1f " % (nome, contaChegada, env.now))
11
12 random.seed(1000)      # semente do gerador de números aleatórios
13 env = simpy.Environment() # cria o environment do modelo
14
15 # cria o processo de chegadas
16 env.process(geraChegadas(env, "Cliente", 2))
17
18 # roda a simulação por 10 unidades de tempo
19 env.run(until=10)
```

Ao executar o programa, temos a saída:

```
1 Cliente 1 chega em: 3.0
2 Cliente 2 chega em: 5.2
3 Cliente 3 chega em: 5.4
4 Cliente 4 chega em: 6.3
5 Cliente 5 chega em: 7.6
6 Cliente 6 chega em: 9.1
```

Agora sim!

Note que `env.process(geraChegadas(env))` é um comando que **torna** a função `geraChegadas()` um **processo** ou um **gerador de eventos** dentro do `Environment` `env`. Esse processo só começa a ser executado na linha seguinte, quando `env.run(until=10)` informa ao SimPy que todo processo pertencente ao `env` deve ser executado por um **tempo de simulação** igual a 10 minutos.

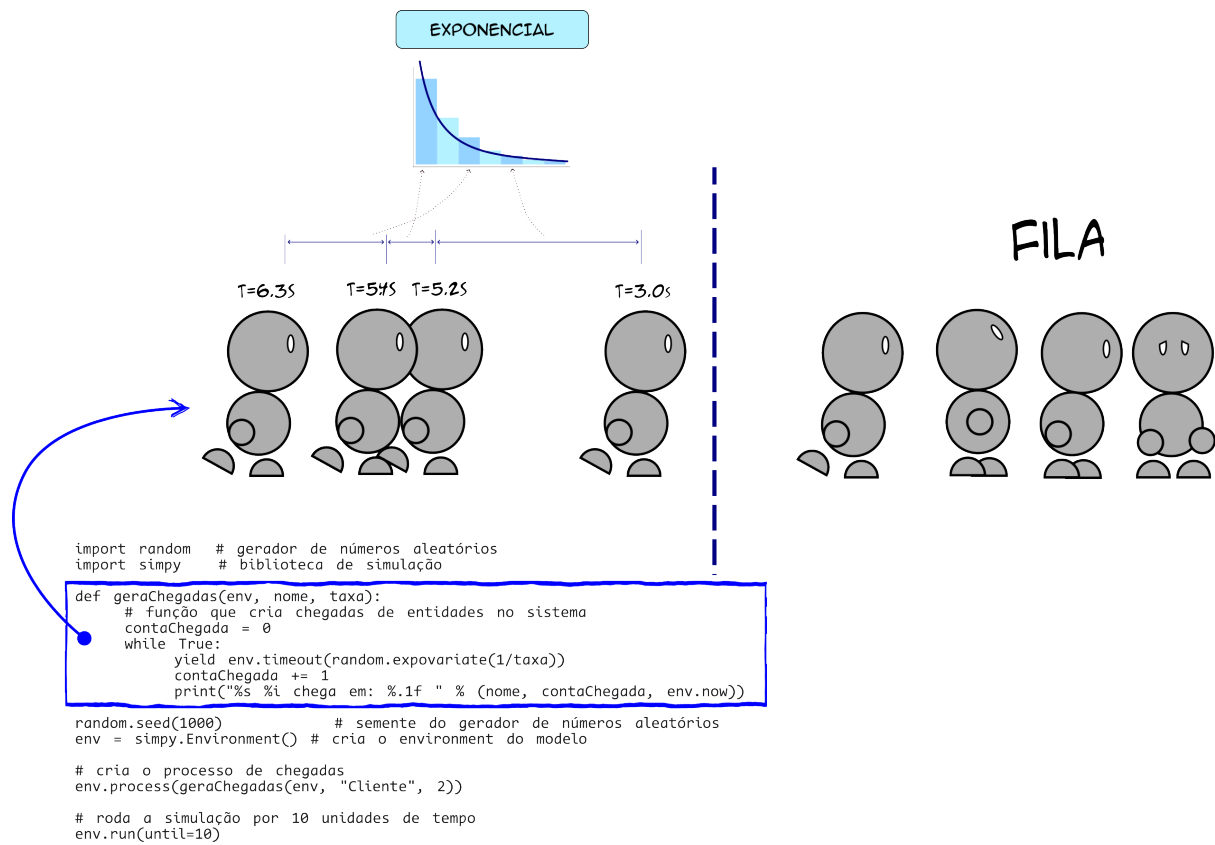


Figura 5.1. Representação das chegadas na fila.

5.6 Conceitos desta seção

Conteúdo	Descrição
<code>env = simpy.Environment()</code>	cria um <code>Environment</code> de simulação
<code>random.expovariate(lambd)</code>	gera números aleatórios exponencialmente distribuídos, com taxa de ocorrência (eventos/unidade de tempo) igual a <code>lambd</code>
<code>yield env.timeout(time)</code>	gera um atraso dado por <code>time</code>
<code>random.seed(seed)</code>	define o gerador de sementes aleatórias para um mesmo valor a cada nova simulação
<code>env.process(geraChegadas(env))</code>	inicia a função <code>geraChegadas</code> como um <i>processo</i> em <code>env</code>
<code>env.run(until=tempoSim)</code>	executa a simulação (executa todos os processos criados em <code>env</code>) pelo tempo <code>tempoSim</code>
<code>env.now</code>	retorna o instante atual da simulação

5.7 Desafios



Desafio 2: é comum que os comandos de criação de entidades nos [softwares proprietários](#)³ tenham a opção de limitar o número máximo de entidades geradas durante a simulação.

Modifique a função `geraChegadas` de modo que ela receba como parâmetro `numeroMaxChegadas` e limite a criação de entidades a este número.



Desafio 3: modifique a função `geraChegadas` de modo que as chegadas entre entidades sejam distribuídas segundo uma distribuição triangular de moda 1, menor valor 0,1 e maior valor 1,1.

³https://pt.wikipedia.org/wiki/Software_proprietário

5.8 Solução dos desafios 2 e 3

Desafio 2: é comum que os comandos de criação de entidades nos softwares proprietários tenham a opção de limitar o número máximo de entidades geradas durante a simulação.

Modifique a função `geraChegadas` de modo que ela receba como parâmetro o `numeroMaxChegadas` e limite a criação de entidades a este número.

Neste caso, o *script* em Python é autoexplicativo, apenas note que limitei o número de chegadas em 5 e fiz isso antes da chamada do processo gerado pela função `geraChegadas()`:

```
1  import random      # gerador de números aleatórios
2  import simpy        # biblioteca de simulação
3
4  def geraChegadas(env, nome, taxa, numeroMaxChegadas):
5      # função que cria chegadas de entidades no sistema
6      contaChegada = 0
7      while (contaChegada < numeroMaxChegadas):
8          yield env.timeout(random.expovariate(1/taxa))
9          contaChegada += 1
10         print("%s %i chega em: %.1f " % (nome, contaChegada, env.now))
11
12 random.seed(1000)    # semente do gerador de números aleatórios
13 env = simpy.Environment() # cria o environment do modelo
14 # cria o processo de chegadas
15 env.process(geraChegadas(env, "Cliente", 2, 5))
16 env.run(until=10) # executa a simulação por 10 unidades de tempo
```

Desafio 3: modifique a função `geraChegadas` de modo que as chegadas entre entidades sejam distribuídas segundo uma distribuição triangular de moda 1, menor valor 0,1 e maior valor 1,1.

Neste caso, precisamos verificar na documentação da biblioteca `random`, quais são nossas opções. A tabela a seguir, resume as distribuições disponíveis:

Função	Distribuição
<code>random.random()</code>	gera números aleatórios no intervalo [0.0, 1.0)
<code>random.uniform(a, b)</code>	uniforme no intervalo [a, b]
<code>random.triangular(low, high, mode)</code>	triangular com menor valor <i>low</i> , maior valor <i>high</i> e moda <i>mode</i>
<code>random.betavariate(alpha, beta)</code>	beta com parâmetros <i>alpha</i> e <i>beta</i>
<code>random.expovariate(lambd)</code>	exponencial com média $1/\text{lambd}$
<code>random.gammavariate(alpha, beta)</code>	gamma com parâmetros <i>alpha</i> e <i>beta</i>
<code>random.gauss(mu, sigma)</code>	normal com média <i>mu</i> e desvio padrão <i>sigma</i>
<code>random.lognormvariate(mu, sigma)</code>	lognormal com média <i>mu</i> e desvio padrão <i>sigma</i>
<code>random.normalvariate(mu, sigma)</code>	equivalente à <code>random.gauss</code> , mas um pouco mais lenta
<code>random.vonmisesvariate(mu, kappa)</code>	distribuição de von Mises ⁴ com parâmetros <i>mu</i> e <i>kappa</i>
<code>random.paretovariate(alpha)</code>	pareto com parâmetro <i>alpha</i>
<code>random.weibullvariate(alpha, beta)</code>	weibull com parâmetros <i>alpha</i> e <i>beta</i>

A biblioteca NumPy, que veremos oportunamente, possui mais opções para distribuições estatísticas. Por enquanto, o desafio 3 pode ser solucionado de maneira literal:

```

1  import random      # gerador de números aleatórios
2  import simpy       # biblioteca de simulação
3
4  def geraChegadas(env, nome, numeroMaxChegadas):
5      # função que cria chegadas de entidades no sistema
6      contaChegada = 0
7      while (contaChegada < numeroMaxChegadas):
8          yield env.timeout(random.triangular(0.1,1,1.1))
9          contaChegada += 1
10         print("%s %i chega em: %.1f " % (nome, contaChegada, env.now))
11
12 random.seed(1000)      # semente do gerador de números aleatórios
13 env = simpy.Environment() # cria o environment do modelo
14 # cria o processo de chegadas
15 env.process(geraChegadas(env, "Cliente", 5))
16 env.run(until=10)

```



Dica: os modelos de simulação, com muitos processos de chegadas e atendimento, tendem a utilizar diversas funções diferentes de distribuição de probabilidades, deixando as coisas meio confusas para o programador. Uma dica bacana é criar uma função que armazene todas as distribuições do modelo em um único lugar, como uma prateleira de distribuições.

Por exemplo, imagine um modelo em SimPy que possui 3 processos: um exponencial com média 10 min, um triangular com parâmetros (10, 20, 30) min e um normal com média 0 e desvio 1 minuto. A função `distribution()` a seguir, armazena todos os geradores de números aleatórios em um único local:

```
1 import random
2
3 def distributions(tipo):
4     return {
5         'arrival': random.expovariate(1/10.0),
6         'singing': random.triangular(10, 20, 30),
7         'applause': random.gauss(10, 1),
8     }.get(tipo, 0.0)
```

O próximo exemplo testa como chamar a função:

```
1 import random
2
3 def distributions(tipo):
4     return {
5         'arrival': random.expovariate(1/10.0),
6         'singing': random.triangular(10, 20, 30),
7         'applause': random.gauss(10, 1),
8     }.get(tipo, 0.0)
9
10 tipo = 'arrival'
11 print(tipo, distributions(tipo))
12
13 tipo = 'singing'
14 print(tipo, distributions(tipo))
15
16 tipo = 'applause'
17 print(tipo, distributions(tipo))
```

O qual produz a saída:

```
1 arrival 6.231712146858156
2 singing 22.192356552471104
3 applause 10.411795571842426
```

Essa foi a nossa dica do dia!

Fique a vontade para implementar funções de geração de números aleatórios ao seu gosto. Note, e isso é importante, que **praticamente todos os seus modelos de simulação em SimPy precisarão deste tipo de função!**

5.9 Teste seus conhecimentos

1. Acrescente ao programa inicial, uma função `distribution` como a proposta na seção “Dica” e faça o tempo entre chegadas sucessivas de entidades chamar a função para obter o valor correto.
2. Considere que 50% das entidades geradas durante a simulação são do sexo feminino e 50% do sexo masculino. Modifique o programa para que ele sorteie o gênero dos clientes. Faça esse sorteio dentro da função `distribution` já criada.

6 Criando, ocupando e desocupando recursos

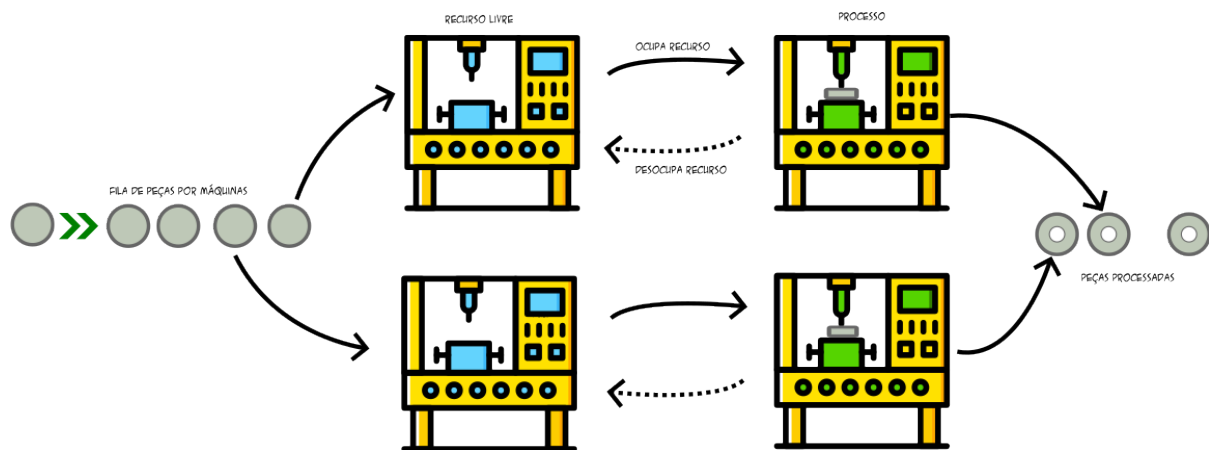


Figura 6.1. Modelo de ocupação das máquinas.

6.1 Criando

Em simulação, é usual representarmos processos que consomem recursos com alguma limitação de capacidade, tais como: máquinas de usinagem, operários em uma fábrica, empilhadeiras em um depósito etc. Quando um recurso é requisitado e não está disponível, há formação de uma fila de espera de entidades pela liberação do recurso.

Por exemplo, considere a simulação de uma fábrica, onde são necessários recursos “máquinas” que serão utilizados nos processos de fabricação.

No SimPy, a sintaxe para criar um recurso é:

```
1 import simpy
2
3 env = simpy.Environment()
4
5 # cria recurso maquinas com capacidade 2
6 maquinas = simpy.Resource(env, capacity=2)
```

Se o parâmetro `capacity` não for fornecido, a função assume `capacity=1`. Note que `maquinas` foi criado dentro do `Environment env`.

6.2 Ocupando

Como comentado ao início da seção, muitos processos em simulação ocupam recursos. Assim, ocupar um recurso em um processo exige a codificação de uma função específica em que um dos argumentos deve ser o próprio recurso a ser utilizado. O trecho de código a seguir, exemplifica para o caso das máquinas já criadas:

```
1  import simpy
2
3  def processo(env, entidade, maquinas):
4      # função que ocupa o recurso e realiza o atendimento
5      pass
6
7  env = simpy.Environment()
8
9  # cria recurso maquinas com capacidade 2
10 maquinas = simpy.Resource(env, capacity=2)
```

É interessante notar que ocupar um recurso no SimPy é feito em duas etapas:

1. **Requisitar** o recurso desejado com um `req = recurso.request()` (o que é equivalente a entrar na fila para de acesso ao recurso);
2. **Ocupar** o recurso com um `yield req`.

Assim, uma chamada ao recurso maquinas ficaria:

```
1  import simpy
2
3  def processo(env, entidade, maquinas):
4      # função que ocupa o recurso e realiza o atendimento
5      print("%s chega em %s" %(entidade, env.now))
6
7      # solicita o recurso e ocupa a fila
8      req = maquinas.request()
9
10     # aguarda em fila até a ocupação do 1o recurso livre
11     yield req
12     print("%s ocupa recurso em %s" %(entidade, env.now))
13
14 env = simpy.Environment()
15
16 # cria recurso maquinas com capacidade 2
17 maquinas = simpy.Resource(env, capacity=2)
```

Enquanto a entidade estiver em fila aguardando a liberação do recurso, ela permanece na linha do comando `yield req`. Quando ela finalmente ocupa o recurso, a execução passa para a linha seguinte (comando `print`, no caso do exemplo).

Se pode parecer estranho que a ocupação de um recurso envolva duas linhas de código, o bom observador deve notar que isso pode dar flexibilidade em situação de lógica intrincada.

6.3 Desocupando

Recurso criado e ocupado é liberado com a função `release(req)`. Considerando, por exemplo, que o processamento de peça leva 5 minutos em uma máquina, nossa função `processo` ficaria:

```
1  import simpy
2
3  def processo(env, entidade, maquinas):
4      # função que ocupa o recurso e realiza o atendimento
5      print("%s chega em %s" %(entidade, env.now))
6
7      # solicita o recurso e ocupa a fila
8      req = maquinas.request()
9
10     # aguarda em fila até a ocupação do 1o recurso livre
11     yield req
12     print("%s ocupa recurso em %s" %(entidade, env.now))
13
14     # executa o processo
15     yield env.timeout(5)
16
17     # libera o recurso
18     yield maquinas.release(req)
19     print("%s libera o recurso em %s" %(entidade, env.now))
20
21 env = simpy.Environment()
22
23 # cria recurso maquinas com capacidade 2
24 maquinas = simpy.Resource(env, capacity=2)
```

Para testarmos nossa função, vamos executar o processo para apenas 4 peças e analisar o resultado. O código a seguir possui um laço `for` que chama a função `processo` 4 vezes no mesmo instante:

```

1  import simpy
2
3  def processo(env, entidade, maquinas):
4      # função que ocupa o recurso e realiza o atendimento
5      print("%s chega em %s" %(entidade, env.now))
6
7      # solicita o recurso e ocupa a fila
8      req = maquinas.request()
9
10     # aguarda em fila até a ocupação do 1o recurso livre
11     yield req
12     print("%s ocupa recurso em %s" %(entidade, env.now))
13
14     # executa o processo
15     yield env.timeout(5)
16
17     # libera o recurso
18     yield maquinas.release(req)
19     print("%s libera o recurso em %s" %(entidade, env.now))
20
21 env = simpy.Environment()
22
23 # cria recurso maquinas com capacidade 2
24 maquinas = simpy.Resource(env, capacity=2)
25 for i in range(1,5):
26     env.process(processo(env, "Peça %s" %i, maquinas))
27 env.run()

```

Quando executado, o programa retorna:

```

1  Peça 1 chega em 0
2  Peça 2 chega em 0
3  Peça 3 chega em 0
4  Peça 4 chega em 0
5  Peça 1 ocupa recurso em 0
6  Peça 2 ocupa recurso em 0
7  Peça 1 libera o recurso em 5
8  Peça 2 libera o recurso em 5
9  Peça 3 ocupa recurso em 5
10 Peça 4 ocupa recurso em 5
11 Peça 3 libera o recurso em 10
12 Peça 4 libera o recurso em 10

```

A saída da simulação permite concluir que as quatro peças chegaram no instante 0, mas como a capacidade do recurso era para apenas 2 peças simultâneas, as peças 3 e 4 tiveram que aguardar em fila até a liberação das máquinas no instante 5.

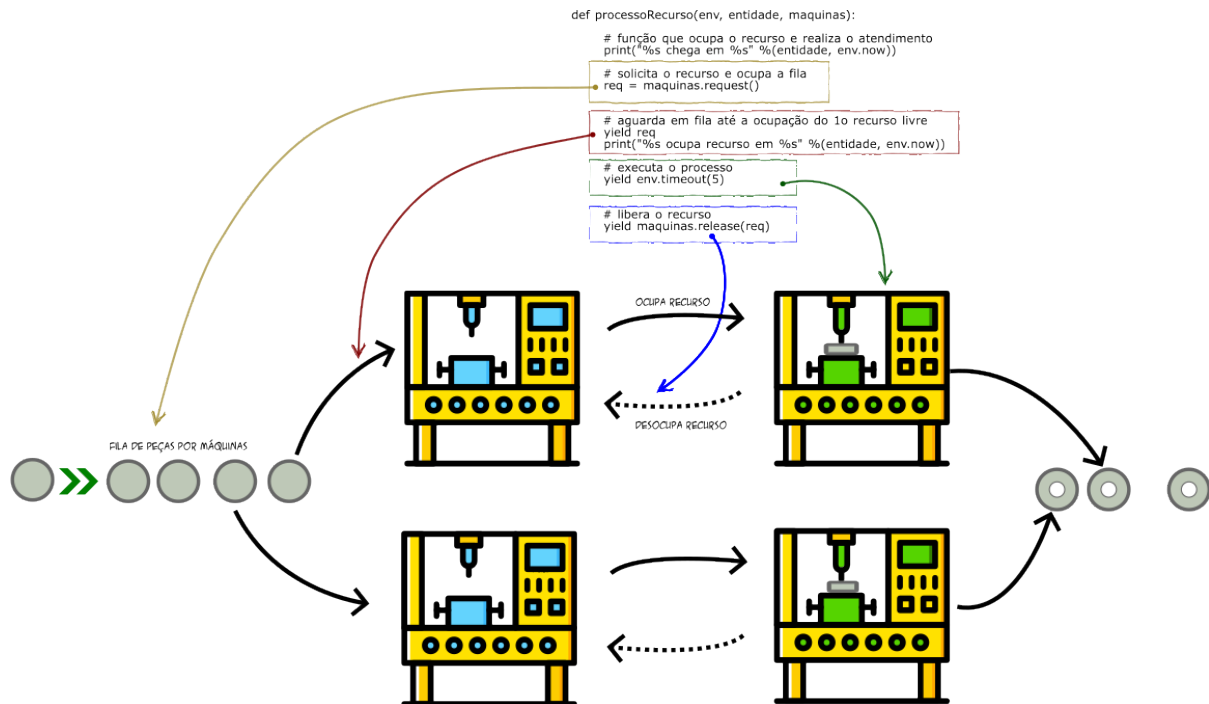


Figura 6.2. Representação do modelo de ocupação das máquinas.

6.4 Status do recurso

O SimPy fornece alguns parâmetros para você acompanhar o status do recursos criados no modelo. Para um recurso `res` definido no programa, podem ser extraídos os seguintes parâmetros durante a simulação:

- `res.capacity`: capacidade do recurso;
- `res.count`: quantas unidades de capacidade do recurso estão ocupadas no momento;
- `res.queue`: lista de objetos (no caso, requisições) que estão em fila no momento. Como `res.queue` é uma lista, o **número** de entidades em fila do recurso é obtido diretamente com o comando `len(res.queue)`;
- `res.users`: lista de objetos (no caso, requisições) que estão ocupando o recurso no momento. Como `res.users` é uma lista, o **número** de entidades em processo no recurso é obtido diretamente com o comando `len(res.users)`.

Ao exemplo anterior acrescentamos uma pequena função, `printStats`, que imprime na tela todos os parâmetros anteriores de um recurso:

```
1  import simpy
2
3  def processoRecurso(env, entidade, maquinas):
4      # função que ocupa o recurso e realiza o atendimento
5      print("%s chega em %s" %(entidade, env.now))
6
7      # solicita o recurso e ocupa a fila
8      req = maquinas.request()
9
10     # aguarda em fila até a ocupação do 1o recurso livre
11     yield req
12
13     print("%s ocupa recurso em %s" %(entidade, env.now))
14
15     # executa o processo
16     yield env.timeout(5)
17
18     # libera o recurso
19     yield maquinas.release(req)
20
21     print("%s libera o recurso em %s" %(entidade, env.now))
22     printStatus(maquinas)
23
24 def printStatus(res):
25     # imprime status do recurso
26     print ("\tCapacidade: %i \tQuantidade ocupada: %i" %(res.capacity, res.count\
27 ))
28     print ("\tEntidades (request) aguardando fila: ", res.queue)
29     print ("\tEntidades em processamento: ", res.users)
30     print ("\tNúmero de entidades em fila: %i e em processamento: %i"
31           % (len(res.queue), len(res.queue)))
32
33
34 env = simpy.Environment()
35
36 # cria recurso maquinas com capacidade 2
37 maquinas = simpy.Resource(env, capacity=2)
38 for i in range(1,5):
39     env.process(processoRecurso(env, "Peça %s" %i, maquinas))
40 env.run()
```

Quando executado, o programa anterior fornece como saída:


```
1 Peça 1 chega em 0
2 Peça 2 chega em 0
3 Peça 3 chega em 0
4 Peça 4 chega em 0
5 Peça 1 ocupa recurso em 0
6 Peça 2 ocupa recurso em 0
7 Peça 1 libera o recurso em 5
8     Capacidade: 2   Quantidade ocupada: 1
9     Entidades (request) aguardando fila: [<Request() object at 0xeffd530>]
10    Entidades em processamento: [<Request() object at 0xeffd5f0>]
11    Número de entidades em fila: 1 e em processamento: 1
12 Peça 2 libera o recurso em 5
13     Capacidade: 2   Quantidade ocupada: 2
14     Entidades (request) aguardando fila: []
15     Entidades em processamento: [<Request() object at 0xeffd5f0>, <Request()\
16 object at 0xeffd530>]
17     Número de entidades em fila: 0 e em processamento: 0
18 Peça 3 ocupa recurso em 5
19 Peça 4 ocupa recurso em 5
20 Peça 3 libera o recurso em 10
21     Capacidade: 2   Quantidade ocupada: 0
22     Entidades (request) aguardando fila: []
23     Entidades em processamento: []
24     Número de entidades em fila: 0 e em processamento: 0
25 Peça 4 libera o recurso em 10
26     Capacidade: 2   Quantidade ocupada: 0
27     Entidades (request) aguardando fila: []
28     Entidades em processamento: []
29     Número de entidades em fila: 0 e em processamento: 0
```

Esta seção para por aqui. Na continuação, construiremos um exemplo completo com geração de entidades e ocupação de recursos, de modo a cruzar tudo o que vimos até agora sobre o SimPy.

6.5 Conceitos desta seção

Conteúdo	Descrição
<code>meuRecurso = simpy.Resource(env, capacity=1)</code>	cria um recurso em env com capacidade = 1
<code>meuRequest = meuRecurso.request()</code>	solicita o recurso meuRecurso (note que o comando ainda não ocupa o recurso)
<code>yield meuRequest</code>	aguarda em fila a liberação do recurso
<code>meuRecurso.release(meuRequest)</code>	libera meuRecurso a partir do meuRequest realizado
<code>env.process(função_geradora)</code>	inicia o processo implementado na função_geradora

7 Juntando tudo em um exemplo: a fila M/M/1

A fila M/M/1 (ver, por exemplo, [Chwif e Medina, 2015](#)¹) representa um sistema simples em que clientes chegam para atendimento em um servidor de fila única, com intervalos entre chegadas sucessivas exponencialmente distribuídos e tempos de atendimentos também exponencialmente distribuídos.

Para este exemplo, vamos considerar que o tempo médio entre chegadas sucessivas é de 1 min (ou seja, uma taxa de chegadas de 1 cliente/min) e o tempo médio de atendimento no servidor é de 0,5 min (ou seja, uma taxa de atendimento de 2 clientes/min). Como um experimento inicial, o modelo deve ser simulado por 5 minutos apenas.

7.1 Geração de chegadas de entidades

Partindo da função `geraChegadas`, codificada na seção “Primeiro passo em SimPy: criando entidades”, precisamos criar uma função ou processo para ocupar, utilizar e desocupar o servidor. Criaremos uma função `atendimentoServidor` responsável por manter os clientes em fila e realizar o atendimento.

Inicialmente, vamos acrescentar as constantes `TEMPO_MEDIO_CHEGADAS` e `TEMPO_MEDIO_ATENDIMENTO`, para armazenar os parâmetros das distribuições dos processos de chegada e atendimento da fila. Adicionalmente, vamos criar o recurso `servidorRes` com capacidade de atender 1 cliente por vez.

```
1  import random                                # gerador de números aleatórios
2  import simpy                                  # biblioteca de simulação
3
4  # tempo médio entre chegadas sucessivas de clientes
5  TEMPO_MEDIO_CHEGADAS = 1.0
6
7  # tempo médio de atendimento no servidor
8  TEMPO_MEDIO_ATENDIMENTO = 0.5
9
10 def geraChegadas(env):
11     # função que cria chegadas de entidades no sistema
12     contaChegada = 0
13     while True:
14         # aguarda um intervalo de tempo exponencialmente distribuído
15         yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_CHEGADAS))
```

¹<https://leanpub.com/msed>

```

16         contaChegada += 1
17         print('%1f Chegada do cliente %d' % (env.now, contaChegada))
18
19     # semente do gerador de números aleatórios
20     random.seed(25)
21
22     # cria o environment do modelo
23     env = simpy.Environment()
24
25     # cria o recurso servidorRes
26     servidorRes = simpy.Resource(env, capacity=1)
27
28     # inicia processo de geração de chegadas
29     env.process(geraChegadas(env))
30
31     # executa o modelo por 5 min
32     env.run(until=5)

```

7.2 Realizando o atendimento no servidor

Se você executar o script anterior, o recurso é criado, mas nada acontece, afinal, não existe ainda nenhum processo requisitando o recurso.

Precisamos, portanto, construir uma nova função que realize o *processo* de atendimento. Usualmente, um processo qualquer tem ao menos as 4 etapas a seguir:

1. Solicitar o servidor;
2. Ocupar o servidor;
3. Executar o atendimento por um tempo com distribuição conhecida;
4. Liberar o servidor para o próximo cliente.

A função `atendimentoServidor`, a seguir, recebe como parâmetros o `env` atual, o nome do cliente e a recurso `servidorRes` para executar todo o processo de atendimento.

```

1  import random                                # gerador de números aleatórios
2  import simpy                                  # biblioteca de simulação
3
4  # tempo médio entre chegadas sucessivas de clientes
5  TEMPO_MEDIO_CHEGADAS = 1.0
6
7  # tempo médio de atendimento no servidor
8  TEMPO_MEDIO_ATENDIMENTO = 0.5
9
10 def geraChegadas(env):

```

```

11     # função que cria chegadas de entidades no sistema
12     contaChegada = 0
13     while True:
14         # aguarda um intervalo de tempo exponencialmente distribuído
15         yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_CHEGADAS))
16         contaChegada += 1
17         print('%0.1f Chegada do cliente %d' % (env.now, contaChegada))
18
19 def atendimentoServidor(env, nome, servidorRes):
20     # função que ocupa o servidor e realiza o atendimento
21     # solicita o recurso servidorRes
22     request = servidorRes.request()
23
24     # aguarda em fila até a liberação do recurso e o ocupa
25     yield request
26     print('%0.1f Servidor inicia o atendimento do %s' % (env.now, nome))
27
28     # aguarda um tempo de atendimento exponencialmente distribuído
29     yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_ATENDIMENTO))
30     print('%0.1f Servidor termina o atendimento do %s' % (env.now, nome))
31
32     # libera o recurso servidorRes
33     yield servidorRes.release(request)
34
35 # semente do gerador de números aleatórios
36 random.seed(25)
37
38 # cria o environment do modelo
39 env = simpy.Environment()
40
41 # cria o recurso servidorRes
42 servidorRes = simpy.Resource(env, capacity=1)
43
44 # inicia processo de geração de chegadas
45 env.process(geraChegadas(env))
46
47 # executa o modelo por 5 min
48 env.run(until=5)

```

Neste momento, nosso script possui uma função geradora de clientes e uma função de atendimento dos clientes, mas o bom observador deve notar que não existe conexão entre elas. Em SimPy, *e vamos repetir isso a exaustão, tudo é processado dentro de um environment*. Assim, o atendimento é um *processo* que deve ser iniciado por cada cliente *gerado* pela função *criaChegadas*. Isto é feito por uma chamada a função `env.process(atendimentoServidor(...))`.

A função *geraChegadas* deve ser alterada, portanto, para receber como parâmetro o recurso *servidorRes*, criado no corpo do programa e para iniciar o processo de atendimento por meio

da chamada à função `env.process`, como representado a seguir:

```
1 def geraChegadas(env):
2     # função que cria chegadas de entidades no sistema
3     contaChegada = 0
4     while True:
5         # aguardo um intervalo de tempo exponencialmente distribuído
6         yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_CHEGADAS))
7         contaChegada += 1
8         print('%1f Chegada do cliente %d' % (env.now, contaChegada))
9
10        # inicia o processo de atendimento
11        env.process(atendimentoServidor(env, "cliente %d" % contaChegada, servido\
12 rRes))
```

Agora execute o script e voilá!

```
1 0.5 Chegada do cliente 1
2 0.5 Servidor inicia o atendimento do cliente 1
3 1.4 Servidor termina o atendimento do cliente 1
4 3.1 Chegada do cliente 2
5 3.1 Servidor inicia o atendimento do cliente 2
6 3.3 Chegada do cliente 3
7 4.1 Servidor termina o atendimento do cliente 2
8 4.1 Servidor inicia o atendimento do cliente 3
9 4.1 Servidor termina o atendimento do cliente 3
10 4.3 Chegada do cliente 4
11 4.3 Servidor inicia o atendimento do cliente 4
12 4.5 Servidor termina o atendimento do cliente 4
```

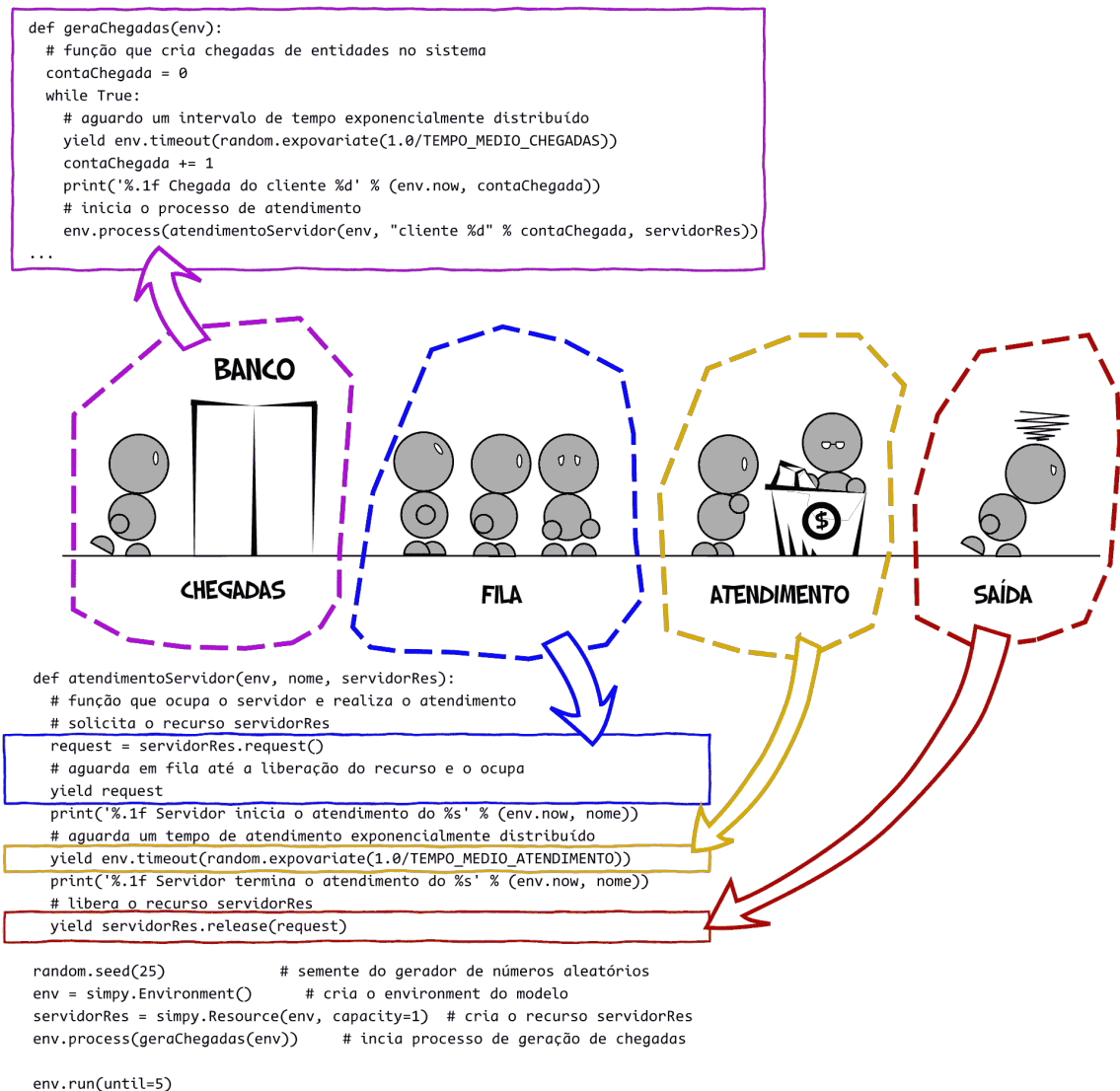


Figura 7.1. Representação do simulador construído.

7.3 Uma representação alternativa para a ocupação e desocupação de recursos

A sequência de ocupação e desocupação do recurso pode ser representada de maneira mais compacta com o laço `with`:

```

1  def atendimentoServidor(env, nome, servidorRes):
2      # função que ocupa o servidor e realiza o atendimento
3      # solicita o recurso servidorRes
4      with servidorRes.request() as request:
5          # aguarda em fila até a liberação do recurso e o ocupa
6          yield request
7          print('%.1f Servidor inicia o atendimento do %s' % (env.now, nome))
8
9          # aguarda um tempo de atendimento exponencialmente distribuído
10         yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_ATENDIMENTO))
11         print('%.1f Servidor termina o atendimento do %s' % (env.now, nome))

```

No script anterior a ocupação e desocupação é garantida dentro do `with`, deixando o código mais compacto e legível. Contudo, a aplicação é limitada a problemas de ocupação e desocupação simples de servidores (veja um contra-exemplo no Desafio 6).

Existem muitos conceitos a serem discutidos sobre os scripts anteriores e, garanto, que eles serão destrinchados nas seções seguintes.

Por hora, e para não esticar demais a atividade, analise atentamente os resultados da execução do script e avance para cima dos nossos desafios.

7.4 Conceitos desta seção

Conteúdo	Descrição
<code>with servidorRes.request() as req:</code>	forma compacta de representar a sequência de ocupação e desocupação do recurso: <code>request()</code> , <code>yield</code> e <code>release()</code> . Tudo que está dentro do <code>with</code> é realizado com o recurso ocupado

7.5 Desafios



Desafio 4: para melhor compreensão do funcionamento do programa, imprima na tela o tempo de simulação e o números de clientes em fila. Quantos clientes existem em fila no instante 4.5?



Desafio 5: calcule o tempo de permanência em fila de cada cliente e imprima o resultado na tela. Para isso, armazene o instante de chegada do cliente na fila em uma variável `chegada`. Ao final do atendimento, armazene o tempo de fila, numa variável `tempoFila` e apresente o resultado na tela.



Desafio 6: um problema clássico de simulação envolve ocupar e desocupar recursos na sequência correta. Considere uma lavanderia com 4 lavadoras, 3 secadoras e 5 cestos de roupas. Quando um cliente chega, ele coloca as roupas em uma máquina de lavar (ou aguarda em fila). A lavagem consome 20 minutos (constante). Ao terminar a lavagem, o cliente retira as roupas da máquina e coloca em um cesto e leva o cesto com suas roupas até a secadora, num processo que leva de 1 a 4 minutos distribuídos uniformemente. O cliente então descarrega as roupas do cesto diretamente para a secadora, espera a secagem e vai embora. Esse processo leva entre 9 e 12 minutos, uniformemente distribuídos. Construa um modelo de simulação que represente o sistema atual?

7.6 Solução dos desafios 4, 5 e 6

Desafio 4: imprima na tela o tempo de simulação e o números de clientes em fila. Quantos clientes existem em fila no instante 4.5?

Para solução do desafio, basta lembrarmos que a qualquer momento, o conjunto de entidades em fila pelo recurso é dado por `servidorRes.queue` e, portanto, o número de entidade em fila é facilmente obtido pela expressão:

```
1 len(servidorRes.queue)
```

Foi acrescentada uma chamadas à função `print`, de modo a imprimir na tela o número de clientes em fila ao término do atendimento de cada cliente:

```
1 def atendimentoServidor(env, nome, servidorRes):
2     # função que ocupa o servidor e realiza o atendimento
3     # solicita o recurso servidorRes
4     request = servidorRes.request()
5
6     # aguarda em fila até a ocupação do 1o recurso livre
7     yield request
8     print('%0.1f Servidor inicia o atendimento do %s' % (env.now, nome))
9
10    # aguarda um tempo de atendimento exponencialmente distribuído
11    yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_ATENDIMENTO))
12    print('%0.1f Servidor termina o atendimento do %s. Clientes em fila: %i'
13          % (env.now, nome, len(servidorRes.queue)))
14
15    # libera o recurso servidorRes
16    yield servidorRes.release(request)
```

Executado o código, descobrimos que no instante 5,5 min, temos 2 clientes em fila:

```
1 0.5 Chegada do cliente 1
2 0.5 Servidor inicia o atendimento do cliente 1
3 1.4 Servidor termina o atendimento do cliente 1. Clientes em fila: 0
4 3.1 Chegada do cliente 2
5 3.1 Servidor inicia o atendimento do cliente 2
6 3.3 Chegada do cliente 3
7 4.1 Servidor termina o atendimento do cliente 2. Clientes em fila: 1
8 4.1 Servidor inicia o atendimento do cliente 3
9 4.1 Servidor termina o atendimento do cliente 3. Clientes em fila: 0
10 4.3 Chegada do cliente 4
11 4.3 Servidor inicia o atendimento do cliente 4
12 4.5 Servidor termina o atendimento do cliente 4. Clientes em fila: 0
```

Portanto, existem 0 cliente em fila no instante 4,5 minutos, nas condições simuladas (note a semente de geração de números aleatórios igual a 2).

Desafio 5: calcule o tempo de permanência em fila de cada cliente e imprima o resultado na tela. Para isso, armazene o instante de chegada do cliente na fila em uma variável `chegada`.

Ao final do atendimento, armazene o tempo de fila, numa variável `tempoFila` e apresente o resultado na tela.

A ideia deste desafio é que você se acostume com esse cálculo tão trivial quanto importante dentro da simulação: o tempo de permanência de uma entidade em algum local. Neste caso, o local é uma fila por ocupação de um recurso.

A lógica aqui é a de um cronometrista que deve disparar o cronômetro na chegada do cliente e pará-lo ao início do atendimento.

Assim, ao chegar, criamos uma variável `chegada` que armazena o instante atual fornecido pelo comando `env.now` do SimPy:

```

1 def atendimentoServidor(env, nome, servidorRes):
2     # função que ocupa o servidor e realiza o atendimento
3     # armazena o instante de chegada do cliente
4     chegada = env.now
5
6     # solicita o recurso servidorRes
7     request = servidorRes.request()
```

Agora, iniciado o atendimento (logo após o `yield` que ocupa o recurso), a variável `tempoFila` armazena o tempo de permanência em fila. Como num cronômetro, o tempo em fila é calculado pelo instante atual do cronômetro menos o instante de disparo dele já armazenado na variável `chegada`:

```

1 def atendimentoServidor(env, nome, servidorRes):
2     # função que ocupa o servidor e realiza o atendimento
3     # armazena o instante de chegada do cliente
4     chegada = env.now
5
6     # solicita o recurso servidorRes
7     request = servidorRes.request()
8
9     # aguarda em fila até a ocupação do 1o recurso livre
10    yield request
11
12    # calcula o tempo em fila
13    tempoFila = env.now - chegada
```

Para imprimir o resultado, basta simplesmente alterar a chamada à função `print` na linha seguinte, de modo que o código final da função `atendimentoServidor` fica:

```

1  def atendimentoServidor(env, nome, servidorRes):
2      # função que ocupa o servidor e realiza o atendimento
3      # armazena o instante de chegada do cliente
4      chegada = env.now
5
6      # solicita o recurso servidorRes
7      request = servidorRes.request()
8
9      # aguarda em fila até a ocupação do 1o recurso livre
10     yield request
11
12     # calcula o tempo em fila
13     tempoFila = env.now - chegada
14     print('%0.1f Servidor inicia o atendimento do %s. Tempo em fila: %0.1f'
15           % (env.now, nome, tempoFila))
16
17     # aguarda um tempo de atendimento exponencialmente distribuído
18     yield env.timeout(random.expovariate(1.0/TEMPO_MEDIO_ATENDIMENTO))
19     print('%0.1f Servidor termina o atendimento do %s. Clientes em fila: %i'
20           % (env.now, nome, len(servidorRes.queue)))
21
22     # libera o recurso servidorRes
23     yield servidorRes.release(request)

```

Agora, a execução do programa mostra na tela o tempo de espera de cada cliente:

```

1  0.5 Chegada do cliente 1
2  0.5 Servidor inicia o atendimento do cliente 1. Tempo em fila: 0.0
3  1.4 Servidor termina o atendimento do cliente 1. Clientes em fila: 0
4  3.1 Chegada do cliente 2
5  3.1 Servidor inicia o atendimento do cliente 2. Tempo em fila: 0.0
6  3.3 Chegada do cliente 3
7  4.1 Servidor termina o atendimento do cliente 2. Clientes em fila: 1
8  4.1 Servidor inicia o atendimento do cliente 3. Tempo em fila: 0.8
9  4.1 Servidor termina o atendimento do cliente 3. Clientes em fila: 0
10 4.3 Chegada do cliente 4
11 4.3 Servidor inicia o atendimento do cliente 4. Tempo em fila: 0.0
12 4.5 Servidor termina o atendimento do cliente 4. Clientes em fila: 0

```

Desafio 6: um problema clássico de simulação envolve ocupar e desocupar recursos na sequência correta. Considere uma lavanderia com 4 lavadoras, 3 secadoras e 5 cestos de roupas. Quando um cliente chega, ele coloca as roupas em uma máquina de lavar (ou aguarda em fila). A lavagem consome 20 minutos (constante). Ao terminar a lavagem, o cliente retira as roupas da máquina e coloca em um cesto e leva o cesto com suas roupas até a secadora, num processo que leva de 1 a 4 minutos distribuídos uniformemente. O cliente então descarrega as roupas do cesto diretamente para a

secadora, espera a secagem e vai embora. Esse processo leva entre 9 e 12 minutos, uniformemente distribuídos. Construa um modelo que represente o sistema descrito.

A dificuldade do desafio da lavanderia é representar corretamente a sequência de ocupação e desocupação dos recursos necessários de cada cliente. Se você ocupá-los/desocupá-los na ordem errada, fatalmente seu programa apresentará resultados inesperados.

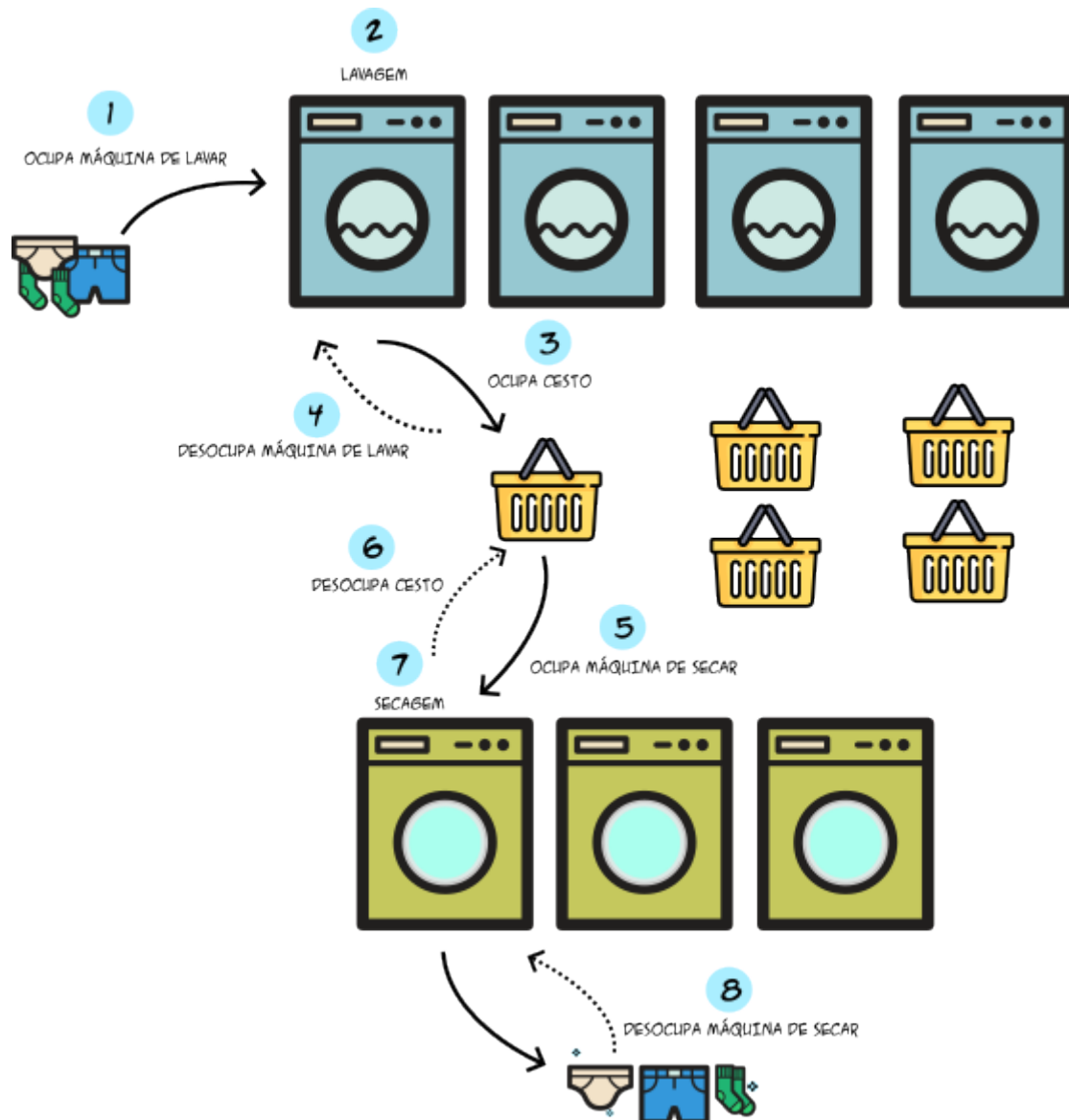


Figura 7.2. Representação do modelo da lavanderia.

Como se trata de um modelo com vários processos e distribuições, vamos seguir a Dica da seção “Solução dos desafios 2 e 3” e construir uma função para armazenar as distribuições do problema, organizando nosso código:

```

1  import random
2  import numpy
3
4  def distributions(tipo):
5      # função que armazena as distribuições utilizadas no modelo
6      return {
7          'chegadas': random.expovariate(1.0/5.0),
8          'lavar': 20,
9          'carregar': random.uniform(1, 4),
10         'descarregar': random.uniform(1, 2),
11         'secar': random.uniform(9, 12),
12     }.get(tipo, 0.0)

```

Como já destacado, a dificuldade é representar a sequência correta de processos do cliente: ele chega, ocupa uma lavadora, lava, ocupa um cesto, libera uma lavadora, ocupa uma secadora, libera o cesto, seca e libera a secadora. Se a sequência foi bem compreendida, a máscara a seguir será de fácil preenchimento:

```

1  import random
2  import numpy
3
4  contaClientes = 0                # conta clientes que chegaram no sistema
5
6  def distributions(tipo):
7      # função que armazena as distribuições utilizadas no modelo
8      return {
9          'chegadas': random.expovariate(1.0/5.0),
10         'lavar': 20,
11         'carregar': random.uniform(1, 4),
12         'descarregar': random.uniform(1, 2),
13         'secar': random.uniform(9, 12),
14     }.get(tipo, 0.0)
15
16  def chegadaClientes(env, lavadoras, cestos, secadoras):
17      # função que gera a chegada de clientes
18      global contaClientes
19
20      pass
21
22      # chamada do processo de lavagem e secagem
23      pass
24
25  def lavaSeca(env, cliente, lavadoras, cestos, secadoras):
26      # função que processa a operação de cada cliente dentro da lavanderia
27
28      # ocupa a lavadora

```

```

29     pass
30
31     # antes de retirar da lavadora, pega um cesto
32     pass
33
34     # libera a lavadora, mas não o cesto
35     pass
36
37     # ocupa a secadora antes de liberar o cesto
38     pass
39
40     # libera o cesto mas não a secadora
41     pass
42
43     # pode liberar a secadora
44     pass
45
46 random.seed(10)
47 env = simpy.Environment()
48 lavadoras = simpy.Resource(env, capacity = 3)
49 cestos = simpy.Resource(env, capacity = 2)
50 secadoras = simpy.Resource(env, capacity = 1)
51 env.process(chegadaClientes(env, lavadoras, cestos, secadoras))
52
53 env.run(until = 40)

```

O programa a seguir apresenta uma possível solução para o desafio, já com diversos comandos de impressão:

```

1  import random
2  import simpy
3
4  contaClientes = 0          # conta clientes que chegaram no sistema
5
6  def distributions(tipo):
7      # função que armazena as distribuições utilizadas no modelo
8      return {
9          'chegadas': random.expovariate(1.0/5.0),
10         'lavar': 20,
11         'carregar': random.uniform(1, 4),
12         'descarregar': random.uniform(1, 2),
13         'secar': random.uniform(9, 12),
14     }.get(tipo, 0.0)
15
16  def chegadaClientes(env, lavadoras, cestos, secadoras):
17      # função que gera a chegada de clientes

```

```

18     global contaClientes
19
20     contaClientes = 0
21     while True:
22         contaClientes += 1
23         yield env.timeout(distributions('chegadas'))
24         print("%.1f chegada do cliente %s" %(env.now, contaClientes))
25         # chamada do processo de lavagem e secagem
26         env.process(lavaSeca(env, "Cliente %s" % contaClientes, lavadoras, cestos\
27 , secadoras))
28
29 def lavaSeca(env, cliente, lavadoras, cestos, secadoras):
30     # função que processa a operação de cada cliente dentro da lavanderia
31     global utilLavadora, tempoEsperaLavadora, contaLavadora
32
33     # ocupa a lavadora
34     req1 = lavadoras.request()
35     yield req1
36     print("%.1f %s ocupa lavadora" %(env.now, cliente))
37     yield env.timeout(distributions('lavar'))
38
39     # antes de retirar da lavadora, pega um cesto
40     req2 = cestos.request()
41     yield req2
42     print("%.1f %s ocupa cesto" %(env.now, cliente))
43     yield env.timeout(distributions('carregar'))
44
45     # libera a lavadora, mas não o cesto
46     lavadoras.release(req1)
47     print("%.1f %s desocupa lavadora" %(env.now, cliente))
48
49     # ocupa a secadora antes de liberar o cesto
50     req3 = secadoras.request()
51     yield req3
52     print("%.1f %s ocupa secadora" %(env.now, cliente))
53     yield env.timeout(distributions('descarregar'))
54
55     # libera o cesto mas não a secadora
56     cestos.release(req2)
57     print("%.1f %s desocupa cesto" %(env.now, cliente))
58     yield env.timeout(distributions('secar'))
59
60     # pode liberar a secadora
61     print("%.1f %s desocupa secadora" %(env.now, cliente))
62     secadoras.release(req3)
63

```



```

64 random.seed(10)
65 env = simpy.Environment()
66 lavadoras = simpy.Resource(env, capacity=3)
67 cestos = simpy.Resource(env, capacity=2)
68 secadoras = simpy.Resource(env, capacity=1)
69 env.process(chegadaClientes(env, lavadoras, cestos, secadoras))
70
71 env.run(until=40)

```

A execução do programa anterior fornece como saída:

```

1  4.2 chegada do cliente 1
2  4.2 Cliente 1 ocupa lavadora
3  12.6 chegada do cliente 2
4  12.6 Cliente 2 ocupa lavadora
5  24.2 Cliente 1 ocupa cesto
6  27.2 Cliente 1 desocupa lavadora
7  27.2 Cliente 1 ocupa secadora
8  28.8 Cliente 1 desocupa cesto
9  32.6 Cliente 2 ocupa cesto
10 36.3 Cliente 2 desocupa lavadora
11 38.7 Cliente 1 desocupa secadora
12 38.7 Cliente 2 ocupa secadora

```

7.7 Teste seus conhecimentos

1. A fila M/M/1 possui expressões analíticas conhecidas. Por exemplo, o tempo médio de permanência no sistema é dado pela expressão: $W = \frac{1}{\mu - \lambda}$. Valide seu modelo, ou seja, calcule o resultado esperado para a expressão e compare com o resultado obtido pelo seu programa.
2. Utilizando a função `plot` da biblioteca `matplotlib`², construa um gráfico que represente a evolução do número de entidades em fila (dica: você precisará armazenar o tempo de espera em uma lista e plotar a lista em um gráfico ao final da simulação).
3. No problema da lavanderia, crie uma situação de desistência, isto é: caso a fila de espera por lavadoras seja de 5 clientes, o próximo cliente a chegar no sistema desiste imediatamente de entrar na lavanderia.

²http://matplotlib.org/users/pyplot_tutorial.html#pyplot-tutorial%29

8 Atributos e variáveis: diferenças em SimPy

Qual a diferença entre atributo e variável para um modelo de simulação? O atributo pertence à entidade, enquanto a variável pertence ao modelo. De outro modo, se um cliente chega a uma loja e compra 1, 2 ou 3 produtos, esse cliente possui um **atributo** imediato: o **número de produtos** comprados. Note que o atributo “número de produtos” é um valor diferente para cada cliente, ou seja: é um valor exclusivo do cliente.

Por outro lado, um parâmetro de saída importante seria o número total de produtos vendidos nesta loja ao longo da duração da simulação. O total de produtos é a soma dos atributos “número de produtos” de cada cliente que comprou algo na loja. Assim, o total vendido é uma **variável** do modelo, que se acumula a cada nova compra, independentemente de quem é o cliente.

Em SimPy a coisa é trivial: toda variável **local** funciona como atributo da entidade gerada e toda variável **global** é naturalmente uma variável do modelo. Não se trata de uma regra absoluta, nem tampouco foi imaginada pelos desenvolvedores da biblioteca, é decorrente da necessidade de se representar os processos do modelo de simulação por meio de **funções** que, por sua vez representam entidades executando alguma coisa.

Usuários de pacotes comerciais (Simul8, Anylogic, GPSS, Arena etc.) estão acostumados a informar explicitamente ao modelo o que é atributo e o que é variável. Em SimPy, basta lembrar que as variáveis globais serão variáveis de todo o modelo e que os atributos de interesse devem ser transferidos de um processo ao outro por transferência de argumentos no cabeçalho das funções.

Voltemos ao exemplo de chegadas de clientes numa loja. Queremos que cada cliente tenha como atributo o número de produtos desejados:

```
1  import random      # gerador de números aleatórios
2  import simpy        # biblioteca de simulação
3
4  contaVendas = 0     # variável global que marca o número de vendas realizadas
5
6  def geraChegadas(env):
7      # função que cria chegadas de entidades no sistema
8      # variável local = atributo da entidade
9      contaEntidade = 0
10     while True:
11         yield env.timeout(1)
12         contaEntidade += 1
13         # atributo do cliente: número de produtos desejados
14         produtos = random.randint(1,3)
15         print("%.1f Chegada do cliente %i\tProdutos desejados: %d"
16               % (env.now, contaEntidade, produtos))
```

```

17
18     # inicia o processo de atendimento do cliente de atributos contaEntidade
19     # e do número de produtos
20     env.process(compra(env, "cliente %d" % contaEntidade, produtos))
21
22 def compra(env, nome, produtos):
23     # função que realiza a venda para as entidades
24     # nome e produtos, são atributo da entidade
25
26     global contaVendas # variável global = variável do modelo
27
28     for i in range(0, produtos):
29         yield env.timeout(2)
30         contaVendas += produtos
31         print("%.1f Compra do %s \tProdutos comprados: %d" % (env.now, nome, prod\
32 utos))
33
34 random.seed(1000) # semente do gerador de números aleatórios
35 env = simpy.Environment() # cria o environment do modelo
36 env.process(geraChegadas(env)) # cria o processo de chegadas
37
38 env.run(until=5) # roda a simulação por 10 unidades de tempo
39 print("\nTotal vendido: %d produtos" % contaVendas)

```

A execução do programa por apenas 5 minutos, apresenta como resposta:

```

1 1.0 Chegada do cliente 1      Produtos desejados: 2
2 2.0 Chegada do cliente 2      Produtos desejados: 3
3 3.0 Compra do cliente 1        Produtos comprados: 2
4 3.0 Chegada do cliente 3      Produtos desejados: 1
5 4.0 Compra do cliente 2        Produtos comprados: 3
6 4.0 Chegada do cliente 4      Produtos desejados: 2
7
8 Total vendido: 5 produtos

```

É importante destacar no exemplo, que o cliente (ou entidade) gerado(a) pela função geraChegadas é enviado(a) para a função compra com seu atributo produtos, como se nota na linha em que o cliente chama o processo de compra:

```

1 env.process(compra(env, "cliente %d" % contaEntidade, produtos))

```

Agora raciocine de modo inverso: seria possível representar o número total de produtos vendidos como uma variável local? Intuitivamente, somos levados a refletir na possibilidade de *transferir* o número total de produtos como uma parâmetro de chamada da função. Mas, reflita mais um tiquinho... É possível passar o total vendido como um parâmetro de chamada da função?

Do modo como o problema foi modelado, isso não é possível, pois cada chegada gera um novo processo compra independente para cada cliente e não há como transferir tal valor de uma chamada do processo para outra. A seção a seguir, apresenta uma alternativa interessante que evita o uso de variáveis globais num modelo de simulação.

8.1 Atributos em modelos orientados ao objeto

Para aqueles que seguem o paradigma da programação orientada ao objeto, um atributo de uma entidade é naturalmente o próprio atributo do objeto gerado pela classe. Uma facilidade que a programação voltada ao objeto possui é que podemos criar atributos para recursos também. Neste caso, basta que o recurso seja criado dentro de uma classe.

Por exemplo, a fila M/M/1 poderia ser modelada por uma classe `Servidor`, em que um dos seus atributos é o próprio `Resource` do SimPy, como mostra o código a seguir:

```
1  import random
2  import simpy
3
4  class Servidor(object):
5      # cria a classe Servidor
6      # note que um dos atributos é o próprio Resource do simpy
7      def __init__(self, env, capacidade, duracao):
8          # atributos do recurso
9          self.env = env
10         self.res = simpy.Resource(env, capacity=capacidade)
11         self.taxaExpo = 1.0/duracao
12         self.clientesAtendidos = 0
13
14     def atendimento(self, cliente):
15         # executa o atendimento
16         print("%.1f Início do atendimento do %s" % (env.now, cliente))
17         yield self.env.timeout(random.expovariate(self.taxaExpo))
18         print("%.1f Fim do atendimento do %s" % (env.now, cliente))
19
20 def processaCliente(env, cliente, servidor):
21     # função que processa o cliente
22
23     print('%.1f Chegada do %s' % (env.now, cliente))
24     with servidor.res.request() as req: # note que o Resource é um atributo também
25         yield req
26
27         print('%.1f Servidor ocupado pelo %s' % (env.now, cliente))
28         yield env.process(servidor.atendimento(cliente))
29         self.clientesAtendidos += 1
30         print('%.1f Servidor desocupado pelo %s' % (env.now, cliente))
31
```

```

32
33 def geraClientes(env, intervalo, servidor):
34     # função que gera os clientes
35     i = 0
36     while True:
37         yield env.timeout(random.expovariate(1.0/intervalo))
38         i += 1
39         env.process(processaCliente(env, 'cliente %d' % i, servidor))
40
41
42 random.seed(1000)
43
44 env = simpy.Environment()
45 # cria o objeto servidor (que é um recurso)
46 servidor = Servidor(env, 1, 1)
47 env.process(geraClientes(env, 3, servidor))
48
49 env.run(until=5)

```

Quando processado por apenas 5 minutos, o modelo anterior fornece:

```

1 4.5 Chegada do cliente 1
2 4.5 Servidor ocupado pelo cliente 1
3 4.5 Início do atendimento do cliente 1
4 4.6 Fim do atendimento do cliente 1
5 4.6 Servidor desocupado pelo cliente 1

```

No caso da programação voltada ao objeto, uma variável do modelo pode pertencer a uma classe, sem a necessidade de que a variável seja global. Por exemplo, o atributo `clientesAtendidos` da classe `Servidor` é uma variável que representa o total de cliente atendidos ao longo da simulação. Caso a representação utilizada não fosse voltada ao objeto, o número de clientes atendidos seria forçosamente uma variável global.

8.2 Conceitos desta seção

Conteúdo	Descrição
representação de atributos	os atributos devem ser representados localmente e transferidos entre funções (ou processos) como parâmetros das funções (ou processos)

Conteúdo	Descrição
representação de variáveis	as variáveis do modelo são naturalmente representadas como variáveis globais ou, no caso da programação voltada ao objeto, como atributos de classes.

8.3 Desafios



Desafio 7: retome o problema da lavanderia (Desafio 6). Estime o tempo médio que os clientes atendidos aguardaram pela lavadora. *Dica:* você precisará de uma variável global para o cálculo do tempo de espera e um atributo para marcar a hora de chegada no sistema.



Desafio 8: no desafio anterior, caso você simule por 10 ou mais horas, deve notar como o tempo de espera pela lavadora fica muito alto. Para identificar o gargalo do sistema, acrescente a impressão do número de clientes que ficaram em fila ao final da simulação. Você consegue otimizar o sistema a partir do modelo construído?

8.4 Solução dos desafios 7 e 8

Desafio 7: retome o problema da lavanderia (Desafio 6). Estime o tempo médio que os clientes atendidos aguardaram pela lavadora. *Dica:* você precisará de uma variável global para o cálculo do tempo de espera e um atributo para marcar a hora de chegada do cliente na lavadora.

O tempo médio de espera por fila de um recurso - no caso lavadoras - é estimado pelo soma do tempo que todos os clientes aguardaram pelo recurso, dividido pelo número de clientes que ocuparam o recurso ao longo da simulação.

Assim, vamos criar duas variáveis globais para armazenar a soma do tempo de espera por lavadora de todos os clientes que ocuparam as lavadoras, bem como o número de clientes que ocuparam as mesmas lavadoras:

```
1 import random
2 import simpy
3
4 contaClientes = 0          # conta clientes que chegaram no sistema
5 tempoEsperaLavadora = 0    # conta tempo de espera total por lavadora
6 contaLavadora = 0          # conta clientes que ocuparam uma lavadora
```

A seguir, precisamos alterar a função `lavaSeca` para calcular corretamente o tempo de espera por lavadora de cada cliente, somar este valor à variável global `tempoEsperaLavadora` e incrementar o número de clientes que ocuparam lavadoras na variável global `contaLavadora` (representação apenas da parte do código que é alterada):

```
1 def lavaSeca(env, cliente, lavadoras, cestos, secadoras):
2     # função que processa a operação de cada cliente dentro da lavanderia
3     global tempoEsperaLavadora, contaLavadora
4
5     # marca atributo chegada com o tempo atual de chegada da entidade
6     chegada = env.now
7     # ocupa a lavadora
8     req1 = lavadoras.request()
9     yield req1
10    # incrementa lavadoras ocupadas
11    contaLavadora += 1
12    # calcula o tempo de espera em fila por lavadora
13    tempoFilaLavadora = env.now - chegada
14    tempoEsperaLavadora += tempoFilaLavadora
15    print("%4.1f %s ocupa lavadora" %(env.now, cliente))
16    yield env.timeout(distributions('lavar'))
```

Ao final do programa, basta acrescentar uma linha para imprimir o tempo médio em fila de espera por lavadoras e o número de vezes que uma lavadora foi ocupada ao longo da simulação:

```

1 random.seed(10)
2 env = simpy.Environment()
3 lavadoras = simpy.Resource(env, capacity=3)
4 cestos = simpy.Resource(env, capacity=2)
5 secadoras = simpy.Resource(env, capacity=1)
6 env.process(chegadaClientes(env, lavadoras, cestos, secadoras))
7
8 env.run(until=40)
9
10 print("\nTempo médio de espera por lavadoras: %.2f min. Clientes atendidos: %i"
11       %(tempoEsperaLavadora/contaLavadora, contaLavadora))

```

Quando executado por 40 minutos, o modelo completo com as alterações anteriores fornece como saída:

```

1 4.2 Chegada do cliente 1
2 4.2 Cliente 1 ocupa lavadora
3 12.6 Chegada do cliente 2
4 12.6 Cliente 2 ocupa lavadora
5 24.2 Cliente 1 ocupa cesto
6 27.2 Cliente 1 desocupa lavadora
7 27.2 Cliente 1 ocupa secadora
8 28.8 Cliente 1 desocupa cesto
9 32.6 Cliente 2 ocupa cesto
10 36.3 Cliente 2 desocupa lavadora
11 38.7 Cliente 1 desocupa secadora
12 38.7 Cliente 2 ocupa secadora
13
14 Tempo médio de espera por lavadoras: 0.00 min. Clientes atendidos: 2

```

Desafio 8: no desafio anterior, caso você simule por 10 ou mais horas, deve notar como o tempo de espera pela lavadora fica muito alto. Para identificar o gargalo do sistema, acrescente a impressão do número de clientes que ficaram em fila ao final da simulação. Você consegue otimizar o sistema a partir do modelo construído?

Quando simulamos o sistema por 10 horas (=10*60 minutos), obtemos como resposta:

```

1 ...
2 599.6 Cliente 75 ocupa cesto
3 600.0 Chegada do cliente 133
4
5 Tempo médio de espera por lavadoras: 138.63 min. Clientes atendidos: 77

```

Para a solução do desafio, basta acrescentar uma linha ao final do programa principal que imprime as filas de por recursos (lavadoras, cestos e secadoras) ao final da simulação:


```

1 env.run(until=600)
2
3 print("\nTempo médio de espera por lavadoras: %.2f min. Clientes atendidos: %i"
4       %(tempoEsperaLavadora/contaLavadora, contaLavadora))
5 print("Fila de clientes ao final da simulação: lavadoras %i cestos %i secadoras %\
6 i"
7       %(len(lavadoras.queue), len(cestos.queue), len(secadoras.queue)))

```

Quando simulado por 600 minutos (ou 10 horas), a saída do modelo fornece:

```

1 ...
2 599.6 Cliente 75 ocupa cesto
3 600.0 Chegada do cliente 133
4
5 Tempo médio de espera por lavadoras: 138.63 min. Clientes atendidos: 77
6 Fila de clientes ao final da simulação: lavadoras 56 cestos 0 secadoras 0

```

Portanto, ao final da simulação, existem 56 clientes aguardando uma lavadora livre, enquanto nenhum cliente aguarda por cestos ou secadoras. Temos um caso clássico de fila **infinita**, isto é: a taxa de horária de atendimento das lavadoras é menor que a taxa horária com que os clientes chegam à lavanderia. Assim, se 1 cliente ocupa em média 20 minutos uma lavadora, a taxa de atendimento em cada lavadora é de $\mu = 0.05$ clientes/min (=1 cliente /20 min), enquanto a taxa de chegadas de clientes na lavandeira é de $\lambda = 0.20$ clientes/min (=1 cliente/5 min).

Como a taxa de atendimento é menor que a taxa de chegadas, a fila cresce indefinidamente. Para termos um sistema equilibrado, precisaríamos de um número de lavadoras tal que se garanta que a taxa de atendimento da soma das lavadoras seja maior que a taxa de chegadas de clientes no sistema ou:

$$\rho = \frac{\lambda}{n \cdot \mu} < 1 \rightarrow n > \frac{\lambda}{\mu} = \frac{0.20}{0.05} = 4$$

Portanto, com 5 (ou mais) lavadoras eliminaríamos o gargalo na lavagem.

O bom da simulação é que podemos testar se a “calculera” anterior faz sentido. Quando simulado para 5 lavadoras, o modelo fornece como saída:

```

1 ...
2 598.8 Cliente 107 ocupa cesto
3 599.0 Cliente 105 desocupa secadora
4
5 Tempo médio de espera por lavadoras: 4.40 min. Clientes atendidos: 108
6 Fila de clientes ao final da simulação: lavadoras 0 cestos 0 secadoras 0

```

Com 5 lavadoras, portanto, já não existe fila residual.

8.5 Teste seus conhecimentos

1. Elabore uma pequena rotina capaz de simular o sistema para números diferentes de recursos (será que o número de cestos e secadoras não está exagerado também?). Manipule o modelo para encontrar o número mínimo de recursos necessário, de modo a não haver gargalos no sistema.

9 Environments: controlando a simulação

No SimPy, o `Environment` é quem coordena a execução do seu programa. Ele avança o relógio de simulação, planeja a ordem de execução dos eventos e executa cada evento planejado pelo programa no instante correto.

9.1 Controle de execução com `env.run(until=fim_da_simulação)`

A maneira mais usual de controle de execução de um modelo de simulação é fornecendo a duração do experimento de simulação. O SimPy, como veremos, vai além e permite alguns outros modos de controle.

Inicialmente, vamos trabalhar com um modelo simples, que gera chegadas de eventos em intervalos constantes entre si:

```
1  import simpy
2
3  def geraChegada(env, p):
4      while True:
5          yield env.timeout(1)
6          print("%3.1f nova chegada" %(env.now))
7
8  env = simpy.Environment()
9  chegadas = env.process(geraChegada(env, "p1"))
10 env.run(until=5)    # executa até o instante 5
```

Quando executado, o modelo anterior fornece:

```
1  1.0 nova chegada
2  2.0 nova chegada
3  3.0 nova chegada
4  4.0 nova chegada
```

No modelo anterior, a última linha informa ao SimPy que a simulação deve ser executada até o instante 5 (implicitamente o SimPy assume que o instante inicial é 0). Esta é a maneira mais usual: o instante final de simulação é um parâmetro de entrada.

O interessante, no modelo anterior, é que se quisermos continuar a execução do instante atual (5, no caso) até o instante 10, por exemplo, podemos simplesmente acrescentar mais uma linha

`env.run(until=10)` informando que a execução **continua de onde está** (instante 5) e termina em 10. Isto pode ser útil em situações em que precisamos modificar algum parâmetro de entrada do modelo ao longo da própria simulação.

Vamos modificar o modelo anterior de modo que nos primeiros 5 minutos o intervalo entre geração de chegadas seja de 1 minuto e, depois, até o instante 10, o intervalo seja de 2 minutos. Para isso, criamos uma variável global `intervalo` que armazenará o intervalo entre chegadas, como mostra o código a seguir:

```

1  import simpy
2
3  intervalo = 1          # valor inicial para o intervalo entre chegada sucessivas
4
5  def geraChegada(env, p):
6      global intervalo
7
8      while True:
9          yield env.timeout(intervalo)
10         print("%3.1f nova chegada" %(env.now))
11
12  env = simpy.Environment()
13  chegadas = env.process(geraChegada(env, "p1"))
14  env.run(until=5)      # executa até o instante 5
15
16  print("\nModificando o intervalo entre chegadas para 2 min")
17
18  # novo intervalo entre chegadas sucessivas
19  intervalo = 2
20  env.run(until=10)    # executa até o instante 10

```

Depois de executado, o modelo anterior fornece:

```

1  1.0 nova chegada
2  2.0 nova chegada
3  3.0 nova chegada
4  4.0 nova chegada
5
6  Modificando o intervalo entre chegadas para 2 min
7  5.0 nova chegada
8  7.0 nova chegada
9  9.0 nova chegada

```

A segunda chamada do `run`, `env.run(until=10)`, executou do instante atual (no caso, 5) até o instante 10. Assim, a opção `until` não representa a duração da simulação, mas *até que instante queremos executá-la*. Isto implica também, que uma nova chamada para `env.run` não reinicializa o tempo de simulação, isto é, não retorna o relógio do simulador para o instante 0.

Para reinicializar a relógio de um modelo em execução, o que seria equivalente a reinicializar a simulação, uma alternativa possível é acrescentar uma nova linha de criação do environment (e, nas linhas seguintes, realizar as chamadas de processo e do `env.run()`). Por exemplo:

```

1  ...
2  env = simpy.Environment()
3  chegadas = env.process(geraChegada(env, "p1"))
4  env.run(until=5)    # executa até o instante 5
5
6  print("\nModificando o intervalo entre chegadas para 2 min")
7
8  # novo intervalo entre chegadas sucessivas
9  intervalo = 2
10
11 # reinicializa o environment
12 env = simpy.Environment()
13
14 # nova chamada do processo chegadas
15 chegadas = env.process(geraChegada(env, "p1"))
16
17 env.run(until=5)    # reexecuta até o instante 5

```

Agora, o modelo reinicializa o relógio, como pode-se verificar pela sua saída:

```

1  1.0 nova chegada
2  2.0 nova chegada
3  3.0 nova chegada
4  4.0 nova chegada
5
6  Modificando o intervalo entre chegadas para 2 min
7  2.0 nova chegada
8  4.0 nova chegada

```

9.2 Parada por execução de todos os processos programados

Quando não se fornece o tempo de simulação (ou ele não é conhecido a priori), podemos interromper a simulação pela própria extinção do processo. No programa anterior, por exemplo, podemos substituir o comando `while True` por um laço `for` executar a simulação com um número fixo de entidades pré estabelecido:

```

1  import simpy
2
3  def geraChegada(env, numEntidades):
4      for i in range(0,numEntidades):
5          yield env.timeout(1)
6          print("%3.1f nova chegada" %(env.now))
7
8  env = simpy.Environment()
9
10 # gera apenas 5 entidades
11 chegadas = env.process(geraChegada(env, 5))
12
13 # executa até o fim de todos os processos do modelo
14 env.run()
```

Quando executado, o modelo anterior fornece como saída:

```

1  1.0 nova chegada
2  2.0 nova chegada
3  3.0 nova chegada
4  4.0 nova chegada
5  5.0 nova chegada
```

Um modelo de simulação pode possuir diversos processos ocorrendo ao mesmo tempo, de modo que o término da simulação só é garantido quando *todos* os processos programados terminarem.

O próximo modelo amplia o exemplo anterior, de modo que dois processos são executados ao mesmo tempo, um com 3 entidades e outro com 5 entidades. Note que os processos também podem se armazenados em uma lista:

```

1  import simpy
2
3  def geraChegada(env, p, numEntidades):
4      for i in range(0,numEntidades):
5          yield env.timeout(1)
6          print("%3.1f nova chegada para o processo %s" %(env.now, p))
7
8  env = simpy.Environment()
9  # chegadas é uma lista que armazena os processos em execução
10 chegadas = [env.process(geraChegada(env, "p1", 5)), env.process(geraChegada(env, \
11 "p2", 3))]
12 env.run()           # executa até o fim de todos os processos do modelo
```

Quando executado, o modelo anterior fornece como saída:

```

1 1.0 nova chegada para o processo p1
2 1.0 nova chegada para o processo p2
3 2.0 nova chegada para o processo p1
4 2.0 nova chegada para o processo p2
5 3.0 nova chegada para o processo p1
6 3.0 nova chegada para o processo p2
7 4.0 nova chegada para o processo p1
8 5.0 nova chegada para o processo p1

```

Neste último exemplo, a simulação terminou apenas quando o processo “p1”, de 5 entidades, foi exaurido.

9.3 Parada por fim de execução de processo específico por `env.run(until=processo)`

Uma outra alternativa de controle de execução é pelo término do próprio processo de execução. Partindo do exemplo anterior, podemos parar a simulação quando o processo que gera 3 entidades termina. Isto é possível com a opção `env.run(until=processo)`:

```

1 import simpy
2
3 def geraChegada(env, p, numEntidades):
4     for i in range(0, numEntidades):
5         yield env.timeout(1)
6         print("%3.1f nova chegada para o processo %s" % (env.now, p))
7
8 env = simpy.Environment()
9 # chegadas é uma lista que armazena os processos em execução
10 chegadas = [env.process(geraChegada(env, "p1", 5)), env.process(geraChegada(env, \
11 "p2", 3))]
12
13 # executa até o fim do processo armazenado em chegadas[1]
14 env.run(until=chegadas[1])

```

Quando executado, o programa anterior fornece:

```

1 1.0 nova chegada para o processo p1
2 1.0 nova chegada para o processo p2
3 2.0 nova chegada para o processo p1
4 2.0 nova chegada para o processo p2
5 3.0 nova chegada para o processo p1
6 3.0 nova chegada para o processo p2

```

No programa anterior, a linha `env.run(until=chegadas[1])` determina que o programa seja executado até que o processo `chegadas[1]` esteja concluído. Note que na lista:

```

1 chegadas = [env.process(geraChegada(env, "p1", 5)), env.process(geraChegada(env, \
2 "p2", 3))]

```

`chegadas[1]` é o processo `env.process(geraChegadas(env, "p2", 3))` que deve terminar após 3 entidades criadas. Verifique na saída do programa que, neste caso, de fato o programa parou após 3 entidades do tipo “p2” geradas.

9.4 Simulação passo a passo: peek & step

O SimPy permite a simulação passo a passo por meio de dois comandos:

- `peek()`: retorna o instante de execução do próximo evento programado. Caso não existam mais eventos programados, retorna infinito (`float('inf')`);
- `step()`: processa o próximo evento. Caso não existam mais eventos, ele retorna um exceção interna `EmptySchedule`.

Um uso interessante da simulação passo a passo é na representação de barras de progresso. O exemplo a seguir faz uso da biblioteca `pyprind`¹ para gerar uma barra de progresso simples (talvez você tenha de instalar a biblioteca `pyprind` - veja no link como proceder):

```

1 import simpy
2 import pyprind
3
4 def geraChegada(env, p):
5     # gera chegadas em intervalos ctes
6     while True:
7         yield env.timeout(1)
8
9 env = simpy.Environment()
10 chegadas = env.process(geraChegada(env, "p1"))
11
12 until = 1000000
13
14 # cria barra de tamanho until
15 pbar = pyprind.ProgBar(until, monitor = True)
16
17 while env.peek() < until:
18     delay = env.now
19     env.step()
20     delay = env.now - delay
21     # atualiza a barra pelo intervalo de tempo processado
22     pbar.update(delay)
23
24 # imprime estatísticas da CPU
25 print(pbar)

```

¹<https://github.com/rasbt/pyprind>


```

1      0%                                100%
2      [#####] | ETA: 00:00:00
3      Title:
4      Started: 10/07/2016 09:58:23
5      Finished: 10/07/2016 09:58:32
6      Total time elapsed: 00:00:00
7      CPU %: 98.80
8      Memory %: 0.32
9

```

Existem outras possibilidades de uso do `peek()` & `step()`. Por exemplo, o Spyder (IDE sugerida para desenvolvimento dos programas deste livro) possui opções de controle de execução passo-a-passo para *debugging*² no menu “Debug”. Assim, podemos colocar um *breakpoint* na linha `env.step()` do programa e acompanhar melhor sua execução—coisa boa quando o modelo tem ainda algum *bug*.

9.5 Conceitos desta seção

Conteúdo	Descrição
<code>env.run(until=tempo_de_simulação)</code>	solicita que o modelo seja executado pelo tempo definido em ‘tempo_de_simulação’. Note que a unidade é intrínseca ao problema simulado.
<code>env.run(until=processo)</code>	solicita que o modelo seja simulado até que o processo termine sua execução. (Caso processo tenha um laço infinito, o programa será simulado indefinidamente ou até que Ctrl+C seja acionado no teclado).
<code>peek()</code>	retorna o instante de execução do próximo evento programado. Caso não existam mais eventos programados, retorna infinito (<code>float('inf')</code>);
<code>step()</code>	processa o próximo evento. Caso não existam mais eventos, ele retorna um exceção interna <code>EmptySchedule</code> .

9.6 Desafios



Desafio 9: Considere que cada entidade gerada no primeiro exemplo desta seção tem um peso em gramas dado por uma distribuição normal de média 10 e desvio padrão igual a 3. Crie um critério de parada para quando a média dos pesos das entidades geradas esteja no intervalo entre 9,5 e 10,5.

²<https://pythonhosted.org/spyder/debugging.html>



Desafio 10: Modifique o critério anterior para que a parada ocorra quando a média for 10 com um intervalo de confiança de amplitude 0,5 e nível de significância igual a 95%.
Dica: utilize a biblioteca `numpy` para isso (consulte o [StackOverflow!](https://stackoverflow.com/questions/tagged/simpy)³).

³[http://stackoverflow.com/questions/tagged/simpy](https://stackoverflow.com/questions/tagged/simpy)

9.7 Solução dos desafios 9 e 10

Desafio 9: Considere que cada entidade gerada no primeiro exemplo desta seção tem um peso em gramas dado por uma distribuição normal de média 10 e desvio padrão igual a 3. Crie um critério de parada para quando a média dos pesos das entidades geradas esteja no intervalo entre 9,5 e 10,5.

Este primeiro desafio envolve poucas modificações no programa original. Acrescentamos três variáveis novas: `media`, `contador` e `pesoTotal`; o laço `while` foi substituído pelo critério de parada e algumas linhas foram acrescentadas para o cálculo da média de peso até a última entidade gerada. O peso de cada entidade é sorteado pela função `random.normalvariate(mu, sigma)` da biblioteca `random`.

```
1  import simpy
2  import random
3
4  def geraChegada(env, numEntidades):
5      media, contador, pesoTotal = 0, 0, 0
6
7      while media > 10.5 or media < 9.5:      # critério de parada
8          yield env.timeout(1)
9          contador += 1                      # conta entidades geradas
10         peso = random.normalvariate(10, 3) # sorteia o peso da entidade
11         pesoTotal += peso                  # acumula o peso total até agora
12         media = pesoTotal/contador        # calcula média dos pesos
13         print("%4.1f nova chegada\tPeso: %4.1f kg\tMédia atual: %4.1f"
14               %(env.now, peso, media))
15
16
17 random.seed(100)
18 env = simpy.Environment()
19
20 # gera apenas 5 entidades
21 chegadas = env.process(geraChegada(env, 5))
22
23 # executa até o fim de todos os processos do modelo
24 env.run()
```

Quando executado, o modelo anterior apresenta como resultado:

1	1.0 nova chegada	Peso: 6.7 kg	Média atual: 6.7
2	2.0 nova chegada	Peso: 14.7 kg	Média atual: 10.7
3	3.0 nova chegada	Peso: 12.1 kg	Média atual: 11.2
4	4.0 nova chegada	Peso: 13.3 kg	Média atual: 11.7
5	5.0 nova chegada	Peso: 6.0 kg	Média atual: 10.6
6	6.0 nova chegada	Peso: 13.5 kg	Média atual: 11.0
7	7.0 nova chegada	Peso: 5.8 kg	Média atual: 10.3

Desafio 10: Modifique o critério anterior para que a parada ocorra quando a média for 10 kg, com um intervalo de confiança de amplitude 0,5 e nível de significância igual a 95%. Dica: utilize a biblioteca `numpy` para isso.

Esta situação exige um pouco mais no processo de codificação, contudo é algo muito utilizado em modelos de simulação de eventos discretos.

Como agora queremos o Intervalo de Confiança de uma dada amostra, os valores dos pesos serão armazenados em uma lista (`pesosList`, no caso do desafio).

A biblioteca `numpy`⁴ fornece um meio fácil de se estimar a média e o desvio padrão de uma amostra de valores armazenada numa lista:

- `numpy.mean(pesosList)`: estima a média da lista `pesosList`;
- `numpy.std(pesosList)`: estima o desvio-padrão da lista `pesosList`.

Para o cálculo do intervalo de confiança, devemos lembrar que, para amostras pequenas, a sua expressão é dada por:

$$[\bar{x} - t_{1-\alpha, n-1} \frac{s}{\sqrt{n}}, \bar{x} + t_{1-\alpha, n-1} \frac{s}{\sqrt{n}}]$$

A biblioteca `scipy`⁵ (pronuncia-se “Sigh Pie”) é pacote de código aberto para Matemática, Ciências e Engenharia. Ela possui diversas funções estatísticas dentro do seu módulo `scipy.stats`⁶, dentre elas, a distribuição t de student, necessária para o cálculo do intervalo de confiança.

Inicialmente, devemos importar a biblioteca `scipy.stats`:

```
1 import simpy
2 import random
3
4 # biblioteca numpy para computação científica http://www.numpy.org/
5 import numpy
6 import scipy.stats # biblioteca scipy
```

Como está será uma operação rotineira nos nossos modelos de simulação, o ideal é encapsular o código em uma função específica:

⁴<http://www.numpy.org/>

⁵<https://docs.scipy.org/doc/scipy/index.html>

⁶<https://docs.scipy.org/doc/scipy/reference/stats.html>

```

1 def intervaloConfMedia(a, conf=0.95):
2     # retorna a média e a amplitude do intervalo de confiança de a
3     media, sem = numpy.mean(a), scipy.stats.sem(a)
4     m = scipy.stats.t.ppf((1+conf)/2., len(a)-1)
5     h = m * sem
6     return media, h
7

```

A função anterior calcula a média e amplitude de um intervalo de confiança, a partir da lista de valores e do nível de confiança desejado.

O novo programa então ficaria:

```

1 import simpy
2 import random
3
4 # biblioteca numpy para computação científica http://www.numpy.org/
5 import numpy
6 import scipy # biblioteca scipy
7
8 def intervaloConfMedia(a, conf=0.95):
9     # retorna a média e a amplitude do intervalo de confiança de a
10    media, sem = numpy.mean(a), scipy.stats.sem(a)
11    m = scipy.stats.t.ppf((1+conf)/2., len(a)-1)
12
13    return media, h
14
15 def geraChegada(env):
16     # lista para armazenar os valores de pesos gerados
17     pesosList = []
18
19     while True:
20         yield env.timeout(1)
21
22         # adiciona à lista o peso da entidade atual
23         pesosList.append(random.normalvariate(10, 5))
24
25         # calcula a amplitude do intervalo, com nível de significância 95%
26         if len(pesosList) > 1:
27             media, amplitude = intervaloConfMedia(pesosList, 0.95)
28             print("%4.1f Média atual: %.2f kg\tAmplitude atual: %.2f kg"
29                 %(env.now, media, amplitude))
30
31             # se a amplitude atende ao critério estabelecido, interrompe o proces\
32 so
33             if amplitude < 0.5:
34                 print("\n%4.1f Intervalo de confiança atingido após %s valores! [\

```

```

35  "%.2f, %.2f]"
36          % (env.now, len(pesosList), media-amplitude, media+amplitude))
37
38          #termina o laço while
39          break
40
41
42  random.seed(100)
43  env = simpy.Environment()
44  chegadas = env.process(geraChegada(env))
45
46  # executa até o fim de todos os processos
47  env.run()

```

O programa anterior leva 411 amostras para atingir o intervalo desejado:

```

1  ...
2  410.0 Média atual: 10.17 kg      Amplitude atual: 0.50 kg
3  411.0 Média atual: 10.18 kg      Amplitude atual: 0.50 kg
4
5  411.0 Intervalo de confiança atingido após 411 valores! [9.68, 10.68]

```

Existem diversas maneiras de se estimar o intervalo de confiança utilizando-se as bibliotecas do Python. A maneira aqui proposta se baseia no `numpy` e no `scipy.stats`. Eventualmente, se tais bibliotecas não estão instaladas no seu ambiente Python, eu antecipo: isso pode ser um baita problema para você.

A questão central é que, usualmente, os modelos de simulação possuem grande demanda por processamento computacional na armazenagem de valores e estimativa de estatísticas durante ou ao final da simulação. A biblioteca `numpy` facilita bastante esta tarefa, principalmente quando se considera o suporte dado pelos usuários do [Stack Overflow](https://stackoverflow.com/).⁷

Como sugestão, habitue-se a construir funções padronizadas para monitoramento e cálculos estatísticos, de modo que você poderá reaproveitá-las em novos programas. Em algum momento, inclusive, você pode [criar sua própria biblioteca](#)⁸ de funções para análise de saída de modelos de simulação e compartilhar com a comunidade de software livre.

⁷<http://stackoverflow.com/search?q=numpy>

⁸<http://stackoverflow.com/questions/15746675/how-to-write-a-python-module>

10 Outros tipos de recursos: com prioridade e preemptivos

Além do recurso como definido nas seções anteriores, o SimPy possui dois tipos específicos de recursos: com prioridade e “preemptivos”.

10.1 Recursos com prioridade: PriorityResource

Um recurso pode ter uma fila de entidades desejando ocupá-lo para executar determinado processo. Existindo a fila, o recurso será ocupado respeitando a ordem de chegada das entidades (ou a regra FIFO).

Contudo, existem situações em que algumas entidades possuem *prioridades* sobre as outras, de modo que elas desrespeitam a regra do primeiro a chegar é o primeiro a ser atendido.

Por exemplo, considere um consultório de pronto atendimento de um hospital em que 70% do pacientes são de prioridade baixa (pulseira verde), 20% de prioridade intermediária (pulseira amarela) e 10% de prioridade alta (pulseira vermelha). Existem 2 médicos que realizam o atendimento e que sempre verificam inicialmente a ordem de prioridade dos pacientes na fila. Os pacientes chegam entre si em intervalos exponencialmente distribuídos, com média de 5 minutos e o atendimento é também exponencialmente distribuído, com média de 9 minutos por paciente.

No exemplo, os médicos são recursos, mas também respeitam uma regra específica de prioridade. Um médico ou recurso deste tipo, é criado pelo comando:

```
1 medicos = simpy.PriorityResource(env, capacity=capacidade_desejada)
```

Para a solução do exemplo, o modelo aqui proposto terá 3 funções: uma para sorteio do tipo de pulseira, uma para geração de chegadas de pacientes e outra para atendimento dos pacientes.

Como uma máscara inicial do modelo, teríamos:

```
1 import simpy
2 import random
3
4 def sorteiaPulseira():
5     # retorna a cor da pulseira e sua prioridade
6     pass
7
8 def chegadaPacientes(env, medicos):
9     # gera pacientes exponencialmente distribuídos
10    # sorteia a pulseira
```

```

11     # inicia processo de atendimento
12     pass
13
14 def atendimento(env, paciente, pulseira, prio, medicos):
15     # ocupa um médico e realiza o atendimento do paciente
16     pass
17
18 random.seed(100)
19 env = simpy.Environment()
20 medicos = simpy.PriorityResource(env, capacity=2) # cria os 2 médicos
21 chegadas = env.process(chegadaPacientes(env, medicos))
22
23 env.run(until=20)

```

O preenchimento da máscara pode ser feito de diversas maneiras, uma possibilidade seria:

```

1  import simpy
2  import random
3
4  def sorteiaPulseira():
5      # retorna a cor da pulseira e sua prioridade
6      r = random.random()          # sorteia número aleatório entre 0 e 1
7      if r <= .70:                  # 70% é pulseira verde
8          return "pulseira verde", 3
9      elif r <= .90:                # 20% (=90-70) é pulseira amarela
10         return "pulseira amarela", 2
11         return "pulseira vermelha", 1    # 10% (=100-90) é pulseira vermelha
12
13 def chegadaPacientes(env, medicos):
14     #gera pacientes exponencialmente distribuídos
15
16     i = 0
17     while True:
18         yield env.timeout(random.expovariate(1/5))
19         i += 1
20
21         # sorteia a pulseira
22         pulseira, prio = sorteiaPulseira()
23         print("%4.1f Paciente %2i com %s chega" %(env.now, i, pulseira))
24
25         # inicia processo de atendimento
26         env.process(atendimento(env, "Paciente %2i" %i, pulseira, prio, medicos))
27
28 def atendimento(env, paciente, pulseira, prio, medicos):
29     # ocupa um médico e realiza o atendimento do paciente
30

```



```

31     with medicos.request(priority=prio) as req:
32         yield req
33         print("%4.1f %s com %s inicia o atendimento" %(env.now, paciente, pulseir\
34 a))
35         yield env.timeout(random.expovariate(1/9))
36         print("%4.1f %s com %s termina o atendimento" %(env.now, paciente, pulsei\
37 ra))
38
39 random.seed(100)
40 env = simpy.Environment()
41 # cria os médicos
42 medicos = simpy.PriorityResource(env, capacity=2)
43 chegadas = env.process(chegadaPacientes(env, medicos))
44
45 env.run(until=20)

```

O importante a ser destacado é que a prioridade é informada ao request do recurso medicos pelo argumento priority:

```

1  with medicos.request(priority=prio) as req:
2      yield req

```

Para o SimPy, **quando menor o valor fornecido para o parâmetro priority, maior a prioridade** daquela entidade na fila. Assim, a função sorteiaPulseira retorna 3 para a pulseira verde (de menor prioridade) e 1 para a vermelha (de maior prioridade).

Quando o modelo anterior é executado, fornece como saída:

```

1  0.8 Paciente 1 com pulseira verde chega
2  0.8 Paciente 1 com pulseira verde inicia o atendimento
3  8.2 Paciente 2 com pulseira amarela chega
4  8.2 Paciente 2 com pulseira amarela inicia o atendimento
5 11.0 Paciente 3 com pulseira verde chega
6 11.4 Paciente 4 com pulseira verde chega
7 11.7 Paciente 5 com pulseira vermelha chega
8 11.8 Paciente 1 com pulseira verde termina o atendimento
9 11.8 Paciente 5 com pulseira vermelha inicia o atendimento
10 15.5 Paciente 5 com pulseira vermelha termina o atendimento
11 15.5 Paciente 3 com pulseira verde inicia o atendimento
12 18.8 Paciente 3 com pulseira verde termina o atendimento
13 18.8 Paciente 4 com pulseira verde inicia o atendimento

```

Percebemos que o paciente 5 chegou no instante 11,7 minutos, depois do pacientes 3 e 4, mas iniciou seu atendimento assim que um médico ficou livre no instante 11,8 minutos (exatamente aquele que atendia ao Paciente 1).

10.2 Recursos que podem ser interrompidos: PreemptiveResource

Considere, no exemplo anterior, que o paciente de pulseira vermelha tem uma prioridade tal que ele interrompe o atendimento atual do médico e imediatamente é atendido. Os recursos com [preemptividade](https://pt.wikipedia.org/wiki/Preemptividade)¹ são recursos que aceitam a interrupção da tarefa em execução para iniciar outra de maior prioridade.

Um recurso capaz de ser interrompido é criado pelo comando:

```
1 medicos = simpy.PreemptiveResource(env, capacity=capacidade)
```

Assim, o modelo anterior precisa ser modificado de modo a criar os médicos corretamente:

```
1 random.seed(100)
2 env = simpy.Environment()
3 # cria os médicos
4 medicos = simpy.PreemptiveResource(env, capacity=2)
5 chegadas = env.process(chegadaPacientes(env, medicos))
6
7 env.run(until=20)
```

Agora, devemos modificar a função `atendimento` para garantir que quando um recurso for requisitado por um processo de menor prioridade, ele causará uma interrupção no Python, o que obriga a utilização de bloco de controle de interrupção `try: ...except`.

Quando um recurso deve ser interrompido, o SimPy retorna um interrupção do tipo `simpy.Interrupt`, como mostrado no código a seguir (note o bloco `try: ...except` dentro da função `atendimento`):

```
1 def atendimento(env, paciente, pulseira, prio, medicos):
2     # ocupa um médico e realiza o atendimento do paciente
3
4     with medicos.request(priority=prio) as req:
5         yield req
6         print("%4.1f %s com %s inicia o atendimento" %(env.now, paciente, pulseir\
7 a))
8         try:
9             yield env.timeout(random.expovariate(1/9))
10            print("%4.1f %s com %s termina o atendimento" %(env.now, paciente, pu\
11 lseira))
12        except:
13            print("%4.1f %s com %s tem atendimento interrompido" %(env.now, pacie\
14 nte, pulseira))
15
```

¹<https://pt.wikipedia.org/wiki/Preemptividade>

```

16 random.seed(100)
17 env = simpy.Environment()
18 # cria os médicos
19 medicos = simpy.PreemptiveResource(env, capacity=2)
20 chegadas = env.process(chegadaPacientes(env, medicos))
21
22 env.run(until=20)

```

Quando simulado por apenas 20 minutos, o modelo acrescido das correções apresentadas fornece a seguinte saída:

```

1  0.8 Paciente  1 com pulseira verde chega
2  0.8 Paciente  1 com pulseira verde inicia o atendimento
3  8.2 Paciente  2 com pulseira amarela chega
4  8.2 Paciente  2 com pulseira amarela inicia o atendimento
5 11.0 Paciente  3 com pulseira verde chega
6 11.4 Paciente  4 com pulseira verde chega
7 11.7 Paciente  5 com pulseira vermelha chega
8 11.7 Paciente  1 com pulseira verde tem atendimento interrompido
9 11.7 Paciente  5 com pulseira vermelha inicia o atendimento
10 15.3 Paciente  5 com pulseira vermelha termina o atendimento
11 15.3 Paciente  3 com pulseira verde inicia o atendimento
12 18.7 Paciente  3 com pulseira verde termina o atendimento
13 18.7 Paciente  4 com pulseira verde inicia o atendimento

```

Note que o Paciente 5 interrompe o atendimento do Paciente 1, como desejado.

Contudo, a implementação anterior está cheia de limitações: pacientes com pulseira amarela não deveriam interromper o atendimento, mas na implementação proposta eles devem interromper o atendimento de pacientes de pulseira verde. Para estas situações, o request possui um argumento preempt que permite ligar ou desligar a opção de preemptividade:

```

1 with medicos.request(priority=prio, preempt=preempt) as req:
2     yield req

```

O modelo alterado para interromper apenas no caso de pulseiras vermelhas, ficaria (note que o argumento preempt é agora fornecido diretamente a partir da função sorteiaPulseira):

```

1  import simpy
2  import random
3
4  def sorteiaPulseira():
5      # retorna a cor da pulseira e sua prioridade
6      r = random.random() # sorteia número aleatório ente 0 e 1
7      if r <= .70: # 70% é pulseira verde
8          return "pulseira verde", 3, False
9      elif r <= .90: # 20% (=90-70) é pulseira amarela
10         return "pulseira amarela", 2, False
11         return "pulseira vermelha", 1, True # 10% (=100-90) é pulseira vermelha
12
13 def chegadaPacientes(env, medicos):
14     #gera pacientes exponencialmente distribuídos
15
16     i = 0
17     while True:
18         yield env.timeout(random.expovariate(1/5))
19         i += 1
20
21         # sorteia a pulseira
22         pulseira, prio, preempt = sorteiaPulseira()
23         print("%4.1f Paciente %2i com %s chega" %(env.now, i, pulseira))
24
25         # inicia processo de atendimento
26         env.process(atendimento(env, "Paciente %2i" %i, pulseira, prio, preempt, \
27 medicos))
28
29 def atendimento(env, paciente, pulseira, prio, preempt, medicos):
30     # ocupa um médico e realiza o atendimento do paciente
31
32     with medicos.request(priority=prio, preempt=preempt) as req:
33         yield req
34         print("%4.1f %s com %s inicia o atendimento" %(env.now, paciente, pulseir\
35 a))
36         try:
37             yield env.timeout(random.expovariate(1/9))
38             print("%4.1f %s com %s termina o atendimento" %(env.now, paciente, pu\
39 lseira))
40         except:
41             print("%4.1f %s com %s tem atendimento interrompido" %(env.now, pacie\
42 nte, pulseira))
43
44 random.seed(100)
45 env = simpy.Environment()
46 # cria os médicos

```

```

47 medicos = simpy.PreemptiveResource(env, capacity=2)
48 chegadas = env.process(chegadaPacientes(env, medicos))
49
50 env.run(until=20)

```

O modelo anterior, quando executado por apenas 20 minutos, fornece como saída:

```

1  0.8 Paciente  1 com pulseira verde chega
2  0.8 Paciente  1 com pulseira verde inicia o atendimento
3  8.2 Paciente  2 com pulseira amarela chega
4  8.2 Paciente  2 com pulseira amarela inicia o atendimento
5 11.0 Paciente  3 com pulseira verde chega
6 11.4 Paciente  4 com pulseira verde chega
7 11.7 Paciente  5 com pulseira vermelha chega
8 11.7 Paciente  1 com pulseira verde tem atendimento interrompido
9 11.7 Paciente  5 com pulseira vermelha inicia o atendimento
10 15.3 Paciente  5 com pulseira vermelha termina o atendimento
11 15.3 Paciente  3 com pulseira verde inicia o atendimento
12 18.7 Paciente  3 com pulseira verde termina o atendimento
13 18.7 Paciente  4 com pulseira verde inicia o atendimento

```

10.3 Conceitos desta seção

Conteúdo	Descrição
<code>meuRecurso = simpy.PriorityResource(env, capacity=1)</code>	Cria um recurso com prioridade e capacidade = 1
<code>meuRequest = meuRecurso.request(env, priority=prio)</code>	Solicita o recurso <code>meuRecurso</code> (note que ele ainda não ocupa o recurso) respeitando a ordem de prioridade primeiro e a regra FIFO a seguir
<code>meuRecursoPreempt = simpy.PreemptiveResource(env, capacity=1)</code>	Cria um recurso em <code>env</code> que pode ser interrompido por entidades de prioridade maior
<code>meuRequest = meuRecursoPreempt.request(env, priority=prio, preempt=preempt)</code>	Solicita o recurso <code>meuRecurso</code> (note que ele ainda não ocupa o recurso) respeitando a ordem de prioridade primeiro e a regra FIFO a seguir. Caso <code>preempt</code> seja <code>False</code> o o recurso não é interrompido
<code>try:...except simpy.Interrupt:</code>	Chamada de interrupção utilizada na lógica <code>try:...except:</code>

10.4 Desafios



Desafio 11: acrescente ao último programa proposto o cálculo do tempo de atendimento que ainda falta de atendimento para o paciente que foi interrompido por outro e imprima o resultado na tela.



Desafio 12: quando um paciente é interrompido, ele deseja retornar ao atendimento de onde parou. Altere o programa para que um paciente de pulseira verde interrompido possa retornar para ser atendido no tempo restante do seu atendimento. Dica: altere a numeração de prioridades de modo que um paciente verde interrompido tenha prioridade superior ao de um paciente verde que acabou de chegar.

10.5 Solução dos desafios 11 e 12

Desafio 11: acrescente ao último programa proposto o cálculo do tempo de atendimento que ainda falta para o paciente que foi interrompido por outro e imprima o resultado na tela.

Neste caso, precisamos acrescentar o cálculo do tempo faltante para o paciente na função `atendimento`:

```

1  def atendimento(env, paciente, pulseira, prio, preempt, medicos):
2      # ocupa um médico e realiza o atendimento do paciente
3
4      with medicos.request(priority=prio, preempt=preempt) as req:
5          yield req
6          inicioAtendimento = env.now
7          print("%4.1f %s com %s inicia o atendimento" %(env.now, paciente, pulseir\
8  a))
9          try:
10             # sorteia o tempo de atendimento
11             tempoAtendimento = random.expovariate(1/9)
12             yield env.timeout(tempoAtendimento)
13             print("%4.1f %s com %s termina o atendimento"
14                   %(env.now, paciente, pulseira))
15         except simpy.Interrupt:
16             # recalcula o tempo de atendimento
17             tempoAtendimento -= env.now-inicioAtendimento
18             print("%4.1f %s com %s tem atendimento interrompido"
19                   %(env.now, paciente, pulseira))
20             print("%4.1f %s ainda precisa de %4.1f min de atendimento"
21                   %(env.now, paciente, tempoAtendimento))

```

Quando o modelo é executado por apenas 20 minutos, com a alteração apresentada da função `atendimento`, temos como saída:

```

1  0.8 Paciente 1 com pulseira verde chega
2  0.8 Paciente 1 com pulseira verde inicia o atendimento
3  8.2 Paciente 2 com pulseira amarela chega
4  8.2 Paciente 2 com pulseira amarela inicia o atendimento
5  11.0 Paciente 3 com pulseira verde chega
6  11.4 Paciente 4 com pulseira verde chega
7  11.7 Paciente 5 com pulseira vermelha chega
8  11.7 Paciente 1 com pulseira verde tem atendimento interrompido
9  11.7 Paciente 1 ainda precisa de 0.1 min de atendimento
10 11.7 Paciente 5 com pulseira vermelha inicia o atendimento
11 15.3 Paciente 5 com pulseira vermelha termina o atendimento

```

```

12 15.3 Paciente 3 com pulseira verde inicia o atendimento
13 18.7 Paciente 3 com pulseira verde termina o atendimento
14 18.7 Paciente 4 com pulseira verde inicia o atendimento

```

Desafio 12: quando um paciente é interrompido, ele deseja retornar ao atendimento de onde parou. Altere o programa para que um paciente de pulseira verde interrompido possa retornar para ser atendido no tempo restante do seu atendimento. Dica: altere a numeração de prioridades de modo que um paciente verde interrompido tenha prioridade superior ao de um paciente verde que acabou de chegar.

Novamente, as alterações no modelo anterior resumem-se à função `atendimento`: precisamos aumentar a prioridade de um paciente interrompido em relação aos pacientes que acabam de chegar com a mesma pulseira, afinal, ele tem prioridade em relação a um paciente recém chegado de gravidade equivalente. Além disso, tal paciente, deve ser atendido pelo tempo *restante* de atendimento, de modo que a função deve receber como parâmetro esse tempo.

O artifício utilizado neste segundo caso foi acrescentar um parâmetro opcional à função, `tempoAtendimento`, de modo que se ele não é fornecido (caso de um paciente novo), a função sorteia um tempo exponencialmente distribuído, com média de 9 minutos. De outro modo, se o parâmetro é fornecido, isso significa que ele é um parceiro interrompido e, portanto, já tem um tempo restante de atendimento calculado.

O código a seguir, representa uma possível solução para a *nova* função `atendimento` do desafio:

```

1  def atendimento(env, paciente, pulseira, prio, preempt, medicos, tempoAtendimento\
2  =None):
3      # ocupa um médico e realiza o atendimento do paciente
4
5      with medicos.request(priority=prio, preempt=preempt) as req:
6          yield req
7          inicioAtendimento = env.now
8          print("%4.1f %s com %s inicia o atendimento" %(env.now, paciente, pulseir\
9  a))
10         try:
11             # sorteia o tempo de atendimento
12             if not tempoAtendimento:
13                 tempoAtendimento = random.expovariate(1/9)
14             yield env.timeout(tempoAtendimento)
15             print("%4.1f %s com %s termina o atendimento"
16                   %(env.now, paciente, pulseira))
17         except simpy.Interrupt:
18             # recalcula o tempo de atendimento
19             tempoAtendimento -= env.now-inicioAtendimento
20             print("%4.1f %s com %s tem atendimento interrompido"
21                   %(env.now, paciente, pulseira))
22             print("%4.1f %s ainda precisa de %4.1f min de atendimento"
23                   %(env.now, paciente, tempoAtendimento))

```



```
24
25         # aumenta a prioridade reduzindo o valor
26         prio -= 0.01
27         env.process(atendimento(env, paciente, pulseira, prio, preempt, medic\
28 os, tempoAtendimento))
29
30 random.seed(100)
31 env = simpy.Environment()
32 # cria os médicos
33 medicos = simpy.PreemptiveResource(env, capacity=2)
34 chegadas = env.process(chegadaPacientes(env, medicos))
35
36 env.run(until=20)
```

Quando executado por apenas 20 minutos, o modelo completo - acrescido da nova função atendimento, fornece como saída:

```
1  0.8 Paciente  1 com pulseira verde chega
2  0.8 Paciente  1 com pulseira verde inicia o atendimento
3  8.2 Paciente  2 com pulseira amarela chega
4  8.2 Paciente  2 com pulseira amarela inicia o atendimento
5 11.0 Paciente  3 com pulseira verde chega
6 11.4 Paciente  4 com pulseira verde chega
7 11.7 Paciente  5 com pulseira vermelha chega
8 11.7 Paciente  1 com pulseira verde tem atendimento interrompido
9 11.7 Paciente  1 ainda precisa de  0.1 min de atendimento
10 11.7 Paciente  5 com pulseira vermelha inicia o atendimento
11 15.3 Paciente  5 com pulseira vermelha termina o atendimento
12 15.3 Paciente  1 com pulseira verde inicia o atendimento
13 15.5 Paciente  1 com pulseira verde termina o atendimento
14 15.5 Paciente  3 com pulseira verde inicia o atendimento
15 18.8 Paciente  3 com pulseira verde termina o atendimento
16 18.8 Paciente  4 com pulseira verde inicia o atendimento
```

Note que agora, o Paciente 1, diferentemente do que ocorre na saída do desafio 11, é atendido antes do Paciente 3, representando o fato de que, mesmo interrompido, ele voltou para o início da fila.

11 Interrupções de processos: `simpy.Interrupt`

Você está todo feliz e contente atravessando a galáxia no seu [X-Wing](https://en.wikipedia.org/wiki/X-wing_fighter)¹ quando... PIMBA! Seu [dróide astromecânico](https://pt.wikipedia.org/wiki/R2-D2)² pifa e só lhe resta interromper a viagem para consertá-lo, antes que apareça um maldito caça [TIE](https://en.wikipedia.org/wiki/TIE_fighter)³ das forças imperiais.

Nesta seção iremos interromper processos já em execução e depois retomar a operação inicial. A aplicação mais óbvia é para a quebra de equipamentos durante a operação, como no caso do R2D2.

A interrupção de um processo em SimPy é realizada por meio de um comando `Interrupt` aplicado ao processo já iniciado. O cuidado aqui é que quando um recurso é interrompido por outro processo ele causa uma interrupção, de fato, no Python, o que nos permite utilizar o bloco de controle de interrupção `try: ... except`, o que não deixa de ser uma boa coisa, dada a sua facilidade.

11.1 Criando quebras de equipamento

Voltando ao exemplo do X-Wing, considere que a cada 10 horas o R2D2 interrompe a viagem para uma manutenção de 5 horas e que a viagem toda levaria (sem não houvessem paralisações) 50 horas.

Inicialmente, vamos criar duas variáveis globais: uma para representar se o X-Wing está operando - afinal, não queremos interrompê-lo quando ele já estiver em manutenção - e outra para armazenar o tempo ainda restante para a viagem.

```
1 import simpy
2
3 viajando = False      # variável global que avisa se o x-wing está operando
4 duracaoViagem = 30    # variável global que marca a duração atual da viagem
```

O próximo passo, é criar uma função que represente a viagem do x-wing, garantindo não só que ela dure o tempo correto, mas também que lide com o processo de interrupção:

¹https://en.wikipedia.org/wiki/X-wing_fighter

²<https://pt.wikipedia.org/wiki/R2-D2>

³https://en.wikipedia.org/wiki/TIE_fighter

```

1  import simpy
2
3  viajando = False          # variável global que avisa se o x-wing está operando
4  duracaoViagem = 30        # variável global que marca a duração atual da viagem
5
6  def viagem(env, tempoParada):
7      #processo de viagem do x-wing
8      global viajando, duracaoViagem
9
10     partida = env.now      # início da viagem
11     while duracaoViagem > 0: # enquanto ainda durar a viagem, execute:
12         try:
13             viajando = True
14             # (re)início da viagem
15             inicioViagem = env.now
16             print("%5.1f Viagem iniciada" %(env.now))
17             # tempo de viagem restante
18             yield env.timeout(duracaoViagem)
19             duracaoViagem -= env.now - inicioViagem
20
21         except simpy.Interrupt:
22             # se o processo de viagem foi interrompido execute
23             # atualiza o tempo restante de viagem
24             duracaoViagem -= env.now - inicioViagem
25             print("%5.1f Falha do R2D2\tTempo de viagem restante: %4.1f horas"
26                   %(env.now, duracaoViagem))
27             # tempo de manutenção do R2D2
28             yield env.timeout(tempoParada)
29
30     # ao final avisa o término da viagem e sua duração
31     print("%5.1f Viagem concluída\tDuração total da viagem: %4.1f horas"
32           %(env.now, env.now-partida))
33
34
35 env = simpy.Environment()
36 viagem = env.process(viagem(env, 15))
37
38 env.run()
```

O importante no programa anterior é notar o bloco `try:...except:`, interno ao laço `while duracaoViagem > 0`, que mantém o nosso X-Wing em processo enquanto a variável `duracaoViagem` for maior que zero. O `except` aguarda um novo comando, o `simpy.Interrupt`, que nada mais é do que uma interrupção causada por algum outro processo dentro do `Environment`.

Quando executado, o programa fornece uma viagem tranquila:

```

1  0.0 Viagem iniciada
2  30.0 Viagem concluída Duração total da viagem: 30.0 horas

```

A viagem é tranquila, pois não criamos ainda um “gerador de interrupções”, que nada mais é do que um processo em SimPy que cria a interrupção da viagem.

Note, na penúltima linha do código anterior, que o processo em execução foi armazenado na variável `viagem` e o que devemos fazer é interrompê-lo de 10 em 10 horas. Para tanto, a função `paradaTecnica` a seguir, verifica se o processo de viagem está em andamento e paralisa a operação depois de 10 horas:

```

1  def paradaTecnica(env, intervaloQuebra, viagem):
2      # processo de paradas entre intervalos de quebra
3      global viajando, duracaoViagem
4
5      while duracaoViagem > 0:
6          # este processo só ocorre durante a viagem
7
8          yield env.timeout(intervaloQuebra) # aguarda a próxima quebra do R2D2
9          if viajando:                       # R2D2 somente quebra durante a viagem
10             viagem.interrupt()             # interrompe o processo viagem
11             viajando = False               # desliga a viagem
12
13  env = simpy.Environment()
14  viagem = env.process(viagem(env, 15))
15  env.process(paradaTecnica(env, 10, viagem))
16
17  env.run()

```

A função `paradaTecnica`, portanto, recebe como parâmetro o próprio *objeto* que representa o processo viagem e, por meio do comando:

```

1  viagem.interrupt()

```

Provoca uma interrupção no processo, a ser reconhecida pela função `viagem` na linha:

```

1  except simpy.Interrupt:

```

Adicionalmente, o processo parada técnica também deve ser iniciado ao início da simulação, de modo que a parte final do modelo fica:

```

1 env = simpy.Environment()
2 viagem = env.process(viagem(env, 15))
3 env.process(paradaTecnica(env, 10, viagem))
4
5 env.run()

```

Quando executado, o modelo completo fornece como saída:

```

1 0.0 Viagem iniciada
2 10.0 Falha do R2D2      Tempo de viagem restante: 20.0 horas
3 25.0 Viagem iniciada
4 30.0 Falha do R2D2      Tempo de viagem restante: 15.0 horas
5 45.0 Viagem iniciada
6 50.0 Falha do R2D2      Tempo de viagem restante: 10.0 horas
7 65.0 Viagem iniciada
8 70.0 Falha do R2D2      Tempo de viagem restante: 5.0 horas
9 85.0 Viagem iniciada
10 90.0 Falha do R2D2      Tempo de viagem restante: 0.0 horas
11 105.0 Viagem concluida  Duração total da viagem: 105.0 horas

```

Alguns aspectos importantes do código anterior:

1. A utilização de variáveis globais foi fundamental para informar ao processo de parada o status do processo de viagem. É por meio de variáveis globais que um processo “sabe” o que está ocorrendo no outro;
2. Como a execução `env.run()` não tem um tempo final pré-estabelecido, a execução dos processos é terminada quando o laço `while` resulta em um Falso:

```

1 while duracaoViagem > 0

```

Note que esse `while` deve existir nos dois processos em execução, caso contrário, o programa seria executado indefinidamente;

3. Dentro da função `paradaTecnica` a variável global `viajando` impede que ocorram duas quebras ao mesmo tempo. Naturalmente o leitor atento sabe que isso jamais ocorreria, afinal, o tempo de duração da quebra é inferior ao intervalo entre quebras. Mas fica o exercício: execute o mesmo programa, agora para uma duração de quebra de 15 horas e veja o que acontece.
4. Se um modelo possui uma lógica de interrupção `.interrupt()` e não possui um comando `except simpy.Interrupt` para lidar com a paralisação do processo, o SimPy finalizará a simulação retornando o erro:

```

1 Interrupt: Interrupt(None)

```

Na próxima seção é apresentada uma alternativa para manter o processamento da simulação.

11.2 Interrompendo um processo sem captura por `try...except`

Caso o modelo possua um comando de interrupção de processo, mas não exista nenhuma lógica de captura com o bloco `try... except`, a simulação será finalizada com a mensagem de erro:

```
1 Interrupt: Interrupt(None)
```

O SimPy cria, para cada processo, uma propriedade chamada `defused` que permite contornar a paralisação. Assim, pode-se interromper um processo, sem que essa interrupção provoque estrago algum ao processamento do modelo. Para *desativar* a paralisação do modelo (e manter apenas a paralisação do processo), basta tornar a propriedade `defused = True`:

```
1 processVar.interrupt()
2 processVar.defused = True
```

Um exemplo bem prático: você está na sua rotina de exercícios matinais, quando... PIMBA:

```
1 import simpy
2
3 def forca(env):
4     # processo a ser interrompido
5     while True:
6         yield env.timeout(1)
7         print('%d Eu estou com a Força e a Força está comigo.' % env.now)
8
9 def ladoNegro(env, proc):
10    yield env.timeout(3)
11    print('%d Venha para o lado negro da força, nós temos CHURROS!' % env.now)
12
13    # interrompe o processo proc
14    proc.interrupt()
15
16    # desarme do processo para evitar a interrupção da simulação
17    proc.defused = True
18    print('%d Ponto para o Império!' % env.now)
19
20 env = simpy.Environment()
21
22 forcaProc = env.process(forca(env))
23 ladoNegroProc = env.process(ladoNegro(env, forcaProc))
24
25 env.run()
```

O modelo anterior deve ser autoexplicativo: o processo `força` é interrompido pelo processo `ladoNegro` depois de 3 unidades de tempo. Note a linha `proc.defused = True` que, ao “desarmar” o processo, impede que o Python interrompa o programa todo.

Quando executado, o modelo fornece como resultado:

```
1 1 Eu estou com a Força e a Força está comigo.
2 2 Eu estou com a Força e a Força está comigo.
3 3 Venha para o lado negro da força, nós temos CHURROS!
4 3 Ponto para o Império!
```

11.3 Conceitos desta seção

Conteúdo	Descrição
<code>processVar = env.process(função_processo(env))</code>	armazena o processo da função <code>processo</code> na variável <code>processVar</code>
<code>processVar.interrupt()</code>	interrompe o processo armazenado na variável <code>processVar</code>
<code>processVar.defused = True</code>	impede que o Python interrompa toda a simulação quando o <code>processVar</code> é interrompido
<code>try...except simpy.Interrupt</code>	lógica <code>try...except</code> necessária para interrupção do processo

11.4 Desafios



Desafio 13: Considere que existam dois tipos de paradas: uma do R2D2 e outra do canhão de combate. A parada do canhão de combate ocorre sempre depois de 25 horas de viagem (em quebra ou não) e seu reparo dura 2 horas. Contudo, para não perder tempo, a manutenção do canhão só é realizada quando o R2D2 quebra.



Desafio 14: Você não acha que pode viajar pelo espaço infinito sem encontrar alguns TIEs⁴ das forças imperiais, não é mesmo? Considere que a cada 25 horas, você se depara com um TIE imperial. O ataque dura 30 minutos e, se nesse tempo você não estiver com o canhão funcionando, a sua próxima viagem é para o encontro do mestre Yoda.

⁴https://en.wikipedia.org/wiki/TIE_fighter

Dica: construa uma função `executaCombate` que controla todo o processo de combate. Você vai precisar também de uma variável global que informa se o X-Wing está ou não em combate.

11.5 Solução dos desafios 13 e 14

Desafio 13: Considere que existam dois tipos de paradas: uma do R2D2 e outra do canhão de combate. A parada do canhão de combate ocorre sempre depois de 25 horas de viagem (em quebra ou não) e seu reparo dura 2 horas. Contudo, para não perder tempo, a manutenção do canhão só é realizada quando o R2D2 quebra.

Inicialmente, precisamos de uma variável global para verificar a situação do canhão:

```

1  import simpy
2
3  canhao = True           # variável global que avisa se o canhão está funcionando
4  viajando = False       # variável global que avisa se o x-wing está operando
5  duracaoViagem = 30     # variável global que marca a duração atual da viagem

```

A função `viagem`, agora deve lidar com um tempo de parada do canhão. Contudo, não existe uma interrupção para o canhão, pois nosso indomável piloto jedi apenas verifica a situação do canhão ao término do concerto do R2D2:

```

1  def viagem(env, tempoParada, tempoParadaCanhao):
2      # processo de viagem do x-wing
3      global viajando, duracaoViagem, canhao
4
5      partida = env.now      # início da viagem
6      while duracaoViagem > 0: # enquanto ainda durar a viagem, execute:
7          try:
8              viajando = True
9              # (re)início da viagem
10             inicioViagem = env.now
11             print("%5.1f Viagem iniciada" %(env.now))
12             # tempo de viagem restante
13             yield env.timeout(duracaoViagem)
14             duracaoViagem -= env.now - inicioViagem
15
16         except simpy.Interrupt:
17             # se o processo de viagem foi interrompido execute
18             # atualiza o tempo restante de viagem
19             duracaoViagem -= env.now - inicioViagem
20             print("%5.1f Falha do R2D2\tTempo de viagem restante: %4.1f horas"
21                   %(env.now, duracaoViagem))
22             # tempo de manutenção do R2D2
23             yield env.timeout(tempoParada)
24             print("%5.1f R2D2 operante" %env.now)
25             # se o canhão não estiver funcionando
26             if not canhao:

```

```

27         print("%5.1f R2, ligue o canhão!" % env.now)
28         # tempo de manutenção do canhão
29         yield env.timeout(tempoParadaCanhao)
30         canhao = True
31         print("%5.1f Pi..bi...bi\tTradução: canhão operante!" % env.now)
32
33     # ao final avisa o término da viagem e sua duração
34     print("%5.1f Viagem concluída\tDuração total da viagem: %4.1f horas"
35           %(env.now, env.now-partida))

```

Além da função de parada técnica (já pertencente ao nosso modelo desde a seção anterior), precisamos de uma função que tire de operação o canhão, ou seja, apenas desligue a variável global `canhao`:

```

1  def quebraCanhao(env, intervaloCanhao):
2      # processo de quebra do canhao entre intervalos definidos
3      global canhao, duracaoViagem
4
5      while duracaoViagem > 0:                # este processo só ocorre durante a v\
6  viagem
7          yield env.timeout(intervaloCanhao) # aguarda a próxima quebra do canhao
8          if canhao:
9              canhao = False
10         print("%5.1f Pi uiui...bi\tTradução: canhão inoperante!" % (env.now))

```

Por fim, o processo de quebra do canhão deve ser iniciado juntamente com o resto do modelo de simulação:

```

1  env = simpy.Environment()
2  viagem = env.process(viagem(env, 15, 2))
3  env.process(paradaTecnica(env, 10, viagem))
4  env.process(quebraCanhao(env, 25))
5
6  env.run()

```

Quando o modelo completo é executado, ele fornece como saída:

```
1  0.0 Viagem iniciada
2  10.0 Falha do R2D2      Tempo de viagem restante: 20.0 horas
3  25.0 Pi uiui...bi      Tradução: canhão inoperante!
4  25.0 R2D2 operante
5  25.0 R2, ligue o canhão!
6  27.0 Pi..bi...bi       Tradução: canhão operante!
7  27.0 Viagem iniciada
8  30.0 Falha do R2D2      Tempo de viagem restante: 17.0 horas
9  45.0 R2D2 operante
10 45.0 Viagem iniciada
11 50.0 Pi uiui...bi      Tradução: canhão inoperante!
12 50.0 Falha do R2D2      Tempo de viagem restante: 12.0 horas
13 65.0 R2D2 operante
14 65.0 R2, ligue o canhão!
15 67.0 Pi..bi...bi       Tradução: canhão operante!
16 67.0 Viagem iniciada
17 70.0 Falha do R2D2      Tempo de viagem restante: 9.0 horas
18 75.0 Pi uiui...bi      Tradução: canhão inoperante!
19 85.0 R2D2 operante
20 85.0 R2, ligue o canhão!
21 87.0 Pi..bi...bi       Tradução: canhão operante!
22 87.0 Viagem iniciada
23 90.0 Falha do R2D2      Tempo de viagem restante: 6.0 horas
24 100.0 Pi uiui...bi     Tradução: canhão inoperante!
25 105.0 R2D2 operante
26 105.0 R2, ligue o canhão!
27 107.0 Pi..bi...bi      Tradução: canhão operante!
28 107.0 Viagem iniciada
29 110.0 Falha do R2D2     Tempo de viagem restante: 3.0 horas
30 125.0 Pi uiui...bi     Tradução: canhão inoperante!
31 125.0 R2D2 operante
32 125.0 R2, ligue o canhão!
33 127.0 Pi..bi...bi      Tradução: canhão operante!
34 127.0 Viagem iniciada
35 130.0 Falha do R2D2     Tempo de viagem restante: 0.0 horas
36 145.0 R2D2 operante
37 145.0 Viagem concluída  Duração total da viagem: 145.0 horas
38 150.0 Pi uiui...bi     Tradução: canhão inoperante!
```

Desafio 14: Você não acha que pode viajar pelo espaço infinito sem encontrar alguns TIEs⁵ das forças imperiais, não é mesmo? Considere que a cada 25 horas, você se depara com um TIE imperial. O ataque dura 30 minutos e, se nesse tempo você não estiver com o canhão funcionando, a sua próxima viagem é para o encontro do mestre Yoda.

⁵https://en.wikipedia.org/wiki/TIE_fighter

Dica: construa uma função `executaCombate` que controla todo o processo de combate. Você vai precisar também de uma variável global que informa se o X-Wing está ou não em combate.

Inicialmente, precisamos de uma variável global para verificar a situação do combate:

```

1  import simpy
2
3  emCombate = False      # variável global que avisa se o x-wing está em combate
4  canhao = True         # variável global que avisa se o canhão está funcionando
5  viajando = False      # variável global que avisa se o x-wing está operando
6  duracaoViagem = 30    # variável global que marca a duração atual da viagem

```

A função `executaCombate` a seguir, inicia o processo de combate e verifica quem foi o vitorioso:

```

1  def executaCombate(env, intervaloCombate, duracaoCombate):
2      # processo de execução do combate
3
4      global emCombate, canhao, duracaoViagem
5
6      while duracaoViagem > 0: # este processo só ocorre durante a viagem
7          # aguarda o próximo encontro com as forças imperiais
8          yield env.timeout(intervaloCombate)
9          # inicio do combate
10         emCombate = True
11         if not canhao:
12             print("%5.1f Fim da viagem\t0 lado negro venceu" %env.now)
13             break
14         else:
15             print("%5.1f Combate iniciado\tReze para que o canhão não quebre!"
16                   %env.now)
17             try:
18                 yield env.timeout(duracaoCombate)
19                 emCombate = False
20                 print("%5.1f Fim do combate\tR2D2 diz: tá tudo tranquilo, tá tudo\
21 normalizado."
22                       %env.now)
23             except simpy.Interrupt:
24                 print("%5.1f OPS... O canhão quebrou durante o combate." %env.now)
25                 print("%5.1f Fim da viagem\t0 lado negro venceu" %env.now)
26                 break

```

A quebra de canhão agora deve verificar se a nave está em combate, pois, neste caso, o X-Wing será esmagado pelo TIE:

```

1 def quebraCanhao(env, intervaloCanhao, combate):
2     # processo de quebra do canhao entre intervalos definidos
3     global canhao, duracaoViagem, emCombate
4
5     while duracaoViagem > 0:                # este processo só ocorre durante a v\
6 iagem
7         yield env.timeout(intervaloCanhao) # aguarda a próxima quebra do canhao
8         if canhao:
9             canhao = False
10            print("%5.1f Pi uiui...bi\tTradução: canhão inoperante!" % (env.now))
11            if emCombate:
12                combate.interrupt()

```

Finalmente, a inicialização do modelo deve contar com uma chamada para o processo `executaCombate`:

```

1 env = simpy.Environment()
2 viagem = env.process(viagem(env, 15, 2))
3 combate = env.process(executaCombate(env, 20, 0.5))
4 env.process(paradaTecnica(env, 10, viagem))
5 env.process(quebraCanhao(env, 25, combate))
6
7 env.run(until=combate)

```

A última linha do código anterior tem algo importante: optei por executar a simulação enquanto durar o processo de combate, afinal, ele mesmo só é executado caso a X-Wing esteja ainda em viagem.

Por fim, quando executado, o modelo completo fornece como saída:

```

1 0.0 Viagem iniciada
2 10.0 Falha do R2D2      Tempo de viagem restante: 20.0 horas
3 20.0 Combate iniciado  Reze para que o canhão não quebre!
4 20.5 Fim do combate    R2D2 diz: tá tudo tranquilo, tá tudo normalizado.
5 25.0 Pi uiui...bi      Tradução: canhão inoperante!
6 25.0 R2D2 operante
7 25.0 R2, ligue o canhão!
8 27.0 Pi..bi...bi       Tradução: canhão operante!
9 27.0 Viagem iniciada
10 30.0 Falha do R2D2     Tempo de viagem restante: 17.0 horas
11 40.5 Combate iniciado  Reze para que o canhão não quebre!
12 41.0 Fim do combate    R2D2 diz: tá tudo tranquilo, tá tudo normalizado.
13 45.0 R2D2 operante
14 45.0 Viagem iniciada
15 50.0 Pi uiui...bi      Tradução: canhão inoperante!
16 50.0 Falha do R2D2     Tempo de viagem restante: 12.0 horas
17 61.0 Fim da viagem     O lado negro venceu

```

11.6 Teste seus conhecimentos

1. Não colocamos distribuições aleatórias nos processos. Acrescente distribuições nos diversos processos e verifique se o modelo precisa de alterações.
2. Acrescente a tentativa da destruição da Estrela da Morte ao final da viagem: com 50% de chances, nosso intrépido jedi consegue acertar um tiro na entrada do reator e explodir a Estrela da Morte (se ele erra, volta ao combate). Replique algumas vezes o modelo e estime a probabilidade de sucesso da operação.

12 Armazenagem e seleção de objetos específicos com Store, FilterStore e PriorityStore

O SimPy possui uma ferramenta para armazenamento de [objetos](#)¹ - como valores, recursos etc. - chamada Store; um comando de acesso a objetos específicos dentro do Store por meio de filtro, o FilterStore e um comando de acesso de objetos por ordem de prioridade, o PriorityStore. O programador experiente vai notar certa similaridade da família Store com o [dicionário](#)² do Python.

Vamos desvendar o funcionamento do Store a partir de um exemplo bem simples: simulando o processo de atendimento em uma barbearia com três barbeiros. Quando você chega a uma barbearia e tem uma ordem de preferência entre os barbeiros, isto é: barbeiro 1 vem antes do 2, que vem antes do 3, precisará selecionar o *recurso* barbeiro na ordem certa de sua preferência, mas lembre-se: cada cliente tem seu gosto e gosto não se discute, simula-se!

12.1 Construindo um conjunto de objetos com Store

Inicialmente, considere que os clientes são atribuídos ao primeiro barbeiro que encontrar-se livre, indistintamente. Se todos os barbeiros estiverem ocupados, o cliente aguarda em fila.

O comando que constrói um armazém de objetos é o `simpy.Store()`:

```
meuStore = simpy.Store(env, capacity=capacidade)
```

Para manipular o Store criado, temos três comandos à disposição:

- `meuStore.items`: adiciona objetos ao meuStore;
- `yield meuStore.get()`: retira o primeiro objeto disponível de meuStore ou, caso o meuStore esteja vazio, aguarda até que algum objeto seja colocado no Store;
- `yield meuStore.put(umObjeto)`: coloca um objeto no meuStore ou, caso o meuStore esteja cheio, aguarda até que surja um espaço vazio para colocar o objeto.



Observação: se a capacidade não for fornecida, o SimPy assumirá que a capacidade do Store é ilimitada.

Para a barbearia, criaremos um Store que armazenará o nome dos barbeiros, aqui denominados de 0, 1 e 2:

¹http://wiki.python.org.br/ProgramacaoOrientadaObjetoPython#A2._Objetos_e_Tipos_de_dados

²<https://docs.python.org/pt-br/3/tutorial/datastructures.html#dictionaries>

```

1 env = simpy.Environment()
2
3 # cria 3 barbeiros diferentes
4 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
5
6 # cria um Store para armazenar os barbeiros
7 barbeariaStore = simpy.Store(env, capacity=3)
8 barbeariaStore.items = [0, 1, 2]

```

No código anterior, criamos uma lista com três recursos que representarão os barbeiros. A seguir, criamos uma Store, chamada `barbeariaStore`, de capacidade 3 e adicionamos, na linha seguinte, uma lista com os três números que representam os próprios barbeiros.

Em resumo, nosso Store contém apenas os números 0, 1 e 2.

Considere que o intervalo entre chegadas sucessivas de clientes é exponencialmente distribuído com média de 5 minutos e que cada barbeiro leva um tempo normalmente distribuído com média 10 e desvio padrão de 2 minutos para cortar o cabelo.

Uma possível máscara para o modelo de simulação seria:

```

1 import simpy
2 import random
3
4 TEMPO_CHEGADAS = 5      # intervalo entre chegadas de clientes
5 TEMPO_CORTE = [10, 2]  # tempo médio de corte
6
7 def chegadaClientes(env, barbeariaStore):
8     # gera clientes exponencialmente distribuídos
9     # encaminha para o processo de atendimento
10
11 def atendimento(env, cliente, barbeariaStore):
12     # ocupa um barbeiro específico e realiza o corte
13
14 random.seed(100)
15 env = simpy.Environment()
16
17 # cria 3 barbeiros diferentes
18 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
19
20 # cria um Store para armazenar os barbeiros
21 barbeariaStore = simpy.Store(env, capacity=3)
22 barbeariaStore.items = [0, 1, 2]
23
24 # inicia processo de chegadas de clientes
25 env.process(chegadaClientes(env, barbeariaStore))
26 env.run(until = 20)

```

A função para gerar clientes é semelhante a tantas outras que já fizemos neste livro:


```

1 def chegadaClientes(env, barbeariaStore):
2     # gera clientes exponencialmente distribuídos
3     # encaminha para o processo de atendimento
4     i = 0
5     while True:
6         yield env.timeout(random.expovariate(1/TEMPO_CHEGADAS))
7         i += 1
8         print("%5.1f Cliente %i chega." %(env.now, i))
9         env.process(atendimento(env, i, barbeariaStore))

```

A função de atendimento, traz a novidade de que primeiro devemos *retirar* um objeto/barbeiro do Store e, ao final do atendimento, devolvê-lo ao Store:

```

1 def atendimento(env, cliente, barbeariaStore):
2     # ocupa um barbeiro específico e realiza o corte
3     chegada = env.now
4     # retira o barbeiro do Store
5     barbeiroNum = yield barbeariaStore.get()
6     espera = env.now - chegada
7     print("%5.1f Cliente %i inicia.\t\tBarbeiro %i ocupado.\tTempo de fila: %2.1f\t"
8         "
9         %(env.now, cliente, barbeiroNum, espera))
10    with barbeirosList[barbeiroNum].request() as req:
11        yield req
12        yield env.timeout(random.normalvariate(*TEMPO_CORTE))
13        print("%5.1f Cliente %i termina.\tBarbeiro %i liberado."
14            %(env.now, cliente, barbeiroNum))
15    # devolve o barbeiro ao Store
16    barbeariaStore.put(barbeiroNum)

```

Quando estamos retirando um objeto do barbeariaStore, estamos apenas retirando o nome (ou identificador) do barbeiro disponível. Algo como retirar um cartão com o nome do barbeiro de uma pilha de barbeiros disponíveis. Se o cartão é retirado, significa que, enquanto ele não for devolvido à pilha, nenhum cliente poderá ocupá-lo, pois ele não se encontra na pilha de barbeiros disponíveis.

Para ocuparmos o recurso (ou barbeiro) selecionado corretamente, utilizamos o identificador como índice da lista barbeiroList. Assim, temporariamente, o barbeiro retirado do Store fica indisponível para outros clientes, pois a linha:

```

1 barbeiroNum = yield barbeariaStore.get()

```

Só retornará um barbeiro dentre aqueles que ainda estão à disposição. Ao ser executado por apenas 20 minutos, o modelo de simulação completo da barbearia fornece como saída:

```

1  11.8 Cliente 1 chega.
2  11.8 Cliente 1 inicia.      Barbeiro 0 ocupado.      Tempo de fila: 0.0
3  17.6 Cliente 2 chega.
4  17.6 Cliente 2 inicia.      Barbeiro 1 ocupado.      Tempo de fila: 0.0
5  19.5 Cliente 1 termina.     Barbeiro 0 liberado.

```

No caso do exemplo, o Store armazenou basicamente uma lista de números [0,1,2], que representam os nomes dos barbeiros. Poderíamos sofisticar um pouco mais o exemplo e criar um dicionário (em Python) para manipular os nomes reais dos barbeiros. Por exemplo, se os nomes dos barbeiros são: João, José e Mário, poderíamos montar o `barbeirosStore` com seus respectivos nomes e evitar o uso de números:

```

1  random.seed(100)
2  env = simpy.Environment()
3
4  # cria 3 barbeiros diferentes e armazena em um dicionário
5  barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
6  barbeirosNomes = ['João', 'José', 'Mário']
7  barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
8
9  # cria um Store para armazenar os barbeiros
10 barbeariaStore = simpy.Store(env, capacity=3)
11 barbeariaStore.items = barbeirosNomes
12
13 # inicia processo de chegadas de clientes
14 env.process(chegadaClientes(env, barbeariaStore))
15 env.run(until = 20)

```

O exemplo anterior apenas reforça que Store é um local para se armazenar objetos de qualquer tipo (semelhante ao dict do Python).



Store opera segundo uma regra FIFO (*Firt-in-First-out*), ou seja: o primeiro objeto a entrar no Store por meio de um `.put()` será o primeiro objeto a sair dele com um `.get()`.

12.2 Selecionando um objeto específico com FilterStore()

Considere agora o caso bastante comum em que precisamos selecionar um recurso específico (segundo alguma regra) dentro de um conjunto de recursos disponíveis. Na barbearia, por exemplo, cada cliente agora tem um barbeiro preferido e, se ele não está disponível, o cliente prefere aguardar sua liberação.

Neste caso, vamos assumir que a preferência de um cliente é uniformemente distribuída entre os três barbeiros.

Precisamos, portanto, de um modo de selecionar um objeto específico dentro do Store. O SimPy tem um comando para construir um conjunto de objetos filtrável, o `FilterStore`:

```
1 meuFilterStore = simpy.FilterStore(env, capacity=capacidade)
```

A grande diferença para o Store é que podemos utilizar uma [função anônima do Python](#)³ dentro do comando `.get()` e assim puxar um objeto de dentro do FilterStore segundo alguma regra codificada por nós.

Inicialmente, criaremos um FilterStore de barbeiros, que armazenará apenas os números 1, 2 e 3:

```
1 random.seed(150)
2 env = simpy.Environment()
3
4 # cria 3 barbeiros diferentes
5 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
6
7 # cria um Store para armazenar os barbeiros
8 barbeariaStore = simpy.FilterStore(env, capacity=3)
9 barbeariaStore.items = [0, 1, 2]
10
11 # inicia processo de chegadas de clientes
12 env.process(chegadaClientes(env, barbeariaStore))
13 env.run(until = 20)
```

A função geradora de clientes terá uma ligeira modificação, pois temos de atribuir a cada cliente um barbeiro específico, respeitando o sorteio de uma distribuição uniforme:

```
1 import simpy
2 import random
3
4 TEMPO_CHEGADAS = 5          # intervalo entre chegadas de clientes
5 TEMPO_CORTE = [10, 5]      # tempo médio de corte
6 PREF_BARBEIRO = [0, 2]     # preferência de barbeiros
7
8 def chegadaClientes(env, barbeariaStore):
9     # gera clientes exponencialmente distribuídos
10    # sorteia o barbeiro
11    # inicia processo de atendimento
12    i = 0
13    while True:
14        yield env.timeout(random.expovariate(1/TEMPO_CHEGADAS))
15        i += 1
16        barbeiroEscolhido = random.randint(*PREF_BARBEIRO)
17        print("%5.1f Cliente %i chega.\t\tBarbeiro %i escolhido."
18              %(env.now, i, barbeiroEscolhido))
19        env.process(atendimento(env, i, barbeiroEscolhido, barbeariaStore))
```

³<http://pt.stackoverflow.com/questions/50422/como-declarar-uma-função-anônima-no-python>

Na função anterior, o *atributo* `barbeiroEscolhido` armazena o número do barbeiro sorteado e envia a informação para a função que representa o processo de atendimento.

A função `atendimento` utilizará uma função anônima para buscar o barbeiro certo dentro do `FilterStore` criado:

```

1 def atendimento(env, cliente, barbeiroEscolhido, barbeariaStore):
2     # ocupa um barbeiro específico e realiza o corte
3     chegada = env.now
4     barbeiroNum = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEsc\
5 olhido)
6     espera = env.now - chegada
7     print("%5.1f Cliente %i inicia.\t\tBarbeiro %i ocupado.\tTempo de fila: %2.1f"
8           %(env.now, cliente, barbeiroEscolhido, espera))
9     with barbeirosList[barbeiroNum].request() as req:
10         yield req
11         yield env.timeout(random.normalvariate(*TEMPO_CORTE))
12         print("%5.1f Cliente %i termina.\tBarbeiro %i liberado."
13               %(env.now, cliente, barbeiroEscolhido))
14     barbeariaStore.put(barbeiroNum)

```

Para selecionar o número certo do barbeiro, existe uma função `lambda` inserida dentro do `.get()`:

```

1 barbeiroNum = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEscolhi\
2 do)

```

Esta função percorre os objetos dentro da `barbeariaStore` até encontrar um que tenha o número do respectivo barbeiro desejado pelo cliente. Note que também poderíamos ter optado por uma construção alternativa utilizando o nome dos barbeiros e não os números—neste caso, uma alternativa seria seguir o exemplo da seção anterior e utilizar um dicionário para associar o nome dos barbeiros aos respectivos recursos.

Quando executado, o modelo anterior fornece:

```

1   8.7 Cliente 1 chega.           Barbeiro 1 escolhido.
2   8.7 Cliente 1 inicia.         Barbeiro 1 ocupado.      Tempo de fila: 0.0
3   9.7 Cliente 2 chega.           Barbeiro 0 escolhido.
4   9.7 Cliente 2 inicia.         Barbeiro 0 ocupado.      Tempo de fila: 0.0
5  12.4 Cliente 3 chega.           Barbeiro 1 escolhido.
6  15.5 Cliente 1 termina.         Barbeiro 1 liberado.
7  15.5 Cliente 3 inicia.         Barbeiro 1 ocupado.      Tempo de fila: 3.0

```

Repare que o cliente 3 chegou num instante em que o barbeiro 1 estava ocupado atendendo o cliente 1, assim ele foi obrigado a esperar em fila por 3 minutos, até que o cliente 1 liberasse o Barbeiro 1.

12.3 Criando um Store com prioridade: PriorityStore

Como sabemos, um Store segue a regra FIFO, de modo que o primeiro objeto a entrar no Store será o primeiro a sair do Store. É possível quebrar essa regra por meio do PriorityStore:

```
1 meuPriorityStore = simpy.PriorityStore(env, capacity=inf)
```

Para acrescentar um objeto qualquer ao meuPriorityStore já criado, a sequência de passos é primeiro criar um objeto PriorityItem—que representará o objeto a ser armazenado—para depois inseri-lo com um comando put, como representado no exemplo a seguir:

```
1 # criar o PriorityItem
2 meuObjetoPriority = simpy.PriorityItem(priority=priority, item=meuObjeto)
3 # adicionar o PriorityItem ao PriorityStore
4 meuPriorityStore.put(meuObjetoPriority)
```



Observação: como no caso do PriorityResource, quanto menor o valor de priority, maior a preferência pelo objeto.

No caso dos barbeiros: “João, José e Mário”, considere que a ordem de prioridades é a própria ordem alfabética dos nomes. Assim, inicialmente, construiremos dois dicionários para armazenar essas informações sobre os barbeiros:

```
1 random.seed(100)
2 env = simpy.Environment()
3
4 # cria 3 barbeiros diferentes e armazena em um dicionário
5 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
6 barbeirosNomes = ['João', 'José', 'Mário']
7 # lista com a ordem de prioridades dos barbeiros
8 barbeirosPrioList = ['0', '1', '2']
9
10 # dicionário de recursos e prioridades
11 barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
12 barbeirosPrioDict = dict(zip(barbeirosNomes, barbeirosPrioList))
```

A partir dos dicionários anteriores, podemos construir um PriorityStore que armazena os nomes dos barbeiros e suas prioridades:

```

1  # cria um Store para armazenar os barbeiros
2  barbeariaStore = simpy.PriorityStore(env, capacity=3)
3  for nome in barbeirosNomes:
4      barbeiro = simpy.PriorityItem(priority=barbeirosPrioDict[nome], item=nome)
5      barbeariaStore.put(barbeiro)
6
7  # inicia processo de chegadas de clientes
8  env.process(chegadaClientes(env, barbeariaStore))
9  env.run(until = 20)

```

A função atendimento é muito semelhante às anteriores, basta notar que o comando get vai buscar não o nome, mas um *objeto* PriorityItem dentro do PriorityStore. Este objeto possui dois atributos: item—no nosso caso, o nome do barbeiro - e priority—a prioridade do objeto.

A seguir, apresenta-se uma possível implementação da função atendimento. Note que o objeto barbeiro é um PriorityItem e que, para sabermos o seu nome, precisamos do comando barbeiro.item:

```

1  def atendimento(env, cliente, barbeariaStore):
2      # ocupa um barbeiro específico e realiza o corte
3      chegada = env.now
4      barbeiro = yield barbeariaStore.get()
5      # extrai o nome do barbeiro
6      barbeiroNome = barbeiro.item
7      espera = env.now - chegada
8      print("%5.1f Cliente %i inicia.\t\tBarbeiro %s ocupado.\tTempo de fila: %2.1f"
9            %(env.now, cliente, barbeiroNome, espera))
10     with barbeirosDict[barbeiroNome].request() as req:
11         yield req
12         yield env.timeout(random.normalvariate(*TEMPO_CORTE))
13         print("%5.1f Cliente %i termina.\t\tBarbeiro %s liberado."
14               %(env.now, cliente, barbeiroNome))
15     barbeariaStore.put(barbeiro)

```

O modelo de simulação completo, quando simulado por apenas 20 minutos, fornece como saída:

```

1  0.8 Cliente 1 chega.
2  0.8 Cliente 1 inicia.           Barbeiro João ocupado.  Tempo de fila: 0.0
3  3.8 Cliente 2 chega.
4  3.8 Cliente 2 inicia.           Barbeiro José ocupado.  Tempo de fila: 0.0
5  10.4 Cliente 3 chega.
6  10.4 Cliente 3 inicia.          Barbeiro Mario ocupado. Tempo de fila: 0.0
7  12.7 Cliente 2 termina.         Barbeiro José liberado.
8  13.9 Cliente 1 termina.         Barbeiro João liberado.
9  14.2 Cliente 4 chega.
10 14.2 Cliente 4 inicia.          Barbeiro João ocupado.  Tempo de fila: 0.0

```

```

11 14.5 Cliente 5 chega.
12 14.5 Cliente 5 inicia.      Barbeiro José ocupado.  Tempo de fila: 0.0
13 17.8 Cliente 3 termina.     Barbeiro Mario liberado.

```



Observação 1: internamente, o SimPy trata a “família” Store como recursos com capacidade ilimitada de armazenamento de objetos.

Observação 2: uma implementação alternativa para o problema anterior, seria armazenar no PriorityStore não o nome do barbeiro, mas o próprio recurso criado. (Veja o tópico: “Teste seus conhecimentos”, na próxima seção.

12.4 Conceitos desta seção

Conteúdo	Descrição
<code>meuStore = simpy.Store(env, capacity=capacity)</code>	cria um <i>Store</i> meuStore: um armazém de objetos com capacidade <code>capacity</code> . Caso o parâmetro <code>capacity</code> não seja fornecido, o SimPy considera <code>capacity=inf</code> .
<code>yield meuStore.get()</code>	retira o primeiro objeto disponível de meuStore ou, caso o meuStore esteja vazio, aguarda até que algum objeto esteja disponível.
<code>yield meuStore.put(umObjeto)</code>	coloca um objeto no meuStore ou, caso o meuStore esteja cheio, aguarda até que surja um espaço vazio para colocar o objeto.
<code>meuFilterStore = simpy.FilterStore(env, capacity=capacity)</code>	cria um <i>Store</i> meuStore: um armazém de objetos filtráveis com capacidade <code>capacity</code> . Caso o parâmetro <code>capacity</code> não seja fornecido, o SimPy considera <code>capacity=inf</code> .
<code>yield meuFilterStore.get(filter=<function <lambda>>)</code>	retira o 1º objeto do meuFilterStore que retorne True para a função anônima fornecida por filter.
<code>meuPriorityStore = simpy.PriorityStore(env, capacity=inf)</code>	cria um <i>PriorityStore</i> meuPriorityStore—um armazém de objetos com ordem de prioridade e capacidade <code>capacity</code> . Caso o parâmetro <code>capacity</code> não seja fornecido, o SimPy considera <code>capacity=inf</code> .

Conteúdo	Descrição
<code>meuObjetoPriority = simpy.PriorityItem(priority=priority, item=meuObjeto)</code>	cria um objeto <code>meuObjetoPriority</code> a partir de um objeto <code>meuObjeto</code> existente, com prioridade para ser armazenado em um <code>PriorityStore</code> . A <code>priority</code> deve ser um objeto ordenável.
<code>meuObjetoPriority.item</code>	retorna o atributo <code>item</code> do objeto <code>meuObjetoPriority</code> , definido no momento de criação do <code>PriorityItem</code> .
<code>meuObjetoPriority.priority</code>	retorna o atributo <code>priority</code> do objeto <code>meuObjetoPriority</code> , definido no momento de criação do <code>PriorityItem</code> .
<code>meuPriorityStore.put(meuObjetoPriority)</code>	coloca o objeto <code>meuObjetoPriority</code> no <code>PriorityStore</code> <code>meuPriorityStore</code> ou, caso o <code>meuPriorityStore</code> esteja cheio, aguarda até que surja um espaço vazio para colocar o objeto.
<code>yield meuPriorityStore.get()</code>	retorna o primeiro objeto disponível em <code>meuPriorityStore</code> respeitando a ordem de prioridade atribuída ao <code>PriorityItem</code> (objetos com valor menor de prioridade são escolhidos primeiro). Caso o <code>meuPriorityStore</code> esteja vazio, aguarda até que surja um espaço vazio para colocar o objeto.

12.5 Desafios



Desafio 15: considere que na barbearia, 40% dos clientes escolhem seu barbeiro favorito, sendo que, 30% preferem o barbeiro A, 10% preferem o barbeiro B e nenhum prefere o barbeiro C (o proprietário do salão). Construa um modelo de simulação representativo deste sistema.



Desafio 16: acrescente ao modelo da barbearia, a possibilidade de desistência e falta do barbeiro. Neste caso, existe 5% de chance de um barbeiro faltar em determinado dia. Neste caso, considere 3 novas situações:

- Se o barbeiro favorito faltar, o respectivo cliente vai embora;
- O cliente que não possui um barbeiro favorito olha a fila de clientes: se houver mais de 6 clientes em fila, ele desiste e vai embora;
- O cliente que possui um barbeiro favorito, não esperará se houver mais de 3 clientes esperando seu barbeiro favorito.

12.6 Solução dos desafios 15 e 16

Desafio 15: considere que na barbearia, 40% dos clientes escolhem seu barbeiro favorito, sendo que, 30% preferem o barbeiro A, 10% preferem o barbeiro B e nenhum prefere o barbeiro C (o proprietário do salão). Construa um modelo de simulação representativo deste sistema.

Como existe preferência pelo barbeiro, naturalmente a escolha mais simples é trabalharmos com o `FilterStore`. O código a seguir, cria uma lista de barbeiros com os nomes, outra com os respectivos `Resources`, um dicionário para localizarmos o barbeiro por seu nome e, por fim, um `FilterStore` com os nomes dos barbeiros:

```
1 random.seed(50)
2 env = simpy.Environment()
3
4 # cria 3 barbeiros diferentes e armazena em um dicionário
5 barbeirosNomes = ['Barbeiro A', 'Barbeiro B', 'Barbeiro C']
6 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
7 barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
8
9 # cria um FilterStore para armazenar os barbeiros
10 barbeariaStore = simpy.FilterStore(env, capacity=3)
11 barbeariaStore.items = barbeirosNomes
12
13 # inicia processo de chegadas de clientes
14 env.process(chegadaClientes(env, barbeariaStore))
15 env.run(until = 20)
```

Quando um cliente chega, existe 40% de chance dele preferir o barbeiro A e 10% de preferir o barbeiro B. O código a seguir atribui o barbeiro utilizando-se da função `random`:

```
1 def chegadaClientes(env, barbeariaStore):
2     # gera clientes exponencialmente distribuídos
3     i = 0
4     while True:
5         yield env.timeout(random.expovariate(1/TEMPO_CHEGADAS))
6         i += 1
7         # tem preferência por barbeiro?
8         r = random.random()
9         if r <= 0.30:
10             barbeiroEscolhido = 'Barbeiro A'
11         elif r <= 0.40:
12             barbeiroEscolhido = 'Barbeiro B'
13         else:
14             barbeiroEscolhido = 'Sem preferência'
```

```

15     print("%5.1f Cliente %i chega.\t\t%s." %(env.now, i, barbeiroEscolhido))
16     # inicia processo de atendimento
17     env.process(atendimento(env, i, barbeiroEscolhido, barbeariaStore))

```

Por fim, o processo de atendimento deve diferenciar os clientes que possuem um barbeiro favorito, pois neste caso temos que criar uma função anônima lambda para resgatar o barbeiro correto do FilterStore:

```

1  def atendimento(env, cliente, barbeiroEscolhido, barbeariaStore):
2      # ocupa um barbeiro específico e realiza o corte
3      chegada = env.now
4      if barbeiroEscolhido != 'Sem preferência':
5          # retira do FilterStore o barbeiro escolhido no processo anterior
6          barbeiro = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEs\
7 colhido)
8      else:
9          # cliente sem preferência, retira o primeiro barbeiro do FilterStore
10         barbeiro = yield barbeariaStore.get()
11     espera = env.now - chegada
12     print("%5.1f Cliente %i inicia.\t\t%s ocupado.\tTempo de fila: %2.1f"
13           %(env.now, cliente, barbeiro, espera))
14     # ocupa o recurso barbeiro
15     with barbeirosDict[barbeiro].request() as req:
16         yield req
17         tempoCorte = random.normalvariate(*TEMPO_CORTE)
18         yield env.timeout(tempoCorte)
19         print("%5.1f Cliente %i termina.\t%s liberado." %(env.now, cliente, barbe\
20 iro))
21     # devolve o barbeiro para o FilterStore
22     barbeariaStore.put(barbeiro)

```

Note, no código anterior, que, caso o cliente não tenha barbeiro preferido, o get do FilterStore é utilizado sem nenhuma função lambda dentro do parêntesis. Por fim, o código completo:

```

1  import simpy
2  import random
3
4  TEMPO_CHEGADAS = 5          # intervalo entre chegadas de clientes
5  TEMPO_CORTE = [10, 2]      # tempo médio de corte
6
7  def chegadaClientes(env, barbeariaStore):
8      # gera clientes exponencialmente distribuídos
9      i = 0
10     while True:
11         yield env.timeout(random.expovariate(1/TEMPO_CHEGADAS))
12         i += 1

```

```

13     # tem preferência por barbeiro?
14     r = random.random()
15     if r <= 0.30:
16         barbeiroEscolhido = 'Barbeiro A'
17     elif r <= 0.40:
18         barbeiroEscolhido = 'Barbeiro B'
19     else:
20         barbeiroEscolhido = 'Sem preferência'
21     print("%5.1f Cliente %i chega.\t\t%s." %(env.now, i, barbeiroEscolhido))
22     # inicia processo de atendimento
23     env.process(atendimento(env, i, barbeiroEscolhido, barbeariaStore))
24
25 def atendimento(env, cliente, barbeiroEscolhido, barbeariaStore):
26     # ocupa um barbeiro específico e realiza o corte
27     chegada = env.now
28     if barbeiroEscolhido != 'Sem preferência':
29         # retira do FilterStore o barbeiro escolhido no processo anterior
30         barbeiro = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEs\
31 colhido)
32     else:
33         # cliente sem preferência, retira o primeiro barbeiro do FilterStore
34         barbeiro = yield barbeariaStore.get()
35     espera = env.now - chegada
36     print("%5.1f Cliente %i inicia.\t\t%s ocupado.\tTempo de fila: %2.1f"
37           %(env.now, cliente, barbeiro, espera))
38     # ocupa o recurso barbeiro
39     with barbeirosDict[barbeiro].request() as req:
40         yield req
41         tempoCorte = random.normalvariate(*TEMPO_CORTE)
42         yield env.timeout(tempoCorte)
43         print("%5.1f Cliente %i termina.\t\t%s liberado." %(env.now, cliente, barbe\
44 iro))
45     # devolve o barbeiro para o FilterStore
46     barbeariaStore.put(barbeiro)
47
48
49 random.seed(50)
50 env = simpy.Environment()
51
52 # cria 3 barbeiros diferentes e armazena em um dicionário
53 barbeirosNomes = ['Barbeiro A', 'Barbeiro B', 'Barbeiro C']
54 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(3)]
55 barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
56
57 # cria um FilterStore para armazenar os barbeiros
58 barbeariaStore = simpy.FilterStore(env, capacity=3)

```

```

59 barbeariaStore.items = barbeirosNomes
60
61 # inicia processo de chegadas de clientes
62 env.process(chegadaClientes(env, barbeariaStore))
63 env.run(until = 20)

```

Quando executado, o código anterior fornece:

```

1   3.4 Cliente 1 chega.      Barbeiro A.
2   3.4 Cliente 1 inicia.    Barbeiro A ocupado.      Tempo de fila: 0.0
3   8.5 Cliente 2 chega.      Sem preferência.
4   8.5 Cliente 2 inicia.    Barbeiro B ocupado.      Tempo de fila: 0.0
5   9.0 Cliente 3 chega.      Barbeiro A.
6   9.8 Cliente 4 chega.      Sem preferência.
7   9.8 Cliente 4 inicia.    Barbeiro C ocupado.      Tempo de fila: 0.0
8  11.8 Cliente 1 termina.    Barbeiro A liberado.
9  11.8 Cliente 3 inicia.    Barbeiro A ocupado.      Tempo de fila: 2.8
10 16.6 Cliente 2 termina.    Barbeiro B liberado.
11 19.0 Cliente 4 termina.    Barbeiro C liberado.

```

Desafio 16: acrescente ao modelo da barbearia, a possibilidade de desistência e falta do barbeiro. Neste caso, existe 5% de chance de um barbeiro faltar em determinado dia. Neste caso, considere 3 novas situações:

- Se o barbeiro favorito faltar, o respectivo cliente vai embora;
- O cliente que não possui um barbeiro favorito olha a fila de clientes: se houver mais de 6 clientes em fila, ele desiste e vai embora;
- O cliente que possui um barbeiro favorito, não esperará se houver mais de 3 clientes esperando seu barbeiro favorito.

Como, neste caso, temos que identificar quantos clientes estão aguardando o respectivo barbeiro favorito, uma saída seria utilizar um dicionário para armazenar o número de clientes em fila (outra possibilidade seria um Store específico para a fila):

```

1  random.seed(25)
2  env = simpy.Environment()
3
4  # cria 3 barbeiros diferentes e armazena em um dicionário
5  barbeirosNomes = ['Barbeiro A', 'Barbeiro B', 'Barbeiro C']
6
7  # falta de um barbeiro
8  if random.random() <= 0.05:
9      barbeirosNomes.remove(random.choice((barbeirosNomes)))
10
11 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(len(barbeirosNome\

```

```

12 s)])
13 barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
14
15 # dicionário para armazenar o número de clientes em fila de favoritos
16 filaDict = {k:0 for k in barbeirosNomes}
17
18 # cria um FilterStore para armazenar os barbeiros
19 barbeariaStore = simpy.FilterStore(env, capacity=3)
20 barbeariaStore.items = barbeirosNomes
21
22 # inicia processo de chegadas de clientes
23 env.process(chegadaClientes(env, barbeariaStore))
24 env.run(until = 30)

```

Para garantir a falta de um barbeiro em 5% das simulações, foi novamente utilizado o comando `random.random` e adicionalmente o comando `[random.choice]` (<https://docs.python.org/dev/library/random>) que seleciona uniformemente um elemento da lista `barbeirosNomes`:

```

1 if random.random() <= 0.05:
2     barbeirosNomes.remove(random.choice((barbeirosNomes)))

```

Na linha anterior, além de sortearmos um dos barbeiros, ele é removido da lista de barbeiros, o que facilita o processo de desistência do cliente. O processo de chegadas de clientes não precisa ser modificado em relação ao código anterior, contudo, o processo de atendimento precisa armazenar o número de clientes em fila por barbeiro—para isso criamos um dicionário—e o número de clientes em fila total. Assim, criamos uma variável global que armazena o número total de clientes em fila. Uma possível codificação para a função de atendimento seria:

```

1 def atendimento(env, cliente, barbeiroEscolhido, barbeariaStore):
2     # ocupa um barbeiro específico e realiza o corte
3     global filaAtual      # número de clientes em fila
4
5     chegada = env.now
6     if barbeiroEscolhido != 'Sem preferência':
7         if barbeiroEscolhido not in barbeirosDict:
8             # caso o barbeiro tenha faltado, desiste do atendimento
9             print("%5.1f Cliente %i desiste.\t%s ausente."
10                  %(env.now, cliente, barbeiroEscolhido))
11             env.exit()
12         if filaDict[barbeiroEscolhido] > 3:
13             # caso a fila seja maior do que 6, desiste do atendimento
14             print("%5.1f Cliente %i desiste.\t%s com mais de 3 clientes em fila."
15                  %(env.now, cliente, barbeiroEscolhido))
16             env.exit()
17         # cliente atual entra em fila e incrementa a fila do barbeiro favorito
18         filaAtual += 1

```

```

19     filaDict[barbeiroEscolhido] = filaDict[barbeiroEscolhido] + 1
20     barbeiro = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEs\
21 colhido)
22     filaDict[barbeiroEscolhido] = filaDict[barbeiroEscolhido] - 1
23     else:
24         # cliente sem preferência, verifica o tamanho total da fila
25         if filaAtual > 6:
26             # caso a fila seja maior do que 6, desiste do atendimento
27             print("%5.1f Cliente %i desiste.\tFila com mais de 6 clientes em fila\
28 ."
29                 %(env.now, cliente))
30             env.exit()
31         else:
32             # cliente entra em fila e pega o primeiro barbeiro livre
33             filaAtual += 1
34             barbeiro = yield barbeariaStore.get()
35
36         # cliente já tem barbeiro, então sai da fila
37         filaAtual -= 1
38
39     espera = env.now - chegada
40     print("%5.1f Cliente %i inicia.\t\t%s ocupado.\tTempo de fila: %2.1f"
41           %(env.now, cliente, barbeiro, espera))
42     # ocupa o recurso barbeiro
43     with barbeirosDict[barbeiro].request() as req:
44         yield req
45         tempoCorte = random.normalvariate(*TEMPO_CORTE)
46         yield env.timeout(tempoCorte)
47         print("%5.1f Cliente %i termina.\t%s liberado." %(env.now, cliente, barbe\
48 iro))
49         # devolve o barbeiro para o FilterStore
50         barbeariaStore.put(barbeiro)

```

```

1  import simpy
2  import random
3
4  TEMPO_CHEGADAS = 5           # intervalo entre chegadas de clientes
5  TEMPO_CORTE = [10, 2]       # tempo médio de corte
6  filaAtual = 0                # armazena o tamanho atual da fila de clientes
7
8  def chegadaClientes(env, barbeariaStore):
9      # gera clientes exponencialmente distribuídos
10     i = 0
11     while True:
12         yield env.timeout(random.expovariate(1/TEMPO_CHEGADAS))
13         i += 1

```

```

14     # tem preferência por barbeiro?
15     r = random.random()
16     if r <= 0.30:
17         barbeiroEscolhido = 'Barbeiro A'
18     elif r <= 0.40:
19         barbeiroEscolhido = 'Barbeiro B'
20     else:
21         barbeiroEscolhido = 'Sem preferência'
22     print("%5.1f Cliente %i chega.\t\t%s." %(env.now, i, barbeiroEscolhido))
23     # inicia processo de atendimento
24     env.process(atendimento(env, i, barbeiroEscolhido, barbeariaStore))
25
26 def atendimento(env, cliente, barbeiroEscolhido, barbeariaStore):
27     # ocupa um barbeiro específico e realiza o corte
28     global filaAtual
29
30     chegada = env.now
31     if barbeiroEscolhido != 'Sem preferência':
32         if barbeiroEscolhido not in barbeirosDict:
33             # caso o barbeiro tenha faltado, desiste do atendimento
34             print("%5.1f Cliente %i desiste.\t\t%s ausente."
35                 %(env.now, cliente, barbeiroEscolhido))
36             env.exit()
37         if filaDict[barbeiroEscolhido] > 3:
38             # caso a fila seja maior do que 6, desiste do atendimento
39             print("%5.1f Cliente %i desiste.\t\t%s com mais de 3 clientes em fila."
40                 %(env.now, cliente, barbeiroEscolhido))
41             env.exit()
42         # cliente atual entra em fila e incrementa a fila do barbeiro favorito
43         filaAtual += 1
44         filaDict[barbeiroEscolhido] = filaDict[barbeiroEscolhido] + 1
45         barbeiro = yield barbeariaStore.get(lambda barbeiro: barbeiro==barbeiroEs\
46 colhido)
47         filaDict[barbeiroEscolhido] = filaDict[barbeiroEscolhido] - 1
48     else:
49         # cliente sem preferência, verifica o tamanho total da fila
50         if filaAtual > 6:
51             # caso a fila seja maior do que 6, desiste do atendimento
52             print("%5.1f Cliente %i desiste.\t\tFila com mais de 6 clientes em fila\
53 ."
54                 %(env.now, cliente))
55             env.exit()
56         else:
57             # cliente entra em fila e pega o primeiro barbeiro livre
58             filaAtual += 1
59             barbeiro = yield barbeariaStore.get()

```

```

60
61     # cliente já tem barbeiro, então sai da fila
62     filaAtual -= 1
63
64     espera = env.now - chegada
65     print("%5.1f Cliente %i inicia.\t\t%s ocupado.\tTempo de fila: %2.1f"
66           %(env.now, cliente, barbeiro, espera))
67     # ocupa o recurso barbeiro
68     with barbeirosDict[barbeiro].request() as req:
69         yield req
70         tempoCorte = random.normalvariate(*TEMPO_CORTE)
71         yield env.timeout(tempoCorte)
72         print("%5.1f Cliente %i termina.\t%s liberado." %(env.now, cliente, barbe\
73 iro))
74     # devolve o barbeiro para o FilterStore
75     barbeariaStore.put(barbeiro)
76
77 random.seed(25)
78 env = simpy.Environment()
79
80 # cria 3 barbeiros diferentes e armazena em um dicionário
81 barbeirosNomes = ['Barbeiro A', 'Barbeiro B', 'Barbeiro C']
82
83 # falta de um barbeiro
84 if random.random() <= 0.05:
85     barbeirosNomes.remove(random.choice((barbeirosNomes)))
86
87 barbeirosList = [simpy.Resource(env, capacity=1) for i in range(len(barbeirosNome\
88 s))]
89 barbeirosDict = dict(zip(barbeirosNomes, barbeirosList))
90
91 # dicionário para armazenar o número de clientes em fila de favoritos
92 filaDict = {k:0 for k in barbeirosNomes}
93
94 # cria um FilterStore para armazenar os barbeiros
95 barbeariaStore = simpy.FilterStore(env, capacity=3)
96 barbeariaStore.items = barbeirosNomes
97
98 # inicia processo de chegadas de clientes
99 env.process(chegadaClientes(env, barbeariaStore))
100 env.run(until = 30)

```

Quando executado, o modelo anterior fornece:

1	13.1 Cliente 1 chega.	Sem preferência.	
2	13.1 Cliente 1 inicia.	Barbeiro A ocupado.	Tempo de fila: 0.0
3	14.3 Cliente 2 chega.	Barbeiro A.	
4	26.6 Cliente 1 termina.	Barbeiro A liberado.	
5	26.6 Cliente 2 inicia.	Barbeiro A ocupado.	Tempo de fila: 12.3
6	29.6 Cliente 3 chega.	Sem preferência.	
7	29.6 Cliente 3 inicia.	Barbeiro B ocupado.	Tempo de fila: 0.0

12.7 Teste seus conhecimentos

1. Considere que a barbearia opera 6 horas por dia. Acrescente ao seu modelo às estatísticas de clientes atendidos, clientes que desistiram (e por qual razão), ocupação dos barbeiros e tempo médio de espera em fila por barbeiro.
2. Dada a presente demanda da barbearia, quantos barbeiros devem estar trabalhando, caso o proprietário pretenda que o tempo médio de espera em fila seja inferior a 15 minutos?

13 Enchendo ou esvaziando caixas, tanques, estoques ou objetos com Container()

Um tipo especial de recurso no SimPy é o container. Intuitivamente, um container seria um tanque ou caixa em que se armazenam coisas. Você pode encher ou esvaziar em quantidade, como se fosse um tanque de água ou uma caixa de laranjas.

A sua utilização é bastante simples, por exemplo, podemos modelar um tanque de 100 unidades de capacidade (m^3 , por exemplo), com um estoque inicial de 50 unidades, por meio do seguinte código:

```
1 import simpy
2
3 env = simpy.Environment()
4 # cria um tanque de 100 m3 de capacidade, com 50 m3 no início da simulação
5 tanque = simpy.Container(env, capacity=100, init=50)
```

O container possui três comandos importantes:

- Para encher: `tanque.put(quantidade)`
- Para esvaziar: `tanque.get(quantidade)`
- Para obter o nível atual: `tanque.level`

13.1 Enchendo o meu container yield meuContainer.put(quantidade)

Considere que um posto de gasolina possui um tanque com capacidade de 100 m^3 (ou 100.000 litros) de combustível e que o tanque já contém 50 m^3 armazenado.

Criaremos uma função, `enchimentoTanque`, que enche o tanque com 50 m^3 sempre que um novo caminhão de reabastecimento de combustível chega ao posto:

```

1  import simpy
2  import random
3
4  TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5
6  def enchimentoTanque(env, qtd, tanque):
7      # enche o tanque
8      print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
9            % (env.now, qtd, tanque.level))
10     yield tanque.put(qtd)
11     print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
12           % (env.now, qtd, tanque.level))
13
14 random.seed(150)
15 env = simpy.Environment()
16
17 # cria um tanque de 100 m3, com 50 m3 no início da simulação
18 tanque = simpy.Container(env, capacity=100, init=50)
19 env.process(enchimentoTanque(env, TANQUE_CAMINHAO, tanque))
20
21 env.run(until = 500)

```

A saída do programa é bastante simples, afinal o processo de enchimento do tanque é executado apenas uma vez:

```

1  0 Novo caminhão de combustível com 50.0 m3. Nível atual:  50.0 m3
2  0 Tanque enchido com 50.0 m3. Nível atual: 100.0 m3

```

Se você iniciar o tanque do posto a sua plena capacidade (100 m^3), o caminhão tentará abastecer, mas não conseguirá por falta de espaço, virtualmente aguardando espaço no tanque na linha de código:

```

1  yield tanque.put(qtd)

```

13.2 Esvaziando o meu container: yield meuContainer.get(quantidade)

Considere que o posto atende automóveis que chegam em intervalos constantes de 5 minutos entre si e que cada veículo abastece 100 litros ou $0,10\text{ m}^3$.

Partindo do modelo anterior, vamos criar duas funções: uma para gerar os veículos e outra para transferir o combustível do tanque para o veículo.

Uma possível máscara para o modelo seria:

```

1  import simpy
2  import random
3
4  TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5  TANQUE_VEICULO = 0.10        # capacidade do veículo
6  TEMPO_CHEGADAS = 5          # tempo entre chegadas sucessivas de veículos
7
8  def chegadasVeiculos(env, tanque):
9      # gera chegadas de veículos por produto
10
11  def esvaziamentoTanque(env, qtd, tanque):
12      # esvazia o tanque
13
14  def enchimentoTanque(env, qtd, tanque):
15      # enche o tanque
16      print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
17            % (env.now, qtd, tanque.level))
18      yield tanque.put(qtd)
19      print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
20            % (env.now, qtd, tanque.level))
21
22  random.seed(150)
23  env = simpy.Environment()
24  # cria um tanque de 100 m3, com 50 m3 no início da simulação
25  tanque = simpy.Container(env, capacity=100, init=50)
26  env.process(chegadasVeiculos(env, tanque))
27
28  env.run(until = 20)

```

A função `chegadasVeiculos` gera os veículos que buscam abastecimento no posto e que, a seguir, chamam a função `esvaziamentoTanque` responsável por provocar o esvaziamento do tanque do posto na quantidade desejada pelo veículo:

```

1  def chegadasVeiculos(env, tanque):
2      # gera chegadas de veículos por produto
3      while True:
4          yield env.timeout(TEMPO_CHEGADAS)
5          # carrega veículo
6          env.process(esvaziamentoTanque(env, TANQUE_VEICULO, tanque))

```

A função que representa o processo de esvaziamento do tanque é semelhante a de enchimento da seção anterior, a menos da opção `get(qtd)`, que retira a quantidade `qtd` do container `tanque`:

```

1 def esvaziamentoTanque(env, qtd, tanque):
2     # esvazia o tanque
3     print("%d Novo veículo de %3.2f m3.\t Nível atual: %5.1f m3"
4           % (env.now, qtd, tanque.level))
5     yield tanque.get(qtd)
6     print("%d Veículo atendido de %3.2f.\t Nível atual: %5.1f m3"
7           % (env.now, qtd, tanque.level))

```

O modelo de simulação completo do posto de gasolina fica:

```

1 import simpy
2 import random
3
4 TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5 TANQUE_VEICULO = 0.10        # capacidade do veículo
6 TEMPO_CHEGADAS = 5          # tempo entre chegadas sucessivas de veículos
7
8 def chegadasVeiculos(env, tanque):
9     # gera chegadas de veículos por produto
10    while True:
11        yield env.timeout(TEMPO_CHEGADAS)
12        # carrega veículo
13        env.process(esvaziamentoTanque(env, TANQUE_VEICULO, tanque))
14
15 def esvaziamentoTanque(env, qtd, tanque):
16     # esvazia o tanque
17     print("%d Novo veículo de %3.2f m3.\t Nível atual: %5.1f m3"
18           % (env.now, qtd, tanque.level))
19     yield tanque.get(qtd)
20     print("%d Veículo atendido de %3.2f m3.\t Nível atual: %5.1f m3"
21           % (env.now, qtd, tanque.level))
22
23 def enchimentoTanque(env, qtd, tanque):
24     # enche o tanque
25     print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
26           % (env.now, qtd, tanque.level))
27     yield tanque.put(qtd)
28     print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
29           % (env.now, qtd, tanque.level))
30
31 random.seed(150)
32 env = simpy.Environment()
33 # cria um tanque de 100 m3, com 50 m3 no início da simulação
34 tanque = simpy.Container(env, capacity=100, init=50)
35 env.process(chegadasVeiculos(env, tanque))
36
37 env.run(until = 200)

```

Quando por 200 minutos, o modelo anterior fornece como saída:

```

1 5 Novo veículo de 0.10 m3.      Nível atual: 50.0 m3
2 5 Veículo atendido de 0.10 m3.  Nível atual: 49.9 m3
3 10 Novo veículo de 0.10 m3.     Nível atual: 49.9 m3
4 10 Veículo atendido de 0.10.    Nível atual: 49.8 m3
5 15 Novo veículo de 0.10 m3.     Nível atual: 49.8 m3
6 15 Veículo atendido de 0.10 m3. Nível atual: 49.7 m3

```

13.3 Criando um sensor para o nível atual do container

Ainda no exemplo do posto, vamos chamar um caminhão de reabastecimento sempre que o tanque atingir o nível de 50 m^3 . Para isso, criaremos uma função `sensorTanque` capaz de reconhecer o instante exato em que o nível do tanque abaixou do valor desejado e, portanto, deve ser enviado um caminhão de reabastecimento.

Inicialmente, para identificar se o nível do tanque abaixou além no nível mínimo, precisamos verificar qual o nível atual. Contudo, esse processo de verificação não é contínuo no tempo e deve ter o seu intervalo entre verificações pré-definido no modelo.

Assim, são necessários dois parâmetros: um para o nível mínimo e outro para o intervalo entre verificações do nível do tanque. Uma possível codificação para a função `sensorTanque` seria:

```

1 NIVEL_MINIMO = 50          # nível mínimo de reabastecimento do tanque
2 TEMPO_CONTROLE = 1        # tempo entre verificações do nível do tanque
3
4 def sensorTanque(env, tanque):
5     # quando o tanque baixar se certo nível, dispara o enchimento
6     while True:
7         if tanque.level <= NIVEL_MINIMO:
8             # dispara pedido de enchimento
9             yield env.process(enchimentoTanque(env, TANQUE_CAMINHAO, tanque))
10        # aguarda um tempo para fazer a nova chegada do nível do tanque
11        yield env.timeout(TEMPO_CONTROLE)

```

A função `sensorTanque` é um laço infinito (`while True`) que a cada 1 minuto (configurável na constante `TEMPO_CONTROLE`) verifica se o nível atual do tanque está abaixo ou igual ao nível mínimo (configurável na constante `NIVEL_MINIMO`).

O modelo completo com a implementação do sensor fica:

```

1  import simpy
2  import random
3
4  TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5  TANQUE_VEICULO = 0.10        # capacidade do veículo
6  TEMPO_CHEGADAS = 5          # tempo entre chegadas sucessivas de veículos
7  NIVEL_MINIMO = 50            # nível mínimo de reabastecimento do tanque
8  TEMPO_CONTROLE = 1          # tempo entre verificações do nível do tanque
9
10 def sensorTanque(env, tanque):
11     # quando o tanque baixar se certo nível, dispara o enchimento
12     while True:
13         if tanque.level <= NIVEL_MINIMO:
14             # dispara pedido de enchimento
15             yield env.process(enchimentoTanque(env, TANQUE_CAMINHAO, tanque))
16             # aguarda um tempo para fazer a nova chegada do nível do tanque
17             yield env.timeout(TEMPO_CONTROLE)
18
19 def chegadasVeiculos(env, tanque):
20     # gera chegadas de veículos por produto
21     while True:
22         yield env.timeout(TEMPO_CHEGADAS)
23         # carrega veículo
24         env.process(esvaziamentoTanque(env, TANQUE_VEICULO, tanque))
25
26 def esvaziamentoTanque(env, qtd, tanque):
27     # esvazia o tanque
28     print("%d Novo veículo de %3.2f m3.\t Nível atual: %5.1f m3"
29           % (env.now, qtd, tanque.level))
30     yield tanque.get(qtd)
31     print("%d Veículo atendido de %3.2f m3.\t Nível atual: %5.1f m3"
32           % (env.now, qtd, tanque.level))
33
34 def enchimentoTanque(env, qtd, tanque):
35     # enche o tanque
36     print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
37           % (env.now, qtd, tanque.level))
38     yield tanque.put(qtd)
39     print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
40           % (env.now, qtd, tanque.level))
41
42 random.seed(150)
43 env = simpy.Environment()
44
45 # cria um tanque de 100 m3, com 50 m3 no início da simulação
46 tanque = simpy.Container(env, capacity=100, init=50)

```

```

47 env.process(chegadasVeiculos(env, tanque))
48 env.process(sensorTanque(env, tanque))
49
50 env.run(until = 20)

```

Note a criação do processo do sensorTanque na penúltima linha do programa:

```

1 env.process(sensorTanque(env, tanque))

```

Este processo garante que o sensor estará operante ao longo de toda a simulação. Quando executado, o programa anterior retorna:

```

1 0 Novo caminhão com 50.0 m3.      Nível atual: 50.0 m3
2 0 Tanque enchido com 50.0 m3.     Nível atual: 100.0 m3
3 5 Novo veículo de 0.10 m3.        Nível atual: 100.0 m3
4 5 Veículo atendido de 0.10 m3.    Nível atual: 99.9 m3
5 10 Novo veículo de 0.10 m3.       Nível atual: 99.9 m3
6 10 Veículo atendido de 0.10 m3.   Nível atual: 99.8 m3
7 15 Novo veículo de 0.10 m3.       Nível atual: 99.8 m3
8 15 Veículo atendido de 0.10 m3.   Nível atual: 99.7 m3

```



Observação 1: Note que o enchimento ou esvaziamento dos tanques é instantâneo, isto é: não existe nenhuma taxa de enchimento ou esvaziamento associada aos processos. Cabe ao programador modelar situações em que a taxa de transferência é relevante (veja o Desafio 17, a seguir).



Observação 2: O tanque pode ser esvaziado ou enchido simultaneamente. Novamente cabe ao programador modelar a situação em que isto não se verifica (veja o Desafio 18, a seguir).

13.4 Conceitos desta seção

Conteúdo	Descrição
<code>meuContainer = simpy.Container(env, capacity=capacity, init=init)</code>	cria um <i>container</i> meuContainer com capacidade <code>capacity</code> e quantidade inicial de <code>init</code>

Conteúdo	Descrição
<code>yield meuContainer.put(quantidade)</code>	adiciona uma dada quantidade ao <code>meuContainer</code> , se houver espaço suficiente, caso contrário aguarda até que o espaço esteja disponível
<code>yield meuContainer.get(quantidade)</code>	retira uma dada quantidade do <code>meuContainer</code> , se houver quantidade suficiente, caso contrário aguarda até que a quantidade esteja disponível
<code>meuContainer.level</code>	retorna a quantidade disponível atualmente em <code>meuContainer</code>

13.5 Desafios



Desafio 17: considere, no exemplo do posto, que a taxa de enchimento do tanque é de 1 litro/min e a de esvaziamento é de 2 litros/min. Altere o modelo para que ele incorpore os tempos de enchimento e esvaziamento, bem como forneça o tempo que o veículo aguardou na fila por atendimento.



Desafio 18: continuando o exemplo, modifique o modelo de modo que ele represente a situação em que o tanque não pode ser enchido e esvaziado simultaneamente.

13.6 Solução dos desafios 17 e 18

Desafio 17: considere, no exemplo do posto, que a taxa de enchimento do tanque é de $5 \text{ m}^3/\text{min}$ e a de esvaziamento é de $1 \text{ m}^3/\text{min}$. Altere o modelo para que ele incorpore os tempos de enchimento e esvaziamento, bem como forneça o tempo que o veículo aguardou na fila por atendimento.

Neste caso, são criadas duas constantes:

```
1 TAXA_VEICULO = 1           # taxa de bombeamento do veículo
2 TAXA_CAMINHAO = 5         # taxa de bombeamento do caminhão
```

As funções de enchimento e esvaziamento do tanque devem ser modificadas para considerar o tempo de espera que os veículos e caminhões aguardam até que o processo de bombeamento tenha terminado, como representado nas funções a seguir:

```
1 def esvaziamentoTanque(env, qtd, tanque):
2     # esvazia o tanque
3     print("%d Novo veículo de %3.2f m3.\t Nível atual: %5.1f m3"
4           % (env.now, qtd, tanque.level))
5     yield tanque.get(qtd)
6     # aguarda o tempo de bombeamento
7     yield env.timeout(qtd/TAXA_VEICULO)
8     print("%d Veículo atendido de %3.2f m3.\t Nível atual: %5.1f m3"
9           % (env.now, qtd, tanque.level))
10
11 def enchimentoTanque(env, qtd, tanque):
12     # enche o tanque
13     print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
14           % (env.now, qtd, tanque.level))
15     yield tanque.put(qtd)
16     # aguarda o tempo de bombeamento
17     yield env.timeout(qtd/TAXA_CAMINHAO)
18     print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
19           % (env.now, qtd, tanque.level))
```

Em cada função foi acrescentada uma linha:

```
1 yield env.timeout(qtd/taxa)
```

Que representa o tempo que deve-se aguardar pelo bombeamento do produto.

O modelo completo ficaria:

```

1  import simpy
2  import random
3
4  TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5  TANQUE_VEICULO = 0.10        # capacidade do veículo
6  TEMPO_CHEGADAS = 5          # tempo entre chegadas sucessivas de veículos
7  NIVEL_MINIMO = 50            # nível mínimo de reabastecimento do tanque
8  TEMPO_CONTROLE = 1           # tempo entre verificações do nível do tanque
9  TAXA_VEICULO = 1             # taxa de bombeamento do veículo
10 TAXA_CAMINHAO = 5            # taxa de bombeamento do caminhão
11
12 def sensorTanque(env, tanque):
13     # quando o tanque baixar se certo nível, dispara o enchimento
14     while True:
15         if tanque.level <= NIVEL_MINIMO:
16             # dispara pedido de enchimento
17             yield env.process(enchimentoTanque(env, TANQUE_CAMINHAO, tanque))
18             # aguarda um tempo para fazer a nova checagem do nível do tanque
19             yield env.timeout(TEMPO_CONTROLE)
20
21 def chegadasVeiculos(env, tanque):
22     # gera chegadas de veículos por produto
23     while True:
24         yield env.timeout(TEMPO_CHEGADAS)
25         # carrega veículo
26         env.process(esvaziamentoTanque(env, TANQUE_VEICULO, tanque))
27
28 def esvaziamentoTanque(env, qtd, tanque):
29     # esvazia o tanque
30     print("%d Novo veículo de %3.2f m3.\t Nível atual: %5.1f m3"
31           % (env.now, qtd, tanque.level))
32     yield tanque.get(qtd)
33     # aguarda o tempo de bombeamento
34     yield env.timeout(qtd/TAXA_VEICULO)
35     print("%d Veículo atendido de %3.2f m3.\t Nível atual: %5.1f m3"
36           % (env.now, qtd, tanque.level))
37
38 def enchimentoTanque(env, qtd, tanque):
39     # enche o tanque
40     print("%d Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
41           % (env.now, qtd, tanque.level))
42     yield tanque.put(qtd)
43     # aguarda o tempo de bombeamento
44     yield env.timeout(qtd/TAXA_CAMINHAO)
45     print("%d Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
46           % (env.now, qtd, tanque.level))

```

```

47
48 random.seed(150)
49 env = simpy.Environment()
50 # cria um tanque de 100 m3, com 50 m3 no início da simulação
51 tanque = simpy.Container(env, capacity=100, init=50)
52 env.process(chegadasVeiculos(env, tanque))
53 env.process(sensorTanque(env, tanque))
54
55 env.run(until=20)

```

Quando executado por apenas 20 minutos, o modelo do desafio fornece como saída:

```

1 0 Novo caminhão com 50.0 m3.      Nível atual: 50.0 m3
2 5 Novo veículo de 0.10 m3.       Nível atual: 100.0 m3
3 5 Veículo atendido de 0.10 m3.   Nível atual: 99.9 m3
4 10 Tanque enchido com 50.0 m3.   Nível atual: 99.9 m3
5 10 Novo veículo de 0.10 m3.      Nível atual: 99.9 m3
6 10 Veículo atendido de 0.10 m3.  Nível atual: 99.8 m3
7 15 Novo veículo de 0.10 m3.      Nível atual: 99.8 m3
8 15 Veículo atendido de 0.10 m3.  Nível atual: 99.7 m3

```

O leitor atento deve ter notado que o caminhão de reabastecimento enche o tanque *antes* mesmo de aguardar o bombeamento, pois a saída do programa indica que um caminhão chegou no instante 0 e que no instante 5 o tanque já possui 100 m^3 à disposição:

```

1 0 Novo caminhão com 50.0 m3.      Nível atual: 50.0 m3
2 5 Novo veículo de 0.10 m3.       Nível atual: 100.0 m3

```

A situação inicialmente *estranha* ainda é reforçada pelo fim da operação de enchimento do tanque no instante 10:

```

1 10 Tanque enchido com 50.0 m3.

```

Isto significa que o produto estava disponível nos tanques *antes* mesmo de ter sido bombeado para o mesmo. Fica como desafio ao leitor *atento* encontrar uma solução para o problema (dica: que tal pensar em um tanque virtual que antecipe as operações antes delas serem executadas de fato?)

Desafio 18: continuando o exemplo, modifique o modelo de modo que ele represente a situação em que o tanque não pode ser enchido e esvaziado simultaneamente.

Neste caso, o tanque quando bombeando para um sentido (encher ou esvaziar), fica impedido de ser utilizado no outro sentido. Este tipo de situação é bem comum em operações envolvendo tanques de produtos químicos.

Uma possível solução para o problema é utilizar um Store para armazenar o Container que representa o tanque. Assim, quando um caminhão de reabastecimento chega ele *retira* do Store o tanque e, caso um veículo chegue nesse instante, não conseguirá abastecer pois não encontrará nenhum tanque no Store.

Inicialmente, vamos modificar o programa principal, criando um Store para o tanque:

```

1  random.seed(150)
2  env = simpy.Environment()
3  # cria um tanque de 100 m3, com 50 m3 no início da simulação
4  tanque = simpy.Container(env, capacity=100, init=50)
5
6  # cria um Store para armazenar o tanque
7  tanqueStore = simpy.Store(env, capacity=1)
8  tanqueStore.items = [tanque]
9
10 env.process(chegadasVeiculos(env, tanqueStore))
11 env.process(sensorTanque(env, tanque, tanqueStore))
12
13 env.run(until = 20)
```

Todas as funções agora devem ser modificadas para receber como argumento o tanqueStore criado e não mais o Container tanque. (Apenas a função sensorTanque ainda precisa do Container tanque pois ela não manipula o Store, apenas verifica o nível do tanque).

Assim, o modelo final pode ser codificado da seguinte forma:

```

1  import simpy
2  import random
3
4  TANQUE_CAMINHAO = 50          # capacidade de abastecimento do caminhão
5  TANQUE_VEICULO = 0.10        # capacidade do veículo
6  TEMPO_CHEGADAS = 5           # tempo entre chegadas sucessivas de veículos
7  NIVEL_MINIMO = 50             # nível mínimo de reabastecimento do tanque
8  TEMPO_CONTROLE = 1           # tempo entre verificações do nível do tanque
9  TAXA_VEICULO = 1              # taxa de bombeamento do veículo
10 TAXA_CAMINHAO = 5             # taxa de bombeamento do caminhão
11
12 def sensorTanque(env, tanque, tanqueStore):
13     # quando o tanque baixar se certo nível, dispara o enchimento
14     while True:
15         if tanque.level <= NIVEL_MINIMO:
16             # dispara pedido de enchimento
17             yield env.process(enchimentoTanque(env, TANQUE_CAMINHAO, tanqueStore))
18             # aguarda um tempo para fazer a nova chegada do nível do tanque
19             yield env.timeout(TEMPO_CONTROLE)
20
```

```
21 def chegadasVeiculos(env, tanqueStore):
22     # gera chegadas de veículos por produto
23     while True:
24         yield env.timeout(TEMPO_CHEGADAS)
25         # carrega veículo
26         env.process(esvaziamentoTanque(env, TANQUE_VEICULO, tanqueStore))
27
28 def esvaziamentoTanque(env, qtd, tanqueStore):
29     # esvazia o tanque
30     # seleciona o tanque para esvaziamento
31     tanque = yield tanqueStore.get()
32     print("%5.1f Novo veículo de %3.2f m3.\t\t Nível atual: %5.1f m3"
33           % (env.now, qtd, tanque.level))
34     yield tanque.get(qtd)
35     # aguarda o tempo de bombeamento
36     yield env.timeout(qtd/TAXA_VEICULO)
37     print("%5.1f Veículo atendido de %3.2f m3.\t Nível atual: %5.1f m3"
38           % (env.now, qtd, tanque.level))
39     yield tanqueStore.put(tanque)
40
41 def enchimentoTanque(env, qtd, tanqueStore):
42     # enche o tanque
43     # seleciona o tanque para enchimento
44     tanque = yield tanqueStore.get()
45     print("%5.1f Novo caminhão com %4.1f m3.\t Nível atual: %5.1f m3"
46           % (env.now, qtd, tanque.level))
47     yield tanque.put(qtd)
48     #aguarda o tempo de bombeamento
49     yield env.timeout(qtd/TAXA_CAMINHAO)
50     print("%5.1f Tanque enchido com %4.1f m3.\t Nível atual: %5.1f m3"
51           % (env.now, qtd, tanque.level))
52     yield tanqueStore.put(tanque)
53
54 random.seed(150)
55 env = simpy.Environment()
56 # cria um tanque de 100 m3, com 50 m3 no início da simulação
57 tanque = simpy.Container(env, capacity=100, init=50)
58
59 # cria um Store para armazenar o tanque
60 tanqueStore = simpy.Store(env, capacity=1)
61 tanqueStore.items = [tanque]
62
63 env.process(chegadasVeiculos(env, tanqueStore))
64 env.process(sensorTanque(env, tanque, tanqueStore))
65
66 env.run(until=20)
```

Quando executado, o modelo do desafio retorna:

1	0.0 Novo caminhão com 50.0 m3.	Nível atual: 50.0 m3
2	10.0 Tanque enchido com 50.0 m3.	Nível atual: 100.0 m3
3	10.0 Novo veículo de 0.10 m3.	Nível atual: 100.0 m3
4	10.1 Veículo atendido de 0.10 m3.	Nível atual: 99.9 m3
5	10.1 Novo veículo de 0.10 m3.	Nível atual: 99.9 m3
6	10.2 Veículo atendido de 0.10 m3.	Nível atual: 99.8 m3
7	15.0 Novo veículo de 0.10 m3.	Nível atual: 99.8 m3
8	15.1 Veículo atendido de 0.10 m3.	Nível atual: 99.7 m3

Note que agora o veículo só é atendido depois que o tanque é enchido. Outra observação interessante: o que ocorreria neste modelo caso um veículo retire o tanque do Store e o mesmo tanque não tenha combustível suficiente?

13.7 Teste seus conhecimentos

1. Modifique o problema para considerar que existam 3 bombas de combustível no posto, capazes de atender aos veículos simultaneamente a partir do mesmo tanque.
2. Construa um gráfico (utilizando a biblioteca [matplotlib](http://matplotlib.org/)¹) do nível do tanque ao longo do tempo.
3. Modifique o modelo de modo que o posto tenha agora dois tanques, um para gasolina e outro para gasolina premium. 25% dos veículos procuram a gasolina premium e 75% a regular.

¹<http://matplotlib.org/>

14 Criando lotes (ou agrupando) entidades durante a simulação

Uma situação bastante comum em modelos de simulação é o agrupamento de entidades em lotes ou o seu oposto: o desmembramento de um lote em diversas entidades separadas. É usual em softwares de simulação proprietários existir um comando (ou bloco) específico para isso. Por exemplo, o Arena possui o “Batch/Separate”, o Simul8 o “Batching” etc.

Vamos partir de um exemplo simples, em que uma célula de produção deve realizar a tarefa de montagem de um certo componente a partir do encaixe de uma peça A com duas peças B. O operador da célula leva em média 5 minutos para montar o componente, segundo uma distribuição normal com desvio padrão de 1 minuto. Os processos de chegadas dos lotes A e B são distintos entre si, com tempos entre chegadas sucessivas uniformemente distribuídos no intervalo entre 40 a 60 minutos.

14.1 Uma tática para agrupamento de lotes utilizando o Container

Uma maneira de resolver o problema é criar um Container de estoque temporário para cada peça. Assim, criamos dois estoques, respectivamente para as peças A e B, de modo que o componente só poderá iniciar sua montagem se cada estoque contiver ao menos o número de peças necessárias para sua montagem.

Começemos criando uma possível máscara para o problema:

```
1  import simpy
2  import random
3
4  TEMPO_CHEGADAS = [40, 50]      # intervalo entre chegadas de peças
5  TEMPO_MONTAGEM = [5, 1]        # tempo de montagem do componente
6  componentesProntos = 0          # variável para o total de componentes produzidos
7
8  def chegadaPecas(env, pecasContainerDict, tipo, tamLote):
9      # gera lotes de pecas em intervalos uniformemente distribuídos
10     # encaminha para o estoque
11     pass
12
13  def montagem(env, pecasContainerDict, numA, numB):
14     # montagem do componente
15     global componentesProntos
16     pass
```



```

17
18 random.seed(100)
19 env = simpy.Environment()
20
21 # cria estoques de peças
22 pecasContainerDict = {}
23 pecasContainerDict['A'] = simpy.Container(env)
24 pecasContainerDict['B'] = simpy.Container(env)
25
26 # inicia processos de chegadas de peças
27 env.process(chegadaPecas(env, pecasContainerDict, 'A', 10))
28 env.process(chegadaPecas(env, pecasContainerDict, 'B', 10))
29
30 # inicia processo de montagem
31 env.process(montagem(env, pecasContainerDict, 1, 2))
32 env.run(until=80)

```

Na máscara anterior, foram criadas duas funções: `chegaPecas`, que gera os lotes de peças A e B e armazena nos respectivos estoques e `montagem`, que retira as peças do estoque e monta o componente.

Note que criei um dicionário no Python: `pecasContainerDict`, para armazenar o `Container` de cada peça:

```

1 # cria estoques de peças
2 pecasContainerDict = {}
3 pecasContainerDict['A'] = simpy.Container(env)
4 pecasContainerDict['B'] = simpy.Container(env)

```

A função de geração de peças de fato, gera lotes e armazena dentro do `Container` o número de peças do lote:

```

1 def chegadaPecas(env, pecasContainerDict, tipo, tamLote):
2     # gera lotes de peças em intervalos uniformemente distribuídos
3     # encaminha para o estoque
4     while True:
5         pecasContainerDict[tipo].put(tamLote)
6         print("%5.1f Chegada de lote\t%s\tPeças: %i"
7               %(env.now, tipo, tamLote))
8         yield env.timeout(random.uniform(*TEMPO_CHEGADAS))

```

Note que, diferentemente das funções de geração de entidades criadas nas seções anteriores deste livro, a função `chegadaPecas` não encaminha a entidade criada para uma nova função, iniciando um novo processo (de atendimento, por exemplo). A função apenas armazena uma certa quantidade de peças, `tamLote`, dentro do respectivo `Container` na linha:

```
1 pecasContainerDict[tipo].put(tamLote)
```

O processo de montagem também recorre ao artifício de um laço infinito, pois, basicamente, representa uma operação que está sempre pronta para executar a montagem, desde que existam o número de peças mínimas à disposição nos respectivos estoques:

```
1 def montagem(env, pecasContainerDict, numA, numB):
2     # montagem do componente
3     global componentesProntos
4     while True:
5         # marca o instante em que a célula esta livre para a montagem
6         chegada = env.now
7         yield pecasContainerDict['A'].get(numA)
8         yield pecasContainerDict['B'].get(numB)
9         # armazena o tempo de espera por peças e inicia a montagem
10        espera = env.now - chegada
11        print("%5.1f Inicia montagem\tEstoque A: %i\tEstoque B: %i\tEspera: %4.1f"
12              %(env.now, pecasContainerDict['A'].level, pecasContainerDict['B'].
13                level, espera))
14        yield env.timeout(random.normalvariate(*TEMPO_MONTAGEM))
15        # acumula componente montado
16        componentesProntos += 1
17        print("%5.1f Fim da montagem\tEstoque A: %i\tEstoque B: %i\tComponentes: \
18              %i\t"
19              %(env.now, pecasContainerDict['A'].level, pecasContainerDict['B'].lev\
20                el,
21                componentesProntos))
```

A parte central da função anterior é garantir que o processo só possa se iniciar caso existam peças suficientes para o componente final. Isto é garantido pelo comando `get` aplicado a cada Container de peças necessárias:

```
1     yield pecasContainerDict['A'].get(numA)
2     yield pecasContainerDict['B'].get(numB)
```

Quando executado, o modelo completo fornece como saída:

1	0.0	Chegada de lote	tipo A: +10 peças		
2	0.0	Chegada de lote	tipo B: +10 peças		
3	0.0	Inicia montagem	Estoque A: 9	Estoque B: 8	Espera: 0.0
4	6.6	Fim da montagem	Estoque A: 9	Estoque B: 8	Componentes: 1
5	6.6	Inicia montagem	Estoque A: 8	Estoque B: 6	Espera: 0.0
6	12.3	Fim da montagem	Estoque A: 8	Estoque B: 6	Componentes: 2
7	12.3	Inicia montagem	Estoque A: 7	Estoque B: 4	Espera: 0.0
8	18.4	Fim da montagem	Estoque A: 7	Estoque B: 4	Componentes: 3
9	18.4	Inicia montagem	Estoque A: 6	Estoque B: 2	Espera: 0.0
10	22.1	Fim da montagem	Estoque A: 6	Estoque B: 2	Componentes: 4
11	22.1	Inicia montagem	Estoque A: 5	Estoque B: 0	Espera: 0.0
12	28.2	Fim da montagem	Estoque A: 5	Estoque B: 0	Componentes: 5
13	41.5	Chegada de lote	tipo A: +10 peças		
14	44.5	Chegada de lote	tipo B: +10 peças		
15	44.5	Inicia montagem	Estoque A: 14	Estoque B: 8	Espera: 16.3
16	48.9	Fim da montagem	Estoque A: 14	Estoque B: 8	Componentes: 6
17	48.9	Inicia montagem	Estoque A: 13	Estoque B: 6	Espera: 0.0
18	53.1	Fim da montagem	Estoque A: 13	Estoque B: 6	Componentes: 7
19	53.1	Inicia montagem	Estoque A: 12	Estoque B: 4	Espera: 0.0
20	59.1	Fim da montagem	Estoque A: 12	Estoque B: 4	Componentes: 8
21	59.1	Inicia montagem	Estoque A: 11	Estoque B: 2	Espera: 0.0
22	64.7	Fim da montagem	Estoque A: 11	Estoque B: 2	Componentes: 9
23	64.7	Inicia montagem	Estoque A: 10	Estoque B: 0	Espera: 0.0
24	70.0	Fim da montagem	Estoque A: 10	Estoque B: 0	Componentes: 10

O que o leitor deve ter achado interessante é o modo passivo da função montagem que, por meio de um laço infinito `while True` aguarda o aparecimento de peças suficientes nos estoques para iniciar a montagem. Interessante também é notar que não alocamos recursos para a operação e isso significa que o modelo de simulação atual não permite a montagem simultânea de componentes (veja o tópico “Teste seus conhecimentos” na próxima seção).

14.2 Agrupando lotes por atributo da entidade utilizando o FilterStore

Outra situação bastante comum em modelos de simulação é quando precisamos agrupar entidades por atributo. Por exemplo, os componentes anteriores são de duas cores: brancos ou verdes, de modo que a célula de montagem agora deve pegar peças A e B com as cores corretas.

Como agora existe um atributo (no caso, cor) que diferencia uma peça da outra, precisaremos de um `FilterStore`, para garantir a escolha certa da peça no estoque. Contudo, devemos lembrar que o `FilterStore`, diferentemente do `Container`, não permite que se armazene ou retire múltiplos objetos ao mesmo tempo. O comando `put` (ou mesmo o `get`), é limitado a um objeto por vez. Por fim, a montagem do componente agora é pelo atributo “cor”, o que significa que a função montagem deve ser chamada uma vez para cada valor do atributo (no caso duas vezes: “branco” ou “verde”).

De modo semelhante ao exemplo anterior, uma máscara para o problema seria:

```

1  import simpy
2  import random
3
4  TEMPO_CHEGADAS = [40, 50]      # intervalo entre chegadas de peças
5  TEMPO_MONTAGEM = [5, 1]        # tempo de montagem do componente
6  componentesProntos = 0          # variável para o total de componentes produzidos
7
8  def chegadaPecas(env, pecasFilterStoreDict, tipo, tamLote):
9      # gera lotes de pecas em intervalos uniformemente distribuídos
10     # sorteia a cor das peças
11     # coloca um número tamLote de peças dentro do FilterStore
12     pass
13
14  def montagem(env, pecasFilterStoreDict, numA, numB, cor):
15     # montagem do componente
16     global componentesProntos
17     pass
18
19  random.seed(100)
20  env = simpy.Environment()
21
22  # cria um dicionário para armazenar os FilterStore
23  pecasFilterStoreDict = {}
24  pecasFilterStoreDict['A'] = simpy.FilterStore(env)
25  pecasFilterStoreDict['B'] = simpy.FilterStore(env)
26
27  # inicia processos de chegadas de pecas
28  env.process(chegadaPecas(env, pecasFilterStoreDict, 'A', 10))
29  env.process(chegadaPecas(env, pecasFilterStoreDict, 'B', 10))
30
31  # inicia processos de montagem de pecas
32  env.process(montagem(env, pecasFilterStoreDict, 1, 2, 'branco'))
33  env.process(montagem(env, pecasFilterStoreDict, 1, 2, 'verde'))
34
35  env.run(until=80)

```

Note que foi criado um dicionário `pecasFilterStore` armazena um `FilterStore` para cada tipo de peça.

Vamos agora construir a função `chegadaPecas`, considerando que ela deve sortear a cor do lote de peças e enviar todas as peças do lote (uma por vez) para o respectivo `FilterStore`.

Para sortear a cor do lote, uma opção é utilizar o comando `random.choice`¹, enquanto o envio de múltiplas peças para o `FilterStore` pode ser feito por um laço `for`, como mostra o código a seguir:

¹<https://docs.python.org/3/library/random.html#random.choice>

```

1 def chegadaPecas(env, pecasFilterStoreDict, tipo, tamLote):
2     # gera lotes de pecas em intervalos uniformemente distribuídos
3     while True:
4         # sorteia a cor das peças
5         cor = random.choice(("branco", "verde"))
6         # coloca um número tamLote de peças dentro do FilterStore
7         for i in range(tamLote):
8             yield pecasFilterStoreDict[tipo].put(cor)
9         print("%5.1f Chegada de lote\ttipo: %s\t\tCor: %s"
10              %(env.now, tipo, cor))
11         yield env.timeout(random.uniform(*TEMPO_CHEGADAS))

```

A função montagem, de modo semelhante a função anterior, é formada por um laço infinito do tipo while... True, mas deve retirar múltiplas peças de cada estoque, respeitando o atributo “cor”. O código a seguir, soluciona este problema novamente com laços do tipo for e uma função anônima para buscar a cor correta da peça dentro do FilterStore:

```

1 def montagem(env, pecasFilterStoreDict, numA, numB, cor):
2     # montagem do componente
3     global componentesProntos
4
5     while True:
6         # marca o instante em que a célula está livre para a montagem
7         chegada = env.now
8         for i in range(numA):
9             yield pecasFilterStoreDict['A'].get(lambda c: c==cor)
10        for i in range(numB):
11            yield pecasFilterStoreDict['B'].get(lambda c: c==cor)
12        # armazena o tempo de espera por peças e inicia a montagem
13        espera = env.now - chegada
14        print("%5.1f Inicia montagem\tCor: %s\tEspera: %4.1f"
15              %(env.now, cor, espera))
16        yield env.timeout(random.normalvariate(*TEMPO_MONTAGEM))
17        # acumula componente montado
18        componentesProntos += 1
19        print("%5.1f Fim da montagem\tCor: %s\tComponentes: %i\tEstoque A: %i\tEs\
20 toque B: %i"
21              %(env.now, cor, componentesProntos, len(pecasFilterStoreDict['A']).it\
22               ems),
23              len(pecasFilterStoreDict['B'].items)))

```

Dois pontos merecem destaque na função anterior:

1. A função montagem, deve ser chamada duas vezes na inicialização da simulação, uma para cada cor. Isto significa que nossa implementação permite a montagem simultânea de peças de cores diferentes. Caso seja necessário contornar este problema, basta a criação de um recurso “montador” (veja o tópico “Teste seus conhecimentos” na próxima seção”;

2. Na última linha, para contabilizar o total de peças ainda em estoque, como o `FilterStore` não possui um método `.level`, utilizou-se a função `len()` aplicada a todos os itens do `FilterStore`.

Quando executado por apenas 80 minutos, o programa anterior fornece como saída:

```

1   0.0 Chegada de lote   tipo: A           Cor: branco
2   0.0 Chegada de lote   tipo: B           Cor: verde
3   44.5 Inicia montagem  Cor: verde        Espera: 44.5
4   44.5 Chegada de lote   tipo: A           Cor: verde
5   47.7 Inicia montagem  Cor: branco       Espera: 47.7
6   47.7 Chegada de lote   tipo: B           Cor: branco
7   50.3 Fim da montagem  Cor: verde        Componentes: 1   Estoque A: 18   Estoque B\
8   : 16
9   50.3 Inicia montagem  Cor: verde        Espera: 0.0
10  51.4 Fim da montagem  Cor: branco       Componentes: 2   Estoque A: 17   Estoque B\
11  : 14
12  51.4 Inicia montagem  Cor: branco       Espera: 0.0
13  54.8 Fim da montagem  Cor: verde        Componentes: 3   Estoque A: 16   Estoque B\
14  : 12
15  54.8 Inicia montagem  Cor: verde        Espera: 0.0
16  57.0 Fim da montagem  Cor: branco       Componentes: 4   Estoque A: 15   Estoque B\
17  : 10
18  57.0 Inicia montagem  Cor: branco       Espera: 0.0
19  59.2 Fim da montagem  Cor: verde        Componentes: 5   Estoque A: 14   Estoque B\
20  : 8
21  59.2 Inicia montagem  Cor: verde        Espera: 0.0
22  61.3 Fim da montagem  Cor: branco       Componentes: 6   Estoque A: 13   Estoque B\
23  : 6
24  61.3 Inicia montagem  Cor: branco       Espera: 0.0
25  64.6 Fim da montagem  Cor: branco       Componentes: 7   Estoque A: 12   Estoque B\
26  : 4
27  64.6 Inicia montagem  Cor: branco       Espera: 0.0
28  65.9 Fim da montagem  Cor: verde        Componentes: 8   Estoque A: 11   Estoque B\
29  : 2
30  65.9 Inicia montagem  Cor: verde        Espera: 0.0
31  68.8 Fim da montagem  Cor: branco       Componentes: 9   Estoque A: 10   Estoque B\
32  : 0
33  71.1 Fim da montagem  Cor: verde        Componentes: 10  Estoque A: 9    Estoque B\
34  : 0

```

Naturalmente, existem outras soluções, mas optei por um caminho que mostrasse algumas limitações para um problema bastante comum em modelos de simulação.

14.3 Desafios



Desafio 19: Considere, no primeiro exemplo, que o componente possui mais duas partes, C e D que devem ser previamente montadas entre si para, a seguir, serem encaixadas nas peças A e B. Os tempos de montagem são todos semelhantes. (Dica: generalize a função montagem apresentada no exemplo).



Desafio 20: Nos exemplos anteriores, os processos de montagem são paralelos. Considere que existe apenas um montador compartilhado para todos processos. Generalize a função montagem do desafio anterior, de modo que ela receba como parâmetro o respectivo recurso utilizado no processo.

14.4 Solução dos desafios 19 e 20

Desafio 19: Considere, no primeiro exemplo, que o componente possui mais duas partes, C e D que devem ser previamente montadas entre si para, a seguir, serem encaixadas nas peças A e B. Os tempos de montagem são todos semelhantes. (Dica: generalize a função montagem apresentada no exemplo).

Para este desafio, não precisamos alterar o processo de chegadas, apenas vamos chamá-lo mais vezes, para as peças tipo A, B, C e D.

Inicialmente, portanto, precisamos criar um Container para cada etapa da montagem, bem como chamar a função de geração de lotes para as 4 partes iniciais do componente:

```
1 random.seed(100)
2 env = simpy.Environment()
3
4 # cria estoques de peças
5 tipoList = ['A', 'B', 'C', 'D', 'AB', 'CD', 'ABCD']
6 pecasContainerDict = {}
7 for tipo in tipoList:
8     pecasContainerDict[tipo] = simpy.Container(env)
9
10 # inicia processos de chegadas de pecas
11 for i in "ABCD":
12     env.process(chegadaPecas(env, pecasContainerDict, i, 10))
```

O bacana desse desafio é explorar o potencial de *generalidade* do SimPy, afinal, os processos de montagem são semelhantes, o que muda apenas é o quais as peças estão sendo unidas.

Imagine por um momento, que podemos querer unir 3 ou mais peças. A lógica em si não é muito diferente daquela feita na seção anterior, muda apenas a necessidade de se lidar com um número de peças diferentes. Assim, para *generalizar* a função montagem, proponho utilizar o operador ****kwargs**² que envia um conjunto de parâmetros para uma função como um dicionário.

A ideia é chamar o processo de montagem de maneira flexível, por exemplo:

```
1 # inicia processos de montagem
2 env.process(montagem(env, pecasContainerDict, 'AB', A=1, B=2))
3 env.process(montagem(env, pecasContainerDict, 'CD', C=1, D=2))
4 env.process(montagem(env, pecasContainerDict, 'ABCD', AB=1, CD=1))
```

A função montagem é executada para diferentes configurações de peças a serem montadas. Por exemplo, a primeira linha representa uma montagem de uma peça A com duas B; a segunda, uma peça C e duas D e a terceira linha representa uma peça do tipo AB com uma do tipo CB, formando o componente ABCD.

O código a seguir, apresenta uma solução que dá razoável generalidade para a função montagem:

²<http://stackoverflow.com/questions/3394835/args-and-kwargs/3394898#3394898>


```

1  def montagem(env, pecasContainerDict, keyOut, **kwargs):
2      # montagem do componente
3
4      while True:
5          # marca o instante em que a célula esta livre para a montagem
6          chegada = env.now
7
8          # pega uma peça de cada um dos items do dicionário kwargs
9          for key, value in kwargs.items():
10             yield pecasContainerDict[key].get(value)
11
12         # armazena o tempo de espera por peças e inicia a montagem
13         espera = env.now - chegada
14         print("%5.1f Inicia montagem\t%s\tEspera: %4.1f" %(env.now, keyOut, esper\
15 a))
16         yield env.timeout(random.normalvariate(*TEMPO_MONTAGEM))
17         # acumula componente montado no Container de saída keyOut
18         yield pecasContainerDict[keyOut].put(1)
19         print("%5.1f Fim da montagem\t%s\tEstoque: %i"
20               %(env.now, keyOut, pecasContainerDict[keyOut].level))

```

No código anterior, os componentes montados são colocados no Container definido no parâmetro keyOut e, note como a parte relativa ao parâmetro **kwargs é tratada como um mero dicionário pelo Python.

Completo, o modelo de simulação do desafio 19 ficaria:

```

1  import simpy
2  import random
3
4  TEMPO_CHEGADAS = [40, 50]      # intervalo entre chegadas de peças
5  TEMPO_MONTAGEM = [5, 1]       # tempo de montagem do componente
6
7  def chegadaPecas(env, pecasContainerDict, tipo, tamLote):
8      # gera lotes de pecas em intervalos uniformemente distribuídos
9      # encaminha para o estoque
10     while True:
11         pecasContainerDict[tipo].put(tamLote)
12         print("%5.1f Chegada de lote\t%s\tPeças: %i"
13               %(env.now, tipo, tamLote))
14         yield env.timeout(random.uniform(*TEMPO_CHEGADAS))
15
16
17  def montagem(env, pecasContainerDict, keyOut, **kwargs):
18      # montagem do componente
19
20     while True:

```

```

21     # marca o instante em que a célula esta livre para a montagem
22     chegada = env.now
23
24     # pega uma peça de cada um dos items do dicionário kwargs
25     for key, value in kwargs.items():
26         yield pecasContainerDict[key].get(value)
27
28     # armazena o tempo de espera por peças e inicia a montagem
29     espera = env.now - chegada
30     print("%5.1f Inicia montagem\t%s\tEspera: %4.1f" %(env.now, keyOut, esper\
31 a))
32     yield env.timeout(random.normalvariate(*TEMPO_MONTAGEM))
33     # acumula componente montado no Container de saída keyOut
34     yield pecasContainerDict[keyOut].put(1)
35     print("%5.1f Fim da montagem\t%s\tEstoque: %i"
36           %(env.now, keyOut, pecasContainerDict[keyOut].level))
37
38 random.seed(100)
39 env = simpy.Environment()
40
41 # cria estoques de peças
42 tipolist = ['A', 'B', 'C', 'D', 'AB', 'CD', 'ABCD']
43 pecasContainerDict = {}
44 for tipo in tipolist:
45     pecasContainerDict[tipo] = simpy.Container(env)
46
47 # inicia processos de chegadas de peças
48 for i in "ABCD":
49     env.process(chegadaPecas(env, pecasContainerDict, i, 10))
50
51 # inicia processos de montagem
52 env.process(montagem(env, pecasContainerDict, 'AB', A=1, B=2))
53 env.process(montagem(env, pecasContainerDict, 'CD', C=1, D=2))
54 env.process(montagem(env, pecasContainerDict, 'ABCD', AB=1, CD=1))
55
56 env.run(until=40)

```

Quando executado por apenas 40 minutos, o modelo anterior fornece como saída:

1	0.0	Chegada de lote	A	Peças: 10
2	0.0	Chegada de lote	B	Peças: 10
3	0.0	Chegada de lote	C	Peças: 10
4	0.0	Chegada de lote	D	Peças: 10
5	0.0	Inicia montagem	AB	Espera: 0.0
6	0.0	Inicia montagem	CD	Espera: 0.0
7	5.7	Fim da montagem	AB	Estoque: 1
8	5.7	Inicia montagem	AB	Espera: 0.0
9	6.1	Fim da montagem	CD	Estoque: 0
10	6.1	Inicia montagem	ABCD	Espera: 6.1
11	6.1	Inicia montagem	CD	Espera: 0.0
12	9.4	Fim da montagem	AB	Estoque: 1
13	9.4	Inicia montagem	AB	Espera: 0.0
14	9.7	Fim da montagem	CD	Estoque: 1
15	9.7	Inicia montagem	CD	Espera: 0.0
16	12.3	Fim da montagem	ABCD	Estoque: 1
17	12.3	Inicia montagem	ABCD	Espera: 0.0
18	13.8	Fim da montagem	AB	Estoque: 1
19	13.8	Inicia montagem	AB	Espera: 0.0
20	13.9	Fim da montagem	CD	Estoque: 1
21	13.9	Inicia montagem	CD	Espera: 0.0
22	18.2	Fim da montagem	ABCD	Estoque: 2
23	18.2	Inicia montagem	ABCD	Espera: 0.0
24	19.2	Fim da montagem	CD	Estoque: 1
25	19.2	Inicia montagem	CD	Espera: 0.0
26	19.4	Fim da montagem	AB	Estoque: 1
27	19.4	Inicia montagem	AB	Espera: 0.0
28	21.8	Fim da montagem	ABCD	Estoque: 3
29	21.8	Inicia montagem	ABCD	Espera: 0.0
30	25.2	Fim da montagem	CD	Estoque: 1
31	25.3	Fim da montagem	AB	Estoque: 1
32	26.6	Fim da montagem	ABCD	Estoque: 4
33	26.6	Inicia montagem	ABCD	Espera: 0.0
34	34.5	Fim da montagem	ABCD	Estoque: 5

Desafio 20: Nos exemplos anteriores, os processos de montagem são paralelos. Considere que existe apenas um montador compartilhado para todos processos. Generalize a função montagem do desafio anterior, de modo que ela receba como parâmetro o respectivo recurso utilizado no processo.

Neste desafio precisamos apenas garantir que a função montagem receba como parâmetro o respectivo recurso a ser utilizado no processo:

```

1  def montagem(env, pecasContainerDict, montador, keyOut, **kwargs):
2      # montagem do componente
3
4      while True:
5          # marca o instante em que a célula esta livre para a montagem
6          with montador.request() as req:
7              # ocupa o recurso montador
8              yield req
9              chegada = env.now
10             # pega uma peça de cada um dos items do dicionário kwargs
11             for key, value in kwargs.items():
12                 yield pecasContainerDict[key].get(value)
13
14             # armazena o tempo de espera por peças e inicia a montagem
15             espera = env.now - chegada
16             print("%5.1f Inicia montagem\t%s\tEspera: %4.1f"
17                   %(env.now, keyOut, espera))
18             yield env.timeout(random.normalvariate(*TEMPO_MONTAGEM))
19             # acumula componente montado no Container de saída keyOut
20             yield pecasContainerDict[keyOut].put(1)
21             print("%5.1f Fim da montagem\t%s\tEstoque: %i"
22                   %(env.now, keyOut, pecasContainerDict[keyOut].level))

```

Repare que a saída de tempo de espera que a função fornece é, na verdade, o tempo de espera do *montador* e não o tempo de espera em fila pelo recurso (veja o tópico “Teste seus conhecimentos” a seguir).

A chamada de execução do modelo não é muito diferente do desafio anterior, apenas precisamos criar um recurso único que realiza todos os processos:

```

1  random.seed(100)
2  env = simpy.Environment()
3
4  # cria estoques de peças
5  tipolist = ['A', 'B', 'C', 'D', 'AB', 'CD', 'ABCD']
6  pecasContainerDict = {}
7  for tipo in tipolist:
8      pecasContainerDict[tipo] = simpy.Container(env)
9
10
11 # inicia processos de chegadas de peças
12 for i in "ABCD":
13     env.process(chegadaPecas(env, pecasContainerDict, i, 10))
14
15 # cria os recursos de montagem
16 montador = simpy.Resource(env, capacity=1)
17

```

```

18 # inicia processos de montagem
19 env.process(montagem(env, pecasContainerDict, montador, 'AB', A=1, B=2))
20 env.process(montagem(env, pecasContainerDict, montador, 'CD', C=1, D=2))
21 env.process(montagem(env, pecasContainerDict, montador, 'ABCD', AB=1, CD=1))
22
23 env.run(until=40)

```

Quando o modelo anterior é executado por apenas 40 minutos, temos como saída:

1	0.0	Chegada de lote	A	Peças: 10
2	0.0	Chegada de lote	B	Peças: 10
3	0.0	Chegada de lote	C	Peças: 10
4	0.0	Chegada de lote	D	Peças: 10
5	0.0	Inicia montagem	AB	Espera: 0.0
6	5.7	Fim da montagem	AB	Estoque: 1
7	5.7	Inicia montagem	CD	Espera: 0.0
8	11.8	Fim da montagem	CD	Estoque: 1
9	11.8	Inicia montagem	ABCD	Espera: 0.0
10	15.5	Fim da montagem	ABCD	Estoque: 1
11	15.5	Inicia montagem	AB	Espera: 0.0
12	21.6	Fim da montagem	AB	Estoque: 1
13	21.6	Inicia montagem	CD	Espera: 0.0
14	25.2	Fim da montagem	CD	Estoque: 1
15	25.2	Inicia montagem	ABCD	Espera: 0.0
16	29.6	Fim da montagem	ABCD	Estoque: 2
17	29.6	Inicia montagem	AB	Espera: 0.0
18	33.8	Fim da montagem	AB	Estoque: 1
19	33.8	Inicia montagem	CD	Espera: 0.0
20	39.7	Fim da montagem	CD	Estoque: 1
21	39.7	Inicia montagem	ABCD	Espera: 0.0

Note como a produção de componentes ABCD caiu de 5 peças no desafio 19, para apenas 2 neste desafio. Isso é naturalmente explicado, pois agora os processos não são mais paralelos e devem ser executados por um único montador.

14.5 Teste seus conhecimentos

1. Acrescente o cálculo do tempo médio em espera por fila de montador ao modelo do desafio 20;
2. Acrescente o cálculo do WIP - *Work In Progress* do processo ou o *trabalho em andamento*, isto é, quantas peças estão em produção ao longo do tempo de simulação;
3. Utilize a biblioteca [matplotlib](http://matplotlib.org/)³ e construa um gráfico para evolução do estoque de cada peça ao longo da simulação, bem como do WIP.

³<http://matplotlib.org/>

15 Criando, manipulando e disparando eventos com event()

Nesta seção, discutiremos comandos que lhe darão o poder de criar, manipular ou disparar seus próprios eventos, de modo independente dos processos já discutidos nas seções anteriores.

Mas com todo o poder, vem também a responsabilidade!

Atente-se para o fato de que, sem o devido cuidado, o seu modelo pode ficar um pouco confuso. Isto porque um evento pode ser criado a qualquer momento e fora do contexto original do processo em execução, naturalmente aumentando a complexidade do código.

15.1 Criando um evento isolado com event()

Considere um problema simples de controle de turno de abertura e fechamento de uma ponte elevatória. A ponte abre para automóveis, opera por 5 minutos, fecha e permite a passagem de embarcações no cruzamento por mais 5 minutos.

Naturalmente, esse modelo poderia ser implementado com os comandos já discutidos neste livro, contudo, a ideia desta seção é demonstrar como criar um evento específico que informe à ponte que ela deve fechar, algo semelhante a um sinal semafórico.

Em SimPy, um evento é criado pelo comando `env.event()`:

```
1 abrePonte = env.event()           # cria o evento abrePonte
```

Criar um evento, não significa executá-lo. Criar um evento significa apenas criá-lo na memória. Para processar um evento, isto é, marcá-lo como executado, utilizamos o método `.succeed()`:

```
1 # marca o evento abrePonte como executado
2 yield abrePonte.succeed()
```

Podemos utilizar o evento criado de diversas formas em um modelo. Por exemplo, com o comando `yield` podemos fazer um processo aguardar até que o evento criado seja processado, com a linha:

```
1 # aguarda o evento para abertura da ponte
2 yield abrePonte
```

Retornando ao exemplo da ponte, criaremos um processo que representará o funcionamento da ponte. Inicialmente a ponte estará fechada e aguardará até que o evento `abrePonte` seja processado:

```
1 def ponteElevatoria(env):
2     # opera a ponte elevatória
3     global abrePonte
4
5     print('%2.0f A ponte está fechada =( ' %(env.now))
6
7     # aguarda o evento para abertura da ponte
8     yield abrePonte
9     print('%2.0f A ponte está aberta =)' %(env.now))
```

Note que abrePonte é tratado como uma variável global e isso significa que alguma outra função deve criá-lo e processá-lo, de modo que nossa função ponteElevatoria abra a ponte no instante correto da simulação.

Assim, criaremos uma função geradora turno que representará o processo de controle do turno de abertura/fechamento da ponte e que será responsável por criar e processar o evento de abertura da mesma:

```
1 def turno(env):
2     # abre e fecha a ponte
3     global abrePonte
4
5     while True:
6         # cria evento para abertura da ponte
7         abrePonte = env.event()
8
9         # inicia o processo da ponte elvatória
10        env.process(ponteElevatoria(env))
11
12        # mantém a ponte fechada por 5 minutos
13        yield env.timeout(5)
14
15        # processa o evento de abertura da ponte
16        yield abrePonte.succeed()
17
18        # mantém a ponte aberta por 5 minutos
19        yield env.timeout(5)
```

Note, na função anterior, que o evento é criado, mas **não é processado** imediatamente. De fato, ele só é processado quando o método abrePonte.succeed() é executado, após o tempo de espera de 5 minutos.

Como queremos que a ponte funcione continuamente, um novo evento deve ser criado para representar o novo ciclo de abertura e fechamento. Isso está representado no início do laço com a linha:

```

1  # cria evento para abertura da ponte
2  abrePonte = env.event()

```

Precisamos deixar isso bem claro, paciente leitor: uma vez processado com o método `.succeed()`, o evento é extinto e caso seja necessário executá-lo novamente, teremos de recriá-lo com `env.event()`.

Juntando tudo num único modelo de abre/fecha da ponte elevatória, temos:

```

1  import simpy
2
3  def turno(env):
4      # abre e fecha a ponte
5      global abrePonte
6
7      while True:
8          # cria evento para abertura da ponte
9          abrePonte = env.event()
10
11         # inicia o processo da ponte elvatória
12         env.process(ponteElevatoria(env))
13
14         # mantém a ponte fechada por 5 minutos
15         yield env.timeout(5)
16
17         # processa o evento de abertura da ponte
18         yield abrePonte.succeed()
19
20         # mantém a ponte aberta por 5 minutos
21         yield env.timeout(5)
22
23  def ponteElevatoria(env):
24      # opera a ponte elevatória
25      global abrePonte
26
27      print('%2.0f A ponte está fechada =( ' %(env.now))
28
29      # aguarda o evento para abertura da ponte
30      yield abrePonte
31      print('%2.0f A ponte está aberta =)' %(env.now))
32
33  env = simpy.Environment()
34
35  # inicia o processo de controle do turno
36  env.process(turno(env))
37
38  env.run(until=20)

```


Quando executado por 20 minutos, o modelo anterior fornece:

```
1 0 A ponte está fechada =(
2 5 A ponte está aberta  =)
3 10 A ponte está fechada =(
4 15 A ponte está aberta  =)
```

No exemplo anterior, fizemos uso de uma variável global, `abrePonte`, para enviar a informação de que o evento de abertura da ponte foi disparado.

Isso é bom, mas também pode ser ruim =).

Note que o evento de abertura da ponte é criado e processado dentro da função `turno(env)`, portanto, **fora** da função que controla o processo de abertura e fechamento da ponte, `ponteElevatoria(env)`. Caso você não tome o devido cuidado, as coisas podem realmente ficar confusas no seu modelo.

O método `.succeed()` ainda pode enviar um valor como parâmetro:

```
1 meuEvento.succeed(value=valor)
```

Poderíamos, por exemplo, enviar à função `ponteElevatoria` o tempo previsto para que a ponte fique aberta. O modelo a seguir transfere o tempo de abertura (5 minutos) à função `ponteElevatoria`, que o armazena na variável `tempoAberta`:

```
1 import simpy
2
3 def turno(env):
4     # abre e fecha a ponte
5     global abrePonte
6
7     while True:
8         # cria evento para abertura da ponte
9         abrePonte = env.event()
10
11        # inicia o processo da ponte elevatoria
12        env.process(ponteElevatoria(env))
13
14        # mantém a ponte fechada por 5 minutos
15        yield env.timeout(5)
16
17        # dispara o evento de abertura da ponte
18        yield abrePonte.succeed(value=5)
19
20        # mantém a ponte aberta por 5 minutos
21        yield env.timeout(5)
22
23 def ponteElevatoria(env):
```

```

24     # opera a ponte elevatória
25     global abrePonte
26
27     print('%2.0f A ponte está fechada =( ' %(env.now))
28
29     # aguarda o evento para abertura da ponte
30     tempoAberta = yield abrePonte
31     print('%2.0f A ponte está aberta =) e fecha em %2.0f minutos'
32           %(env.now, tempoAberta))
33
34 env = simpy.Environment()
35
36 # inicia o processo de controle do turno
37 env.process(turno(env))
38
39 env.run(until=20)

```

Dentro da função `ponteElevatoria`, a linha:

```

1  # aguarda o evento para abertura da ponte
2  tempoAberta = yield abrePonte

```

Aguarda até que o evento `abrePonte` seja executado e resgata seu valor (o tempo que a ponte deve permanecer aberta) na variável `tempoAberta`.

Concluindo, o potencial de uso do comando `event()` é extraordinário, mas, por experiência própria, garanto que seu uso descuidado pode tornar qualquer código ininteligível, candidato ao [Campeonato Mundial de Código C Ofuscado](http://www.ioccc.org/)¹ (sim, isso existe!) ou mesmo algo semelhante a utilizar desvios de laço `goto`...to em um programa (des)estruturado).



15.2 Conceitos desta seção

¹<http://www.ioccc.org/>

Conteúdo	Descrição
<code>meuEvento = env.event()</code>	cria um novo <i>evento</i> meuEvento durante a simulação, mas não o processa.
<code>yield meuEvento</code>	aguarda até que o evento meuEvento seja processado
<code>yield meuEvento.succeed(value=valor)</code>	processa o evento meuEvento, isto é, dispara o evento no tempo atual e inicia o seu processamento, retornando o parâmetro opcional <code>valor</code> .

15.3 Desafios



Desafio 21: crie um processo de geração de automóveis que desejam cruzar a ponte, durante o horário de pico que dura 4 horas. Os intervalos entre chegadas sucessivas de veículos para travessia são exponencialmente distribuídos com média de 10 segundos (ou 6 veículos/min), e a ponte permite a travessia de 10 veículos por minuto. Após 4 horas de operação, quantos veículos estão em espera por travessia da ponte?



Desafio 22: para o sistema anterior, construa um gráfico para o número de veículos em fila em função do tempo de abertura da ponte para travessia de automóveis. Qual o tempo mínimo que você recomendaria para abertura da ponte?

15.4 Solução dos desafios 21 e 22

Desafio 21: crie um processo de geração de automóveis que desejam cruzar a ponte, durante o horário de pico que dura 4 horas. Os intervalos entre chegadas sucessivas de veículos para travessia são exponencialmente distribuídos com média de 10 segundos (ou 6 veículos/min), e a ponte permite a travessia de 10 veículos por minuto. Após 4 horas de operação, quantos veículos estão em espera por travessia da ponte?

Em relação ao modelo anterior, este novo sistema possui um processo de geração de chegadas de veículos e espera por abertura da ponte. Uma alternativa de implementação é utilizar um Container para armazenar os veículos em espera pela abertura da ponte, como no código a seguir:

```
1 import simpy
2 import random
3
4 CAPACIDADE_TRAVESSIA = 10    # capacidade da ponte em por min
5
6 def chegadaVeiculos(env, filaTravessia):
7     while True:
8         # aguarda a chegada do próximo veículo
9         yield env.timeout(random.expovariate(6.0))
10        # acrescenta o veículo na fila de travessia
11        yield filaTravessia.put(1)
```

No código anterior, filaTravessia é um Container que representa o conjunto de veículos em espera por travessia da ponte. Cada veículo gerado é imediatamente transferido para o Container.

A função turno é semelhante à anterior, apenas com a inclusão do tempo de abertura da ponte, como o parâmetro tempo_ponte:

```
1 def turno(env, filaTravessia, tempo_ponte):
2     # abre e fecha a ponte
3     global abrePonte
4
5     while True:
6         # cria evento para abertura da ponte
7         abrePonte = env.event()
8         # inicia o processo da ponte elvatória
9         env.process(ponteElevatoria(env, filaTravessia))
10        # mantém a ponte fechada por 5 minutos
11        yield env.timeout(5)
12        # dispara o evento de abertura da ponte
13        yield abrePonte.succeed(value=tempo_ponte)
14        # mantém a ponte aberta por 5 minutos
15        yield env.timeout(tempo_ponte)
```

Note que o parâmetro `tempo_ponte` é enviado como um valor para a função `ponteElevatoria`, que agora deve lidar com a travessia dos veículos quando aberta. Neste caso, basta um comando `get` no `Container` que representa a fila de travessia dos veículos:

```

1 def ponteElevatoria(env, filaTravessia, tempo_ponte):
2     # opera a ponte elevatória
3     global abrePonte, naoAtendidos
4
5     print('%2.0f A ponte está fechada =( ' %(env.now))
6     # aguarda o evento para abertura da ponte
7     tempoAberta = yield abrePonte
8     print('%2.0f A ponte está aberta =) e fecha em %2.0f minutos'
9           %(env.now, tempoAberta))
10
11     # aguarda a chegada de mais veículos na fila de espera
12     yield env.timeout(tempoAberta)
13
14     # calcula quantos veículos podem atravessar a ponte
15     numVeiculos = min(int(tempoAberta*CAPACIDADE_TRAVESSIA), filaTravessia.level)
16
17     # retira os veículos da fila
18     filaTravessia.get(numVeiculos)
19     print('%2.0f Travessia de %i veículos\tFila atual: %i'
20           %(env.now, numVeiculos, filaTravessia.level))

```

Analisando o código anterior, assim que a ponte abre, a linha:

```

1 # calcula quantos veículos podem atravessar a ponte
2 numVeiculos = min(int(tempoAberta*CAPACIDADE_TRAVESSIA), filaTravessia.level)

```

Estima quantos veículos, dentre aqueles que estão no `Container`, podem realizar a travessia. A seguir, os veículos são retirados do `Container` por meio de um comando `get` com parâmetro no número de veículos a serem retirados.

Ao final, deve-se criar o `Container`, realizar as chamadas dos processos e executar o modelo por 4 horas (ou 240 minutos):

```

1 random.seed(100)
2 env = simpy.Environment()
3
4 # container para representar a fila de automóveis aguardando a travessia
5 filaTravessia = simpy.Container(env)
6
7 # inicia o processo de controle do turno
8 env.process(turno(env, filaTravessia, 5))
9 env.process(chegadaVeiculos(env, filaTravessia))
10
11 env.run(until=240)

```

Quando executado, o modelo completo fornece como saída (resultados compactados):

```

1  0 A ponte está fechada =(
2  5 A ponte está aberta =) e fecha em 5 minutos
3 10 A ponte está fechada =(
4  ...
5 225 A ponte está aberta =) e fecha em 5 minutos
6 230 A ponte está fechada =(
7 230 Travessia de 50 veículos    Fila atual: 170
8 235 A ponte está aberta =) e fecha em 5 minutos

```

Portanto, considerando-se as condições simuladas, o modelo indica que 170 veículos ainda estão em espera em fila ao final da última abertura da ponte dentro do horário de pico. Portanto, o tempo de abertura da ponte parece ser insuficiente durante o horário de pico.

Desafio 22: para o sistema anterior, construa um gráfico para o número de veículos em fila em função do tempo de abertura da ponte para travessia de automóveis. Qual o tempo mínimo que você recomendaria de abertura da ponte.

Como o desafio deseja uma avaliação da fila ao final do horário de pico para diferentes valores de abertura da ponte, o primeiro passo é construir um laço para que o modelo possa ser executado para diferentes valores do tempo de abertura da ponte:

```

1  # lista para armazenar o resultado do cenário simulado
2  resultado = []
3
4  for tempo_ponte in range(5, 10):
5      random.seed(100)
6      env = simpy.Environment()
7
8      # número de veículos não atendidos ao final da simulação
9      naoAtendidos = 0
10
11     # container para representar a fila de automóveis aguardando a travessia
12     filaTravessia = simpy.Container(env)
13
14     # inicia o processo de controle do turno
15     env.process(turno(env, filaTravessia, tempo_ponte))
16     env.process(chegadaVeiculos(env, filaTravessia))
17
18     env.run(until=240)
19
20     # armazena o número de veículos do cenário atual em uma lista
21     resultado.append((tempo_ponte, naoAtendidos))

```

Note, no código anterior, que foi criada uma lista, `resultado`, para armazenar o `Tuple` com o tempo de abertura da ponte simulado e o resultado do número de veículos não atendidos ao final da simulação.

A função `ponteElevatoria` precisa de apenas de uma pequena modificação para assegurar que a variável global `naoAtendidos` receba corretamente o número de veículos não atendidos imediatamente após ao fechamento da ponte:

```

1  def ponteElevatoria(env, filaTravessia):
2      # opera a ponte elevatória
3      global abrePonte, naoAtendidos
4
5      print('%2.0f A ponte está fechada =( ' %(env.now))
6      # aguarda o evento para abertura da ponte
7      tempoAberta = yield abrePonte
8      print('%2.0f A ponte está aberta =) e fecha em %2.0f minutos'
9          %(env.now, tempoAberta))
10
11     # aguarda a chegada de mais veículos na fila de espera
12     yield env.timeout(tempoAberta)
13     # calcula quantos veículos podem atravessar a ponte
14     numVeiculos = min(int(tempoAberta*CAPACIDADE_TRAVESSIA), filaTravessia.level)
15     # retira os veículos da fila
16     filaTravessia.get(numVeiculos)
17     # armazena o número de veículos não atendidos ao final da abertura da ponte
18     naoAtendidos = filaTravessia.level
19     print('%2.0f Travessia de %i veículos\tFila atual: %i'
20         %(env.now, numVeiculos, filaTravessia.level))

```

O próximo passo é acrescentar, ao final da simulação, um gráfico do número de veículos em função do tempo de abertura da ponte. Essa operação é facilitada pelo uso da biblioteca **matplotlib** no conjunto de dados armazenado na lista `resultado`:

```

1  # lista para armazenar o resultado do cenário simulado
2  resultado = []
3
4  for tempo_ponte in range(5, 11):
5      random.seed(100)
6
7      env = simpy.Environment()
8
9      # número de veículos não atendidos ao final da simulação
10     naoAtendidos = 0
11
12     # container para representar a fila de automóveis aguardando a travessia
13     filaTravessia = simpy.Container(env)
14

```

```
15     # inicia o processo de controle do turno
16     env.process(turno(env, filaTravessia, tempo_ponte))
17     env.process(chegadaVeiculos(env, filaTravessia))
18
19     env.run(until=240)
20
21     # armazena o número de veículos do cenário atual em uma lista
22     resultado.append((tempo_ponte, naoAtendidos))
23
24 import matplotlib.pyplot as plt
25
26 # descompacta os valores armazenados na lista resultado e plota em um gráfico de \
27 barras
28 plt.bar(*zip(*resultado), align='center')
29 plt.xlabel('Tempo de abertura da ponte (min)')
30 plt.ylabel('Número de veículos em espera ao final\nda última abertura da ponte')
31 plt.xlim(4.5,10.5)
32 plt.grid(True)
33 plt.show()
```

Quando executado, o modelo anterior fornece como resultado o seguinte gráfico:

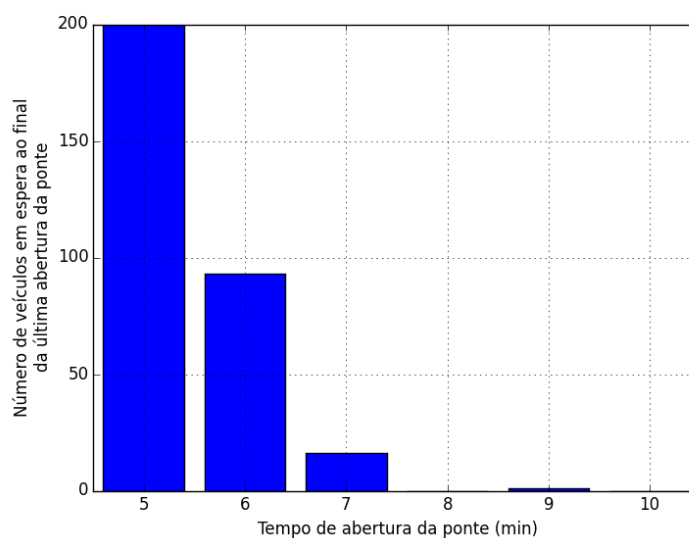


Figura 15.1. Número de veículos em espera ao final da simulação para cada cenário simulado do tempo de abertura da ponte.

Em uma primeira análise, portanto, o tempo de abertura de 8 minutos seria suficiente para atender aos veículos durante o horário de pico. Contudo, nossa análise está limitada a uma replicação apenas, o que torna a conclusão, eventualmente, precipitada (veja o item 2 do tópico “Teste seus conhecimentos” a seguir).

15.5 Teste seus conhecimentos

1. Por que utilizamos na função `ponteElevatoria` a variável global `naoAtendidos`? Não seria suficiente armazenar na fila `resultados` diretamente o número de veículos no Container `filaTravessia`, pelo comando `filaTravessia.level`?
2. Como nos lembram Chwif e Medina (2024)²: “nunca se deve tomar decisões baseadas em apenas uma replicação de um modelo de simulação”. Afinal, basta modificar a semente geradora de número aleatórios, para que o resultado do gráfico seja outro (teste no seu modelo!). Modifique o modelo para que ele simule um número de replicações configurável para cada tempo de abertura da ponte. Adicionalmente, garanta que tempos de abertura diferentes utilizem a mesma sequência de números aleatórios. (Dica: para esta segunda parte, armazene as sementes geradoras em uma lista).

³ Chwif, L., and A. C. Medina. 2024. [Modelagem e Simulação de Eventos Discretos: Teoria e Aplicações](#)⁴. 5ª ed. Leanpub.

²Em português, ainda é comum o estrangeirismo “framework” entre profissionais da Ciência da Computação.

³Em português, ainda é comum o estrangeirismo “framework” entre profissionais da Ciência da Computação.

⁴<https://leanpub.com/msed>

16 Aguardando múltiplos eventos ao mesmo tempo com AnyOf e AllOf

Uma funcionalidade importante do SimPy é permitir que uma entidade aguarde até que dois ou mais eventos ocorram para então prosseguir com o processo. O SimPy possui duas opções muito interessantes para isso:

- `simpy.AnyOf(env, eventos)`: aguarda até que um dos eventos tenham ocorrido—AnyOf é equivalente ao símbolo de “|” (ou or);
- `simpy.AllOf(env, eventos)`: aguarda até que todos os eventos tenham ocorrido—AllOf é equivalente ao símbolo de “&” (ou and).

Para compreender o funcionamento dos comandos anteriores, partiremos de um exemplo baseado numa *obscura* fábula infantil: [a Lebre e a Tartaruga](#)¹.



Neste exemplo, sortearmos um tempo de corrida para cada bicho e identificaremos quem foi o vencedor. Para tanto, além do sorteio, criaremos dois eventos que representam a corrida de cada bicho:

```
1 def corrida(env):
2     # a lebre x tartaruga!
3     # sorteia aleatoriamente os tempos dos animais
4     lebreTempo = random.normalvariate(5,2)
5     tartarugaTempo = random.normalvariate(5,2)
6     # cria os eventos de corrida de cada animal
7     lebreEvent = env.timeout(lebreTempo, value='lebre')
8     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
```

Na função `corrida`, criamos os eventos `lebreEvent` e `tartarugaEvent`, que simulam, respectivamente, as corridas da lebre e da tartaruga. Mas, lembre-se: apesar de criados, os eventos ainda não foram *necessariamente* executados. Como não existe um `yield` aplicado aos eventos, eles foram criados

¹https://en.wikipedia.org/wiki/The_Tortoise_and_the_Hare

na memória do Python, mas só serão considerados *processados* após o tempo de simulação avançar o suficiente para que os tempos de timeout tenham passado.

De outra forma, os eventos `lebreEvent` e `tartarugaEvent` foram *disparados*, mas não processados, pois a função `corrida` ainda não tem um comando que aguarda o término do processamento desses eventos.

16.1 Aguardando até que, ao menos, um evento termine com AnyOf

Para garantir que a função `corrida` aguarde até que, ao menos, um dos corredores termine a prova, uma opção é acrescentar um `yield AnyOf()` (que pode ser substituído por “[”) após a criação dos eventos:

```
1     # começou!
2     start = env.now
3     print('%3.1f Iniciada a corrida!' %(env.now))
4     # simule até que o ao menos um dos eventos termine
5     resultado = yield lebreEvent | tartarugaEvent
6     tempo = env.now - start
```

O `yield` garante que a função aguardará até que um dos dois eventos—`lebreEvent` ou `tartarugaEvent`—termine e a variável `resultado` armazenará qual desses eventos terminou primeiro (ou mesmo os dois, em caso de empate). Assim, para sabermos quem venceu, basta explorarmos o valor contido na variável `resultado`.

O código a seguir completa o modelo, testando qual dos dois eventos está na variável `resultado`:

```
1  import simpy
2  import random
3
4  def corrida(env):
5      # a lebre x tartaruga!
6      # sorteia aleatoriamente os tempos dos animais
7      lebreTempo = random.normalvariate(5,2)
8      tartarugaTempo = random.normalvariate(5,2)
9      # cria os eventos de corrida de cada animal
10     lebreEvent = env.timeout(lebreTempo, value='lebre')
11     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
12
13     # começou!
14     start = env.now
15     print('%3.1f Iniciada a corrida!' %(env.now))
16     # simule até que alguém chegue primeiro
17     resultado = yield lebreEvent | tartarugaEvent
18     tempo = env.now - start
```

```

19
20     # quem venceu?
21     if lebreEvent not in resultado:
22         print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo))
23     elif tartarugaEvent not in resultado:
24         print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
25     else:
26         print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
27
28 random.seed(10)
29 env = simpy.Environment()
30 proc = env.process(corrida(env))
31 env.run(until=10)

```

Quando o modelo anterior é executado, o bicho pega:

```

1 0.0 Iniciada a corrida!
2 5.3 A tartaruga venceu em 5.3 minutos

```

Note que a linha:

```
1 resultado = yield lebreEvent | tartarugaEvent
```

Poderia ter sido substituída pela linha:

```
1 resultado = yield simpy.AnyOf(env, lebreEvent, tartarugaEvent)
```

16.2 Aguardando todos os eventos com Allof

Para não deixar ninguém triste, poderíamos forçar um empate na corrida, aguardando que a função `corrida` aguarde até que os dois eventos sejam concluídos para decretar o vencedor. Para isso, basta substituir a linha:

```
1 resultado = yield lebreEvent | tartarugaEvent
```

por:

```
1 resultado = yield lebreEvent & tartarugaEvent
```

Quando simulado, o novo modelo fornece como saída:

```

1 0.0 Iniciada a corrida!
2 5.4 Houve um empate em 5.4 minutos

```

O que ocorreu? Neste caso, o comando `AllOf` (ou “&”) aguardou até que os dois eventos terminassem para liberar o processamento da linha seguinte de código e nosso desvio de condição `if` identificou que a variável `resultado` possuía os dois eventos armazenados.

16.3 Compreendendo melhor as saídas dos comandos `AllOf` e `AnyOf`

Agora que já sabemos o que fazem os comandos `AllOf` e `AnyOf`, vamos explorar nessa seção um pouco mais sobre o que exatamente esses comandos retornam.

Inicialmente, imprima os valores dos eventos e da variável `resultado`, para descobrir seus conteúdos:

```

1 import simpy
2 import random
3
4 def corrida(env):
5     # a lebre x tartaruga!
6     # sorteia aleatoriamente os tempos dos animais
7     lebreTempo = random.normalvariate(5,2)
8     tartarugaTempo = random.normalvariate(5,2)
9     # cria os eventos de corrida de cada animal
10    lebreEvent = env.timeout(lebreTempo, value='lebre')
11    tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
12    print('lebreEvent= ', lebreEvent)
13    print('tartarugaEvent= ', tartarugaEvent)
14    # começou!
15    start = env.now
16    print('%3.1f Iniciada a corrida!' %(env.now))
17    # simule até que alguém chegue primeiro
18    resultado = yield lebreEvent & tartarugaEvent
19    print('resultado = ', resultado)
20    tempo = env.now - start
21
22    # quem venceu?
23    if lebreEvent not in resultado:
24        print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo))
25    elif tartarugaEvent not in resultado:
26        print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
27    else:
28        print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
29

```

```

30 random.seed(10)
31 env = simpy.Environment()
32 proc = env.process(corrida(env))
33 env.run(until=10)

```

Quando executado, o programa fornece:

```

1 lebreEvent= <Timeout(5.428964407135667, value=lebre) object at 0xa592470>
2 tartarugaEvent= <Timeout(5.33749212083634, value=tartaruga) object at 0xa5920f0>
3 0.0 Iniciada a corrida!
4 resultado = <ConditionValue {<Timeout(5.33749212083634, value=tartaruga) object \
5 at 0xa5920f0>: 'tartaruga',
6 <Timeout(5.428964407135667, value=lebre) object at 0xa592470>: 'lebre'}>
7 5.4 Houve um empate em 5.4 minutos

```

Pela saída anterior, descobrimos, inicialmente, que os eventos são *objetos* do tipo `Timeout` e que armazenam tanto o tempo de espera, quanto o valor (ou `value`) fornecido na chamada da função `env.timeout`.

Um pouco mais abaixo, a saída revela que a variável `resultado` é um objeto da classe `ConditionValue` que, aparentemente, contém um dicionário de eventos em seu interior. Para acessar esse dicionário, o SimPy fornece o método `.todict()`:

```

1 resultado.todict()

```

Que retorna:

```

1 {<Timeout(5.428964407135667, value=lebre) object at 0xa18e30>: 'lebre',
2 <Timeout(5.33749212083634, value=tartaruga) object at 0xa18eb0>: 'tartaruga'}

```

Que nada mais é do que um dicionário padrão do Python, onde as `keys` são os eventos e os `items` são os valores dos eventos.

16.4 Conceitos desta seção

Conteúdo	Descrição
<code>AnyOf(env, eventos)</code>	aguarda até que um dos eventos tenham ocorrido - <code>AnyOf</code> é equivalente ao símbolo de “ ” (ou or).
<code>AllOf(env, eventos)</code>	aguarda até que todos os eventos tenham ocorrido - <code>AllOf</code> é equivalente ao símbolo de “&” (ou and).

16.5 Desafios



Desafio 23: Considere que existe uma probabilidade de que a Lebre, por alguma razão mal explicada (eu realizaria um teste antidoping nos competidores), resolva que é uma boa ideia tirar uma soneca de 5 minutos em algum instante entre 2 e 10 minutos do início da corrida. Modele esta nova situação (dica: crie um função *soneca* que gera um evento que pode ocasionar a parada da Lebre ainda durante a corrida).



Desafio 24: É interessante notar, que mesmo quando um dos competidores *perde* a corrida, de fato, o respectivo evento **não** é cancelado. Altere o modelo anterior para marcar o horário de chegada dos dois competidores, garantindo que os eventos `lebreEvent` e `tartarugaEvent` sejam executados até o fim.

16.6 Solução dos desafios 23 e 24

Desafio 23: Considere que existe uma probabilidade de que a Lebre, por alguma razão mal explicada (eu realizaria um teste antidoping nos competidores), resolva que é uma boa ideia tirar uma soneca de 5 minutos em algum instante entre 2 e 10 minutos do início da corrida. Modele esta nova situação (dica: crie um função soneca que gera um evento que pode ocasionar a parada da Lebre ainda durante a corrida).

Para este desafio será criada uma função para gerar a soneca e uma função adicional para identificar o vencedor, pois agora a tartaruga pode (ou não) vencer enquanto a lebre estiver dormindo.

Começando pela função que determina o vencedor:

```
1 def imprimeVencedor(start, resultado, lebreEvent, tartarugaEvent):
2     # determina o vencedor da corrida
3     tempo = env.now - start
4     # quem venceu?
5     if lebreEvent not in resultado:
6         print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo))
7     elif tartarugaEvent not in resultado:
8         print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
9     else:
10        print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
```

A função soneca a seguir, cria um evento sonecaEvent, sorteia o instante da soneca e processa o evento no instante correto:

```
1 def soneca(env):
2     global sonecaEvent
3
4     # cria o evento da soneca
5     sonecaEvent = env.event()
6     # sorteia o instante da soneca
7     inicioSoneca = random.uniform(2, 10)
8     # aguarda instante da soneca
9     yield env.timeout(inicioSoneca)
10    # durma!
11    sonecaEvent.succeed()
```

A função corrida, agora deve lidar com as diversas situações possíveis: uma soneca da lebre durante a corrida, a lebre acordando, a tartaruga ou a lebre vencendo. Essas diversas situações podem ser facilmente modeladas por um conjunto de lógicas de desvio condicional `if` concatenadas com operações do tipo `AnyOf`:


```

1  def corrida(env):
2      # a lebre x tartaruga!
3      global sonecaEvent
4
5      # sorteia aleatoriamente os tempos dos animais
6      lebreTempo = random.normalvariate(5,2)
7      tartarugaTempo = random.normalvariate(5,2)
8      # cria os eventos de corrida de cada animal
9      lebreEvent = env.timeout(lebreTempo, value='lebre')
10     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
11
12     # começou!
13     start = env.now
14     print('%3.1f Iniciada a corrida!' %(env.now))
15     # simule até que alguém chegue primeiro ou alguém pegue no sono...
16     resultado = yield lebreEvent | tartarugaEvent | sonecaEvent
17
18     if sonecaEvent in resultado:
19         # a lebre dormiu!
20         lebreTempo -= env.now - start
21         print('%3.1f A lebre capotou de sono?!?!' %(env.now))
22         lebreAcorda = env.timeout(5, value='lebre')
23         resultado = yield lebreAcorda | tartarugaEvent
24
25         if tartarugaEvent in resultado:
26             # a tartaruga venceu durante o sono da lebre
27             imprimeVencedor(start, resultado, None, tartarugaEvent)
28             env.exit()
29         else:
30             # a lebre acordou antes da corrida terminar
31             print('%3.1f A lebre acordou, amigo!' %(env.now))
32             lebreEvent2 = env.timeout(lebreTempo, value='lebre')
33             resultado = yield lebreEvent2 | tartarugaEvent
34             # alguém venceu!
35             imprimeVencedor(start, resultado, lebreEvent2, tartarugaEvent)
36     else:
37         # alguém venceu, antes da lebre pegar no sono!
38         imprimeVencedor(start, resultado, lebreEvent, tartarugaEvent)

```

Não devemos esquecer de iniciar o processo da soneca:

```

1 random.seed(100)
2 env = simpy.Environment()
3 # inicia processo da soneca
4 env.process(soneca(env))
5 # inicia a corrida
6 env.process(corrida(env))
7
8 env.run(until=10)

```

Note, na função `corrida`, que quando a tartaruga vence enquanto a lebre está dormindo, a função `imprimeVencedor` é chamada com o parâmetro `lebreEvent=None`, já informando à função que a lebre perdeu.

Quando o modelo anterior é simulado, fornece como resultado:

```

1 0.0 Iniciada a corrida!
2 3.2 A lebre capotou de sono?!?!
3 7.6 A tartaruga venceu em 7.6 minutos

```

Você pode alterar o valor da semente geradora de números aleatórios—na linha `random.seed(...)`—e apostar com os amigos quem vencerá a próxima corrida!

Desafio 24: É interessante notar, que mesmo quando um dos competidores *perde* a corrida, de fato, o respectivo evento **não** é cancelado. Altere o modelo anterior para marcar o horário de chegada dos dois competidores, garantindo que os eventos `lebreEvent` e `tartarugaEvent` sejam executados até o fim.

Uma possível solução para o desafio é transformar o a função `imprimeVencedor` em um processo que, após informar o vencedor, continua até que o outro competidor passe pela linha de chegada. Por exemplo, podemos uma alternativa seria:

```

1 def imprimeVencedor(env, start, resultado, lebreEvent, tartarugaEvent):
2     # determina o vencedor da corrida
3     tempo = env.now - start
4     # quem venceu?
5     if lebreEvent not in resultado:
6         print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo))
7         if lebreEvent:
8             yield lebreEvent
9         print('%3.1f A lebre chega em 2º lugar' %(env.now))
10    elif tartarugaEvent not in resultado:
11        print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
12        yield tartarugaEvent
13        print('%3.1f A tartaruga chega em 2º lugar' %(env.now))
14    else:
15        print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))

```

A função corrida deve agora utilizar comandos do tipo `env.process(imprimeVencedor)` para iniciar o processo que determina quem venceu e continua a corrida até que o outro competidor chegue. Note, no código a seguir, que apenas no caso da lebre ser pega ainda dormindo, a modificação do código é um pouco mais trabalhosa:

```

1  def corrida(env):
2      # a lebre x tartaruga!
3      global sonecaEvent
4
5      # sorteia aleatoriamente os tempos dos animais
6      lebreTempo = random.normalvariate(5,2)
7      tartarugaTempo = random.normalvariate(5,2)
8      # cria os eventos de corrida de cada animal
9      lebreEvent = env.timeout(lebreTempo, value='lebre')
10     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
11
12     # começou!
13     start = env.now
14     print('%3.1f Iniciada a corrida!' %(env.now))
15     # simule até que alguém chegue primeiro ou alguém pegue no sono...
16     resultado = yield lebreEvent | tartarugaEvent | sonecaEvent
17
18     if sonecaEvent in resultado:
19         # a lebre dormiu!
20         lebreTempo -= env.now - start
21         print('%3.1f A lebre capotou de sono?!?!' %(env.now))
22         lebreAcorda = env.timeout(lebreTempo, value='acorda')
23         resultado = yield lebreAcorda | tartarugaEvent
24
25     if tartarugaEvent in resultado:
26         # a tartaruga venceu durante o sono da lebre
27         env.process(imprimeVencedor(env, start, resultado, None, tartarugaEvent))
28
29         yield lebreAcorda
30         print('%3.1f A lebre acordou, amigo!' %(env.now))
31         yield env.timeout(lebreTempo, value='lebre')
32         print('%3.1f A lebre chega em 2º lugar' %(env.now))
33         # termina o processo
34         env.exit()
35     else:
36         # a lebre acordou antes da corrida terminar
37         print('%3.1f A lebre acordou, amigo!' %(env.now))
38         lebreEvent2 = env.timeout(lebreTempo, value='lebre')
39         resultado = yield lebreEvent2 | tartarugaEvent
40         # alguém venceu!
41         env.process(imprimeVencedor(env, start, resultado, lebreEvent2, tartarugaEvent))

```

```
42 rugaEvent))
43     else:
44         # alguém venceu, antes da lebre pegar no sono!
45         env.process(imprimeVencedor(env, start, resultado, lebreEvent, tartarugaE\
46 vent))
```

Quando executado, o modelo completo fornece como saída:

```
1 0.0 Iniciada a corrida!
2 3.2 A lebre capotou de sono?!?!
3 7.6 A tartaruga venceu em 7.6 minutos
4 8.2 A lebre acordou, amigo!
5 9.3 A lebre chega em 2º lugar
```

O que é importante destacar é que o comando `AnyOf` **não** paralisa um evento que ainda não foi processado. No saída exemplo, a tartaruga venceu e o processamento da linha:

```
1 yield lebreAcorda
```

Fez o modelo aguardar até que o evento da lebre acordar fosse processado (no instante determinado quando o evento foi criado). Não houve, portanto, um novo evento criado para a lebre, apenas aguardamos a conclusão do evento já engatilhado na memória, mas não concluído.

16.7 Teste seus conhecimentos

1. Generalize a função `corrida` para um número qualquer de competidores utilizando o operador `**kwargs` visto no Desafio 19.
2. Construa um gráfico com a evolução da prova, isto é, a distância percorrida por cada competidor *vs.* tempo de prova.

17 Propriedades úteis dos eventos

Um evento possui algumas propriedades que fazem a alegria de qualquer leitor:

- `Event.value`: o valor que foi passado para o evento no momento de sua criação;
- `Event.triggered`: `True`, caso o `Event` já tenha sido engatilhado, isto é, ele está na fila de eventos do SimPy e programado para ocorrer em determinado instante da simulação; `False`, caso contrário;
- `Event.processed`: `True`, caso o `Event` já tenha sido executado e `False`, caso contrário.

Existe uma dificuldade inicial que deve ser obrigatoriamente superada pelo programador: compreender a sequência de criação, disparo e execução de um evento em SimPy.

No momento da sua criação, todo evento surge como um objeto na memória do SimPy e, inicialmente, ele encontra-se no estado *não engatilhado* (`Event.triggered = False`). Quando o evento é programado para ocorrer em determinado instante da simulação, ele passa ao estado *engatilhado* (`Event.triggered = True`). Quando o evento é finalmente executado no instante determinado, seu estado passa a processado: (`Event.processed = True`).

Por exemplo, quando você adiciona ao modelo a linha de comando:

```
1 yield env.timeout(10)
```

O SimPy processará a linha na seguinte sequência de passos:

1. Cria na memória um novo evento dentro do `Environment env`;
2. Engatilha o evento para ser processado dali a 10 unidades de tempo (minutos, por exemplo);
3. Quando a simulação atinge o instante de processamento esperado (passados os 10 minutos), o evento é processado. Automaticamente, o comando `yield` recebe o sinal de sucesso do processamento e o fluxo de execução do modelo retoma seu curso normal, processando a linha seguinte do modelo.

A Figura 1 elenca os diversos tipos de eventos que discutimos ao longo deste livro e sua sequência usual de criação->engatilhamento->processamento. Note como eles estão intrinsecamente ligados à questão do tempo de simulação.

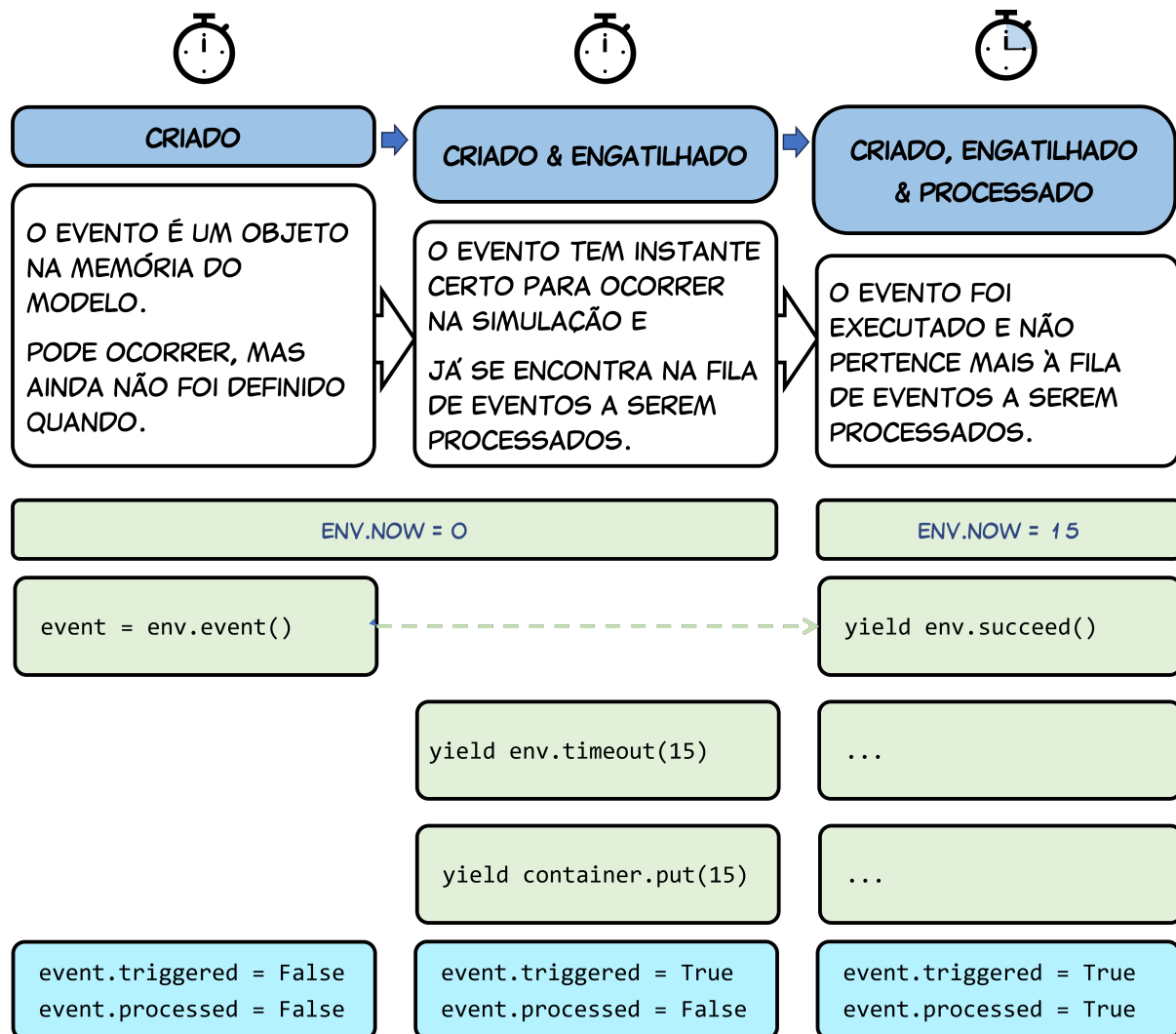


Figura 17.1. Criação, engatilhamento e processamento de eventos.

Antes de avançar—e com o intuito de facilitar o aprendizado do *lebrístico* leitor—acrescentaremos ao modelo da Lebre e da Tartaruga uma função para imprimir as propriedades dos eventos. Basicamente ela recebe uma lista de eventos e imprime na tela as propriedades de cada evento da lista:

```

1 def printEventStatus(env, eventList):
2     # imprime o status das entidades
3     for event in eventList:
4         print('%3.1f value: %s\t programado: %s\t processado: %s'
5               %(env.now, event.value, event.triggered, event.processed))

```

Basicamente, devemos criar uma lista que armazena os eventos de corrida da Lebre e da Tartaruga e imprimir esta lista por meio da função `printEventStatus`:

```

1  # cria os eventos de corrida de cada animal
2  lebreEvent = env.timeout(lebreTempo, value='lebre')
3  tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
4
5  # cria lista de eventos
6  eventList = [lebreEvent, tartarugaEvent]
7  printEventStatus(env, eventList)

```

O modelo completo, com a impressão das propriedades dos eventos, ficaria:

```

1  import simpy
2  import random
3
4  def corrida(env):
5      # a lebre x tartaruga!
6      # sorteia aleatoriamente os tempos dos animais
7      lebreTempo = random.normalvariate(5,2)
8      tartarugaTempo = random.normalvariate(5,2)
9
10     # cria os eventos de corrida de cada animal
11     lebreEvent = env.timeout(lebreTempo, value='lebre')
12     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
13     eventList = [lebreEvent, tartarugaEvent]
14     printEventStatus(env, eventList)
15
16     # começou!
17     start = env.now
18     print('%3.1f Iniciada a corrida!' %(env.now))
19     # simule até que alguém chegue primeiro
20     resultado = yield lebreEvent | tartarugaEvent
21     tempo = env.now - start
22     # quem venceu?
23     if lebreEvent not in resultado:
24         print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo)) \
25
26     elif tartarugaEvent not in resultado:
27         print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
28     else:
29         print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
30     printEventStatus(env, eventList)
31
32  def printEventStatus(env, eventList):
33      # imprime o status das entidades
34      for event in eventList:
35          print('%3.1f value: %s\t programado: %s\t processado: %s'
36                %(env.now, event.value, event.triggered, event.processed))

```

```

37
38 random.seed(10)
39 env = simpy.Environment()
40 env.process(corrida(env))
41 env.run(until=10)

```

Quando simulado, o modelo fornece como resultado:

```

1 0.0 value: lebre          programado: True      processado: False
2 0.0 value: tartaruga      programado: True      processado: False
3 0.0 Iniciada a corrida!
4 5.3 A tartaruga venceu em 5.3 minutos
5 5.3 value: lebre          programado: True      processado: False
6 5.3 value: tartaruga      programado: True      processado: True

```

Repare que, no instante 0.0, os eventos do tipo `timeout` são programados imediatamente após sua criação. Quando a tartaruga vence, por exemplo, o seu evento é considerado processado e do lebre, não. Contudo, o que aconteceria se simulássemos o modelo por mais tempo? De outro modo, se deixássemos o modelo processando por mais tempo, a lebre apareceria na linha de chegada?

No modelo a seguir, acrescentamos à função `corrida` um evento `timeout`, logo após a chegada, que representaria a comemoração da tartaruga por 4 minutos:

```

1 def corrida(env):
2     # a lebre x tartaruga!
3     # sorteia aleatoriamente os tempos dos animais
4     lebreTempo = random.normalvariate(5,2)
5     tartarugaTempo = random.normalvariate(5,2)
6
7     # cria os eventos de corrida de cada animal
8     lebreEvent = env.timeout(lebreTempo, value='lebre')
9     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
10    eventList = [lebreEvent, tartarugaEvent]
11    printEventStatus(env, eventList)
12
13    # começou!
14    start = env.now
15    print('%3.1f Iniciada a corrida!' %(env.now))
16    # simule até que alguém chegue primeiro
17    resultado = yield lebreEvent | tartarugaEvent
18    tempo = env.now - start
19    # quem venceu?
20    if lebreEvent not in resultado:
21        print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo)) \
22
23    elif tartarugaEvent not in resultado:

```



```

24     print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
25 else:
26     print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
27     printEventStatus(env, eventList)
28
29     # a tartaruga inicia a comemoração!
30     yield env.timeout(4)
31     printEventStatus(env, eventList)

```

Como resultado, agora o modelo fornece como saída:

```

1 0.0 value: lebre          programado: True      processado: False
2 0.0 value: tartaruga     programado: True      processado: False
3 0.0 Iniciada a corrida!
4 5.3 A tartaruga venceu em 5.3 minutos
5 5.3 value: lebre         programado: True      processado: False
6 5.3 value: tartaruga     programado: True      processado: True
7 9.3 value: lebre         programado: True      processado: True
8 9.3 value: tartaruga     programado: True      processado: True

```

Repare que no instante 9.3 minutos o evento da lebre está processado. Isto é importantíssimo e não avance sem compreender o que está acontecendo. Quando a linha:

```

1 # simule até que alguém chegue primeiro
2 resultado = yield lebreEvent | tartarugaEvent

```

Aguardou até que um dos eventos tivesse terminado, ela **não cancelou** o outro evento que, por ser um timeout, continuou na fila de eventos até que o SimPy pudesse processá-lo. (Note que o instante 9.3 não representa o instante de processamento do processo, provavelmente a lebre cruzou a linha de chegada após isso).



Até a presente versão do SimPy, não existe a possibilidade de se cancelar um evento já programado.

17.1 Conceitos desta seção

Conteúdo	Descrição
<code>Event.value</code>	o valor que foi passado para o evento no momento de sua criação.

Conteúdo	Descrição
<code>Event.triggered</code>	<code>True</code> , caso o <code>Event</code> já tenha sido engatilhado, isto é, ele está na fila de eventos do SimPy e programado para ocorrer em determinado instante da simulação; <code>False</code> , caso contrário.
<code>Event.processed</code>	<code>True</code> , caso o <code>Event</code> já tenha sido executado e <code>False</code> , caso contrário.

17.2 Desafios



Desafio 25: modifique o modelo anterior para que ele aguarde até a chegada da lebre.

17.3 Solução do desafio 25

Desafio 25: modifique o modelo anterior para que ele aguarde até a chegada da lebre.

Para este desafio, basta alterarmos a parte do modelo que lida com a decisão de quem venceu:

```

1     # quem venceu?
2     if lebreEvent not in resultado:
3         print('%3.1f A tartaruga venceu em %3.1f minutos' %(env.now, tempo))
4         printEventStatus(env, eventList)
5         # aguarda o fim do evento da lebre
6         yield lebreEvent
7         printEventStatus(env, eventList)
8         print('%3.1f A lebre chega em segundo...' %(env.now))
9     elif tartarugaEvent not in resultado:
10        print('%3.1f A lebre venceu em %3.1f minutos' %(env.now, tempo))
11        printEventStatus(env, eventList)
12        # aguarda o fim do evento da tartaruga
13        yield tartarugaEvent
14        printEventStatus(env, eventList)
15        print('%3.1f A tartaruga chega em segundo...' %(env.now))
16    else:
17        print('%3.1f Houve um empate em %3.1f minutos' %(env.now, tempo))
18        printEventStatus(env, eventList)

```

Quando simulado, o modelo anterior fornece:

```

1  0.0 value: lebre          programado: True      processado: False
2  0.0 value: tartaruga     programado: True      processado: False
3  0.0 Iniciada a corrida!
4  5.3 A tartaruga venceu em 5.3 minutos
5  5.3 value: lebre          programado: True      processado: False
6  5.3 value: tartaruga     programado: True      processado: True
7  5.4 value: lebre          programado: True      processado: True
8  5.4 value: tartaruga     programado: True      processado: True
9  5.4 A lebre chega em segundo...

```

Nessa replicação a lebre perdeu por 0,1 minutos apenas. Um cochilada imperdoável da lebre, MEU AMIGO!

Na próxima seção, veremos uma tática mais interessante para resolver o mesmo problema a partir da novidade a ser apresentada, os `callbacks`.

17.4 Teste seus conhecimentos

1. Crie um processo chuva que ocorre entre intervalos exponencialmente distribuídos com média de 5 minutos e dura, em média, 5 minutos exponencialmente distribuídos também. Quando a chuva começa, os corredores são 50% mais lentos durante o período. (Dica: quando a chuva começar, construa novos eventos `timeout` e despreze os anteriores, mas calcule antes quais as velocidades dos corredores).
2. Modifique o modelo para que ele execute um número configurável de replicações e forneça como resposta a porcentagem das vezes em que cada competidor venceu.

18 Adicionando callbacks aos eventos

SimPy possui uma ferramenta tão curiosa quanto poderosa: os `callbacks`. Um `callback` é uma função que você *acrescenta* ao final de um evento. Por exemplo, considere que quando o evento da tartaruga (ou da lebre) termina, desejamos imprimir o vencedor na tela. Assim, quando o evento é *processado*, desejamos que ele processe a seguinte função, que recebe o evento como único parâmetro de entrada:

```
1 def campeao(event):
2     # imprime a faixa de campeão
3     print('%3.1f \o/ Tan tan tan (música do Senna) A %s é a campeã!\n'
4           %(env.now, event.value))
```

Toda função para ser anexada como um `callback`, deve possuir como parâmetro de chamada apenas um único evento. Note também, que o valor de `env.now` impresso na tela é possível pois `env` é uma variável global para o Python. (Caso você ainda tenha alguma dificuldade para entender como o Python lida com variáveis globais e locais, um bom caminho é [ler este bom guia](#)¹.

Para anexarmos a função `campeão` a um evento, utilizaremos o método `callbacks.append( função_criada )`:

```
1     # cria os eventos de corrida de cada animal
2     lebreEvent = env.timeout(lebreTempo, value='lebre')
3     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
4
5     # acrescenta os callbacks
6     lebreEvent.callbacks.append(campeao)
7     tartarugaEvent.callbacks.append(campeao)
```

O código completo do modelo com `callbacks`, ficaria:

¹http://www.python-course.eu/python3_global_vs_local_variables.php

```

1  import simpy
2  import random
3
4  def corrida(env):
5      # a lebre x tartaruga!
6      # sorteia aleatoriamente os tempos dos animais
7      # cria os eventos que disparam as corridas
8      lebreTempo = random.normalvariate(5,2)
9      tartarugaTempo = random.normalvariate(5,2)
10     # cria os eventos de corrida de cada animal
11     lebreEvent = env.timeout(lebreTempo, value='lebre')
12     tartarugaEvent = env.timeout(tartarugaTempo, value='tartaruga')
13
14     # acrescenta os callbacks
15     lebreEvent.callbacks.append(campeao)
16     tartarugaEvent.callbacks.append(campeao)
17
18     # começou!
19     print('%3.1f Iniciada a corrida!' %(env.now))
20     # simule até que alguém chegue primeiro
21     yield lebreEvent | tartarugaEvent
22
23 def campeao(event):
24     # imprime a faixa de campeão
25     print('%3.1f \o/ Tan tan tan (música do Senna) A %s é a campeã!\n'
26           %(env.now, event.value))
27
28
29 random.seed(10)
30 env = simpy.Environment()
31 proc = env.process(corrida(env))
32 env.run(until=10)

```

Note como o código ficou razoavelmente mais compacto, por eliminamos toda a codificação referente aos testes `if...then...else` para determinar que é o campeão.

Quando executado, o modelo fornece como resultado:

```

1  0.0 Iniciada a corrida!
2  5.3 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
3  5.4 \o/ Tan tan tan (música do Senna) A lebre é a campeã!

```

OPS!

Aos 5.4 minutos a lebre, que chegou depois, foi declarada campeã também. Como isso aqui não é a [Federação Paulista de Futebol](https://pt.wikipedia.org/wiki/Campeonato_Paulista_de_Futebol)², temos de corrigir o modelo e garantir que vença sempre quem chegar primeiro.

²https://pt.wikipedia.org/wiki/Campeonato_Paulista_de_Futebol_de_1973

Uma solução prática seria criar uma variável global que se torna True quando o primeiro corredor ultrapassa a linha, de modo que a função `campeao` consiga distinguir se já temos um vencedor ou não:

```
1 import simpy
2 import random
3
4 vencedor = False    # se já temos um vencedor na corrida
```

A função `campeao` agora deve lidar com uma lógica de desvio de fluxo para determinar se evento representa um campeão ou não:

```
1 def campeao(event):
2     global vencedor
3
4     if not vencedor:
5         # imprime a faixa de campeão
6         print('%3.1f \o/ Tan tan tan (música do Senna) A %s é a campeã!'
7               %(env.now, event.value))
8         # atualiza a variável global vencedor
9         vencedor = True
10    else:
11        # imprime o perdedor
12        print('%3.1f A %s chega em segundo lugar...'
13              %(env.now, event.value))
```

Quando simulado, o modelo fornece como saída:

```
1 0.0 Iniciada a corrida!
2 5.3 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
3 5.4 A lebre chega em segundo lugar...
```

Você pode adicionar quantas funções de `callback` quiser ao seu evento, mas lembre-se que manipular um modelo diretamente por eventos tende a deixar o código ligeiramente confuso. Neste caso, a boa prática recomenda não economizar nos comentários.

18.1 Todo processo é um evento

Quando um processo é gerado pelo comando `env.process()`, o processo gerado é automaticamente tratado como um evento pelo SimPy. Você pode igualmente adicionar `callbacks` aos processos em execução ou mesmo retornar um valor.

Por exemplo, vamos acrescentar uma função de `callback` para ao processo `corrida` que informa o final da corrida para o público. A função `final`, a seguir, informa ao público que a corrida chegou ao fim:

```

1 def final(event):
2     # imprime o aviso de final da corrida
3     print('%3.1f Ok pessoal, a corrida acabou.' %env.now)

```

Acrescentar um callback ao processo, é bem semelhante a acrescentar um evento qualquer, basta armazenar o processo em uma variável:

```

1 random.seed(10)
2 env = simpy.Environment()
3 proc = env.process(corrida(env))
4
5 # adiciona ao processo proc a função callback final
6 proc.callbacks.append(final)
7
8 # executa até que o processo corrida termine
9 env.run(proc)

```

Note que, além de adicionarmos a função `final` como callback da função `corrida`, modificamos o comando `env.run()` para que ele simule até que a função `corrida` termine seu processamento. (Experimente substituir a linha `env.run(proc)` por `env.run(until=10)` e verifique o que acontece).

Quando executado, o modelo fornece como saída:

```

1 0.0 Iniciada a corrida!
2 5.3 \o/ Tan tan tan (musica do Senna) A tartaruga é a campeã!
3 5.3 Ok pessoal, a corrida acabou.

```

Uma boa pedida para callbacks é construir funções que calculem estatísticas de termino de processamento ou, como veremos na próxima seção, quando se deseja trabalhar com falhas, paralisações, interrupções de serviços etc.

18.2 Conceitos desta seção

Conteúdo	Descrição
<code>Event.callbacks.append(callbackFunction)</code>	adiciona um callback, representado pela função <code>callbackFunction</code> do modelo. Após o processamento do evento (<code>Event.processed = True</code>) a função de callback é executada. A função <code>callbackFunction</code> , obrigatoriamente deve ter como parâmetro apenas um evento.

18.3 Desafios



Desafio 26: acrescente à função `final` um comando para armazenar quem venceu e em que tempo. Simule para 10 replicações e calcule a média e a porcentagem de vitórias de cada corredor.

18.4 Solução do desafio 26

Desafio 26: acrescente à função `final` um comando para armazenar quem venceu e em que tempo. Simule para 10 replicações e calcule a média e a porcentagem de vitórias de cada corredor.

Neste caso, podemos criar uma lista `resultadoList` para armazenar a `Tuple` (tempo, vencedor) de cada replicação. Os valores podem ser armazenados no instante em que a função `campeão` identifica o vencedor, isto é:

```
1 def campeão(event):
2     global vencedor
3
4     if not vencedor:
5         # imprime a faixa de campeão
6         print('%3.1f \o/ Tan tan tan (musica do Senna) A %s é a campeã!'
7               %(env.now, event.value))
8         # atualiza a variável global vencedor
9         vencedor = True
10        # armazena o resultado na lista resultadoList
11        resultadoList.append((env.now, event.value))
12
13    else:
14        # imprime o perdedor
15        print('%3.1f A %s chega em segundo lugar...'
16              %(env.now, event.value))
```

Como o desafio pede que sejam executadas 10 replicações da corrida, devemos modificar a chamada do modelo para refletir isso. Inicialmente, criamos a lista `resultadoList` e, a seguir, criamos um laço para executar o modelo 10 replicações:

```
1 # lista para armazenar os resultados das corridas/replicações
2 resultadoList = []
3 random.seed(10)
4
5 # processa 10 replicações
6 for i in range(10):
7     # True, se já temos um vencedor na corrida, False caso contrário
8     vencedor = False
9     env = simpy.Environment()
10    proc = env.process(corrida(env))
11    proc.callbacks.append(final)
12    # executa até que o processo corrida termine
13    env.run(proc)
```

Note que a variável global vencedor é atualizada dentro do laço (na seção seguinte, ela era criada logo no cabeçalho do modelo), afinal, a cada replicação precisamos garantir que as variáveis do modelo sejam reinicializadas.

Uma vez que as 10 replicações tenham sido executadas, basta calcular as estatísticas desejadas. O número de vitórias é facilmente extraída da lista de vencedores:

```
1 # separa a lista de resultados em duas listas
2 tempos, vencedores = zip(*resultadoList)
3 # conta o número de vitórias
4 lebreWin = vencedores.count('lebre')
5 tartarugaWin = vencedores.count('tartaruga')
6
7 # calcula a percentagem de vitórias da lebre
8 lebreP = lebreWin/(lebreWin+tartarugaWin)
```

Para o cálculo da média do tempo de corrida, temos diversas possibilidades. A escolhida aqui foi utilizar a função mean da biblioteca numpy:

```
1 # importa a função para cálculo da média a partir biblioteca numpy
2 from numpy import mean
3
4 print('Tempo médio de corrida: %3.1f\tLebre venceu: %3.1f%\tTartaruga venceu: %3\
5 .1f%'\
6       % (mean(tempos), lebreP*100, (1-lebreP)*100))
```

Quando executado, o modelo fornece como resposta:

```
1 0.0 Iniciada a corrida!
2 5.3 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
3 5.3 Ok pessoal, a corrida acabou.
4 0.0 Iniciada a corrida!
5 5.1 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
6 5.1 Ok pessoal, a corrida acabou.
7 0.0 Iniciada a corrida!
8 4.4 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
9 4.4 Ok pessoal, a corrida acabou.
10 0.0 Iniciada a corrida!
11 6.1 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
12 6.1 Ok pessoal, a corrida acabou.
13 0.0 Iniciada a corrida!
14 5.4 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
15 5.4 Ok pessoal, a corrida acabou.
16 0.0 Iniciada a corrida!
17 0.5 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
18 0.5 Ok pessoal, a corrida acabou.
```

```
19 0.0 Iniciada a corrida!
20 4.8 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
21 4.8 Ok pessoal, a corrida acabou.
22 0.0 Iniciada a corrida!
23 3.5 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
24 3.5 Ok pessoal, a corrida acabou.
25 0.0 Iniciada a corrida!
26 4.9 \o/ Tan tan tan (música do Senna) A lebre é a campeã!
27 4.9 Ok pessoal, a corrida acabou.
28 0.0 Iniciada a corrida!
29 5.4 \o/ Tan tan tan (música do Senna) A tartaruga é a campeã!
30 5.4 Ok pessoal, a corrida acabou.
31
32 Tempo médio de corrida: 4.5      Lebre venceu: 60.0%      Tartaruga venceu: 40.0%
```

18.5 Teste seus conhecimentos

1. Acrescente o cálculo do intervalo de confiança ao resultados do tempo médio de corrida e simule um número de replicações suficientes para garantir que este intervalo seja no máximo de 5% em torno da média, para um nível de significância de 95%.
2. Reformule o modelo para que ele aceite uma lista de corredores diferentes, não só uma lebre e uma tartaruga.

19 Interrupções de eventos

De modo semelhante ao que vimos com recursos, os eventos também podem ser interrompidos em SimPy. Como o SimPy aproveita-se dos comandos de interrupção já existentes no Python, pode-se utilizar o bloco `try... except`¹ e assim capturar a interrupção em qualquer parte do modelo.

Considere um exemplo simples em que um jovem *Jedi*² tem sua seção de meditação matinal interrompida por um cruel *Lord Sith*³ interessado em convertê-lo para o Lado Negro.

Construir um modelo de simulação deste sistema, é tarefa simples: precisamos de uma função que representa o Jedi meditando e outra que interrompe o processo em determinado momento (lembre-se que um processo é também um evento para o SimPy).

O SimPy tem dois métodos diferentes para interromper um evento: `interrupt` ou `fail`.

19.1 Interrompendo um evento com o método `interrupt`

Criaremos duas funções: `forca` que representa o processo de meditação e `ladoNegro` que representa o processo de interrupção da meditação. Inicialmente, interromperemos o processo `forca` por meio do método `interrupt`. Uma possível máscara para o modelo ficaria:

```
1 import simpy
2
3 def forca(env):
4     # processo a ser interrompido
5
6 def ladoNegro(env, proc):
7     # gerador de interrupção do processo proc
8
9 env = simpy.Environment()
10
11 forcaProc = env.process(forca(env))
12 ladoNegroProc = env.process(ladoNegro(env, forcaProc))
```

Para este exemplo, o processo de meditação é bem simples, pois estamos mais interessados em aprender sobre interrupções:

¹<https://docs.python.org/3.5/tutorial/errors.html>

²<https://pt.wikipedia.org/wiki/Jedi>

³<https://pt.wikipedia.org/wiki/Sith>

```

1  def forca(env):
2      # processo a ser interrompido
3      while True:
4          yield env.timeout(1)
5          print('%d Eu estou com a Força e a Força está comigo.' % env.now)

```

Portanto, a cada 1 unidade de tempo, o Jedi fala uma frase para manter-se concentrado. O processo de interrupção por sua vez, recebe como parâmetro de entrada o processo (ou evento) a ser interrompido. Apenas para ilustrar melhor o exemplo, vamos considerar que após 3 unidades de tempo, a função interrompe o processo (ou evento) de meditação:

```

1  def ladoNegro(env, proc):
2      # gerador de interrupção do processo proc
3      yield env.timeout(3)
4      print('%d Venha para o lado negro da força, nós temos CHURROS!' % env.now)
5      # interrompe o processo proc
6      proc.interrupt()
7      print('%d Welcome, young Sith.' % env.now)

```

Portanto, a interrupção de um evento ou processo qualquer é invocada pelo método `.interrupt()`. Por exemplo, dado que processo ou evento `proc`, podemos interrompê-lo com a linha de código:

```

1  # interrompe o processo proc
2  proc.interrupt()

```

Se você executar o modelo anterior, a coisa até começa bem, mas depois surge uma surpresa desagradável:

```

1  1 Eu estou com a Força e a Força está comigo.
2  2 Eu estou com a Força e a Força está comigo.
3  3 Venha para o lado negro da força, nós temos CHURROS!
4  3 Welcome, young Sith.
5  Traceback (most recent call last):
6
7      File "<ipython-input-8-623c3bea2882>", line 1, in <module>
8          runfile('C:/Book/Interruption/meditation.py', wdir='C:/Book/Interruption/')
9
10     File "C:\Anaconda3\lib\site-packages\spyderlib\widgets\externalshell\sitecustom\
11     ize.py", line 714, in runfile
12         execfile(filename, namespace)
13
14     File "C:\Anaconda3\lib\site-packages\spyderlib\widgets\externalshell\sitecustom\
15     ize.py", line 89, in execfile
16         exec(compile(f.read(), filename, 'exec'), namespace)
17

```

```

18 File "C:/Book/Interruption/meditation.py", line 22, in <module>
19     env.run()
20
21 File "C:\Anaconda3\lib\site-packages\simpy\core.py", line 137, in run
22     self.step()
23
24 File "C:\Anaconda3\lib\site-packages\simpy\core.py", line 229, in step
25     raise exc
26
27 Interrupt: Interrupt(None)

```

O que essa longa mensagem de erro nos faz lembrar é que o método `.interrupt()` vai além de interromper um mero evento do SimPy, ele interrompe o programa todo.

Mas, jovem leitor Jedi, temos duas maneiras de contornar o problema: com a lógica de controle de exceção do Python, `try...except`, ou com a propriedade `.defused`, como veremos a seguir.

19.1.1 Método de controle de interrupção 1: lógica de exceção `try...except`

Neste caso, a solução é razoavelmente simples, basta acrescentarmos ao final do programa (ou em outra parte conveniente) uma lógica de exceção do SimPy, `simpy.Interrupt`, como no exemplo a seguir:

```

1  import simpy
2
3  def forca(env):
4      # processo a ser interrompido
5      while True:
6          yield env.timeout(1)
7          print('%d Eu estou com a Força e a Força está comigo.' % env.now)
8
9  def ladoNegro(env, proc):
10     # gerador de interrupção do processo proc
11     yield env.timeout(3)
12     print('%d Venha para o lado negro da força, nós temos CHURROS!' % env.now)
13     # interrompe o processo proc
14     proc.interrupt()
15     print('%d Welcome, young Sith.' % env.now)
16
17  env = simpy.Environment()
18
19  forcaProc = env.process(forca(env))
20  ladoNegroProc = env.process(ladoNegro(env, forcaProc))
21
22  try:

```

```

23     env.run()
24 except simpy.Interrupt:
25     print('%d Eu estou com a Força e a Força está comigo.' % env.now)

```

Quando executado, o modelo anterior fornece:

```

1 1 Eu estou com a Força e a Força está comigo.
2 2 Eu estou com a Força e a Força está comigo.
3 3 Venha para o lado negro da força, nós temos CHURROS!
4 3 Welcome, young Sith.
5 3 Eu estou com a Força e a Força está comigo.

```

É importante notar que depois da interrupção `proc.interrupt()` o modelo ainda executa a última linha do processo `ladoNegro` (basicamente, imprime “Welcome, young Sith”) para, a seguir, executar o comando dentro do `except simpy.Interrupt`.

19.1.2 Método de controle de interrupção 2: alterando o atributo `defused`

No caso anterior, o leitor deve ter notado que, ao interromper o processo, interrompemos a simulação por completo, pois nossa lógica de exceção está ao final do código.

E se quiséssemos apenas paralisar o processo (ou evento) sem que isso impactasse em toda a simulação? Neste caso, SimPy fornece um atributo `defused` para cada evento que, quando alterado para `True`, faz com que a interrupção seja “desarmada”.

Vamos alterar o atributo `defused` do processo interrompido no exemplo anterior:

```

1 import simpy
2
3 def forca(env):
4     # processo a ser interrompido
5     while True:
6         yield env.timeout(1)
7         print('%d Eu estou com a Força e a Força está comigo.' % env.now)
8
9 def ladoNegro(env, proc):
10    # gerador de interrupção do processo proc
11    yield env.timeout(3)
12    print('%d Venha para o lado negro da força, nós temos CHURROS!' % env.now)
13    # interrompe o processo proc
14    proc.interrupt()
15    # defused no processo para evitar a interrupção da simulação
16    proc.defused = True
17    print('%d Welcome, young Sith.' % env.now)
18

```



```
19 env = simpy.Environment()  
20  
21 forcaProc = env.process(forca(env))  
22 ladoNegroProc = env.process(ladoNegro(env, forcaProc))  
23  
24 env.run()
```

Quando executado, o modelo anterior fornece:

```
1 1 Eu estou com a Força e a Força está comigo.  
2 2 Eu estou com a Força e a Força está comigo.  
3 3 Venha para o lado negro da força, nós temos CHURROS!  
4 3 Welcome, young Sith.
```

Novamente a execução do processo de interrupção vai até o fim e a interrupção que poderia causar a paralisação de todo o modelo é desarmada.

Portanto, se o objetivo é *desarmar* a interrupção, basta tornar True o atributo `defused` do evento.

19.2 Interrompendo um evento com o método `fail`

Assim como se provoca uma interrupção, pode-se também provocar uma *falha* no evento. O interessante, neste caso, é que podemos informar a falha (todo)

20 O que são funções geradoras? (ou como funciona o SimPy) - Parte I

O comando `yield` é quem, como você já deve ter notado, dá o *ritmo* do seu modelo em SimPy. Para melhor compreender como funciona um programa em SimPy, precisamos entender, além do próprio comando `yield`, outro conceito fundamental em programação: as funções geradoras.

Começaremos pelo conceito de Iterador.

20.1 Iterador

Na programação voltada ao objeto, *iteradores* são *métodos* que permitem ao programa observar os valores dentro de um dado objeto.

Quando você percorre uma lista com o comando `for`, por exemplo, está intrinsecamente utilizando um iterador:

```
1  lista = [10, 20, 30]
2  for i in lista:
3      print (i)
4
5  10
6  20
7  30
```

No exemplo, `lista` é um *objeto* e o comando `for` é um **iterador** que permite vasculhar cada elemento dentro da lista, retornando sempre o elemento seguinte do objeto.

20.2 Funções geradoras

Elas existem e [estão há tempos entre nós...](https://en.wikipedia.org/wiki/Generator_(computer_programming))¹

Uma **função geradora** é uma classe especial de função que tem como característica retornar, cada vez que é chamada, valores em sequência. O que torna uma função qualquer uma *função geradora* é a presença do comando `yield` em seu corpo.

O comando `yield` funciona, a primeira vez que a função é chamada, algo semelhante a um `return`, mas com um **superpoder**: toda vez que a função é chamada novamente, a execução começa a partir da linha seguinte ao `yield`.

Por exemplo, podemos imprimir os mesmo números da lista anterior, chamando 3 vezes a função `seqNum()`, definida a seguir:

¹[https://en.wikipedia.org/wiki/Generator_\(computer_programming\)](https://en.wikipedia.org/wiki/Generator_(computer_programming))

```
1 def seqNum():
2     n = 10
3     yield n
4     n += 10
5     yield n
6     n += 10
7     yield n
8
9 for i in seqNum():
10    print(i)
11
12 10
13 20
14 30
```

Note que a *função geradora* `seqNum` é um *objeto* e que o loop `for` permite acessar os elementos retornados por cada `yield`.

A primeira vez que o loop `for` chamou a função `seqNum()` o código é executado até a linha do `yield n`, que retorna o valor 10 (afinal, $n=10$ neste momento).

A segunda vez que o loop `for` chama a função, ela não recomeça da primeira linha, de fato, ela é executada a partir da **linha seguinte ao comando** `yield` e retorna o valor 20, pois n foi incrementado nessa segunda passagem.

Na terceira chamada à função, a execução retoma a partir da linha seguinte ao segundo `yield` e o próximo valor de n será o anterior incrementado de 10.

Uma função geradora é, de fato, um *iterador* e você normalmente vai utilizá-la dentro de algum *loop for* como no caso anterior. Outra possibilidade é você chamá-la diretamente pelo comando `next` do Python, como será visto no próximo exemplo.

Exemplo: Que tal uma função que nos diga a posição atual de um Zumbi que só pode andar uma casa por vez no plano? A função geradora a seguir acompanha o andar cambaleante do zumbi:

```
1 import random
2
3 def zombiePos():
4     x, y, = 0, 0 # posição inicial do zumbie
5     while True:
6         yield x, y, "Brains!"
7         x += random.randint(-1, 1)
8         y += random.randint(-1, 1)
9
10 zombie = zombiePos()
11
12 print(next(zombie))
13 print(next(zombie))
14 print(next(zombie))
```

Diferentemente do caso anterior, criamos um zumbi a partir da linha:

```
1 zombie = zombiePos()
```

Cada novo passo do pobre infeliz é obtido pelo comando:

```
1 next(zombie))
```

O bacana, no caso, é que podemos criar 2 zumbis passeando pela relva:

```
1 import random
2
3 def zombiePos():
4     x, y, = 0, 0 # zombie initial position
5     while True:
6         yield x, y, "Brains!"
7         x += random.randint(-1, 1)
8         y += random.randint(-1, 1)
9
10 zombie1 = zombiePos()
11 zombie2 = zombiePos()
12 print(next(zombie1), next(zombie2))
13 print(next(zombie1), next(zombie2))
14 print(next(zombie1), next(zombie2))
```

Na seção a seguir, discutiremos o papel da função geradora em um modelo de simulação. Por ora, sugerimos as seguintes fontes de consulta caso você procure um maior aprofundamento sobre o `yield` e as funções geradoras:

- [PEP 225 - Descritivo técnico do yield no Python](https://www.python.org/dev/peps/pep-0255/)²
- [Utilizando Geradores na Python Brasil](http://wiki.python.org.br/UsandoGenerators)³
- Uma boa explicação na StackOverflow: [What does the yield keyword do?](http://stackoverflow.com/questions/231767/what-does-the-yield-keyword-do)⁴

²<https://www.python.org/dev/peps/pep-0255/>

³<http://wiki.python.org.br/UsandoGenerators>

⁴<http://stackoverflow.com/questions/231767/what-does-the-yield-keyword-do>

21 O que são funções geradoras? (ou como funciona o SimPy?) - Parte II

21.1 SimPy vs. funções geradoras

No episódio anterior...

Uma **função geradora** é uma classe especial de funções que têm como característica retornar, cada vez que são chamadas, valores em sequência. O que torna uma função qualquer uma *função geradora* é a presença do comando `yield` em seu corpo.

Para compreendermos a mecânica do SimPy (e da maioria dos softwares de simulação), devemos lembrar que os processos de um modelo de simulação nada mais são que eventos (ou atividades ou ações) que interagem entre si de diversas maneiras, tais como: congelando outro evento por tempo determinado, disparando novos eventos ou mesmo interrompendo certo evento já em execução.

Já sabemos que as entidades e eventos em SimPy são modelados como **processos** dentro de um dado **environment**. Cada processo é basicamente uma função iniciada por `def` como qualquer outra construída em Python, mas que contém a palavrinha mágica `yield`. Assim, como descrito no item anterior, todo **processo** em SimPy é também uma **função geradora**.

Um evento bastante elementar em SimPy é o `timeout()` ou, na sua forma mais usual:

```
1 yield env.timeout(tempo_de_espera)
```

Imagine por um momento que você é a própria encarnação do SimPy, lidando com diversos eventos, processos, recursos etc. Repentinamente, você, Mr. SimPy, depara-se com a linha de código anterior. Mr. SimPy vai processar a linha em duas etapas principais:

1. A palavra `yield` suspende imediatamente o processo ou, de outro modo, impede que a execução avance para linha seguinte (como esperado em toda função geradora);
2. Com o processo suspenso, a função `env.timeout(tempo_de_espera)` é executada e só após o seu derradeiro término, o processamento retorna para a linha seguinte do programa.

Portanto, quando um processo encontra um `yield`, ele é suspenso até o instante em que o evento deve ocorrer, quando o SimPy então *dispara* o novo evento. O que o SimPy faz no caso é **criar um evento a ser disparado** dali a um tempo igual ao `tempo_de_espera`.

Naturalmente, quando num modelo de simulação temos muito eventos interpostos, cabe ao SimPy coordenar os disparos e suspensões dos eventos corretamente ao longo da simulação,

respeitando um calendário único do programa—é nesta parte que você deve se emocionar com a habilidade dos programadores que codificaram o calendário de eventos dentro do SimPy...

Em resumo, SimPy é um controlador de eventos, gerados pelo seu programa. Ele recebe seus eventos, ordena pelo momento de execução correto (ou prioridade, quando existem eventos simultâneos no tempo) e armazena uma lista de eventos dentro do `environment`. Se uma função dispara um novo evento, cabe ao SimPy adicionar o evento na lista de eventos, de modo ordenado pelo momento de execução (ou da prioridade daquele evento sobre os outros).

22 Simulação Baseada em Agentes usando o SimPy

No futuro, virtualmente todas as simulações de computador serão baseadas em agentes devido à naturalidade da representação do agente e à grande similaridade dos modelos de agentes com o paradigma computacional predominante da programação orientada a objetos (North e Macal, 2007)¹.

A Simulação Baseada em Agentes (*Agent Based Modelling-ABM*) é um paradigma de modelagem em que um sistema real é representado computacionalmente por “agentes”, programados para executar um conjunto de regras, que interagem entre si e com o ambiente. O seu campo de aplicação é bem amplo e, de fato, o seu *superpoder* reside na facilidade de representação de sistemas complexos em que certos processos ou comportamentos emergem da interação entre agentes. Considere-se, por exemplo, como formigas buscam alimento para sobrevivência da colônia. Embora, isoladamente, cada formiga toma sua própria decisão de onde buscar o alimento, essa decisão é afetada pela maior ou menor presença do feromônio depositado por outras formigas no caminho. O feromônio toma assim, o papel da troca de mensagens entre os agentes, fazendo com que, em algum momento, as formigas busquem em fila indiana o alimento para a colônia.

Sendo executada num ambiente a eventos discretos, como o SimPy, por exemplo, a simulação baseada em agentes permite a construção de *modelos híbridos* de simulação, que combinam a modelagem de processos com a representação de agentes².

São aplicações típicas da simulação baseada em agentes:

- Propagação de doenças em uma população (pandemias, por exemplo);
- Deslocamentos populacionais devido, por exemplo, ao aumento do preço da moradia, à variação na oferta de emprego, início de guerras etc.;
- Cadeias de suprimentos (cada elo da cadeia—por exemplo: fornecedores, fábricas, centros de distribuição e pontos de demanda—pode ser representado por um agente. É assim que funciona, por exemplo, o [anyLogistix](https://www.anylogicbrasil.com.br/anylogistix/)³);
- Circulação ou evacuação de pedestres;
- Operações de manufatura, centros de distribuição etc.

Se você deseja mergulhar na “simulação de agentes”—como se fala por essas bandas—minha sugestão de literatura básica é o trabalho de [Mancal & North: Tutorial on agent-based modelling and simulation](#)⁴, publicado em 2010. No trabalho, os autores apresentam os conceitos fundamentais e propõem um conjunto de passos para o desenvolvimento de modelos de agentes.

¹Em português, ainda é comum o estrangeirismo “framework” entre profissionais da Ciência da Computação.

²*Script* é uma sequência de comandos executados no interior de algum arquivo por meio de um interpretador. É chamado de *script*, pois eles são lidos e interpretados pelo Python, linha a linha.

³<https://www.anylogicbrasil.com.br/anylogistix/>

⁴<https://rdcu.be/dX2IV>

Uma abordagem mais prática pode ser encontrada no texto [The Big Book of Simulation Modeling](#)⁵. A obra é voltada para o uso do [AnyLogic](#)⁶, mas você encontrará no texto uma ótima introdução à simulação simulação híbrida.

Um breve relato pessoal: minha experiência, ao lidar com estudantes e/ou profissionais formados no dia-a-dia da execução de projetos de simulação, parece comprovar o impacto transformador da modelagem de sistemas por meio de agentes. Com ou sem formação prévia em simulação, quem experimenta o método tende a adotá-lo como abordagem prioritária. Ainda no terreno pessoal, aparenta ser muito sedutora a facilidade de identificar previamente (i.e.: ainda na fase conceitual) quais são os agentes componentes de um sistema, os ambientes em que eles existem, os mecanismos de trocas entre eles etc. Ademais, para aquelas pessoas com algum contato anterior com a Programação Orientada ao Objeto, a Simulação Baseada em Agentes é como algo natural, como se eles já soubessem, tal a sobreposição de conceitos entre os dois métodos.

22.1 Como construir um modelo de simulação de agentes: passos básicos

Como em qualquer *projeto* que envolva simulação, o primeiro passo é sempre identificar qual *a pergunta a ser respondida pelo simulador*. Sem antes saber o que se deseja do simulador, não se deve perder tempo discutindo técnicas, avaliando softwares, coletando dados etc. Reconhecido o objetivo do projeto, o passo seguinte é a construção do modelo conceitual do sistema em estudo para, a seguir, partir para a etapa de codificação do modelo computacional (algo que neste livro é sempre feito em SimPy.)

Para a simulação baseada em agentes, a etapa de modelagem conceitual é naturalmente direcionada para a identificação dos agentes que possam representar o sistema, o ambiente em que eles vivem e como se relacionam entre si.

A Figura 22.1 apresenta uma sequência mínima de perguntas que você deve responder para modelar conceitualmente seu sistema *antes* de iniciar qualquer linha de código em Python (adaptado de Mancal & North, 2010).

⁵<https://www.anylogic.com/resources/books/big-book-of-simulation-modeling/>

⁶<https://anylogicbrasil.com.br>

ETAPA	PERGUNTAS	RESPOSTAS
OBJETIVOS	• QUAL A PERGUNTA QUE O MODELO DEVE RESPONDER?	
AGENTES E ESTADOS	• QUAIS SÃO OS AGENTES DO MODELO? • QUAIS SÃO OS ESTADOS QUE ESTES AGENTES PODEM ASSUMIR? • COMO UM AGENTE PASSA DE UM ESTADO PARA OUTRO? • QUAIS AGENTES TOMAM DECISÕES?	
DADOS	• QUAIS DADOS DESCREVEM OS AGENTES? • QUAIS ATRIBUTOS DOS AGENTES SERIAM CALCULADOS PELO MODELO? • QUAL SERIA O AMBIENTE DOS AGENTES? • OS AGENTES SE MOVIMENTAM ESPACIALMENTE PELO AMBIENTE?	
COMPORTAMENTO	• QUAIS OS COMPORTAMENTOS DE INTERESSE DOS AGENTES? • QUAIS DECISÕES SÃO TOMADAS PELOS AGENTES?	
INTERAÇÕES	• COMO OS AGENTES INTERAGEM ENTRE SI E COM O AMBIENTE?	
EXPERIMENTAÇÃO	• COMO VOCÊ VALIDARIA O COMPORTAMENTO DESSES AGENTES NO MODELO? • COMO VOCÊ CONSTRUIRIA UM CONJUNTO DE EXPERIMENTOS PARA RESOLVER O PROBLEMA ORIGINAL	

Figura 22.1. Etapas de modelagem conceitual de um simulador baseado em agentes.

Vamos aplicar estes passos modelando um problema clássico da simulação: o modelo SIR.

22.2 Modelo epidêmico baseado em agentes: o modelo SIR⁷

Em um reino moram exatamente 10.000 habitantes distribuídos *uniformemente* por uma área quadrada de 10 km por 10 km. Os habitantes do reino têm contatos diários uns com os outros, limitados às pessoas vivendo dentro de um raio máximo de 1 km de distância de onde vivem.

⁷Baseado em: Borshchev, A. e Grigoryev, Ilya. The Big Book of Simulation Modeling. Multimethod modeling with AnyLogic 8. AnyLogic North America. 2020.



PAUSA DRÁMATICA: tentando superar um longo dia de tédio, uma bruxa má decide enfeitiçar 10 habitantes escolhidos aleatoriamente com um vírus encantado e provocar assim, uma pandemia de pessoas enfeitiçadas!

Ao ser **infectado** ou **enfeitiçado**, o habitante torna-se imediatamente um *agente* propagador do vírus. A duração da fase infecciosa é uniformemente distribuída entre 7 e 21 dias. Durante este período, cada habitante entra, a cada dia, em contato com outros 5 habitantes.

Se uma pessoa infectada entra em contato com uma pessoa que ainda não foi infectada ou **suscetível**, esta ficará infectada 10% das vezes.

Após a fase infecciosa, a pessoa é considerada **recuperada** e torna-se imune ao feitiço.

O objetivo do problema é descobrir as curvas epidêmicas do reino, isto é: o número de pessoas expostas, infectadas e recuperadas ao longo do tempo. Para atingir tal objetivo, construa dois gráficos:

- O mapa do reino, com cada agente posicionado na sua respectiva localização. Para representar as mudanças de estado ao longo da simulação, acrescente um *slider* que permita variar o dia representado no gráfico;
- As curvas epidêmicas (número de pessoas suscetíveis, infectadas e recuperadas) ao longo de um período de 50 dias.

O problema proposto é uma adaptação de modelos comportamentais epidemiológicos, mais precisamente do **modelo SIR**⁸. Embora tais modelos sejam muitas vezes solucionados por meio de modelos de simulação de dinâmica de sistemas, o problema proposto aqui carrega algum realismo que não é bem capturado pelos modelos epidemiológicos clássicos: a localização espacial de cada habitante, o contágio dependente da proximidade entre habitantes, a duração uniformemente distribuída para a fase infecciosa e, mais importante, a presença de uma bruxa.

⁸https://en.wikipedia.org/wiki/Compartmental_models_in_epidemiology

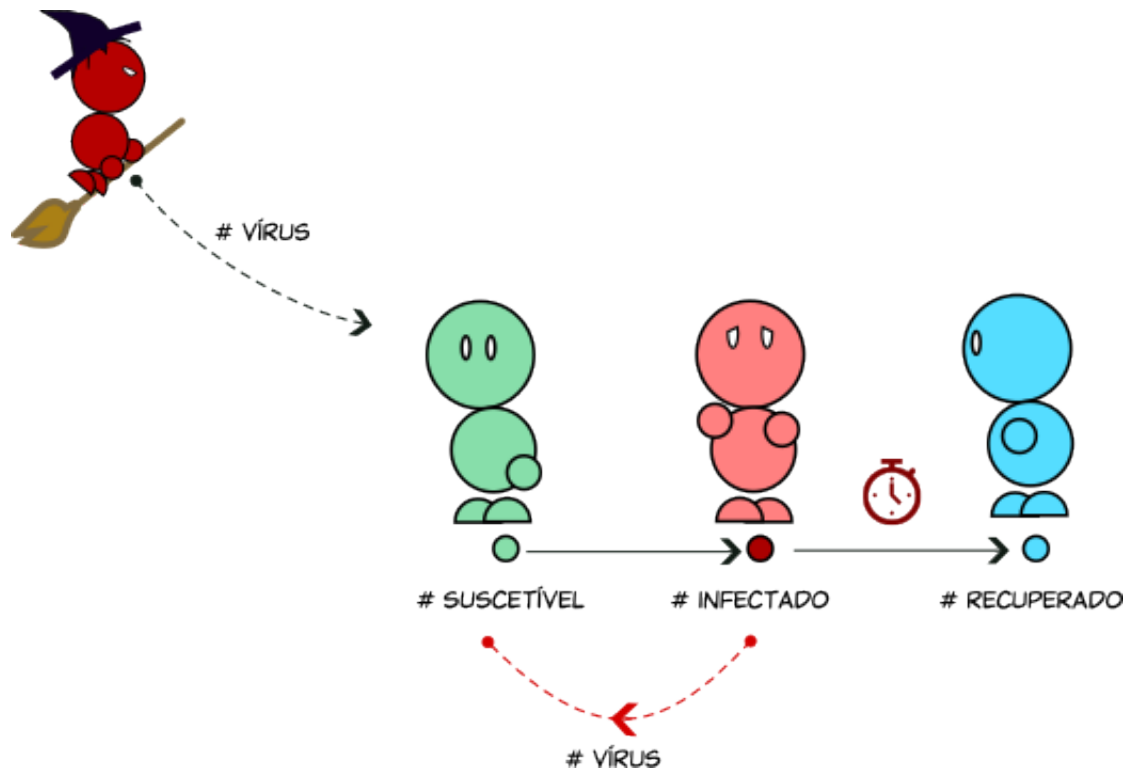


Figura 22.2. Estados dos agentes do problema.

Mãos à obra: comece aplicando os passos sugeridos na Figura 22.1 ao problema proposto!

1. Objetivos

Qual(is) a(s) pergunta(s) que o modelo deve responder?

O objetivo do problema é descobrir a sua curva epidêmica, isto é: o número de pessoas expostas, infectadas e recuperadas ao longo do tempo.

2. Agentes e estados

Quais são os agentes representativos do modelo?

Um agente é uma pessoa ou habitante do reino.

Quais são os estados que estes agentes podem assumir?

*Cada agente pode assumir apenas 3 **estados**: suscetível, infectado ou recuperado.*

Como um agente passa de um estado para outro?

Mudanças de estados:

- *Um agente muda do estado suscetível apenas para o estado infectado. Esta transição ocorre em decorrência do contato com outra pessoa que esteja no estado infectado;*
- *Do estado infectado, o agente muda para o estado recuperado automaticamente após decorrido um certo tempo de infecção.*

Quais agentes tomam decisões?

*Não existem agentes tomadores de decisão no sistema (seria o caso da bruxa, mas ela é um **chiste**⁹ criado pelo autor do problema).*

⁹https://pepsic.bvsalud.org/scielo.php?script=sci_arttext&pid=S1982-12472019000300007

3. Dados

Quais dados descrevem os agentes?

*Para a modelagem do sistema, são relevantes os seguintes dados: * A localização inicial de cada agente; * O número de contatos diários; * A distância máxima que permite o contato; * A duração do período de infecção.*

Quais atributos dos agentes seriam calculados pelo modelo?

Os atributos calculado pelo modelo seriam:

- *O estado do agente (suscetível, infectado ou recuperado) e*
- *O seu estado inicial.*

Qual seria o ambiente dos agentes?

O ambiente é o reino, definido pela área de 10 km x 10 km.

Os agentes se movimentam espacialmente pelo ambiente?

Não existe propriamente algum tipo de interação com o meio ou movimento dos agentes.

4. Comportamento

Quais os comportamentos de interesse dos agentes?

Diariamente, os agentes entram em contato com até 5 pessoas, em um raio de 1 km.

Quais decisões são tomadas pelos agentes?

Os agentes não tomam decisões (seria o caso, por exemplo, se o agente decidisse se isolar durante o período de infecção ou procurar a ajuda hospitalar).

5. Interações

Como os agentes interagem entre si e com o ambiente?

Os agentes entram em contato direto um com os outros, de modo aleatório, dentro de um raio de 1 km.

6. Experimentação

Como você validaria o comportamento desses agentes no modelo?

A validação será por confronto da curva pandêmica real com o resultado da simulação.

Como você construiria um conjunto de experimentos para resolver o problema original?

Será construída a curva pandêmica da população.

22.3 Modelagem do problema no SimPy

O uso do SimPy para modelagem de agentes apoia-se no fato do Python ser uma linguagem que permite a Programação Orientada ao Objeto-POO. Basicamente, as classes da POO são, na simulação baseada em agentes, *classes construtoras de agentes*. É importante notar que, todo modelo de agentes é também um modelo de simulação de eventos discretos, embora o contrário não seja sempre verdadeiro.

Dado que o *script* em Python para modelagem do problema será um pouco longo, a proposta aqui é dividi-lo em 6 seções, facilitando sua discussão por etapas:

- **Importação das bibliotecas** necessárias para o modelo;
- **Parâmetros e variáveis globais** do modelo;
- **Construtores dos Agentes**;
- **Processo de monitoramento da simulação**: coleta de estatísticas realizadas ao longo da execução do modelo;
- **Dashboard**: geração de gráficos;
- **Execução**: criação do environment, dos agentes etc.

Note que as seções propostas são suficientemente genéricas para servir como um *template* de qualquer modelo de simulação em SimPy que você venha a construir.

22.3.1 Importação de bibliotecas

Para este modelo, além das bibliotecas *simpy* e *random*, serão necessárias:

- O API (*Application Programming Interface*) *pyplot*¹⁰, para a geração dos gráficos da pandemia e
- A biblioteca *matplotlib* disponibiliza um *Slider*¹¹, basicamente um botão deslizante que utilizaremos para navegar pelos dias simulados.

O objetivo é desenhar um mapa com os diversos agentes distribuídos pela região e, por meio do *Slider*, navegar pelos dias da pandemia, visualizando o estado dos agentes a cada dia.

Assim, a seção de importação de bibliotecas ficaria:

```
1 import simpy
2 import random
3
4 import matplotlib.pyplot as plt
5 from matplotlib.widgets import Slider
```

22.3.2 Parâmetros de entrada e variáveis globais

Neste caso, alguns parâmetros são fornecidos no problema: população total, largura x altura da área em que os agentes vivem e a duração da epidemia:

```
1 # parâmetros do modelo
2 POPTOTAL = 10000      # população total
3 DUR_SIM = 50          # duração da simulação
4 XMAX = 10             # largura do reino
5 YMAX = 10             # altura do reino
```

As variáveis globais normalmente dizem respeito aos valores que você necessita armazenar *ao longo da simulação*. Alguns destes valores são fáceis de identificar como, por exemplo: o número de agentes infectados, recuperados e suscetíveis:

¹⁰<https://matplotlib.org/stable/tutorials/pyplot.html>

¹¹https://matplotlib.org/stable/gallery/widgets/slider_demo.html

```

1  # variáveis globais
2  infectados = 0          # numero atual de infectados
3  recuperados = 0        # numero atual de recuperados
4  suscetiveis = POPTOTAL # numero atual de suscetíveis

```

Outro conjunto de variáveis globais são valores coletados para análises estatísticas ou *monitoramento* do modelo. Usualmente, os valores apresentados em gráficos ou estatísticas relevantes são coletados ao longo do modelo e processados ao final da simulação. Assim, no caso do exemplo, podemos utilizar *listas* para armazenar as curvas do número de agentes nos diferentes estados a cada dia. Já para a construção do mapa, utilizaremos um *dicionário*, pois precisamos armazenar, para cada agente, sua localização (x, y) e o seu estado atual (suscetível, infectado ou recuperado).

```

1  # listas para o número de pessoas em cada estado a cada dia
2  numS, numI, numR, x = [POPTOTAL], [0], [0], [0]
3
4  # dicionario para armazenar o estado de cada agente ao longo da simulação
5  # para uso na construção do gráfico do mapa de agentes
6  popDayDict = {}

```

Em qualquer simulador de agentes é praticamente certo que os agentes criados devem ser armazenados em alguma estrutura de dados para acesso rápido dos seus estados, atributos, processos etc. Para o modelo do reino, utilizaremos uma *lista* para armazenar a população de agentes criados:

```

1  # lista que armazena a população de agentes
2  popList = []

```

Note que, se quiséssemos criar os agentes *bruxas*, elas poderiam ser armazenadas em uma lista à parte, `bruxasList[]`, por exemplo.

22.3.3 Construtor de agentes

Os agentes são gerados por uma **Classe Agent** e começa aqui a nossa aventura com a **Programação Orientada ao Objeto** em Python. Todo construtor de agentes tem, ao menos, quatro métodos:

- **Inicialização do agente:** responsável por criar o agente, criar/atribuir parâmetros ao agente e inicializar o procedimento de envios de mensagens;
- **Envio de mensagens:** procedimento de envio de mensagens para outros agentes;
- **Recebimento de mensagens:** procedimento que processa a mensagem recebida;
- **Processos específicos:** usualmente, todo agente tem um ou mais processos que são executados internamente, provocados ou não por mensagens recebidas.

O *template* a seguir tem os 4 métodos descritos e pode ser adaptado facilmente para qualquer agente (pessoa, bruxa, rei, automóvel etc.):

```
1 class Agent():
2     def __init__(self, env, agent_id):
3         # cria um agente no environment env
4
5         # parâmetros do agente
6
7         # inicia o processo de envio de mensagens
8         env.process(self.send_msg())
9
10    def send_msg(self):
11        # envia mensagens para outros agentes
12        while True:
13            # aguarda um dia
14
15            # se o agente estiver no estado 1 (infectado) ele tenta infectar
16
17    def receive_msg(self):
18        # recebe mensagens
19
20    def proc_incubacao(self):
21        # processo de incubação
22
23        # muda estado e cor do agente
24
25        # sorteia o tempo de incubação
26
27        # muda o estado e cor do agente
```

A função **init** é sempre executada quando um novo objeto/agente da Classe Agent é criado. Ela é uma função de inicialização do agente, ou seja, onde são colocados todos os comandos de carregam os atributos iniciais ou executam os primeiros processos do agente. Uma breve introdução às classes no Python pode ser consultada na [W3School](https://www.w3schools.com/python/python_classes.asp)¹².

22.3.4 Inicialização

A primeira parte da classe inicializa um novo agente e este é o tipo de coisa que você espera encontrar na maioria dos construtores de agentes em um modelo de simulação:

- O **environment** do agente;
- Um identificador ou **id** do agente;
- O **estado** inicial do agente e, se necessário para saídas gráficas, um atributo associado de **cor** do agente;
- A **localização** do agente: no caso, suas coordenadas cartesianas (x , y). Mas poderiam ser outras formas de representação espacial, tais como: coordenadas georreferenciadas (GIS), polares, matriciais etc.;

¹²https://www.w3schools.com/python/python_classes.asp

- O disparo de um ou mais **processos** iniciais: no caso, o envio de mensagens para outros agentes.

O código a seguir, apresenta a função `__init__` com os parâmetros iniciais de cada agente. Para representação dos estados do agente, o parâmetro `self.state` pode assumir os valores: 0, 1 ou 2, representando os estados **suscetível**, **infectado** e **recuperado**. Adicionalmente, o parâmetro `self.color` armazena uma *string* com as cores de cada estado: 'g' (*green* ou verde), 'r' (*red* ou vermelho) ou 'b' (*blue* ou azul), respectivamente para os estados **suscetível**, **infectado** e **recuperado**.

```

1 class Agent():
2     def __init__(self, env, agent_id):
3         # cria um agente no environment env
4
5         # parâmetros do agente
6         self.env = env                # environment do agente
7         self.agent_id = agent_id      # id do agente
8         self.state = 0                # estado do agente {0, 1 ou 2}
9         self.color = 'g'              # cor do estado {'g', 'r' ou 'b'}
10        self.x = random.uniform(0, XMAX) # coordenada x
11        self.y = random.uniform(0, YMAX) # coordenada y
12
13
14        # inicia o processo de envio de mensagens
15        env.process(self.send_msg())

```

A localização (x, y) de cada agente é obtida por meio do sorteio de duas distribuições uniformes, uma para cada coordenada, limitada a largura e altura da região (linhas 10 e 11).

Como último comando de inicialização, disparamos o processo de envio de mensagens por meio do comando `env.process(self.send_msg())` na linha 15. Note que o método `send_msg` ainda será codificado mais à frente.

22.3.5 Envio e recebimento de mensagens

Dia após dia, cada agente encontra outros 5 agentes em um raio de 1 km. No código a seguir, um “encontro” entre agentes é representado pelo envio de uma mensagem do agente atual para um outro agente selecionado na lista de agentes da população.


```

1     def send_msg(self):
2         # envia mensagens para outros agentes
3         while True:
4             # aguarda um dia
5             yield self.env.timeout(1)
6
7             # se o agente estiver no estado 1 (infectado) ele tenta infectar
8             if (self.state == 1):
9                 i = 0
10                while (i < 5):
11                    randAgent = random.choice(popList)
12                    if ((self.x-randAgent.x)**2 + (self.y-randAgent.y)**2 <= 1):
13                        # envia uma mensagem para algum agente da população
14                        randAgent.receive_msg(self)
15                        i += 1

```

Como feito em diversos exemplos neste livro, um *loop* `while True` (linha 3) governa o processo de envio dessas mensagens, uma vez por dia (linha 5).

É relevante notar, que só é de interesse para o modelo as mensagens enviadas a partir de agentes infectados, pois essas são as únicas que podem causar algum problema, como representado no comando de desvio `if (self.state == 1)` (linha 8).

Por fim, o agente se encontra com 5 outros aleatoriamente escolhidos (linha 7), mas eles devem estar a uma distância máxima de 1 km entre si (linha 12).

Selecionado um agente, ele é armazenado na variável `randAgent` e chegamos na parte mais bacana: o envio da mensagem é, de fato, a execução do comando de recebimento da mensagem no outro agente.

O comando da (linha 14) dispara a função de recebimento de mensagens no agente visitado `randAgent`, com agente atual como o parâmetro de entrada da função. De outro modo, a linha de comando `randAgent.receive_msg(self)` provoca a troca de mensagens entre os agentes, tal qual um e-mail recebido e aberto pelo destinatário.

22.3.6 Processo de infecção

```

1     def receive_msg(self):
2         # recebe mensagens
3         # se o agente que recebeu a mensagem e esta suscetível
4         if (self.state == 0):
5             # contamina o agente atual 10% das vezes
6             if(random.random() <= 0.1):
7                 # altera o estado do agente para 1
8                 self.state = 1
9                 # inicia o processo de incubação
10                self.env.process(self.proc_incubacao())

```

Ao receber a mensagem (ou o encantamento ou o vírus), um agente somente poderá ficar infectado, somente no caso em que ele esteja no estado “0”, suscetível. Mesmo assim, ainda existe uma probabilidade de 10% (linha 7) do agente inicializar o processo de incubação (linha 8).

22.3.7 Processo de incubação

O processo de incubação envolve três etapas: mudança de estado para infectado, sorteio de um tempo de incubação e, ao término deste tempo, mudança para o estado recuperado.

```
1     def proc_incubacao(self):
2         # processo de incubação
3         global suscetiveis
4         global infectados
5         global recuperados
6
7         # muda estado e cor do agente
8         self.state = 1
9         self.color = 'r'
10        suscetiveis -= 1
11        infectados += 1
12
13        # sorteia o tempo de incubação
14        yield self.env.timeout(random.uniform(7, 21))
15
16        # muda estado e cor do agente
17        self.state = 2
18        self.color = 'b'
19        recuperados += 1
20        infectados -= 1
```

Inicialmente, portanto, o agente *muda* seu estado de suscetível para infectado (linhas 8 e 9) e devemos decrementar a variável global do número de agentes no estado suscetível (linha 10) e incrementar a variável global de agentes infectados (linha 11).

Para representar o tempo de incubação, um processo *uniformemente* distribuído entre 7 e 21 dias é disparado na linha 13. Ao fim do processo de incubação, o agente muda seu estado para recuperado e as variáveis globais dos número de agentes recuperados e infectados são atualizadas (linhas 16 a 19).

22.3.8 Monitoramento e exibição de gráficos

O monitoramento da simulação será responsável por armazenar as informações necessárias para a construção dos gráficos das curvas pandêmicas. Serão duas funções: `save_pandemic`, para salvar os valores coletados ao longo da simulação e `dashboard` para apresentar os gráficos na tela.

Um possível *templante* para a função `save_pandemic`, seria:

```

1  def save_pandemic():
2      # a cada dia, armazena as informações das curvas pandêmicas
3      # e estados das pessoas
4
5      # define as variáveis globais
6
7      while True:
8          # carrega as listas de suscetíveis, infectados e recuperados
9
10         # armazena no dicionário da população, a posição e a cor/estado
11         # de cada pessoa em um Tuple (pos x, pos y, cor)
12
13         # aguarda ate o próximo dia
14

```

A função `save_pandemic` é representada por um processo que, a cada dia, adiciona quantos agentes estão no estado suscetível, infectado e recuperado, respectivamente nas listas: `numS`, `numI` e `numR` (linhas 6 a 8). Adicionalmente, para que o mapa de estados dos agentes tenha suas cores armazenadas, um dicionário, `popDayDict`¹³, armazena uma lista de todos os agentes por meio de *Tuples* de valores representando as coordenadas (*x*, *y*) dos agentes e suas respectivas cores (linha 10).

Note como o processo de monitoramento deve ser executado em paralelo ao simulador, como serviço independente de monitoramento que captura valores do *environment* em execução.

```

1  def save_pandemic():
2      # a cada dia, armazena as informações das curvas pandêmicas
3      # e estados das pessoas
4
5      # define as variáveis globais
6      global suscetiveis
7      global infectados
8      global recuperados
9
10     while True:
11         # carrega as listas de suscetíveis, infectados e recuperados
12         numS.append(suscetiveis)      # armazena os suscetíveis do dia
13         numI.append(infectados)       # armazena os infectados do dia
14         numR.append(recuperados)      # armazena os recuperados do dia
15         x.append(env.now)             # armazena o numero do dia
16
17         # armazena no dicionário da população, a posição e a cor/estado
18         # de cada pessoa em uma lista de Tuples (pos x, pos y, cor)
19         popDayDict[env.now] = [ (p.x, p.y, p.color) for p in popList]
20

```

¹³A escolha por um dicionário é meramente didática. Fica como exercício para o entusiasmado leitor: identificar um modo mais fácil de armazenar os valores, outro mais compacto e, por fim, um modo mais obscuro.

```

21     # aguarda ate o próximo dia
22     yield (env.timeout(1))

```

A construção de gráficos é uma necessidade básica de modelos de simulação (como a geração de arquivos de log ou de saídas). Quando a ideia é produzir gráficos melhor elaborados, um bom caminho é *esboçar* a disposição das imagens, gráficos e tabelas, antes de começar a codificação propriamente dita.

Por exemplo, a Figura 22.2 apresenta a ideia de tela de saída do modelo:

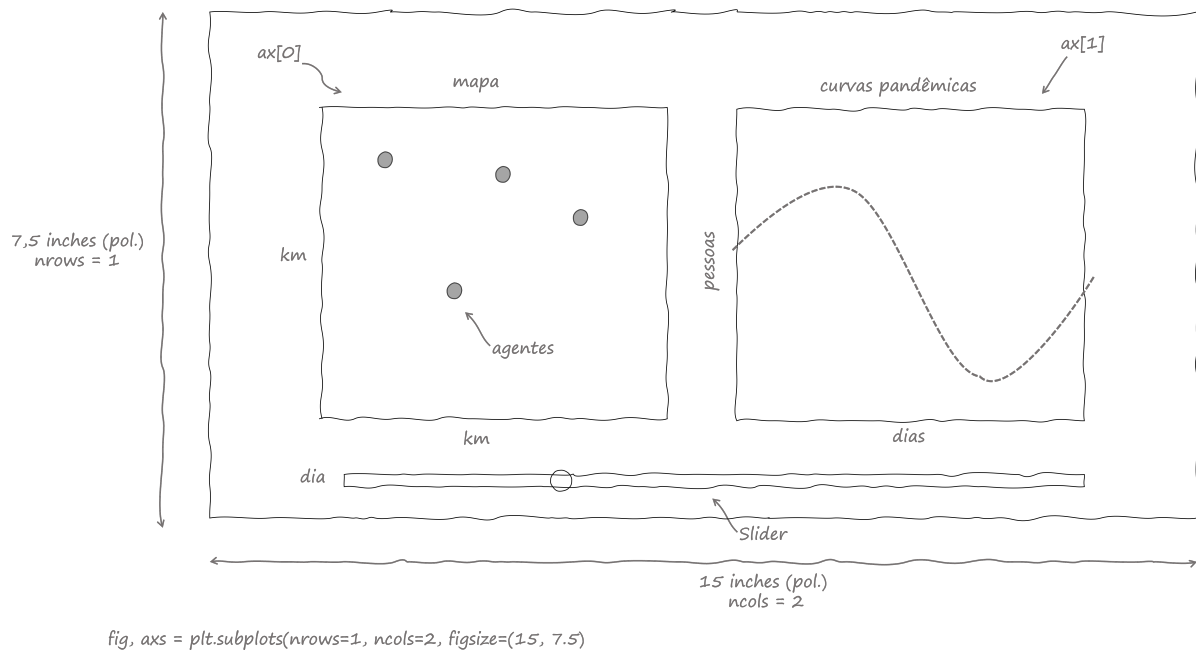


Figura 22.3. Esboço de interface de saída.

O *template* a seguir, detalha todas os comandos necessários para que os gráficos do mapa diário e das curvas pandêmicas apareçam na mesma tela. O leitor já deve saber que o código apresentado é apenas uma dentre as infinitas formas de codificação do problema (fica como exercício: que tal criar os gráficos diretamente uma planilha eletrônica?).

```

1  def dashBoard():
2      # crie uma função chamada quando o slider eh alterado
3      def update_plt(val):
4          # capture o valor desejado do slider
5
6          # cria a figura com dois gráficos um ao lado do outro de tamanho 15x7,5 pol
7
8          # set de parâmetros do gráfico 1, da esquerda
9
10         # plota as curvas pandêmicas

```

```

11
12     # cria e adiciona um slider no gráfico 0
13
14
15     # quando o slider sofrer uma alteração de valor,
16     # chama a função de atualização do gráfico
17
18
19     # plota o gráfico do dia 0 ao iniciar a plotagem
20
21     # apresenta a figura na tela

```

A seguir, é apresentado o código completo da função.

```

1  def dashBoard():
2      # cria uma função chamada quando o slider eh alterado
3      def update_plt(val):
4          # captura o valor desejado do slider
5          day = int(slider_day.val)
6
7          # limpa o gráfico 0 antes de substituir a figura
8          axs[0].cla()
9
10         # recupera a posição e a cor/estado da pessoa
11         x_pos, y_pos, color = zip(*popDayDict[day])
12
13         # plota o agente no espaço e corrige os limites dos eixos e labels
14         axs[0].scatter(x_pos, y_pos, color=color, s=1)
15         axs[0].set_title('Mapa')
16         axs[0].set_xlabel('km')
17         axs[0].set_ylabel('km')
18         axs[0].set_xlim(0, 10)
19         axs[0].set_ylim(0, 10)
20         fig.canvas.draw_idle()
21
22         # pausa pequena, em segundos, para atualizar o gráfico
23         plt.pause(0.1)
24
25     # cria a figura com dois gráficos um ao lado do outro de tamanho 15x7,5 pol
26     fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(15, 7.5))
27
28     # um pequeno ajuste para reservar espaço ao slider
29     plt.subplots_adjust(bottom=0.25)
30
31     # set de parâmetros do gráfico 1, da esquerda
32     axs[1].set_title('Curvas pandêmicas')

```

```

33     axs[1].set_xlabel('Dias')
34     axs[1].set_ylabel('Pessoas')
35     axs[1].set_xlim([0,DUR_SIM])
36     axs[1].set_ylim([0,POPTOTAL])
37
38     # plota as curvas pandêmicas
39     axs[1].plot(x, numS, label='Suscetíveis', color='b', linewidth=2)
40     axs[1].plot(x, numI, label='Infectados', color='r', linewidth=2)
41     axs[1].plot(x, numR, label='Recuperados', color='g', linewidth=2)
42     axs[1].legend()
43
44     # cria e adiciona um slider no gráfico 0
45     # [pos x, pos y, comprimento, altura] em pixels
46     axs[0].slide = plt.axes([0.15, 0.1, 0.75, 0.05])
47     slider_day = Slider(axs[0].slide,
48                         'Dia',                                # texto do slider
49                         valmin=0, valmax=DUR_SIM-1,          # min, max do slider
50                         valinit=0,                            # valor inicial
51                         valstep=1)                            # passo do slider
52
53     # quando o slider sofrer uma alteração de valor,
54     # chama a função de atualização do gráfico
55     slider_day.on_changed(update_plt)
56
57     # plota o gráfico do dia 0 ao iniciar a plotagem
58     update_plt(0)
59
60     # apresenta a figura na tela
61     plt.show()
62

```

Sobre o código anterior, alguns pontos merecem destaque:

- Para que o Slider modifique o gráfico a cada deslize do botão, o **matplotlib** exige que o usuário crie uma função de *atualização de valores*, no caso, a função `update_plt(val)`;
- A primeira plotagem do gráfico da esquerda, `axs[0]`, é feita diretamente por meio da chamada da função `update_plt(0)` com o valor 0 na linha 58;
- O comando `plt.pause(0.1)` é necessário, pois garante um tempo mínimo para que o mapa seja regenerado na tela. Pode ser que em computadores mais lentos ou mais rápidos, este valor tenha de ser aumentado ou diminuído.

22.3.9 Execução (ufa!)

O próximo *template* apresenta as etapas de execução do modelo. Algo que, nesta altura do aprendizado, o nobre leitor já deve conseguir programar esta etapa sem grandes dificuldades:

```
1 env = simpy.Environment()
2
3 # cria os agentes e a população
4 for i in range(POPTOTAL):
5
6     # adiciona o agente a lista da população
7
8 # cria os contatos dos agentes a 1 km de dist
9
10 # a bruxa infecta 10 pessoas aleatoriamente escolhidas
11
12 # inicia o processo de monitoramento de valores
13 env.process(save_pandemic())
14
15 # inicia a execução do modelo
16 env.run(DUR_SIM)
17
18 # inicia a apresentação de resultados
19 dashBoard()
```

Preenchido, o código de execução fica:

```
1 env = simpy.Environment()
2
3 # cria os agentes e a população
4 for i in range(POPTOTAL):
5     agent = Agent(env, i)
6
7     # adiciona o agente a lista da população
8     popList.append(agent)
9
10 # a bruxa infecta 10 pessoas aleatoriamente escolhidas
11 i = 0
12 while (i < 10):
13     escolhido = random.choice(popList)
14     env.process(escolhido.proc_incubacao())
15     i += 1
16
17 # inicia o processo de monitoramento de valores
18 env.process(save_pandemic())
19
20 # inicia a execução do modelo
21 env.run(DUR_SIM)
22
23 # inicia a apresentação de resultados
24 dashBoard()
```

Executado, o modelo retorna uma figura com o mapa de agentes e as curvas pandêmicas:

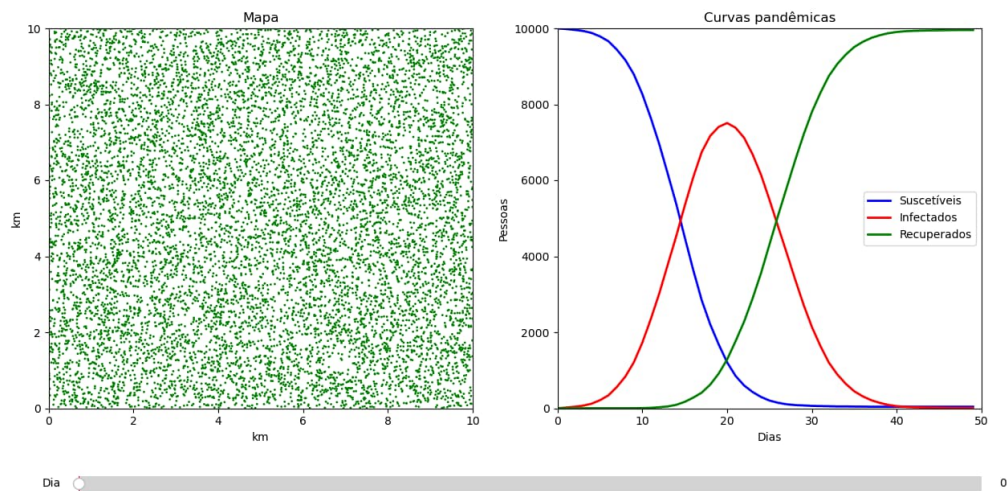


Figura 22.4. Saída do modelo ao final da simulação.

Com o *Slider* é possível navegar pelos dias da pandemia e analisar a propagação do vírus ao longo dos dias (você conseguiria visualizar os 10 primeiros agentes infectados?)

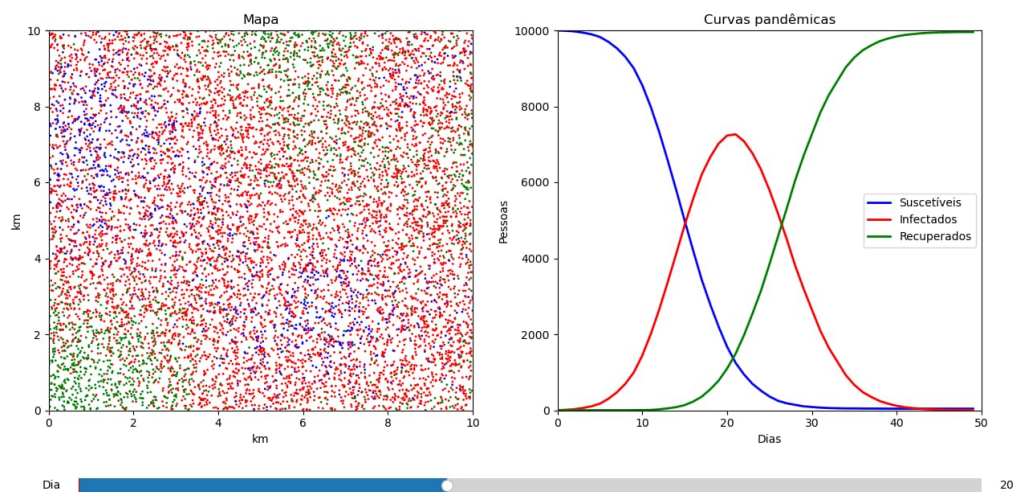


Figura 22.5. Saída do modelo para o dia 20.

É provável que o modelo seja um pouco lento no seu computador. Isto não é algo incomum:

modelos de simulação são compostos por códigos em que muitos comandos se repetem a cada dia, hora, minuto, segundo, milissegundo etc. Embora esta seja uma realidade conhecida por qualquer profissional de simulação, a provável alta carga de processamento não pode ser desculpa para um código descuidado. Antes de partir para o uso de ferramentas de *profiling*, existe algo que você mesmo pode fazer: investigue o modelo em busca de algum processo, comando, cálculo que esteja se repetindo desnecessariamente. Além de fazer bem para o seu processo de aprendizagem, você ainda melhora a qualidade do seu código.

A seção a seguir propõem um método bastante eficaz para resolver esse problema. Mas antes, um resumo do simulador construído:

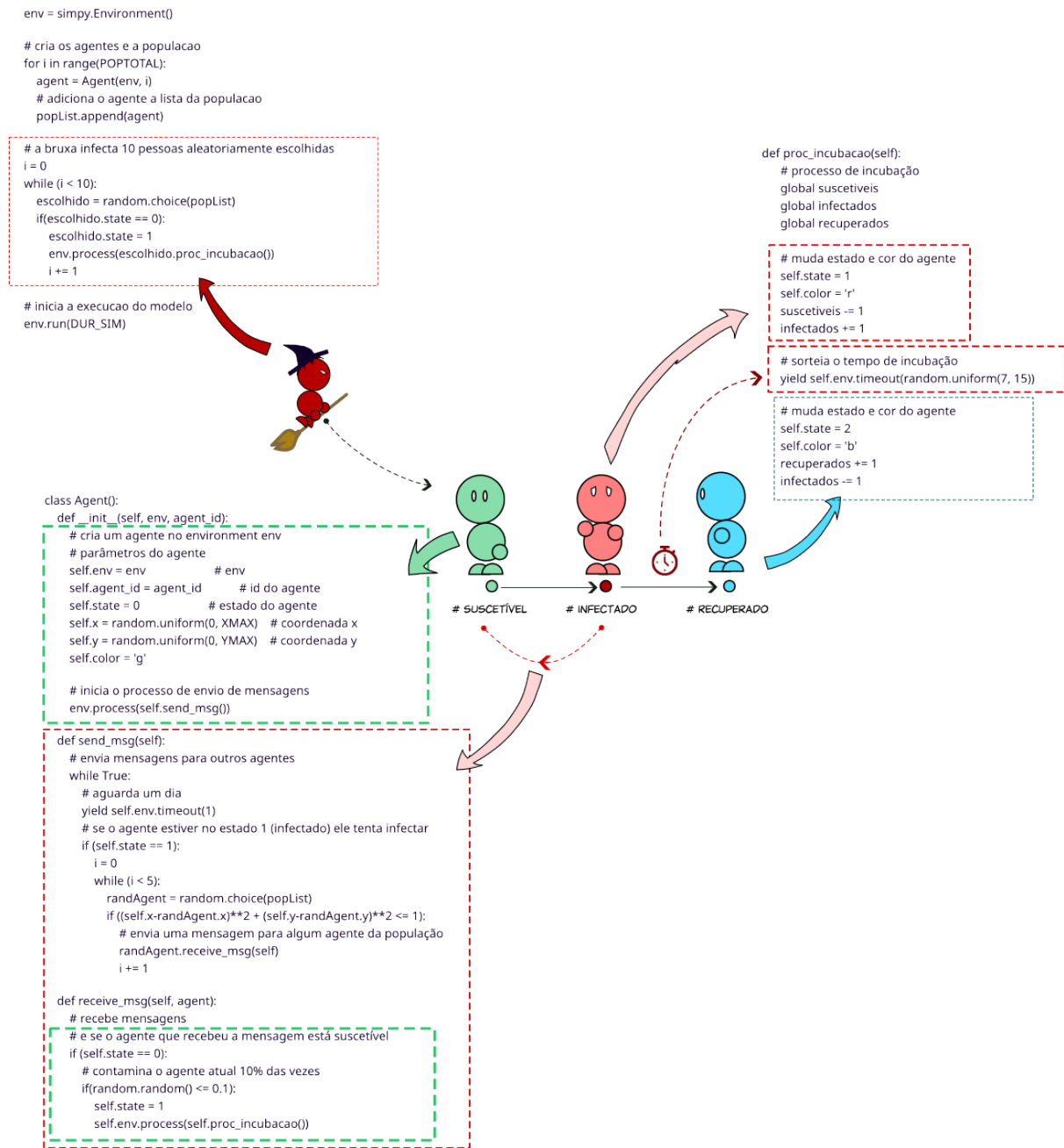


Figura 22.6. Mapa do simulador construído.

22.4 Melhorando o desempenho do código de simulação

Como já descrito, modelos de simulação são exigentes do ponto de vista computacional. Trata-se de algo que os profissionais estão acostumados e, por isso mesmo, procuram, desde a primeira linha de código, caminhos que tornem seus modelos mais rápidos. Você conseguiria identificar o vilão do programa? Aquela parte do código que parece consumir um processamento exagerado

e passível de *otimização*?

Se você pensou no procedimento de sorteio dos vizinhos, acertou! No modelo, os agentes não mudam de posição, então não haveria necessidade de executar essa verificação a todo momento. Trata-se de um trecho de código repetido, a cada dia, 5 vezes ou mais para cada um dos 10.000 agentes e que ainda envolve a operação “elevar ao quadrado”, uma função reconhecidamente exigente do ponto de vista computacional.

Uma possível solução seria, antes de iniciar a simulação, realizar o cálculo de classificação de todos os agentes entre si e armazenar uma lista de vizinhos dentro de cada agente. O problema desta solução é que teríamos de percorrer todos os 10.000 agentes e verificar quais, dentre os outros 9.999 agentes, estão dentro do raio mágico de 1 km. Você mesmo pode testar esta ideia (e vai se surpreender com o resultado!)

Em modelos de simulação baseados em agentes, o procedimento de classificar agentes por proximidade é algo típico, afinal, a distância entre agentes é parâmetro de definição de muitos sistemas reais (por exemplo: distanciamento social em pandemias, regiões de cobertura de ambulâncias e/ou viaturas policiais, nós de cadeias de suprimentos etc.)

Assim, o que será discutido a seguir pode ser algo replicável para a maioria dos seus modelos no futuro.

Antes de implementar alguma modificação, precisamos de um método para avaliar se a mudança de código é realmente vantajosa. A função `time()`, contida na biblioteca `time`, retorna a hora atual do computador e será utilizada para estimar o tempo consumido na execução do simulador e em partes dele¹⁴.

```
1 import simpy
2 import random
3 import time
4
5 import matplotlib.pyplot as plt
6 from matplotlib.widgets import Slider
7
8 start_time = time.time() # hora atual do computador
```

Para saber o tempo de processamento do início da execução do modelo até o instante anterior a execução da função `dashboard`—isto é: o tempo de execução do modelo, excluído o tempo de processamento dos gráficos—basta imprimir a diferença entre o tempo atual e o armazenado ao início do programa:

¹⁴A função `time()` retorna a hora exata do relógio interno do seu computador, ou seja: um valor dissociado do tempo computado pelo SimPy, `env.now()`, que é um relógio virtual.

```

1  # inicia a execução do modelo
2  env.run(DUR_SIM)
3
4  # imprime o tempo de processamento
5  print("Tempo de processamento: %.2f segundos" % (time.time() - start_time))
6
7
8  # inicia a apresentação de resultados
9  dashBoard()

```

Quando o simulador é executado no meu computador, um (Intel(R) Core(TM) Ultra 9 185H 2.30 GHz), o comando forneceu a seguinte saída:

```

1  Tempo de processamento: 20.10 segundos

```

22.4.1 Criando uma malha de conexões entre vizinhos mais próximos

Para evitar o cálculo repetitivo da distância entre agentes, podemos utilizar uma lista para cada agente armazenar suas conexões. O primeiro passo, portanto, é acrescentar um parâmetro que armazenará uma lista de vizinhos próximos em cada agente. No trecho de código a seguir, foi acrescentado o comando `self.vizinhos = []` na inicialização do construtor de agentes:

```

1  class Agent():
2      def __init__(self, env, agent_id):
3          # cria um agente no environment env
4
5          # parâmetros do agente
6          self.env = env                    # environment do agente
7          self.agent_id = agent_id          # id do agente
8          self.state = 0                    # estado do agente {0, 1 ou 2}
9          self.color = 'g'                 # cor do estado {'g', 'r' ou 'b'}
10         self.x = random.uniform(0, XMAX)  # coordenada x
11         self.y = random.uniform(0, YMAX)  # coordenada y
12         self.color = 'g'                   # cor do agente
13         self.vizinhos = []                 # lista de conexões
14
15         # inicia o processo de envio de mensagens
16         env.process(self.send_msg())

```

O processo de troca de mensagens não precisa mais calcular se o agente está próximo ou não, basta selecionar, aleatoriamente, um vizinho pertencente à lista criada no passo anterior. A biblioteca `random` possui um comando pronto para este fim: `random.choice()`, que aceita uma lista como parâmetro de entrada (linha 11):

```

1     def send_msg(self):
2         # envia mensagens para outros agentes
3         while True:
4             # aguarda um dia
5             yield self.env.timeout(1)
6
7         # se o agente estiver no estado 1 (infectado) ele tenta infectar
8         if (self.state == 1):
9             i = 0
10            while (i < 5):
11                randAgent = random.choice(self.vizinhos)
12                randAgent.receive_msg(self)
13                i += 1

```

O próximo e último passo é classificar os vizinhos pertencentes ao raio de 1 km e armazená-los na lista vizinhos de cada agente.

O problema da classificação de vizinhos é bem conhecido da Ciência da Computação e, não raramente, é dos primeiros problemas ao qual os futuros cientistas da computação são apresentados. O “problema do vizinho mais próximo” ou *k-ésimo vizinho mais próximo* (*k-nearest neighbors-KNN*) é um método de classificação de vizinhos por proximidade e muito utilizado na Simulação Baseada em Agentes.

Uma estrutura de dados de particionamento de espaço conhecida como árvore *k-d* (abreviação de “árvore *k*-dimensional”) é usada em Ciência da Computação para organizar pontos em um espaço *k*-dimensional. Árvores *k-d* são uma estrutura de dados útil para muitas aplicações, incluindo fazer nuvens de pontos e realizar pesquisas com uma chave de pesquisa multidimensional (como pesquisas de intervalo e pesquisas do vizinho mais próximo). Um tipo específico de árvore de particionamento de espaço binário é uma árvore *k-d*.

A biblioteca de código aberto *scipy*, voltada para aplicações de Matemática, Ciências e Engenharia, possui um módulo dedicado a estrutura de dados e algoritmos de posicionamento espacial, o *scipy.spatial*¹⁵. Deste módulo, utilizaremos a classe *cKDTree*¹⁶ que é uma implementação da árvore *k-d*.

Inicialmente, deve-se importar o módulo *spatial* da biblioteca *scipy* e a biblioteca *numpy*, pois a *cKDTree*¹⁷ exige como parâmetro de entrada um *array* com as coordenadas população:

```

1 import scipy
2 import random
3 import time
4 from scipy import spatial
5 import numpy as np

```

A construção de um *array* a partir de uma lista é tarefa trivial por meio do comando *np.array*:

¹⁵<https://docs.scipy.org/doc/scipy/reference/spatial.html#module-scipy.spatial>

¹⁶<https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html>

¹⁷A biblioteca *scipy.spatial*¹⁸ possui duas classes: *cKDTree* e *KDTree*. Contudo, a *KDTree* é mantida apenas por compatibilidade com versões anteriores, dado que a *cKDTree* é mais atualizada e mais eficiente.

```

1  # cria o array da população
2  popArray = np.array([[a.x, a.y] for a in popList])

```

A classificação dos vizinhos deve ser feita *antes* do início da simulação. Basicamente são duas etapas:

- Criar uma KDTree a partir de um *array* contendo as coordenadas de cada agente da população;
- Realizar uma consulta (*query_ball_point*) de vizinhos a 1 km de raio.

O comando `cKDTree` tem como seu principal parâmetro um *array* com as coordenadas (*x*, *y*) dos pontos a serem classificados. O comando possui ainda outros parâmetros, mas, por simplicidade, eles serão mantidos no seus valores *default*.

```

1  # monta a KDTree
2  tree = spatial.cKDTree(popArray)

```

Para a identificação dos vizinhos dentre do raio de 1 km, a classe *cKDTree* possui o método `query_ball_point(x, r)`¹⁹ que retorna uma *array* de listas com os *índices* dos pontos vizinhos entre si, dentro de um raio máximo *r* fornecido.

Vamos utilizar o método para a árvore criada *tree*, armazenando os valores na variável *ind* (e imprimir na tela o tempo de processamento até aqui):

```

1  # procura na KDTree os vizinhos que estão a 1 km entre si
2  ind = tree.query_ball_point(popArray, p=2, r=1.0, workers=2)
3
4  print("Tempo de processamento KDTree: %.2f segundos" % (time.time() - start_time))

```

Os parâmetros utilizados da função `query_ball_point(popArray, p=2, r=1.0, workers=2)` da classe *cKDTree*, significam:

- *popArray* é o *array* de pontos a serem comparados;
- *p=2* significa utilizar a distância euclidiana como métrica (*p=1* é a distância de Manhattan);
- *r=1.0* é o raio considerado de distância;
- *workers=2* é o número de *jobs* disparados para processamento paralelo (*workers=-1* implica em utilizar todos os processadores utilizados).

Neste ponto do programa, a variável *ind* contém uma preciosa *array* com as listas de vizinhos próximos para cada agente. Portanto, precisamos carregar cada lista de vizinhos na respectiva lista, `self.vizinhos`, de cada agente. De outro modo, percorrendo o *array* *ind*, a lista 0 deve ser carregada no agente 0, a lista 1 no agente 1 e assim por diante.

¹⁹https://docs.scipy.org/doc/scipy-1.15.0/reference/generated/scipy.spatial.cKDTree.query_ball_point.html#scipy.spatial.cKDTree.query_ball_point

Esta tarefa pode ser facilitada utilizando o comando do Python `enumerate(iterable, start=0)` que retorna um contador (começando em 0, por *default* do parâmetro `start`) e o valor de cada item (no caso cada lista de índices dos agentes).

Assim, para carregar a lista de vizinhos correta em seu respectivo agente, vamos criar um *loop* para percorrer todas as listas de vizinhos contidas na variável `ind`, com o comando `enumerate(ind)`:

```

1  # carrega a lista de vizinhos próximos para cada agente
2  for i, nn_indices in enumerate(ind):
3      popList[i].vizinhos = nn_indices
4      popList[i].vizinhos.remove(i)  # remove o próprio agente da lista de vizinhos
5
6  print("Tempo de processamento vizinhos: %.2f segundos" % (time.time() - start_tim\
7  e))

```

Note, no código anterior, a linha 4: `popList[i].vizinhos.remove(i)`. Ela remove da lista de vizinhos do agente o próprio agente, pois o comando de busca do vizinho mais próximo considera o próprio agente um vizinho dele mesmo.



Se o paciente leitor chegou até aqui, então é hora de executar o simulador e avaliar se essas modificações melhoraram ou pioraram o desempenho!

No meu computador, os tempos de processamento melhoraram e bem: (**A FAZER:** computador errado)

```

1  # Tempo de processamento KDTree: 1.19 segundos
2  Tempo de processamento vizinhos: 1.24 segundos
3  Tempo de processamento total: 8.78 segundos

```

O tempo de processamento foi de 20.10 para 8.78 segundos ou uma melhora de 56%! (**A FAZER:** computador errado) Espero que tenham gostado!

22.5 Como seguir a partir daqui

Muitos modelos baseados em agentes exigem recursos de visualização ou movimentação dos agentes, com controle de velocidades, distanciamento etc. Obviamente, a biblioteca SimPy não possui nenhum recurso deste tipo. Então, como então prosseguir? Uma resposta preguiçosa seria sugerir o uso da bibliotecas como a [PySimpleGui](https://www.pysimplegui.com/)²⁰ voltada a construção de interfaces gráficas (GUI) ou mesmo a [PyGame](https://www.pygame.org/)²¹, voltada à construção de jogos no Python e bastante popular.

Mais recentemente, surgiram alguns projetos voltados a construção de simuladores de agentes em Python e, talvez, o mais promissor seja o [Projeto Mesa](https://mesa.readthedocs.io/stable/)²². Além de um projeto bastante ativo,

²⁰<https://www.pysimplegui.com/>

²¹<https://www.pygame.org/>

²²<https://mesa.readthedocs.io/stable/>

a biblioteca já possui um conjunto robusto de recursos de animação e controle, inclusive com recursos de [Georreferenciamento](#)²³.

Na prateleira de pacotes comerciais, o único simulador de agentes disponível no mercado é o [AnyLogic](#)²⁴.

²³<https://mesa-geo.readthedocs.io/stable/index.html>

²⁴<https://www.anylogicbrasil.com.br/>

22.6 Conceitos desta seção

Conteúdo	Descrição
<code>random.choice(lista)</code>	seleciona aleatoriamente um dos elementos da lista fornecida como parâmetro da função
<code>np.array(objeto)</code>	cria um array a partir do objeto fornecido como parâmetro de entrada. O objeto pode ser qualquer coisa semelhante a um array.
<code>enumerate(iterable, start=0)</code>	A função <code>enumerate(iterable, start=0)</code> adiciona um contador a um objeto inteirável (lista, array, dict etc) e o retorna como uma lista de tuples (índice, valor).
<code>ckdTree(array)</code>	cria uma <i>KDTree</i> a partir da array fornecida como parâmetro de entrada.
<code>query_ball_point(x, r)</code>	Retorna todos os pontos a uma distância máxima r , do(s) ponto(s) x .

22.7 Desafios



Desafio 27: altere o modelo para considerar a situação em que, passado um tempo *uniformemente* distribuído entre 15 e 45 dias, o agente recuperado perde a imunização adquirida pela infecção e volta novamente para o estado suscetível. Simule pelo período de 1 ano. Você consegue interpretar os resultados obtidos?



Desafio 28: altere o modelo anterior para considerar o cenário de vacinação. Neste cenário, uma vacina é implantada no vigésimo dia da pandemia. A partir dali, o ritmo de vacinação programado é de 10% da população por dia, mas apenas entre os habitantes não vacinados. Como agravante, dada uma eficiente campanha de *fake news* promovida pela bruxa, 30% de toda a população se recusa a ser vacinada.

22.8 Solução dos Desafios 27 e 28

Desafio 27: altere o modelo para considerar a situação em que, passado um tempo *uniformemente* distribuído entre 15 e 45 dias, o agente recuperado perde a imunização adquirida pela infecção e volta novamente para o estado suscetível. Simule pelo período de 1 ano.

Neste caso, precisamos modificar o processo de incubação de modo que o agente retorne ao estado suscetível após o tempo sorteado entre 30 e 40 dias. Basicamente, ao terminar o período de incubação e entrar no estado *recuperado*, dispara-se um processo com o tempo desejado de imunização por contágio.

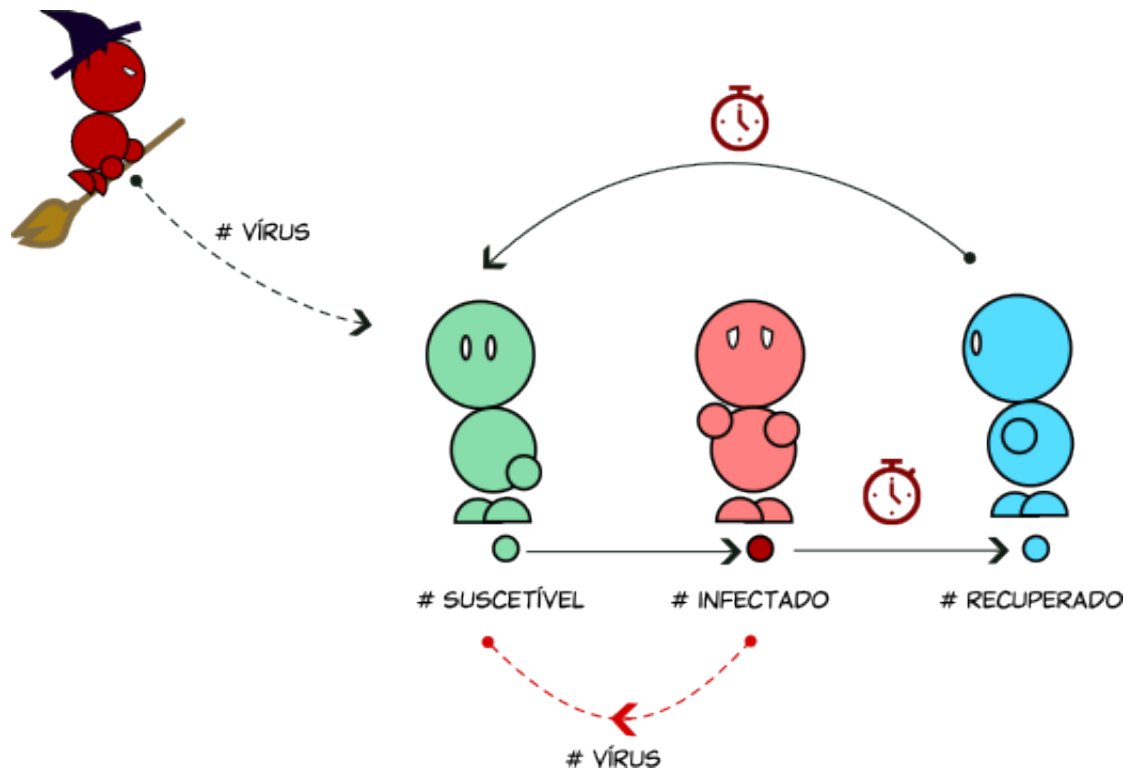


Figura 22.7. Representação do novo sistema com imunidade temporária.

Isto é feito acrescentando-se algumas linhas no método `proc_incubacao(self)` da Classe `Agente`:

```
1     def proc_incubacao(self):
2         # processo de incubação
3         global suscetiveis
4         global infectados
5         global recuperados
6
7         # muda estado e cor do agente
8         self.state = 1
9         self.color = 'r'
10        suscetiveis -= 1
11        infectados += 1
12
13        # sorteia o tempo de incubação
14        yield self.env.timeout(random.uniform(7, 15))
15
16        # muda estado e cor do agente
17        self.state = 2
18        self.color = 'b'
19        recuperados += 1
20        infectados -= 1
21
22        #sorteia tempo de imunidade
23        yield self.env.timeout(random.uniform(15, 45))
24
25        # muda estado e cor do agente para novamente suscetível
26        self.state = 0
27        self.color = 'g'
28        recuperados -= 1
29        suscetiveis += 1
```

Antes de executar o programa, não se esqueça de aumentar o tempo de simulação para 365 dias, como sugerido no texto do Desafio:

```
1 # parâmetros do modelo
2 POPTOTAL = 10000
3 DUR_SIM = 365
4 XMAX = 10
5 YMAX = 10
```

No meu caso, as curvas pandêmicas ficaram bem interessantes:

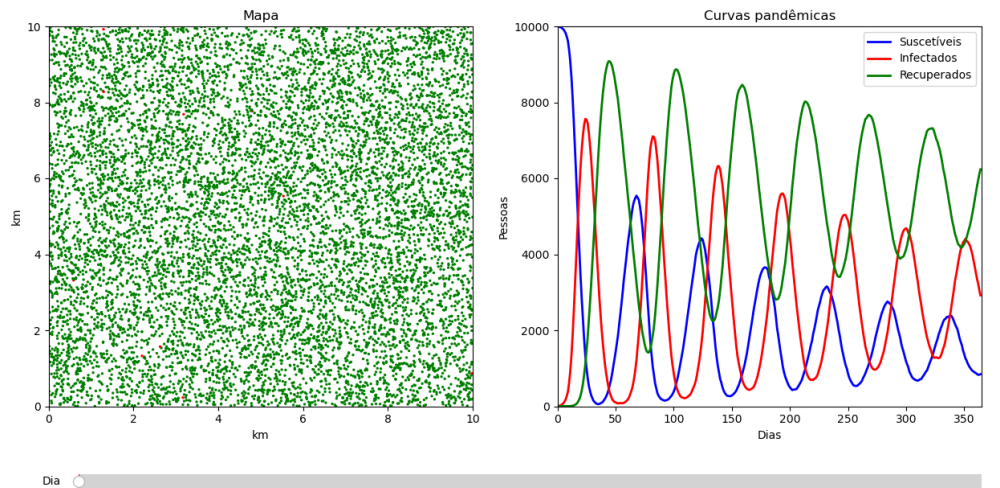


Figura 22.8. Cenário de simulação com tempo de proteção entre 15 e 45 dias.

A interpretação do gráfico é direta: o tempo médio de proteção por contaminação é de 30 dias, mas como o vírus ainda está presente na população ao final de cada ciclo e ele é de fácil transmissão, a pandemia retorna novamente no próximo ciclo (acho que eu já vi isso em algum lugar...). O que se tem no caso é de uma população grande, sob efeito de um vírus em que o tempo de proteção é pequeno em relação ao tempo de incubação.

Podemos pensar em outro caminho para o problema: qual seria o tempo mínimo de proteção para que a chamada “proteção de rebanho”, de fato proteja a população?

Desafio 28: altere o modelo anterior para considerar o cenário de vacinação. Neste cenário, uma vacina é implantada no vigésimo dia da pandemia. A partir dali, o ritmo de vacinação programado é de 10% da população por dia, mas apenas entre os habitantes não vacinados. Como agravante, dada uma eficiente campanha de *fake news* promovida pela bruxa, 30% de toda a população se recusa a ser vacinada.

Para este desafio, vamos acrescentar uma nova curva, “vacinados”, no gráfico. Assim, como feito para as outras curvas, precisamos adicionar listas e variáveis globais para monitorar o número de vacinados:

```

1  # variaveis globais
2  infectados = 0
3  recuperados = 0
4  suscetiveis = POPTOTAL
5  vacinados = 0
6
7  # listas para o número de pessoas em cada estado a cada dia
8  numS, numI, numR, numV, x = [POPTOTAL], [0], [0], [0], [0]
9
10 # lista que armazena a população de agentes
11 popList = []
12 popNVList = []

```

No código anterior, foi criada a variável global `vacinados`, a lista `numV` e uma lista específica, `popNVList`, que armazenará a população ainda não vacinada.

Na função `init` da Classe `Agent`, devemos acrescentar um atributo booleano `vacinado`, que indicará se o agente foi vacinado ou não:

```

1  class Agent():
2      def __init__(self, env, agent_id):
3          # cria um agente no environment env
4
5          # parâmetros do agente
6          self.env = env                # env
7          self.agent_id = agent_id      # id do agente
8          self.state = 0                # estado do agente
9          self.x = random.uniform(0, XMAX) # coordenada x
10         self.y = random.uniform(0, YMAX) # coordenada y
11         self.color = 'g'              # cor do agente
12         self.vizinhos = []            # lista de conexões
13         self.vacinado = False         # se vacinado ou não

```

Quanto o agente recebe uma mensagem/vírus, se ele for vacinado, então ele não inicializa o processo de incubação. Assim, a função `receive_msg`, tem agora um desvio de laço na (linha 6) que garante que apenas os não vacinados são candidatos a serem infectados.

```

1  def receive_msg(self, agent):
2      # recebe mensagens
3      # e se o agente que recebeu a mensagem está suscetível
4      if (self.state == 0):
5          # se o agente não foi vacinado
6          if(self.vacinado == False):
7              # contamina o agente atual 10% das vezes
8              if(random.random() <= 0.1):
9                  self.state = 1
10                 self.env.process(self.proc_incubacao())

```

O processo de incubação agora tem, também, um desvio de laço: caso o agente esteja vacinado (note que ele pode ser vacinado em qualquer estado) ele nunca mais retorna ao estado suscetível.

A função `proc_incubacao` é alterada da (linha 23) em diante:

```
1  def proc_incubacao(self):
2      # processo de incubação
3      global suscetiveis
4      global infectados
5      global recuperados
6
7      # muda estado e cor do agente
8      self.state = 1
9      self.color = 'r'
10     suscetiveis -= 1
11     infectados += 1
12
13     # sorteia o tempo de incubação
14     yield self.env.timeout(random.uniform(7, 21))
15
16     # muda estado e cor do agente
17     self.state = 2
18     self.color = 'b'
19     recuperados += 1
20     infectados -= 1
21
22     #sorteia tempo de imunidade
23     yield self.env.timeout(random.uniform(15, 45))
24
25     # muda estado e cor do agente
26     self.state = 0
27     self.color = 'yellow'
28     recuperados -= 1
29     if(self.vacinado == False):
30         suscetiveis += 1
31         self.color = 'g'
```

No código anterior, o agente retorna para o estado 0, mas com a cor `'yellow'`, indicando que ele está vacinado. Caso, o atributo `vacinado` seja `False`, a variável `suscetiveis` é incrementada e a cor do agente é alterada para `'g'` (verde).

O próximo passo é implementar o processo de vacinação:

```

1  def vacina(env):
2      # variáveis globais utilizadas
3      global vacinados
4      global suscetiveis
5
6      # aguarda 20 dias para iniciar a vacinação
7      yield (env.timeout(20))
8
9      while True:
10         # a cada dia, a vacinação programa
11         # atingir 10% da população não vacinada
12         numProgVacina = int(0.1 * len(popNVList))
13         if(numProgVacina > 0):
14             # o agentes são escolhidos ao acaso
15             popProgVacina = random.sample(popNVList, k=numProgVacina)
16             for p in popProgVacina:
17                 # os agentes são removidos da lista de não vacinados
18                 popNVList.remove(p)
19                 # 30% não aceitam a vacinação (maldita bruxa!)
20                 if(random.random() <= 0.70):
21                     p.vacinado = True
22                     vacinados += 1
23                     if(p.state == 0):
24                         suscetiveis -= 1
25
26         # aguarda ate o próximo dia
27         yield (env.timeout(1))

```

No código anterior, a linha 5 implementa o atraso de 20 dias para o início da vacinação. Iniciada a vacinação, o programa calcula quantas pessoas podem ser vacinada na linha 10. A seguir, o programa sorteia o número de vacinados previsto por meio da função `random.sample(popNVList, k=numProgVacina)` que retorna uma amostra de k agentes da lista *popNVList* escolhidos ao acaso e sem reposição.

É importante notar que, independentemente do agente estar sob efeito de *fake news* ou não, ele é removido da lista de não vacinados. A seguir é realizado o sorteio de que ele deseja ou não ser vacinado e, se ele estiver no estado suscetível, a variável *suscetiveis* é decrementada.

A função de monitoramento `save_pandemic(env)` sofre duas alterações: o acréscimo da variável global *vacinados* e o carregamento do número de vacinados no dia, na lista *numV*:

```

1  def save_pandemic(env):
2      # a cada dia, armazena as informações das curvas pandêmicas
3      # e estados das pessoas
4
5      # define as variáveis globais
6      global suscetiveis
7      global infectados
8      global recuperados
9      global vacinados
10
11     while True:
12         # carrega as listas de suscetíveis, infectados e recuperados
13         numS.append(suscetiveis)    # armazena os suscetíveis do dia
14         numI.append(infectados)     # armazena os infectados do dia
15         numR.append(recuperados)    # armazena os recuperados do dia
16         numV.append(vacinados)      # armazena os vacinados até o dia
17         x.append(env.now)           # armazena o numero do dia
18
19         # armazena no dicionario da população, a posição e a cor/estado
20         # de cada pessoa em um Tuple (pos x, pos y, cor)
21         popDayDict[env.now] = [ (p.x, p.y, p.color) for p in popList]
22
23         # aguarda ate o próximo dia
24         yield (env.timeout(1))

```

Por fim, devemos inicializar o processo de vacinação:

```

1  # inicia o processo de vacinação
2  env.process(vacina(env))
3
4  # inicia a execução do modelo
5  env.run(DUR_SIM)

```

No meu caso, o resultado foi bem interessante:

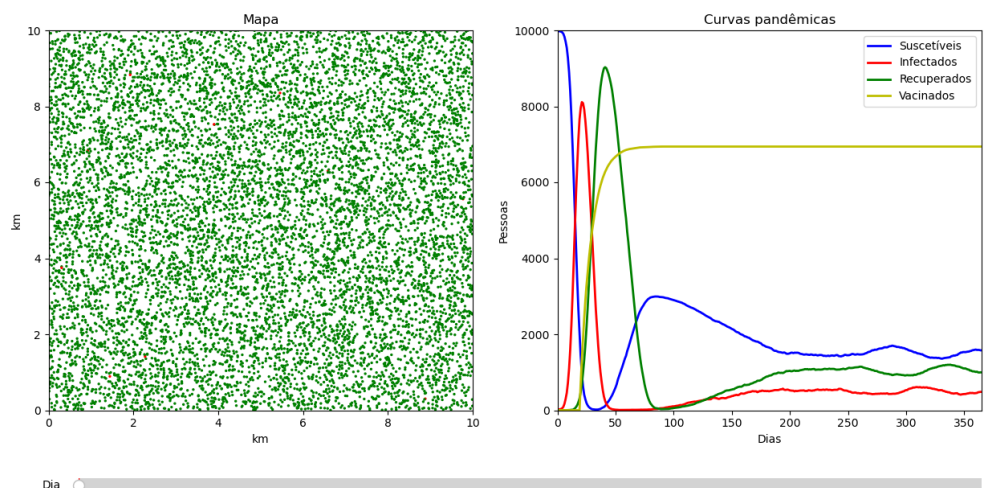


Figura 22.9. Cenário de simulação com vacinação de 70% da população.

É relativamente fácil notar que a campanha de desinformação da bruxa impede a eliminação do vírus no reino. Em situações reais, além do agravante da permanência do vírus em cerca de 30% da população, são facilitadas as condições para mutações do vírus—capazes de ultrapassar a barreira de imunização existente—e a própria contaminação de recém nascidos.

Você mesmo pode testar o modelo é verificar o que aconteceria com, por exemplo, uma campanha de vacinação que atingisse, por exemplo, 95% da população.

22.9 Teste seus conhecimentos

Os problemas de 1 a 4, referem-se ao modelo SIR desenvolvido neste capítulo.

1. Implemente a possibilidade de que a vacina tenha uma taxa de sucesso de 95%.
2. Considere um novo estado de “exposição ao vírus”, em que a pessoa é infectada mas leva um tempo para iniciar o estado infeccioso. Considere que este estado dura de 3 a 6 dias. Como isso afeta as curvas pandêmicas? Observação: neste caso, o modelo é denominado *SEIR: Susceptible Exposed Infectious Recovered*.²⁵
3. Considere que 20% das pessoas infectadas apresentam um agravamento do quadro após um tempo exponencialmente distribuído de média 3 dias e buscam apoio hospitalar. Estas pessoas são internadas e ocupam leitos hospitalares. Durante o período de internação, elas são isoladas e não conseguem infectar ninguém, mas o período de internação varia de 14 a 60 dias. Quantos leitos seriam necessários para atender apenas à pandemia? Assuma, como nível de serviço, a espera média por leito que não deve superar 3 horas.
4. A rede hospitalar descobriu que se uma pessoa aguarda mais do que 4 horas por um leito, ela entra em óbito 99% das vezes. Como isso altera o número de leitos necessários?

²⁵https://www.nature.com/articles/s41592-020-0856-2?_x_tr_sl&_x_tr_tl&_x_tr_hl